University of Pisa and Scuola Superiore Sant'Anna

Master Degree in Computer Science and Networking

DPI over commodity hardware: implementation of a scalable framework using FastFlow

Candidate

Daniele De Sensi

Supervisors

Prof. Marco Danelutto
Dr. Luca Deri

Academic Year 2011/2012

Alla mia famiglia

"Breathe, breathe in the air
Don't be afraid to care"

Pink Floyd

Ringraziamenti

A pochi giorni dalla fine di questo percorso, non posso fare a meno di pensare a tutte le persone che mi sono state accanto in questi anni e che mi hanno aiutato, ognuno a suo modo, a raggiungere questo traguardo.

Innanzitutto esprimo la mia profonda gratitudine al Prof. Marco Danelutto per la disponibilità e la fiducia che mi ha sempre dimostrato. Il suo supporto, formativo e morale, mi ha permesso di svolgere al meglio il mio lavoro di tesi.

Ringrazio inoltre il Dott. Luca Deri e il Dott. Massimo Torquati per l'aiuto fornitomi durante lo svolgimento della tesi e per essere sempre stati disponibili a risolvere i miei dubbi.

Vorrei poi ringraziare LIST S.p.A., per avermi dato la possibilità di approfondire le mie conoscenze, grazie alla borsa di studio sul tema "*Nuove architetture hardware-software ad alte prestazioni*", conferitami durante il periodo compreso tra Febbraio e Luglio 2012.

Il ringraziamento più sentito lo rivolgo a mio padre, mia madre, Fabri e Davide. Senza la loro guida tutto questo non sarebbe stato possibile. Il loro supporto e la loro pazienza mi hanno sempre accompagnato in tutti questi anni. Inoltre ringrazio gli zii, i nonni, i cugini e tutti coloro che mi hanno sempre fatto sentire la loro vicinanza anche se a chilometri di distanza.

Un ringraziamento speciale va a Mari, che mi è stata accanto sin da quando ho iniziato questo cammino. La sua presenza e il suo appoggio sono stati per me di fondamentale importanza.

Ringrazio poi Andre, Cica e Picci, che hanno condiviso con me questi mesi di tesi. Tra una battuta e l'altra ci siamo sorretti a vicenda e siamo riusciti a sorridere anche di fronte a situazioni sfavorevoli.

Un affettuoso ringraziamento a coloro che ho conosciuto in questi ultimi due anni e con i quali ho passato alcuni dei momenti più piacevoli di questo percorso universitario: Alessio, Bob, Davide, Ema, FraPac, Frenci, Gian, Luigi, Simo, Sina, Tixi e Tudor.

Grazie a Bice, Francesca, Nicola, Roberta, Simone e Stefano, per le serate

passate insieme e per l'affetto che mi hanno sempre dimostrato.

Un pensiero sentito va infine a Maria e Gianluca per la loro sincera amicizia.

A loro tutti va il mio sincero ringraziamento.

Daniele

a89bc677da53e61abf3153299ecb935a

Contents

Introduction **i**

1 Thesis context, related work and tools **1**
1.1 Protocol classification 1
1.1.1 Related work 3
1.1.1.1 Hardware based solutions 3
1.1.1.2 Software based solutions 4
1.2 Structured parallel programming 6
1.2.1 Pipeline 9
1.2.2 Farm 10
1.2.3 Considerations about average values 12
1.2.4 Related work 13
1.2.4.1 Muesli 14
1.2.4.2 SkeTo 14
1.2.4.3 SkePu 14
1.3 FastFlow 15

2 Architectural design **19**
2.1 Framework design 19
2.2 Parallel structure design 22

3 Implementation **28**
3.1 Application interface 29
3.2 Network and transport headers processing 31
3.2.1 IP fragmentation support 32
3.3 Flow data management 36
3.3.1 Hash functions analysis 37
3.4 Protocol identification 39
3.4.1 TCP stream management 39
3.4.2 Protocols inspectors 41
3.5 Callbacks mechanisms 43

3.6 Demo application 48
3.7 Parallel structure 49
3.7.1 Interface with the application 49
3.7.2 Implementation details 51

4 Experimental results **54**
4.1 Hash functions analysis 56
4.2 Analysis of Move To Front technique 60
4.3 Comparison of L3 farm scheduling strategies 61
4.4 Speedup 63
4.4.1 Protocol identification 63
4.4.2 Processing of extracted data 64
4.4.3 Application speedup 66
4.5 Assessment 66
4.5.1 Comparison with other sequential software solutions . 67
4.5.2 Comparison with hardware solutions 71

5 Conclusions **74**

A Code of the sequential framework **77**
A.1 API implementation 77
A.2 Flow table 129
A.3 IPv4 defragmentation 155
A.4 TCP stream management 199

B Code of the parallel implementation **212**
B.1 Nodes definition 212
B.2 API implementation 228
B.3 Test 262
B.3.1 Performance test 262
B.3.2 Demo application 267

References **274**

List of acronyms

IP	Internet Protocol
UDP	User Datagram Protocol
TCP	Transmission Control Protocol
P2P	Peer to Peer
VoIP	Voice over IP
ISP	Internet Service Providers
DPI	Deep Packet Inspection
NI	Network Intelligence
MTU	Maximum Transmission Unit
FPGA	Field Programmable Gate Array
CAM	Content Addressable Memories
IPS	Intrusion Prevention System
IDS	Intrusion Detection System
SPSC	Single Producer Single Consumer
NIC	Network Interface Controller
FIFO	First In First Out
OSI	Open Systems Interconnection
URL	Uniform Resource Locator
GPU	Graphics Processing Unit
MPI	Message Parsing Interface
OpenMP	Open Multiprocessing
CUDA	Compute Unified Device Architecture
OpenCL	Open Computing Language
STL	Standard Template Library

List of Figures

1.1 Example of a pattern split between two TCP segment 4
1.2 Pipeline skeleton 9
1.3 Farm skeleton 11
1.4 FastFlow architecture (taken from [1]) 15
1.5 Examples of computational graphs (taken from [1]) 17

2.1 Flow diagram of the framework 20
2.2 Structure of the framework with flow table partitioned among the set of workers (collector not shown) 23
2.3 Structure of the framework when the emitter is a bottleneck (the collector of the second farm is not shown) 25

3.1 Stateful interaction 29
3.2 Stateless interaction 30
3.3 IP fragmentation process 33
3.4 List of IPv4 fragments 34
3.5 List of IPv4 fragmented datagrams 35
3.6 Hash table containing the IPv4 sources which have outstanding fragmented datagrams. For sake of simplicity, the list of outstanding fragmented datagrams is not shown 35
3.7 Interaction between the application and the parallel framework 50

4.1 Hash functions uniformity over the flows contained in **CAIDA** dataset 57
4.2 Hash functions execution times over the flows contained in **CAIDA** dataset 58
4.3 Comparison of the impact of the different hash functions over the framework bandwidth (in millions of packets per second) 59
4.4 Analysis, using Sigcomm dataset, of the impact of MTF strategy over the global bandwidth of the framework varying the average load factor of the table 61

4.5 Comparison of the impact of the different scheduling strategies for the L3 farm 62
4.6 Comparison of the speedup for protocol identification using different datasets 63
4.7 Comparison of the speedup when packet processing capabilities are required 65
4.8 Comparison of the speedup of the HTTP payload pattern matching application 66
4.9 Comparison of the bandwidth (in millions of packets per second) of our framework with respect to nDPI over different datasets 68
4.10 Comparison of the bandwidth (in millions of packets per second) of nDPI with respect to our framework varying the number of workers of the farm 69
4.11 Bandwidth of the HTTP payload pattern matching application 72

Introduction

In the last years we assisted to a large increase of the number of applications running on top of IP networks. Consequently has increased the need to implement very efficient network monitoring solutions that can manage these high data rates.

Network administrators always deal with the problem of classifying and analyzing the type of traffic is traveling on their networks. This can be functional to multiple purposes, like:

Network security We assisted recently to an increase in the complexity of the applications which run on top of IP networks. Consequently, we have seen a shift from so-called "*network-level*" attacks, which target the network the data is transported on (e.g. Denial of Service), to content-based threats which exploit applications vulnerabilities and require sophisticated levels of intelligence to be detected. Many of these threats are designed to bypass traditional firewalls systems and often they cannot be detected by antivirus scanners [2]. Accordingly, it is no more sufficient to have only a software solution on the client side but we also need to run some controls on the network itself. These types of controls needs to identify the application protocol carried inside the packet and possibly to analyze its content in order to detect a potential threat.

Quality of Service and Traffic shaping Another situation in which protocol classification may be useful is when network administrators may want to limit the transmission of packets that might degrade overall network performance [3, 4]. For example, the streaming of large amounts of data for an extended period of time (like the one performed by Peer to Peer applications) may degrade the other users' experience on the network. Consequently, to improve the quality of service, traffic classification engines may identify the problematic applications and then reduce their priority level. Alternatively, it could be used in the

opposite way, ensuring an higher priority to sensitive applications like Voice Over IP (VoIP) or video streaming.

Data leak prevention The analysis of the data traveling over an enterprise network may be useful to detect potential data breach or ex-filtration transmissions and to prevent and block them. In data leakage incidents, sensitive data is disclosed to unauthorized personnel either by malicious intent or inadvertent mistake. These data may include: private or company information, intellectual property, financial or patient information, credit-card data, and other information depending on the business and the industry [5]. This is a very felt issue since the leakage of sensitive data can lead an organization to face with criminal lawsuits [6] which could then cause the bankrupt of the company or takeovers by larger companies. Moreover, leakage of personal data may involve every citizen, since their data are stored electronically by hospitals, universities and other public organizations.

Network access control Traffic classification and inspection mechanisms may be used by network administrators or Internet Service Providers (ISPs) to ensure that their acceptable use policy is enforced, allowing the access to the network only to some kind of traffic. This can be used, for example, for parental control [7] or to limit the access to the network in order to improve employees productivity [8].

For all these tasks, identification of the application protocol (e.g. HTTP, SMTP, SKYPE, etc. . .) is required and, in some cases, extraction and processing of the information contained in its payload is needed.

However, in order to be able to identify the protocol, is no more sufficient to only look at TCP/UDP ports because protocols often run on ports different from the assigned ones.

To tackle these problems, in the recent years, Deep Packet Inspection (DPI) technology has emerged. Differently from classic monitoring solutions [9] that classify traffic and collect information only based on the <SOURCE IP, DESTINATION IP, SOURCE PORT, DESTINATION PORT, L4 PROTOCOL> tuple, DPI inspects the entire payload to identify the exact application protocol.

In most cases, protocol identification is mainly achieved by comparing the packet payload with some well-known protocols patterns or, if the protocol is encrypted and we accept to have a reduction in the accuracy, by using some statistical knowledge about the protocol [10, 11, 12].

However, considering the current networks rates, this kind of processing is not suitable for offline analysis, which would require to store the packets traces for further elaboration. Therefore, we need to manage the incoming

packets as soon they arrive, without relying on the possibility to store the packets that we are not able to manage for later processing.

Moreover, some protocols like HTTP are so extensively used [13] that we may want to analyze the carried content to classify it in more sub protocols (e.g. Facebook, Gmail, Netflix and others can all be viewed as HTTP sub protocols). Furthermore, as we anticipated, some network management tasks require to extract protocol content and metadata. For this reason, Network Intelligence (NI) technology has been recently proposed [14]. This kind of analysis is built on DPI concepts and capabilities and extends them with the possibility to extract and process content and metadata carried by the protocol.

However, since the network could reorder the messages, to correctly extract the data carried by the protocol, some applications may require to the NI engine to manage the expensive task of IP defragmentation and TCP stream reordering and reassembly. In general, this is not an easy task and needs to be carefully designed since it may be vulnerable to some exploits [15, 16].

For these reasons, and considering the high bandwidth of the current networks, we need highly efficient solutions capable of processing millions of packets per second.

This kind of processing is in many cases implemented, at least in part, through dedicated hardware [17, 18, 19]. However, full software solutions may often be more appealing because they are typically more economical and have, in general, the capability to react faster to protocols evolution and changes.

Furthermore, with the shift from single core to multicore processing elements, it should be possible, in principle, to implement high speed DPI on non-dedicated hardware and, at the same time, to provide performances comparable to those of special purpose solutions.

However, common existing DPI software solutions [20, 21, 22, 23] don't take advantage of the underlying multicore architecture, providing only the possibility to process the packets sequentially. Therefore, when multicores have to be exploited, they demand to the application programmer the complicated and error-prone task of parallelization and fine tuning of the application. Furthermore, many DPI research works that can be found in literature [24, 25, 26, 27] and which exploit multicore architectures are often characterized by a poor scalability, due to the overhead required for synchronization or to the load unbalance among the used cores.

Moreover, in order to be able to manage high data rates, some of these solutions don't inspect the entire payload, using "lightweight" approaches which analyze only few bytes of the packet and are, in general, less accurate

than solutions which adopt full packet inspection. Furthermore, these solutions are limited to protocol identification, without providing any mechanism to locate, extract and process the data and metadata contained inside the packets, which are basic requirements for a NI engine.

> The target of this thesis is to explore the possibility to apply *structured parallel programming* theory to support the implementation of an high bandwidth streaming application as the one just described. Using these concepts, we would like to design, realize and validate a DPI and NI framework capable of managing current networks rates using commodity multicore hardware.

Accordingly we will possibly analyze the entire payload in order to reach an high accuracy when identifying the protocol and, at the same time, to give to the application programmer the possibility to specify which protocol metadata to extract and how to process them. However, considering that in general the processing of the extracted data may be computationally expensive, we need to design our framework in such a way that is possible to distribute it over the available processing elements.

> We will show that, using *structured parallel programming* concepts and with an accurate design, we are able to do stateful packet inspection over current network rates using commodity hardware and to achieve a good speedup and results comparable to those obtained by dedicated hardware.

In order to be able to reach our goals, we implemented our framework using the *FastFlow* library [28, 1] which, thanks to its low latency communication mechanisms, allowed us to write an efficient and scalable DPI framework.

Our proof-of-concept framework supports at the moment some of the most common protocols (such as HTTP, POP3, IMAP, DHCP, DNS, MDNS and few others). Moreover, the framework has been designed in such a way that it could be easily extended with new protocols with limited changes in the code.

The rest of the thesis is structured in this way:

- In Chapter 1 we will analyze the context in which this thesis is located, briefly describing some of the techniques commonly used to perform DPI and showing why our work differs from already existing works. Moreover, we will give the definition of *structured parallel programming*, exposing the main features provided by *FastFlow*, the library we used to implement these concepts.

- We will then describe in Chapter 2 the global design of the framework and how, using the concepts introduced in Chapter 1, it efficiently exploits the underlying multicore architecture.

- In Chapter 3 we will discuss the main features of the framework, its internal structure and a sketch of the API offered to the user. We will then describe how we split its parts among the different execution modules, avoiding any type of unnecessary synchronization mechanism in such a way that they can be completely independent from each other.

  Moreover, to validate the framework, we present an application implemented on top of it, which scans all the HTTP messages traveling on the network searching for some specified patterns.

- In Chapter 4 some experimental results will be presented and analyzed, validating our framework and showing that it allows to write scalable DPI applications exploiting the processing power providing by the underlying multiprocessor architecture.

- In Chapter 5 conclusions will be drawn and the limits of the framework will be analyzed. Moreover, we will propose some ideas for possible features which could be added in future to our work.

Chapter 1

Thesis context, related work and tools

In this chapter we will introduce the context of this thesis, analyzing the related work and introducing the concept of *structured parallel programming*, which characterizes our thesis with respect to other existing works. We will then describe *FastFlow*, the library we used to apply these concepts inside the framework.

1.1 Protocol classification

The increasing number of applications running on top of IP networks is making more and more urgent the need to implement very efficient tools for an accurate protocol classification. The ability to identify and classify the packets according to the protocol they carry may be useful for different purposes.

As an example, as far as network security is concerned, in the recent years we have seen a shift from so-called "*network-level*" attacks, which target the network they are transported on (e.g. Denial of Service), to content-based threats which exploit applications vulnerabilities and require sophisticated levels of intelligence to be detected. For some of these threats, it is no more sufficient to have only a software solution on the client side but we also need to run some controls on the network itself [2]. These types of controls needs to identify the application protocol carried inside the packet and possibly to analyze its content in order to detect a potential threat.

For this kind of applications, the accuracy is extremely important, as wrong assumptions on what is happening on the network could lead to nasty effects.

Traffic classification solutions may be roughly divided into two main cat-

egories:

Flow based In this case the packets are grouped in flows. A flow is defined as a set of packets with the same <Source IP address, Destination IP address, Source port, Destination port, Transport protocol identifier>. These flows are bidirectional, therefore packets with the key: <w.x.y.z, a.b.c.d, r, s, t> belong to the same flow of the packets with key: <a.b.c.d, w.x.y.z, s, r, t>.

The recognition of the packet is done by using both the information carried by the current packet and the accumulated information about the flow. However this requires to store some kind of data about the previous packets received for the flow (e.g. received bytes, state of the TCP connection and others).

Packet based In this case each packet is analyzed independently from the others and there is no need to store any information inside the classification engine.

Moreover, traffic classification could also be divided according to the type of mechanisms used to identify the protocol carried inside the packets.

Port based This is one of the simplest and most used techniques. It simply tries to classify the protocol according to the ports used by the application. However, this approach exhibits a low accuracy [29, 30]. Indeed, many applications often use ports different from the standard ones or they use dynamic ports which are not known in advance.

Statistical The solutions based on this technique try to identify the protocol using statistical knowledge about the distribution of the packet length or by analyzing the packets interarrival times. Some of these solutions use machine learning techniques to train the engine with previously captured and classified traffic traces to characterize the statistical properties of the specific protocol.

The advantage of this approach is that it doesn't need to look the packet payload at all. Despite this can be very useful in presence of encrypted protocols, it may not be the best solution for non encrypted protocols since it provides a lower accuracy with respect to that provided by payload based techniques.

Payload based This class of solutions try to identify the protocol by searching inside the payload for well known protocol signatures or by analyzing its content and correlating it with that of the other packets

belonging to the same flow. Accordingly, in some cases we may need to maintain some information between successive received packets of the flow.

Protocol signatures may be both generated by hand or by using machine learning techniques [31], which automatically extract application signatures from IP traffic payload content.

Since payload based technique can analyze the entire payload, it is the most accurate technique and the only one which allows the extraction and processing of the content and metadata carried by the protocol. However, when metadata extraction is performed or if a very high accuracy is needed, TCP/IP normalization may be required by the application. TCP/IP normalization aims at solving the problems relative to IP fragmentation (i.e. when an IP datagram is larger than the Maximum Transmission Unit (MTU) of the outgoing link and hence it is divided in two or more fragments) and TCP reassembly (e.g. when an application message doesn't fit in a single IP datagram and hence it is split across different TCP segments) [32].

Moreover, in some cases IP fragmentation and TCP segmentation may be used by an attacker to evade the DPI engine [33, 34]. As an efficient alternative to TCP/IP normalization, some existing works [35, 36] try to identify this kind of misbehaving situations and to divert them to a slow path engine, which reassemble such flows. Conversely, the other segmented flows are managed using faster techniques which can match the signatures also in presence of fragmentation or segmentation. However, these techniques are not sufficient when the application explicitly requires to process the data contained inside the protocol payload in the same order they are sent.

1.1.1 Related work

We will now describe some of the existing works in the field of protocol classification, analyzing both hardware and software solutions and presenting their peculiar characteristics.

1.1.1.1 Hardware based solutions

Many hardware solutions exist which perform, at least in part, the steps required for protocol classification.

One of the most common cases, is to use dedicated hardware to search for patterns inside the packets both by using Field Programmable Gate Arrays

(FPGAs) [17, 37, 38] or by using Content Addressable Memories (CAMs) [39, 18, 19, 40]. These solution may both use exact matching approaches or accept a certain rate of false positive matching by using, for example, bloom filters.

The patterns searched inside the packet may both be application signatures used to identify the application protocol, or may be patterns identifying security threats, similarly to what is done by Intrusion Detection/Prevention Systems (IPS/IDS) [41, 42].

However, these solutions only perform stateless pattern matching, without any knowledge of the structure of the packet or of the relationship between them. This kind of knowledge should be provided in a real environment, since its absence could lead to the impossibility to find these patterns.

Let us consider for example the scenario depicted in figure 1.1, where the string DIVISIONBELL is split in two different TCP segment.

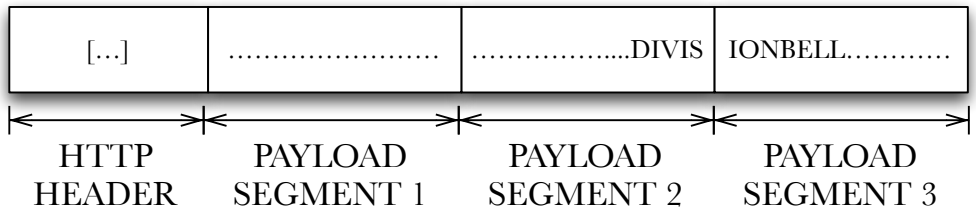

Figure 1.1: Example of a pattern split between two TCP segment

In that case, if TCP analysis and normalization is not provided, the two segments would be analyzed separately and thus the string would not be found by the pattern matching engine. Moreover, this could also happen when these solutions are used to identify the application protocol by signature matching, thus leading to the impossibility to identify some application flows. This is the reason why pattern matching alone is not sufficient and some kind of process and knowledge about the characteristics of the flow is required.

1.1.1.2 Software based solutions

nDPI This is a well known, open source DPI library which supports more than 100 protocols [23]. It has been forked from OpenDPI [20] and optimizes and extends it with new protocols. An inspector is associated to each supported protocol and each of them analyzes the entire packet payload searching for characteristics or signatures which allows it to identify the carried protocol. However, it lacks of support for IP and TCP normalization

and doesn't provide any possibility to specify which data to extract once that the protocol has been identified. Moreover, it doesn't have any support for multiprocessor architectures, demanding to the application programmer the difficult and error prone task of parallelizing his application.

Libprotoident In this work the concept of "Lightweight Packet Inspection" (LPI) is proposed [22]. Instead of analyzing the entire payload, LPI tries to identify the application protocol by simply looking to the first four bytes of the payload. However, many applications use HTTP to carry their data and they in principle could be classified by looking to the `Content-Type` or `User-Agent` fields. Consequently, the lack of the remaining part of the payload makes impossible to analyze the HTTP header contents and sub classify HTTP traffic accordingly. Therefore, when such level of traffic classification is required, this approach is not sufficient. Moreover, also in this case, the library provides only the possibility to process the packets sequentially.

L7-filter This is a packet classifier which use regular expression matching on the application layer data to determine what protocols are being used [21]. However, also in this case, it doesn't provide any way to specify how to extract and process the data carried by the protocol once that the packet has been classified. Moreover, as shown in [43, 44], due to simple regular expression matching, its accuracy is lower with respect to more precise approaches as the one adopted by *nDPI* and *libprotoident*.

Solutions with multicore support Software solutions which exploit network processors or commodity multicore architectures can be found in literature. We will now describe some of them together with the choices that have been taken for their parallelization.

- In [24] a software solution which exploit a Cavium network processor is proposed. The main idea behind this solution is to distribute the packets among different threads by means of a shared queue. However, the access to the queue is protected by lock mechanisms and, as also stated by their developers, this is the reason why they are not able to achieve a good scalability.

- In [27] the different steps performed by the DPI engine are profiled and then distributed among the available cores, trying to keep the work balanced. However this is not a general approach and, if executed on different machine, it would require to do again from scratch profiling, design and implementation of, possibly, a different partitioning among

the activity. Moreover, despite lock free data structures are used, also in this case scalability problems are experienced, even using a relatively low number of cores. In this particular case, this is due to the unbalancing between the parts of the application executed by the different cores.

- In [25] different strategies to schedule the packets among threads are analyzed. These strategies take into account factors like cache affinity and load balancing. Anyway, also considered the best proposed strategy, this solution still suffers a limited scalability.

- Also in [26] a distribution of the work among different threads running on separate cores is proposed. However, also this solution is characterized by serious scalability problems caused by the overhead due to synchronization among the threads.

1.2 Structured parallel programming

As discussed above, existing software solutions often do not provide mechanisms to efficiently exploit the current multiprocessor architectures and, when such possibility is provided, they suffer from scalability problems. Indeed, writing an efficient parallel application is not only matter of adding some threads and force the correctness through mutual exclusion mechanisms, but it needs an accurate design in order to achieve the required efficiency.

For example, according to *libprotoident* creators[1]: *"Determined that adding threading to* libprotoident *was completely not beneficial - in fact, it ended up running much slower than before. This seems to be mainly due to the rules being so simple. There was no performance gain to compensate for the overhead of locking mutexes and synching threads that was introduced."*. Moreover, this is something that have also been experienced by other software solutions [24, 27, 25, 26]. As we will see in section 4.4.3, in some cases this implies a low scalability also when a relatively low number of cores is used. Besides the overhead due to synchronization, this can often be caused by poor design choices which will lead to load unbalancing among the used cores.

Moreover, these solutions provide only the possibility to identify the application protocol, without offering any facility to extract and process the data carried by the protocol once that it has been identified.

[1] http://www.wand.net.nz/content/weekly-report-21102011

For this reason in this thesis we would like to propose and implement a novel approach which, thanks to an accurate design, can take advantage from the underlying architecture and, at the same time, doesn't renounce to the precision of a stateful *payload based* classification approach. Moreover, differently from existing solutions, we would like to provide to the application programmer the possibility to specify the data to be extracted from the packet once that the protocol has been identified.

In order to reach this target, we applied to our work the concepts of *structured parallel programming* methodology [45].

Any application can, in general, be viewed as a set of concurrent activities cooperating to reach a common objective. However, developing an efficient parallel application is often a difficult task. Indeed, the user doesn't have only to deal with the algorithm details, but he needs to also take care of the setup of the concurrent modules in which the application is divided, to map and schedule them on the architecture on which the application will run and to implement correct and efficient techniques to let them communicate. All these activities require a big and error prone programming effort.

When structured parallel programming methodology is used, it's possible to hide a part of this complexity and to let it be managed by the library or the programming language used to describe the application [46, 47]. In general, starting from the sequential description of the application, the user can individuate a graph of activities which models it and which can express the same application as a composition of parallel modules.

Considered that the same concurrent application can, in principle, be modeled by many parallel modules compositions, different performance metrics can be used to evaluate a specific graph of activities and to compare it with alternative solutions. In our specific case, where the application works on a stream of packets, we are interested in the following performance metrics:

Bandwidth B Defined as the average number of packets per second that the DPI framework is able to process.

Service Time T_S Defined as the average time interval between the beginning of the executions of two consecutive stream elements. It is the inverse of the bandwidth, therefore it can be expressed as $T_S = \frac{1}{B}$.

Latency L Defined as the average time needed to complete the processing over a single packet. In our case this metric is particularly important when, before forwarding the packet, the application needs to wait for the result of its processing to decide, for example, if it matches or not the filtering rules. Indeed, if an high latency is required by the

processing of the packet, we could introduce a not acceptable delay in the forwarding of the packet over the network.

However, to avoid this problem, the packet could be passed to the framework and then immediately forwarded without waiting for the result. In this way, also if the first packets of the flow are forwarded independently from the processing result, the filtering would be still applied to the remaining part of the flow. Anyway, also in this case, we would like to have a low latency in such a way that the decision is taken before the termination of the flow.

Nevertheless, in all the other cases, the only metric to be take into consideration should be the bandwidth (and then the service time).

Furthermore, we are interested in the average **Interarrival Time** of the packets to the application T_A. This quantity is the inverse of the rate of packets arriving to the framework. If we are able to structure the framework so that it has a service time $T_S \leq T_A$, then the framework will be able to process all the received packets.

Another important metric we are interested in is the **Speedup**. For a computation composed by n modules it is defined as

$$Speedup(n) = \frac{B_{seq}}{B_n}$$

where B_{seq} is the bandwidth of the sequential framework while B_n is the bandwidth of the parallel framework when n modules are activated. Ideally, we should have $B_n = \frac{B_{seq}}{n}$ and then $Speedup(n) = n$. However this is different for what will really happen because, in the parallel solution we will have, in addition to the latencies of the sequential version, some other latencies caused by factors like communication latencies and memory contentions.

For these reasons, in general we will have $Speedup(n) \leq n$. However, as we will see in Chapter 4, there are some cases in which we could have $Speedup(n) > n$. This could happen when, for example, the solution composed by n modules exploits a better spatial cache locality as a consequence of the reduction of the size of the working set.

Among all the possible graphs that can be used to represent an application, many real application can be described using some recurrent schemes called *skeletons* [45, 48]. Skeletons are programming patterns which allow to model typical forms of parallelism exploitation, expressed in parametric form. They have a precise semantic and are characterized by a specific *cost model*, which can be used to evaluate their performance metrics. Moreover, they can be composed and nested together to represent more complicated parallelism forms.

Skeletons can work over single data elements or over a so called "stream" of tasks. A stream is a possibly infinite sequence of values of the same type. For example, in our case, we can consider each packet arriving to the framework as an element of the stream.

Depending on the type of parallelism modelled, we can classify them in the following categories:

Stream parallel These skeletons exploit the parallelism among computations relative to independent tasks appearing on the input stream of the program.

Data parallel Are those exploiting parallelism in the computation of different sub tasks derived from the same input task.

As we will show in Chapter 2, for our framework we are interested in two particular *stream parallel* skeletons: *farm* and *pipeline*.

1.2.1 Pipeline

This skeleton can be used when the application computes a function $F(x)$ which can be expressed as a composition of n functions:

$$F(x) = F_N(F_{N-1}(\ldots F_2(F_1(x))\ldots))$$

where x is the received task. In this case, the corresponding graph of activities (shown in figure 1.2) is made by one module (in this case called *stage*) for each computed function.

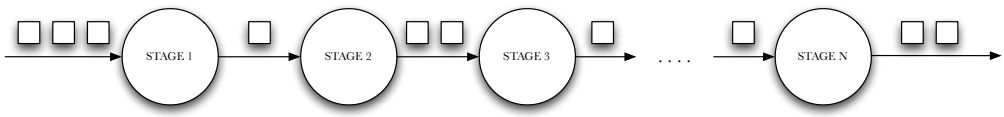

Figure 1.2: Pipeline skeleton

If the function computed by the *i-th* stage has latency L_i, then the service time of the pipeline will be

$$T_S = \max_{1 \le i \le N}(L_i + L_{com_i}) \tag{1.1}$$

where L_{com_i} is the time spent by the *i-th* stage to read the task from the input channel and to send it over the output channel. In general, if an

appropriate support is present, L_{com_i} could be overlapped with the computation latency L_i. However, this is not our case because the framework we used doesn't have any support for this kind of overlapping and only allows to execute the communications sequentially with the computation. For this reason, from now on we will always consider that the communication will never overlap with the computation.

The pipeline latency instead is:

$$L = \sum_{i=1}^{N} (L_i + L_{com_i}) \tag{1.2}$$

Therefore, since the sequential application is characterized by:

$$L = T_S = \sum_{i=1}^{N} L_i \tag{1.3}$$

in general, using the pipeline, we decrease the service time of the application while the latency is increased.

1.2.2 Farm

Is a paradigm based on functional replication, and which can be represented by the graph depicted in figure 1.3. It is composed by: a function replicated into a set of modules $\{W_1, W_2, W_3 \ldots, W_N\}$ called *workers*, an *emitter* E which schedules the tasks to the workers and a *collector* C which collects the produced results. Therefore, in general, the function will be applied at the same time by different workers over different tasks of the input stream.

Different scheduling strategies may be used by the *emitter*. The most common are:

On-demand Using this strategy is possible to maintain a good balancing among the different workers. It could be implemented, for example, by allowing each worker to communicate to the emitter when it finished to process the task and is thus ready to accept a new one.

Round robin This is a simple strategy which sends the task circularly to the available workers. Often it is implemented in such a way that, if the communication channel towards a worker is full, the task is sent to the first worker with at least one available slot in the channel.

However, any other scheduling function different from the proposed ones can be used. We will compare the effect of these different strategies in Chapter 4.

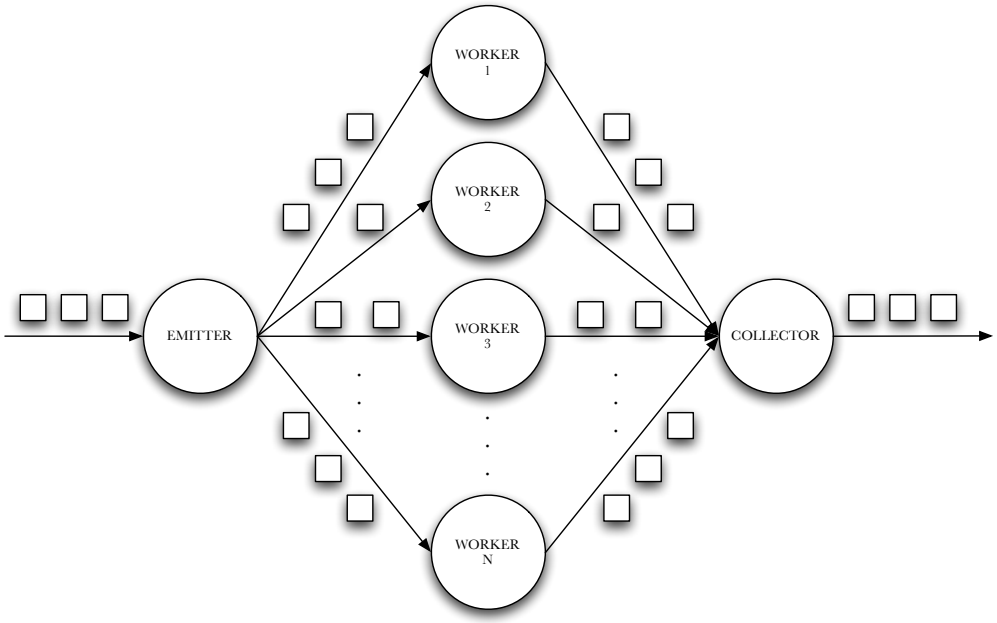

Figure 1.3: Farm skeleton

As we can see from the figure, the farm could be viewed as a 3-stage pipeline. Therefore, if the load of the workers is balanced (i.e. the probability that an input stream element is sent to any worker is $\frac{1}{N}$), the service time of the farm will be:

$$T_S = \max(L_E + L_{com}, \frac{L_W + L_{com}}{N}, L_C + L_{com}) \tag{1.4}$$

with L_E latency of the emitter, L_W the latency of a generic worker and L_C the latency of the collector.

The latency of the farm instead is:

$$L = L_E + L_W + L_C + (3 \times L_{com}) \tag{1.5}$$

Accordingly, also in this case the service time may be reduced while the latency is increased.

Eventually, we would like to find the optimal parallelism degree, i.e. the number of workers $\bar{n}$ such that, if this number of workers is used, the farm has a service time equal to the interarrival time T_A and it is therefore able to manage the whole incoming bandwidth.

Using equation 1.4 we have that $\bar{n}$, the *optimal* number of workers to be used in the farm, is equal to:

$$\bar{n} = \frac{L_W + L_{com}}{\max(T_A, L_E + L_{com}, L_C + L_{com})} \tag{1.6}$$

However, according to *structured parallel programming* concepts, if N is the number of available computational nodes (i.e. of machine's cores in our case) and if $\bar{n} > N$, then we need to restructure the computational graph such that it is composed by a number of nodes equal to N. Indeed, using this methodology, a reduction in the number of the modules is more effective than a multiprogrammed execution of more of them on a single physical node.

This is always true in our case because the modules are running for all the application lifetime processing data received from the input channels. For example, considering the case in which the modules are threads executed over multicore architecture. In this case, if more of them run on the same core, they would interfere and invalidate cache data between each other. The only exception to this may be when a core has more than a single context (i.e. the so-called *hardware multithreading*. However, also in that case, the effectiveness of running multiple threads over different context of the same core depends from the specific case.

Accordingly, if the optimal number of workers is $\bar{n} > N$, we need to reduce them to $n = N-2$, because 2 cores are reserved to emitter and collector nodes.

In general, if a composition of more skeletons is used, we can use an heuristic [49] and reduce the parallelism degree of each of them multiplying their degree by a factor

$$\alpha = \frac{N - ps}{n_\Sigma - ps} \tag{1.7}$$

where N is the number of available physical nodes, n_Σ is the amount of nodes of the graph before the reduction and ps is the number of "service module" (i.e. emitters and collectors).

We will see in Chapter 2 how these concepts have been applied to this thesis in order to efficiently structure the framework accordingly to the incoming bandwidth and to the number of nodes provided by the underlying multicores architecture.

1.2.3 Considerations about average values

When referred to our specific case, it is very difficult to consider average execution times. Indeed, the time spent to process a network packet can be influenced by many different factors like:

- Presence of IP tunneling.
- IP fragmentation.
- Specific transport protocol.
- Packet belonging or not to a flow for which we already identified the application protocol.
- Flow length.
- Out of order TCP segments.
- Specific application protocol.
- Payload length.
- Variance in the execution times of the callbacks specified by the user.

The same reasoning can also be applied considering the average interarrival times. Indeed, in a real network, the utilized bandwidth changes many times during the day. Therefore, if the framework is dimensioned for an average value of the bandwidth, sometimes it will be overdimensioned while other times it will be underdimensioned and, consequently, not able to manage all the traffic passing over the network.

Accordingly, we can still try to compute an average and accepting that in some cases it could be very different from the real situation. Otherwise, we could consider all the latencies as upper bounds on real latencies and thus dimensioning the framework to always be able to manage all the network traffic. In this case however, we have to accept that there will be moments when the resources will be underutilized.

Alternatively, our framework could be modified in such a way that it can adapt, time by time, to the real situation of the network, thus dynamically adding or removing computational nodes to avoid resources underutilization. This is considered as possible "future work", however.

1.2.4 Related work

Many libraries and programming languages which implement the concepts of *structured parallel programming* over shared memory or distributed memory architectures exist. Since it was a prerequisite of the thesis, in our framework we used *FastFlow* [28, 1], which we will describe in detail in section 1.3.

We now present some of the most recent works in this field, describing their main characteristics and showing the reasons *FastFlow* is more appropriate for this kind of high bandwidth applications.

1.2.4.1 Muesli

Muesli [50, 51] is a C++ skeleton library which uses OpenMP [52] and MPI [53] to target shared and distributed memory architectures and combinations of the two. It implements the most commonly used task parallel and data parallel skeletons as C++ template classes. Furthermore, Muesli offers the possibility to nest these skeletons to create more complex execution graphs by using the two tier model introduced by the Pisa Parallel Programming Language (P3L) [47]. Basically, the computation can be structured using task parallel skeletons where each node of the skeleton can internally be implemented as a data parallel skeleton. Concerning the data parallel skeletons, Muesli provides distributed data structures for arrays, matrices and sparse matrices and it has been recently extended to support Graphics Processing Unit (GPU) clusters [54].

However it would not be suitable for our framework since, as shown in [55, 56], OpenMP does not perform as well as FastFlow for fine grained streaming application

1.2.4.2 SkeTo

SkeTo [57] is a parallel skeleton library written in C++ with MPI. Although in some intermediate versions it supported also stream parallel skeletons, it mainly provides data parallel skeletons which can operate on data structures like arrays, matrices, sparse matrices and binary trees. For each data structure, the library consists of two C++ classes; one provides the definition of parallel data structure, and the other provides the parallel skeletons. Differently from most frameworks, where the application is entirely build around skeletons, SkeTo provides the possibility to to use their skeletons by means of library calls performed by the sequential program. Currently, except the one provided by MPI, it provides no explicit support for multicore architectures. Additional work have been done over domain specific strategies [58] and optimizations for the construction of data structures [59].

Since we designed our framework to use stream parallel skeletons (section 2.2), SkeTo would not be suitable for our purposes because it provides only data parallel skeletons.

1.2.4.3 SkePu

SkePU [60] is a C++ template library which provides a simple interface for mainly specifying data parallel skeletons computations over GPUs architectures using CUDA [61] and OpenCL [62]. However, the interface is general enough and SkePU provides also an OpenMP based implementation for all

the proposed skeletons. In addition to the skeleton templates, SkePU also includes an implementation of a vector with an interface similar to the one of the C++ Standard Template Library (STL) and which hides the complexity of GPU memory management. The skeletons in SkePU are represented by objects and contain member functions representing each of the different implementations, CUDA, OpenCL and OpenMP. If the skeleton is called with `operator()`, the library decides which one to use depending on what is available.

However, since its multicore support is built on top of OpenMP, it would not be suitable to manage the fine grained computations performed by our framework.

1.3 FastFlow

The framework we used to implement *structured parallel programming* concepts inside our framework is *FastFlow* [28, 1]. FastFlow is a `C++` framework targeting both shared memory and distributed memory architectures and which we already shown in a previous work [63] to be capable to manage similar kind of monitoring applications.

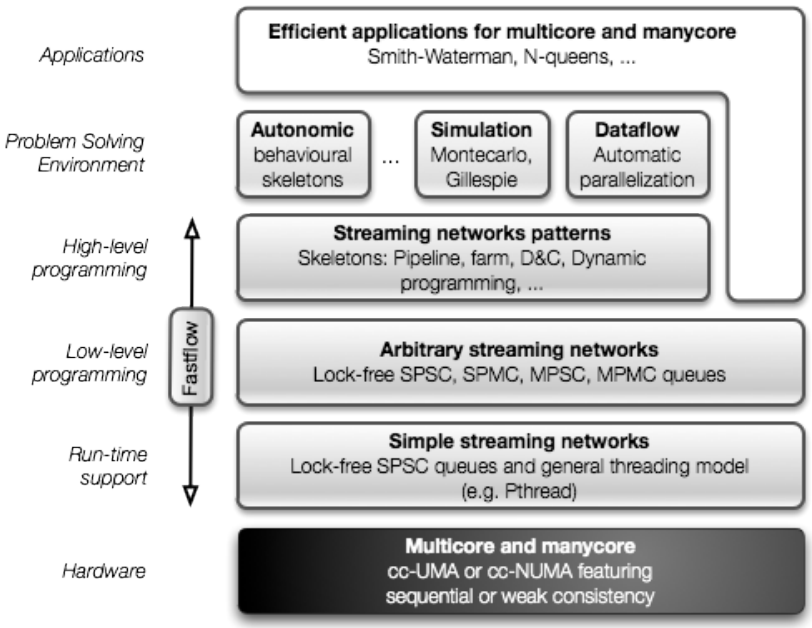

Figure 1.4: FastFlow architecture (taken from [1])

It is composed by several layers which abstract the underlying architecture. The abstraction provided by these layers (figure 1.4) is twofold: to simplify the programming process offering high-level constructs for *data parallel* and *stream parallel* skeletons creation and, at the same time, to give the possibility to fine tune the applications using the mechanisms provided by the lower layers.

At the very base level we found both bounded [64] and unbounded [65] *single producer, single consumer* (SPSC) queues which can be used as communication channels between threads. These queues are characterized by the total absence of *lock* mechanisms and have been realized taking inspiration from *wait-free* protocols described by Lamport in [66] and from FastForward queues outlined in [67].

Furthermore, FastFlow gives the possibility to define the code to be executed in the different computational module. This can be done by defining a class which extend `ff::ff_node` and implementing the virtual function `svc`. In example 1.1 we can see the definition of a simple module which takes an integer from the input channel, increments it and sends it on the output channel.

These nodes can then be easily linked together using the SPSC queues as communication channels in order to create arbitrary computation graphs similar to those depicted 1.5.

Listing 1.1: Node creation

```
#include <ff/node.hpp>
using namespace ff;

class ComputationalNode: public ff_node {
public:
    ComputationalNode(int max_task):ntask(max_task){};

    void* svc(void* task){
        int* real_task=(int*) task;
        ++(*real_task);
        return (void*) real_task;
    }
private:
    int ntask;
};
```

Alternatively, is possible to directly use some implemented and optimized *stream parallel* and *data parallel* skeletons or nesting among them. In this case the user simply specify the nodes and the skeleton to be used and the

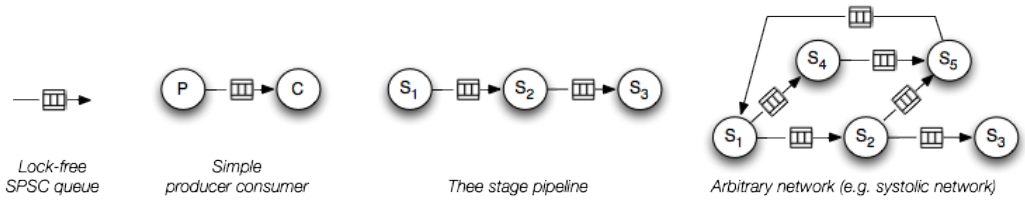

Figure 1.5: Examples of computational graphs (taken from [1])

library will link the nodes together to form the requested computation graph.

Furthermore, FastFlow provides some ways to customize the behaviour of these skeletons, for example by specifying the strategy to be used to distribute tasks in the emitter of the farm. Anyway, if this should still not be sufficient for a specific purpose, the user can use mechanisms provided by the lower layers or can easily extend the library by defining new skeletons.

In listing 1.2 we can see how, using few lines of code, it is possible to parallelize an already existing application. In this example, we define the application as a pipeline composed by two stages. The first of these stages (lines 6-12) reads the packets from the network (line 9) and send them over the communication channel towards the second stage (line 10). On the other hand, the second stage (lines 14-21) process the packets received from the channel (line 18) and indicates to FastFlow that it is ready to receive another task (line 19).

However, instead of processing the packets sequentially, we can decide to process them in parallel by using a farm with four workers (lines 29-32). In this way, the received packets will be transparently scheduled by FastFlow to the different workers. Eventually, we start the execution (line 37) and we wait for its end (line 39).

Although this is a small example, we can already see the advantages of this approach over classic techniques as `pthread` calls, which would require the user to deal with the additional and error prone task of threads synchronization.

Listing 1.2: Definition of the structure of the application

```
1 #include <ff/farm.hpp>
2 #define NWORKERS 4
3
4 using namespace ff;
5
6 class PacketsReader: public ff_node{
7 public:
```

```
8       void* svc(void* task){
9           char* packet=read_packet(network_interface);
10          return (void*) packet;
11      }
12  }
13
14  class PacketsAnalyzer: public ff_node{
15  public:
16      void* svc(void* task){
17          char* packet=(char*) task;
18          //Process packet
19          return GO_ON;
20      }
21  };
22
23  int main(){
24      PacketsReader reader;
25      std::vector<ff_node*> w;
26      ff_farm<> farm;
27      ff_pipeline pipeline;
28
29      for(int i=0; i<NWORKERS; ++i){
30          w.push_back(new PacketsAnalyzer);
31      }
32      farm.add_workers(w);
33
34      pipeline.add_stage(&reader);
35      pipeline.add_stage(&farm);
36
37      pipeline.run();
38      //Do something else
39      pipeline.wait();
40      return 1;
41  }
```

We will see in Chapter 2 how the theory presented in this chapter will be used in the design of our framework and how the tools we introduced will be used to implement it in Chapter 3.

Chapter 2

Architectural design

In this chapter we will see how we designed our work in order to reach our the goals stated in the introduction.

2.1 Framework design

First of all, we assume that the application is able to provide to the framework a stream of IPv4/IPv6 datagrams. This decision has been taken in order to be completely independent from the specific technology that the application uses to capture the packets from the network, thus allowing the framework to be used with different kinds of networks or packets reading mechanisms.

This decision have been taken because current networks are often characterized by a bandwidth in the order of tens of Gigabits per second and, by using traditional communication subsystems, this packets rate will not be completely delivered to the applications [68, 69]. This is mainly due to heavy kernel involvement and extra data copies in the communication path, which increase the latency needed to read a packet from the network. Consequently, leaving to the application programmer the choice of the way in which the packets will be captured, we can exploit possibly available dedicated technologies.

For example, during the past years, different solutions have been proposed and implemented to solve these problems both using specialized Network Interface Controllers (NICs) as Infiniband [70] and Myrinet [71] or by using standard NICs with an appropriate software support [72, 73].

In both cases, zero copy and kernel bypass techniques [74, 75] are used to reduce the latency of the data transfers from the NIC to the application memory and to deliver the whole network bandwidth to the application [76, 77].

In order to provide a sketch of the framework structure, we will now describe the operations that should be performed when a TCP segment is received[1] and which are depicted in the flow diagram in figure 2.1.

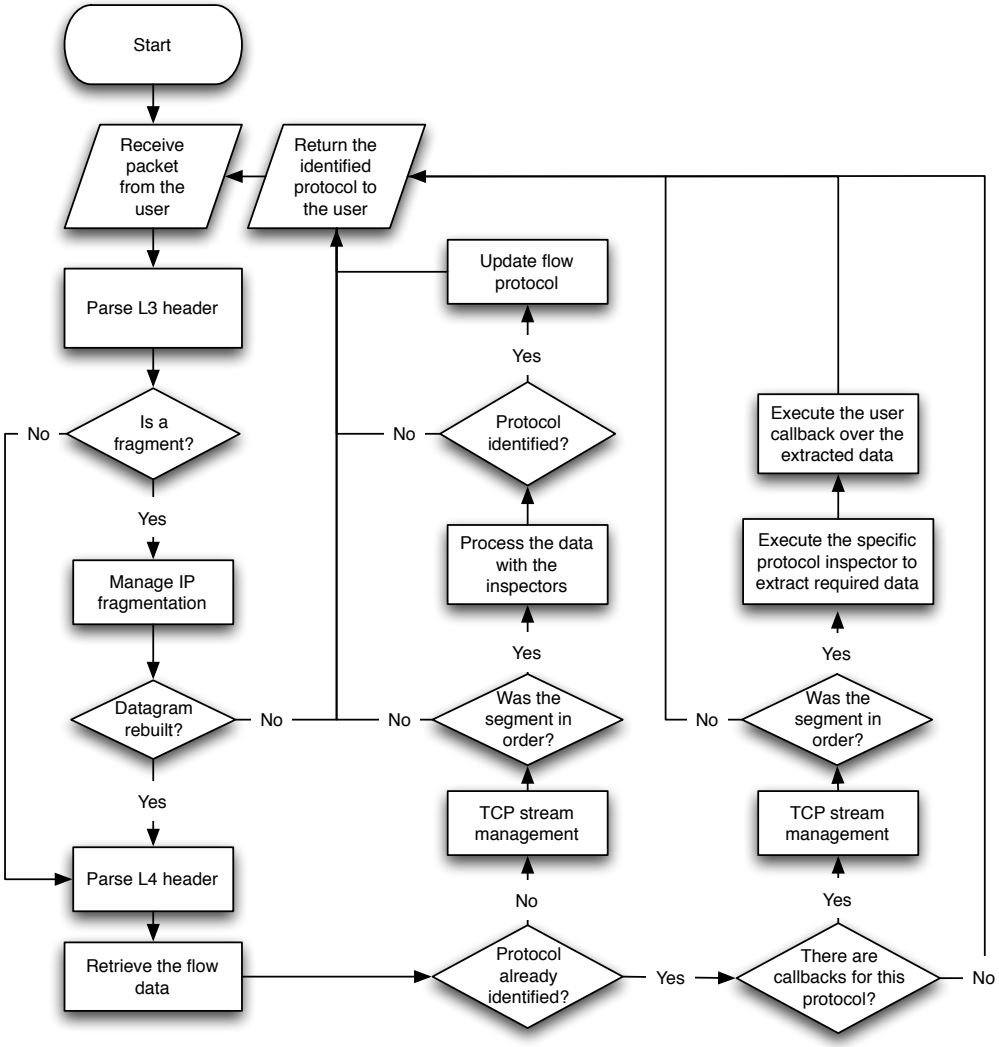

Figure 2.1: Flow diagram of the framework

When a packet containing a TCP segment is passed to the framework, it needs to:

1. Parse the network header and extract the real source and destination

[1]For UDP datagrams the process is the same except for TCP stream management.

addresses if *IP-in-IP* tunnels are present. Then, if the received datagram is a fragment of a bigger datagram, the framework will store the fragment for subsequent reassembly (IP normalization).

Hence, after parsing the network header, if we have a non-fragmented datagram, we parse the transport header in order to get source and destination transport ports.

Therefore, at this point, we have the entire 5-tuple key (composed by <SOURCE IP ADDRESS, DESTINATION IP ADDRESS, SOURCE PORT, DESTINATION PORT, TRANSPORT PROTOCOL IDENTIFIER>) which characterizes the bidirectional flow to which the packet belongs.

2. In order to be able to track the progress of the protocol, we need to store for each flow some data about the previous packets we received for that flow. Notice that this doesn't mean that we need to store all the previous packets belonging to the flow but only a constant size structure containing some elements that will be updated when a new packet arrives (e.g. last TCP sequence number seen or current status of HTTP parser). The only exception to this is when an out of order TCP segment arrives. In this case indeed we need to store the entire segment for future reordering and processing (TCP normalization).

   These data are stored into an internal data structure and are maintained in it for all the flow lifetime. For the moment, we will just say that it has been implemented using an hash table with collision lists. The reasons behind this choice, together with the details about how it have been designed and implemented can be found in Chapter 3.

   When we didn't receive packets for the flow for a certain amount of time or when the TCP connection is terminated, these elements will be deleted.

   Therefore, using the flow key, the framework is able to do a lookup and to retrieve these elements.

3. After we obtained the flow data, if the application protocol has not yet been identified, the framework needs to manage the TCP stream and, if the segment is out of order, to store it for future reassembly. If the segment is in the proper order, the protocols *inspectors* will be executed over the application data. Each inspector, using previously collected elements about the flow and analyzing the current packet will try to infer if the flow carries data of its specific application protocol.

   If, instead, the protocol was already identified and if the application which uses the framework specified some callbacks for that protocol,

we manage the TCP stream and we execute the specific protocol inspector in order to extract the required elements from the payload and to process them using the callback specified by the application.

Otherwise, if no callbacks for this protocol were defined, we simply return the previously identified protocol.

2.2 Parallel structure design

In this section we will present some of the possible solutions that could be adopted, together with the reasons which drove our choice towards the solution we decided to implement for our framework.

The first possibility we analyzed, is to use a farm where each worker executes, over a different received packet, all the steps described in the previous section. However the workers, for each received packet, should access to the shared hash table where the flows are stored. Considered that these flows can be modified, this would require to access the table (or at least each collision list) in a mutual exclusive way. Consequently, this solution would not scale with the number of workers. Moreover, we can't assume that the packets will be processed by the workers in the same order in which they arrived. Therefore, also if the TCP segments belonging to a certain TCP stream will be received in order by the DPI framework, it could be processed out of order by the workers. As we will show in section 4.3, this can have a not negligible impact on the overall performances.

However, analyzing the diagram in figure 2.1, we can see that the operations on different application flows are independent and could be then executed at the same time over distinct flows. Anyway, is in general unfeasible to have a module of the concurrent graph for each flow. For this reason, we decided to assign groups of flows to different modules that will process them in parallel exploiting the multicore hardware available.

Accordingly, we decided to structure the framework as a farm, with the flow table partitioned among a set of workers, as shown in figure 2.2 (collector not shown). In this way, the worker i will access only to the flows in the range $[low_i,\ high_i[$ without any need of mutual exclusion mechanisms or synchronization with any other thread.

Anyhow, we should take care of the way in which the packets are distributed to the workers. If *round robin* or *on demand* scheduling strategies are used, we must consider that a worker could receive packets that it can't manage and which should be forwarded to some other worker. To solve this problem we could let each worker communicate with all the other workers of the farm in such a way it can forward the packets not directed to itself to

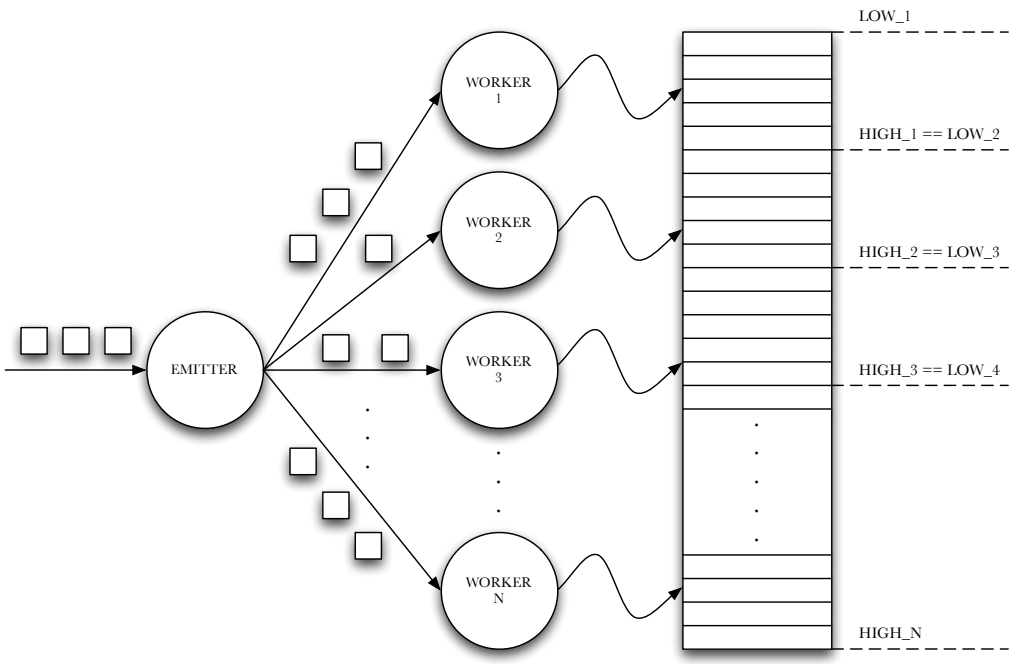

Figure 2.2: Structure of the framework with flow table partitioned among the set of workers (collector not shown)

the corresponding worker. However, also in this case we can't ensure that the packets belonging to the same flow will be processed in the same order they arrived.

For these reasons, we decided to provide the emitter with a scheduling function which distribute to each worker only the packets belonging to the flows contained in its partition. Consequently, the emitter needs first to extract the key of the flow and then, as in the sequential case, to determine the bucket of the table where the flow can be found. Once that the bucket has been found, the emitter can easily derive the partition to which the it belongs and, therefore, can forward the packet to the correct worker.

Moreover, since the communication channels are First In First Out (FIFO) queues, the order of the packets belonging to the same flow is preserved, avoiding thus to reorder data which arrives already ordered to the framework.

It's important to point out that, aside from the communication channels, the different nodes don't share any data structure among each other, allowing thus to advance in their execution without any need of synchronization.

Let's now analyze the service times of the different nodes of the graph.

- The emitter will have a service time

$$T_E = L_{read} + L_{L3_L4} + L_{hash} + L_{com} \tag{2.1}$$

  where L_{read} is the average latency of the *reading callback*, L_{L3_L4} is the average latency for network and transport headers parsing and L_{hash} is the latency of the flow hash function.

- The worker will have a service time

$$T_W = L_{table} + L_{L7} + L_{cb} + L_{com} \tag{2.2}$$

  where L_{table} is the latency required to access the table, L_{L7} is the latency of the protocols inspectors and L_{cb} is the latency of the callbacks that the user specified over the protocols metadata.

- The collector has a service time

$$T_C = L_{proc} + L_{com} \tag{2.3}$$

  with L_{proc} latency of the *processing callback*.

We should now decide the optimal amount of workers $\bar{n}$ to activate for a given interarrival time T_A. Using equation 1.6 we have:

$$\bar{n} = \frac{T_W}{max(T_A, T_E, T_C)} \tag{2.4}$$

Therefore, the entire rate of packets arriving to the framework can be managed only if $T_A > L_{proc} + L_{com}$ and $T_A > L_{read} + L_{L3_L4} + L_{hash} + L_{com}$.

These conditions strongly depend on the specific callbacks specified by the user. However, even considering $L_{read} = 0$, the emitter could still be a bottleneck for the application. Indeed, the latency spent to parse the network and transport headers and to apply the hash function is not negligible and may often be greater than the interarrival time.

If this case is verified, we need to find a way to reduce the service time of the emitter. This can be done by replacing the emitter with a farm where each worker executes the steps that were previously executed by the emitter. Consequently, the resulting graph is a pipeline where each stage is a farm, as shown in figure 2.3.

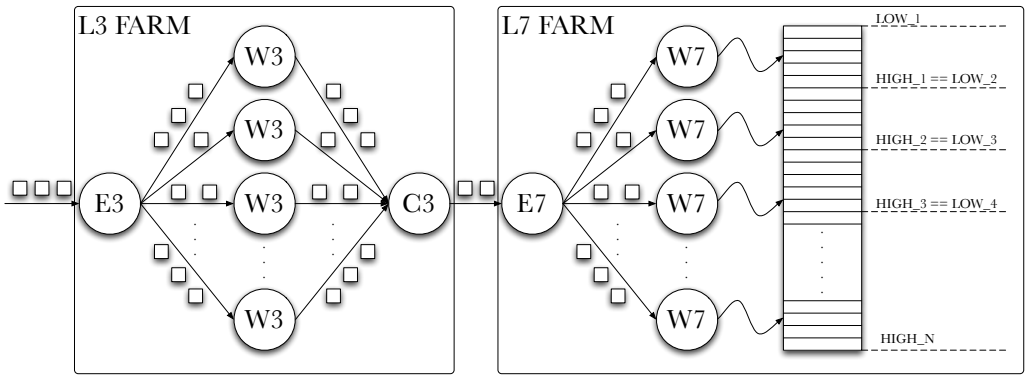

Figure 2.3: Structure of the framework when the emitter is a bottleneck (the collector of the second farm is not shown)

From now on, since the first farm of the pipeline take care of the processing of the network layer (level 3 in the Open Systems Interconnection (OSI) model) and of the transport layer (lever 4), we will refer to it as L3 farm. For similar reasons, we will refer to the second farm as L7 farm.

For L3 farm, any scheduling strategy could be used. However, we would like to have a scheduling strategy which maintains the order of the packets in such a way that they exit from the L3 farm in the same order they arrived. In this way, if the packets belonging to the same flow were already ordered, they will arrive in the same order to L7 farm and thus to the protocols inspectors. Consequently, the framework can avoid the overhead of TCP

reordering when it is not really needed. In section 3.7 we will see how this has been implemented and in section 4.3 we will analyze the performance gain obtained by using an order preserving scheduling strategy.

Using this solution, the different nodes will have the service times shown in the following table:

Node	Service Times
L3 emitter	$T_{E3} = L_{read} + L_{com}$
L3 worker	$T_{W3} = L_{L3_L4} + L_{hash} + L_{com}$
L3 collector	$T_{C3} = L_{com}$
L7 emitter	$T_{E7} = L_{com}$
L7 worker	$T_{W7} = L_{table} + L_{L7} + L_{cb} + L_{com}$
L7 collector	$T_{C7} = L_{proc} + L_{com}$

Consequently, defined $\bar{n_1}$ as the optimal number of workers for L3 farm and $\bar{n_2}$ the optimal number of workers for L7 farm, we will have:

$$\bar{n_1} = \frac{T_{W3}}{max(T_A, T_{E3}, T_{C3})} \tag{2.5}$$

$$\bar{n_2} = \frac{T_{W7}}{max(T_A, T_{E7}, T_{C7})} \tag{2.6}$$

However, also in this case there should be cases in which the ideal service time could not be reached because an emitter or a collector is a bottleneck.

For example, let's consider the case in which we have $T_A < T_{E3} = L_{read} + L_{com}$. In this case, in order to reduce the impact of the communication latency, we could calculate the optimal communication grain g and thus send and process the packets in blocks of size g. Accordingly, we need to find the value g such that $g \times T_A = (g \times L_{read}) + L_{com}$.

Therefore, we have:

$$g = \frac{L_{com}}{T_A - L_E} \tag{2.7}$$

It's important to notice that this equation is meaningful only when $T_A > L_E$. Indeed, in all the other cases the bottleneck cannot be removed by increasing the communication grain because $T_A < L_E$ and consequently, for each granularity g:

$$g \times T_A < (g \times L_{read}) + L_{com}$$

If more than one bottleneck is present, we can evaluate all the values of g and taking the maximum among them to ensure that these bottlenecks are removed.

However, we need to take into account that each task that the L7 emitter receives from the L3 farm can contain packets directed to different L7 workers. For this reason the emitter of the L7 farm, when a task is received, needs to redistribute the packets in temporary tasks and, when one of them has been completely filled, to send it to the corresponding L7 worker. Since the redistribution introduces an additional latency in the L7 emitter, its service time becomes: $T_{E7} = T_{redistribute}(g) + L_{com}$. Consequently, there are cases in which the L7 emitter could be a bottleneck. However, as we will see in Chapter 4, this will happen only in a limited number of cases.

On the other hand, concerning the L3 collector, it will never be a bottleneck because its service time will be always be lesser than the service time of the L3 emitter. Moreover, since the service time of the L3 emitter consists only in the reading of the packet, it can be consider as a lower bound on the service time of the framework.

We will see in Chapter 3 how these concepts have been applied for the implementation of the framework.

Chapter 3

Implementation

We will now describe some of the most important details concerning the implementation of the framework, relying on the design described in Chapter 2.

Concerning the sequential implementation, we will first analyze the interface provided to the application programmer, describing the choices we made to make it as much flexible as possible, in order to satisfy different application requirements and scenarios. Then we will analyze and motivate the main choices we made to implement the steps needed to perform a correct protocol identification. We start describing the processing of the network and transport headers, analyzing in detail the structures used to perform a functional and efficient IP defragmentation. After that, we describe how the management of the flows have been handled by our framework, describing some alternative solutions which will be then evaluated in Chapter 4. Furthermore, we will present the TCP reordering techniques we used and the interaction of the framework with the protocol inspectors. Eventually, we describe the possibility we provide to extract and process specific data and metadata of the protocol, once it has been identified.

Concerning the parallel structure, we will analyze how the concepts described in Chapter 2 have been implemented in this thesis. Also in this case different solutions have been proposed and then evaluated in Chapter 4. It's important to point out that FastFlow allowed us to prototype and test these different alternative solutions with relatively low programming effort. This has been possible because FastFlow provides an higher abstraction level with respect to that provided by directly using `pthreads`, which otherwise would have required much more effort due to the complicated and error prone task of thread synchronizations.

3.1 Application interface

The framework has been written in `C` and it can be used in two different ways:

Stateful mode Targeted for applications which don't have a concept of "flow" and therefore don't store any information about the previously received packets. In this case, the framework will keep an internal table with the information about each individual application flow.

Stateless mode This mode is designed for applications which already store some kind of data about the application flows (e.g. packets and bytes received). In this case, the elements needed by the framework can be stored into the application table. Therefore, the application should be modified in order to keep, with its own flow data, also a pointer to the flow elements created and managed by the library. In this case, however, the retrieval of the data must be entirely managed by the application.

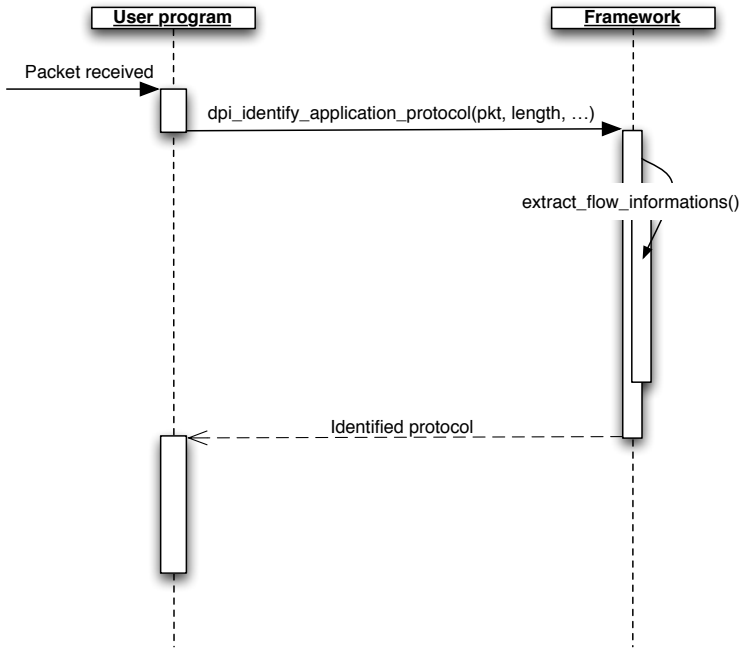

Figure 3.1: Stateful interaction

The two interaction modes are depicted in figures 3.1 and 3.2. We can see that in the stateful interaction the application simply provides to the

framework the packet and then gets the identified protocol. Conversely, in the stateless mode, the parsing of the network and transport headers is decoupled from the identification of the protocol. Indeed, the application needs first invoke the framework to parse the network and transport headers and to get the flow key. Then, using this key, retrieves the flow data from its own table. Using the flow data, it can then call the framework to identify the protocol.

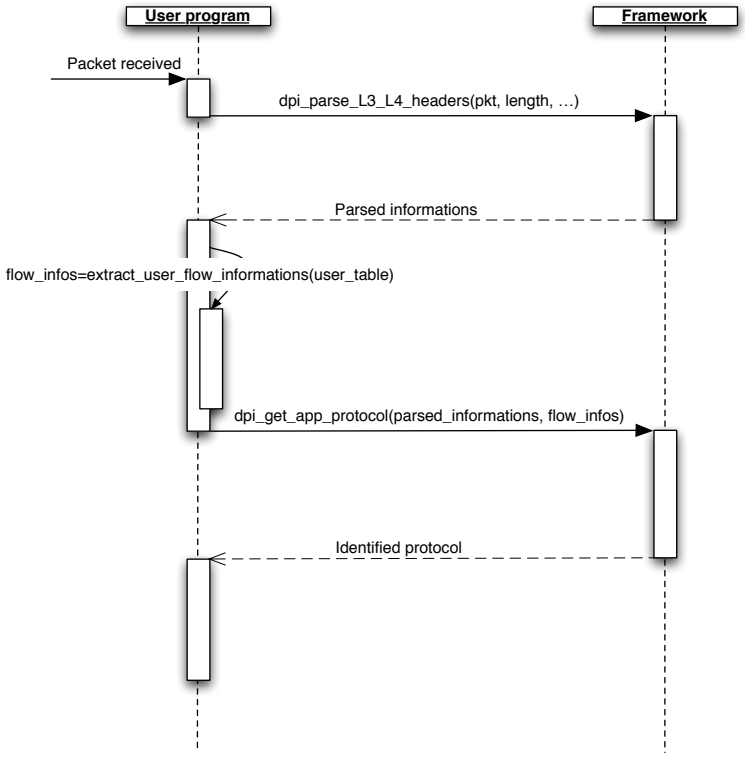

Figure 3.2: Stateless interaction

Almost all the calls we provided need an handle to the framework. This handle can be created with a specific framework call which will take care of creating and initializing its internal structures. By default, the framework activates all the protocols inspectors. Anyway, when is needed, the application can decide at runtime to activate only some protocols inspectors. This is particularly useful in *firewall-like* applications when the set of protocols to which the application is interested can dynamically change during its execution.

IP fragmentation and TCP reassembly supports are also activated by default. It is possible to disable them if needed, for example when they are provided by some other parts of the application or by the specific packet

reading technique used[1].

The framework has an internal clock that is updated each time a packet is passed by the application. It is responsibility of the application to pass to the framework, together with the packet, the time it has been captured (it is sufficient to have a resolution of one second). In this way, the programmer can decide which mechanism is more appropriate or feasible for its application. Indeed, when managing millions of packets per second, adding a timestamp to each packet using classic calls as `gettimeofday` or `time` may result in a bottleneck.

As an alternative, if available, the application programmer could decide to use hardware timestamping provided by the network card or to use some other software level mechanisms (for example, to use the clock cycles counter). Anyway, since this task is left to the programmer, he can use the more appropriate mechanism with respect to the context in which the application is running.

3.2 Network and transport headers processing

First of all, the framework checks if the datagram is an IPv4 or an IPv6 datagram. After that, it starts parsing the header and, in case of presence of tunnels, they are unwrapped and the framework will consider as actual source and destination of the flow those found in the inner datagram. Up to now, we implemented the support for 4in4, 4in6, 6in4 and 6in6 tunnels[2] and all the combinations among them.

After parsing the network header, in case the transport protocol is TCP or UDP, the framework extracts the transport ports, finds the offset at which the application data start and build the flow key.

However, if the packet we received was carrying a fragment of a bigger datagram, before extracting the transport header elements we need to reconstruct the original datagram. Let's now see how this has been implemented in the framework.

[1] For example when the application, instead of reading "raw" data, decides to read from operating system sockets.

[2] An outer IP header is added before the original IP header. The outer IP addresses identify the "endpoints" of the tunnel while the inner IP header addresses identify the original sender and recipient of the datagram [78].

3.2.1 IP fragmentation support

We briefly recall that, when an IPv4 datagram is larger than the MTU of the outgoing link, it will be split in different fragments. After that, an IPv4 header will be attached to each fragment and they will be sent separately over the network (figure 3.3). However, to be able to reconstruct the original datagram, all its fragments will be marked with the same identification number. Moreover, each fragment is characterized by a specific fragment offset, which represents the point in which this fragment must be put inside the reconstructed datagram. This means that if a fragment has an offset x and a length of l bytes (considering only the payload), it carries the bytes that in the original datagram were in the range $[x, x + l[$[3]. From now on we will generically refer to this range as `[Offset, End[` range. Eventually, the last fragment of the datagram can be identified because it is the only one with a specific flag in the IP header (the MF flag) set to zero.

For IPv6 the fragmentation process is slightly different[4]. Anyway, as the reassembly process performed by our framework is similar, we will describe only the one for IPv4.

To be able to reconstruct the original datagram, we need to store its fragments into a set and then, when all the fragments have been received, to put their content together and in the correct order. At this point the framework can analyze the content of the original datagram as if it has never been fragmented. Furthermore, for each datagram we are reconstructing, we need to store also a timer and its original IPv4 header. The timer is needed as, if the datagram is not reconstructed in a certain amount of time, all the pending data that we have about the datagram will be removed from the library [79].

In principle, the fragments will not overlap between each other. However, an attacker could send overlapping fragments to try to exploit vulnerabilities in the defragmentation algorithm [15]. This is the reason why, when we insert a new fragment we need to check that it doesn't overlap with the fragments already present in the set and, if overlaps are present, we need to store only the part of the fragment that we didn't received yet.

Therefore, when a fragment is received, the framework needs to:

1. Obtain the set of fragments that we already received for this datagram.

[3]Actually the offset is expressed in 8-bytes blocks. However, to simplify the exposition, we will consider the offset as expressing the exact byte where the fragment starts.

[4]Differently from IPv4 it can be performed only by end nodes, and involves IPv6 optional *fragmentation header*. Moreover, some of the optional headers present in each fragment must be copied in the final datagram

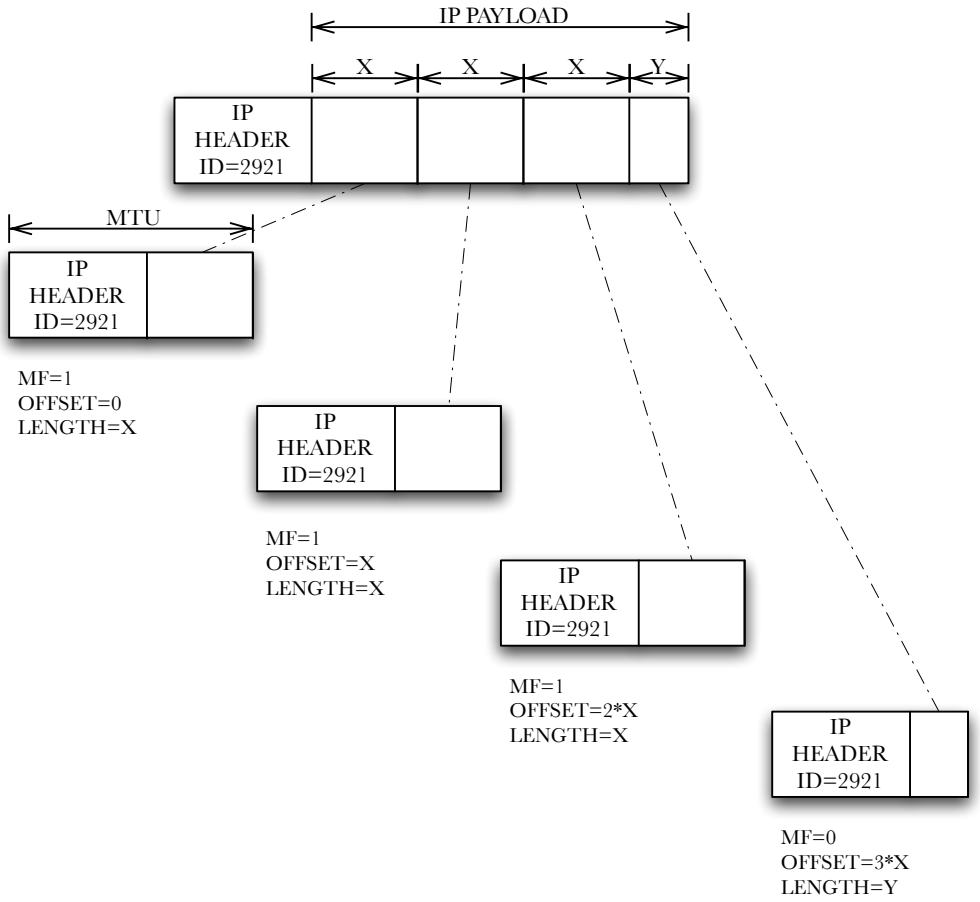

Figure 3.3: IP fragmentation process

2. Check if the fragment we are inserting overlaps with any existing fragment by comparing the `[Offset, End[` ranges.

3. Insert the non overlapping bytes into the set.

To avoid to check all the fragments present in the set, we decided to implement the set as a linked list, sorted according to the `[Offset, End[` range (figure 3.4). In this way, we will check only the fragments that have bytes belonging to the `[Offset, End[` range of the fragment that we are inserting.

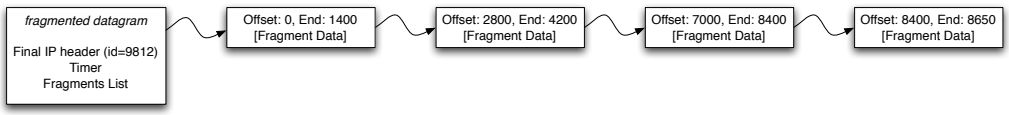

Figure 3.4: List of IPv4 fragments

Anyway we still didn't say how, given a datagram, the framework retrieves the list of its fragments. It's important to point out that we cannot use for this purpose the same structure that we use to keep track of the other flow data. Indeed, at this point, we still don't know to which flow the packet belongs because the transport header could have been split in different IP datagrams.

For this reason, we need a separate data structure. Moreover, we would like to organize this structure in such a way that we can easily know the amount of memory that the framework uses for each IPv4 source and, if a predefined threshold is exceeded, we can delete its oldest outstanding fragmented datagrams. Accordingly, for each source, we have a structure containing the sum of its used memory and the list of its fragmented datagrams (figure 3.5).

Eventually, we need to organize these sources in such a way that when a datagram is received, we can easily retrieve the fragmented datagrams generated by that specific IPv4 source. For this purpose, we decided to store the set of sources into a simple hash table where collisions are resolved through separate chaining (figure 3.6).

In conclusion, putting all together, when a fragment is received the framework will:

1. Check if there are expired fragmented datagrams.

2. Apply an hash function over the IPv4 source address to get the bucket of the hash table where the data about this specific source are stored.

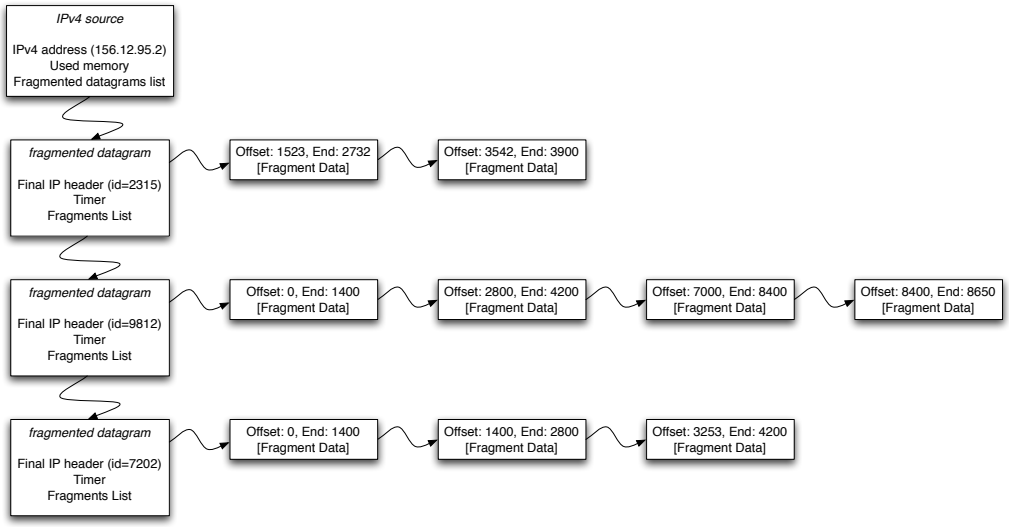

Figure 3.5: List of IPv4 fragmented datagrams

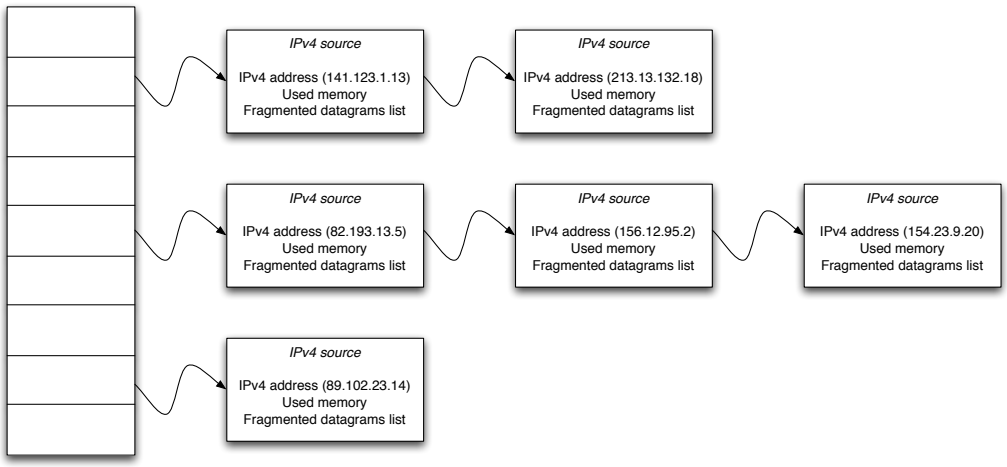

Figure 3.6: Hash table containing the IPv4 sources which have outstanding fragmented datagrams. For sake of simplicity, the list of outstanding fragmented datagrams is not shown

3. Scan the collision list to find the IPv4 source.

4. Scan the list of its outstanding datagrams.

5. Scan the fragments list and put the fragment in the correct position.

To implement point 1, and to avoid to check all the datagrams, the framework keeps a list of timers (one per fragmented datagram). When the first fragment of a datagram is received, we insert a new timer into the head of the list. In this way, the list is automatically kept sorted from the newest to the oldest received fragmented datagram. If some datagrams are expired, they will be located in the last positions of the list. Therefore, when the framework needs to check if there are expired datagrams, it will start checking from the tail of the list avoiding, in general, to scan the it entirely.

3.3 Flow data management

As we anticipated before, if the framework is used in *stateful* mode, it is responsible of storing and retrieving flow data. These data include:

- The application protocol, if already identified.
- Data that the application wants to associate to the flow (we will see in section 3.5 how this is used).
- TCP tracking information, including the segments that have been received out of order.
- Protocol specific tracking data (for example, the state of the HTTP parser).

The interaction of the framework with these data is depicted by the underlined functions in algorithm 1.

Since this structure should be accessed using the flow key, we decided to implement it as an hash table. Accordingly, if we have f flows, a table of size s and we have an uniform hash function, we will have an average search complexity of $O(\frac{f}{s})$.

We decided to manage the collisions using separate chaining by means of double linked lists. Consequently, the creation of a new flow will always have $O(1)$ complexity. The same holds true for the deletion of a flow as a result of a TCP connection termination. In this case indeed it will be done only after that the flow have already been found.

Algorithm 1 Flow management in our framework

Require: A packet p belonging to a flow with key k has been received
  $f \Leftarrow$ FIND(k);
  if f == NULL **then**
    $f \Leftarrow$ CREATEFLOWDATA(p);
  end if
  protocol $\Leftarrow$ IDENTIFYAPPLICATIONPROTOCOL(p, f);
  UPDATE(f, *protocol*);
  if f is a TCP flow and if the connection terminated **then**
    DELETE(f);
  end if

We now introduce the mechanism we used to check time by time if there are expired flows in the table. Each flow, when updated[5], is marked with the timestamp of the last received packet and, when we want to know if a flow is expired, we simply check that the difference between the current time and the timestamp of the flow doesn't exceed a predefined threshold. However, we would like to avoid to check all the flows each time that a new packet arrive. Taking into account that the framework clock has a resolution of one second, this check will be done at most one time per second.

Moreover, we would like to avoid to scan all the flows and to only check the oldest flows in the table. In order to do this, we keep the collision lists sorted from the most recently updated to the less recently updated flow. This can be done by simply moving a flow to the head of its collision list when a packet for that flow is received. In this way, for each collision list, we start executing the check from its tail and, being the list sorted, we remove all the traversed flows, up to the point when the difference between the current timestamp and the timestamp of the flow that we are analyzing is less than the inactivity threshold. We will analyze in section 4.2 the impact of this technique on the overall framework bandwidth.

It's important to notice that, keeping the collisions lists sorted, we also have the advantage to have the most active flows in the first positions of the list, thus reducing the search time for the flows for which we are presumably receiving more packets.

3.3.1 Hash functions analysis

At this point, we have to find a good hash function, in order to keep an uniform distribution of the flows over the table. For this purpose, we proposed,

[5]For TCP flows this is done only if the received TCP segment is in order

implemented and analyzed four different hash functions:

Simple hash This function simply sums together the five fields of the flow key.

FNV1a-32 hash The "Fowler/Noll/Vo" (or FNV) hash functions are designed to be fast while maintaining a low collision rate. Particularly, we used the alternate algorithm for 32 bits keys shown in algorithm 2.

Algorithm 2 FNV1A-32 hash function

```
function FNV(data, data_length)
    hash ⇐ OFFSET_BASIS
    for i = 1 to data_length do
        hash ⇐ hash ⊕ data[i]
        hash ⇐ hash × FNV_PRIME
    end for
     return hash
end function
```

More details on how OFFSET_BASIS and FNV_PRIME are chosen are described in [80].

BKDR hash This hash function comes from Brian Kernighan and Dennis Ritchie's book "The C Programming Language" [81] and is described in algorithm 3.

Algorithm 3 BKDR hash function

```
function BKDR(data, data_length)
    seed ⇐ 131
    hash ⇐ 0
    for i = 1 to data_length do
        hash ⇐ (hash × seed) + data[i]
    end for
     return hash & 0x7FFFFFFF
end function
```

Murmur3 hash We will not show here its pseudocode because is more complex than the previous functions (details about the algorithm can be found in [82]). However, it's important to say that, with respect to the other presented algorithms, MURMUR3 hashes are influenced by a seed that can be randomly chosen in order to make the function more

robust against possible attacks. On the other hand, in FNV and BKDR the seed can only be chosen from a predefined small set of seeds which satisfy some particular properties.

In section 4.1 we will do a comparison between these hash functions, comparing them according to the uniformity of the distribution, to the execution time and to the real impact on the DPI framework.

3.4 Protocol identification

After that the framework parsed the packet and retrieved the data about the previous packets belonging to the same flow, it can proceed and try to identify the application protocol, if not yet identified.

When a flow is created, it is in the NOT DETERMINED state. When a new packet for that flow is received, the flow can remain in the NOT DETERMINED state, it can move to the IDENTIFIED state or it can move to the UNKNOWN state.

A flow remains in the NOT DETERMINED if the framework still judges possible to classify the packet in one of two or protocols and it needs more packets to identify the exact protocol of the flow. To avoid to keep continuously analyzing packets in presence of this kind of ambiguity, the application can specify the maximum amount of packets to analyze for each flow. When this amount is exceeded, the framework moves the flow in the UNKNOWN state and will no more try to identify the protocol.

Alternatively, if after one of more packets the framework identified the protocol, then the flow is moved in the IDENTIFIED state. In this case, if no callbacks are present (section 3.5), when a new packet for this flow is received the framework will simply return the already identified protocol without executing any additional operation on the packet[6].

3.4.1 TCP stream management

TCP is a reliable and stream oriented protocol. For this reason, the applications that use TCP as transport protocol assume that their data will be received in the same order in which they are sent. Therefore, considered the best effort delivery provided by IP networks, if we want to correctly identify the protocol, we need first to put the data in the same order in which

[6]Actually, if the packet belongs to a TCP flow, the framework will still check if the segment carries the FIN flag in such a way that the connection can be correctly closed and the data about the flow removed.

they were sent. It's important to notice that this is not a problem for UDP based protocol as, in that case, when reliability and reordering are needed, they are implemented through application specific mechanisms and so in the framework they are demanded to the specific protocol inspector.

Accordingly, before executing the protocols inspectors on the packet payload, if the received packet contains a TCP segment, the framework needs to update the information on the TCP connection of the flow and, if the segment is out of order, to store it for further reordering.

The connection information stored by the framework includes the expected sequence numbers in both directions, a list of out of order segments and information about connection establishment and termination (for example, if one of the two endpoints sent the FIN segment).

It's important to notice that, at this point, these elements are already available to the framework because they were stored together with the other flows data that we retrieved in the previous step.

We can distinguish two different types of TCP flows: those who start while the framework is working and those who already started before the framework was started.

In the former case is easier to start tracking the connection because the framework can follow the three-way handshake. Therefore, it knows which are the sequence numbers that both endpoints will use and, consequently, it can identify the out of order segments.

The latter case is more complex because, if the first packet received for a flow isn't the first segment of the three-way handshake, then the framework cannot determine if the segment is out of order or not. In this case it needs more packets in order to be able to evince the current state of the connection. Basically, the framework will consider all the received segments as out of order and, instead of storing them for future reordering, it will discard these packets. When the highest acknowledgment number seen in one direction of the connection coincides with the expected sequence number in the other connection direction (and vice versa) the framework knows the current state of both endpoints and it can start to behave as if he has just seen the three-way handshake.

From this point, we can start managing the out of order TCP segments, using techniques similar to those we used for IP defragmentation.

First of all, for each out of order segment we store its content, its sequence number and its length. Then, the segment is inserted into a linked list containing the other out of order segments, checking for possible overlaps with those already present and, in case, inserting only the non overlapping bytes.

When a segment is received in order, if it fills an hole in the stream (i.e. if the first byte of the first out of order segment in the list follows the last byte

of the received segment), the framework compacts the segments and passes them to the inspectors for protocol identification.

3.4.2 Protocols inspectors

After the framework obtained ordered data, it will invoke the protocols inspectors on that data, trying to identify the protocol of the flow.

Up to now our prototype framework implementation supports the following protocols: HTTP (based on the open source parser in [83]), DNS, MDNS, DHCP, DHCPV6, SMTP, POP3, BGP and NTP.

Each of these protocols has its own inspector parsing the application payload to try to understand if the protocol carried by the flow matches. Each inspector has the following input parameters:

- The handle of the framework, that can be used by the inspector to get information about the current state of the framework. For example, when the application programmer specifies some callbacks to be executed when specific protocol metadata are found in the packet, the pointers to these callbacks are stored in the handle of the framework. Accordingly, when the inspector finds such data, it can call the corresponding callback.
- A pointer to the parsed packet.
- A pointer to the application data.
- The length of the application data.
- Information about the the flow, used by the inspector to keep track of the current state of the protocol.

When an inspector is called, it can return three different responses: MATCHES, DOESN'T MATCHES or MORE DATA NEEDED. The latter is returned in case the inspector needs more data to decide if the protocol matches or not. For example, consider the case in which the HTTP inspector is analyzing a TCP flow which started when the framework was still not running. In this case, it could have started analyzing the flow in the middle of a file transfer and, consequently, the first packets that it receives for that flow contains HTTP *body* segments. Therefore, the inspector can't determine if the flow carries HTTP data by simply looking to the first packets of the flow. For this reason it could return a MORE DATA NEEDED response and wait for other packets before deciding.

The responses given by the inspector are in a strict relationship with the possible states of a flow. For each flow the framework keeps a set of possible matching protocols. Each time that an inspector returns a DOESN'T MATCHES response, the corresponding protocol is removed from the set. On the contrary, if a MATCHES response is returned, then the protocol has been identified, the flow will pass from the NOT DETERMINED state to the IDENTIFIED state, and no other inspectors will be ever called for this flow. If, after the maximum number of trials specified by the application, the set still contains two or more protocols or if it is empty (i.e. if all the inspectors returned a DOESN'T MATCHES response), then the framework will move the flow from the NOT DETERMINED state to the UNKNOWN state. This maximum number of trials can be indicated by a specific call of the framework and can be changed during its execution. Since this set is maintained for all the flow lifetime, for the packets successive to the first one, only the inspectors that returned a MORE DATA NEEDED response will be invoked.

The order in which the inspectors is invoked is not the same for all the flows. Indeed, if the source or destination port of the flow is a well known port, the first inspector to be invoked will be the one corresponding to the protocol which usually runs on that specific port. For example, if one of the ports of the flow is port 80, then the first inspector to be invoked will be the HTTP inspector. Then, if it doesn't return a MATCHES response, the framework will invoke the other inspectors.

Algorithm 4 DHCP inspector

```
if (payload_length >= 244 AND
   (source_port == 67 AND dest_port == 68) OR
   (source_port == 68 AND dest_port == 67)) AND
   payload[236] == 0x63 AND
   payload[237] == 0x82 AND
   payload[238] == 0x53 AND
   payload[239] == 0x63 AND
   payload[240] == 0x35 AND
   payload[241] == 0x01) then
      return PROTOCOL_MATCHES
else
      return DOESN'T_MATCHES
end if
```

In algorithm 4 we can see an example of a simple protocol inspector for the DHCP protocol.

As we can see, it tries to determine if the flow matches by simply looking

to some bytes, in specific positions, of the received packet. In the DHCP case these bytes will be always the same independently from the DHCP message. Therefore, after only one packet, the inspector can determine if that packet belongs to a DHCP flow or not. Anyway, for more complicated protocols the identification process turn out to be not so simple and could require many packets to correctly identify the protocol.

The framework can be easily extended with new protocols with few isolated addition to the code. If the programmer wants, he can implement a new inspector by following these simple steps:

1. Give to the protocol the next available numeric identifier.
2. Create a new inspector, by implementing a `C` function with the previously described signature and semantic.
3. If the inspector needs to store information about the application flow, add an appropriate structure in the flow data description.
4. Add the inspector to the set of inspectors which will be called by the framework. This can be done by inserting a pointer to the corresponding function into an appropriate array.
5. If the protocol usually run on one or more predefined ports, specify the association between the ports and the protocol identifier.
6. Recompile the framework.

At this point, when the framework will be executed, it will use this new inspector as any other inspectors already provided by it.

3.5 Callbacks mechanisms

The last point to describe, is about how the application can specify which actions the framework should execute on the data or metadata of the packet once that its protocol has been identified.

For example, let us consider the case in which the application programmer wants to write a monitoring application which extracts the `Host` header from all the HTTP packets and, if it its value is contained in a blacklist, to block the connection.

For this purpose, the framework provides to the application the possibility to define protocol and metadata specific callbacks. For the moment this possibility has been provided only for HTTP protocol and for its different types

of metadata. Anyway, this can be extended to other protocols extending the inspector through a process similar to the one described for the creation of a new inspector.

Let us now see the possibilities provided by the library for the parsing and processing of HTTP metadata. First of all, the application can specify the callbacks by using an appropriate function which requires as parameters:

- The handle of the framework.
- A pointer to some data that will be accessible from the callbacks. For example, considering the HTTP host filtering application, this could be a pointer to the Uniform Resource Locator (URL) blacklist.
- A data structure containing one or more callbacks. Using this parameter the application can specify:
  - The callback to be used on the HTTP *request-URI*.
  - The names of the HTTP *header fields* that the application wants to inspect, along with a callback for each specified *field*. Since the *field* value could be split in more than one TCP segment, the framework will store the different parts in which it is divided and will invoke the callback only when the entire *field* has been reconstructed.
  - A callback to be called when the HTTP *header* has been completely received and processed. This type of callback has been provided because the HTTP *header* may be split in more TCP segments. Therefore, it is needed in order to distinguish the case in which a specific *header field* callback has been not called because the corresponding *field* was not present, from the case in which it has not been called because the header is still not completely received and therefore the *field* may be present in the successive segments.
  - A callback to be called on the *body* of the HTTP message. Like the *header*, also the *body* could be split in different TCP segment. Anyway in this case, instead of waiting to receive and store the entire body, the framework invokes the callback on each body chunk as soon it arrives. This decision has been taken because in case of large files transfer it may be not feasible to store the entire *body* in main memory before invoking the callback. Moreover, in some cases the application might take its decision by analyzing the first bytes of the *body* or might not be willing to wait for its entire reception.

The arrival of the last chunk is signaled through an appropriate callback parameter.

Each callback may have different parameters and must be specified as a `C` function. For example, the HTTP *body* callback has the following parameters, which are provided by the framework and which can be accessed from the code specified inside the callback:

- Information about the HTTP message. It can be used, for example, to check if the message is an HTTP *request* or *response* or to know the `http` *version*.
- A pointer to a chunk of the HTTP *body*.
- The length of the chunk.
- A flag which indicates if this is the last chunk of the HTTP *body*.
- Information extracted from the network and transport headers of the packet (e.g. IP addresses and TCP or UDP ports).
- A pointer to some *flow application data*. This data is stored by the framework, together with the other data about the flow and can be used to keep track of accumulated knowledge about the flow between successive received packets or between successive invoked callbacks. Moreover, this pointer will be returned by the protocol identification call of the framework, together with the identified protocol. In this way the data collected by the callbacks about the flow can be communicated to the rest of the application.

  Furthermore, the application can specify an additional callback to be invoked by the framework on the *flow application data* when the flow expires. Indeed, while the other information about the flow are allocated, managed and deallocated by the framework, *flow application data* is created and managed by the application and thus it should be the application to decide how to manage this data once the flow is terminated.
- A pointer to the global HTTP application data. Differently from the previous parameter, this is a global structures that can be accessed by any HTTP callback, independently from the specific flow.

In listing 3.1 we can see how the application can specify and provide to the framework a simple callback which checks if the HTTP flow is carrying a FLASH video stream.

Listing 3.1: HTTP callback example

```
/**
 * Definition of the callback function with the parameters
 * previously specified.
 **/

#define FLASH_CONTENT_FOUND 0x01

void callback(dpi_http_message_information_t* http_info,
              const u_char* content_type_string,
              u_int32_t content_type_string_length,
              dpi_pkt_infos_t* L3_L4_parsed_information,
              void** app_flow_data,
              void* app_data){

    if((*flow_specific_app_data==NULL) &&
       (strncmp(content_type_string,
                "video/x-flv",
                content_type_string_length)==0)){
        /**
         * Simply to indicate to the main a
         * match has been found.
         **/
        *app_flow_data=FLASH_CONTENT_FOUND;
    }

}

struct packet_information{
    char* pkt; /** Packet **/
    uint len; /** Length **/
    uint ts; /** Timestamp **/
};

int main(int argc, char** argv){
    dpi_http_header_field_callback* cb[1]={&callback};
    /**
     * Indicates to the library that the callback
     * is associated to "Content-Type" header field.
     **/
    const char* ct[1]={"Content-Type"};

    dpi_http_callbacks_t callbacks={
```

```
43          .header_url_callback=NULL, .header_names=ct,
44          .num_header_types=1, .header_types_callbacks=cb,
45          .header_completion_callback=NULL,
46          .http_body_callback=NULL};
47
48      dpi_library_state_t *state=dpi_init_stateful();
49      dpi_http_activate_callbacks(state, &callbacks, NULL);
50
51      struct packet_information packet;
52      dpi_identification_result_t r;
53
54      while(true){
55          packet=receive_packet();
56
57          r=dpi_stateful_identify_application_protocol(
58              state, packet.pkt, packet.len, packet.ts);
59
60          if(r.status>=0 &&
61             r.protocol==HTTP &&
62             r.app_flow_data==FLASH_CONTENT_FOUND){
63              printf("Flash stream found.\n");
64          }
65      }
66
67      dpi_terminate(state);
68  }
```

The struct returned by the protocol identification function contains:

- The status of the processing. When an error occurs (e.g. if the packet is truncated) it will be less than zero. If it is greater or equal than zero, the processing succeeded and this field provides additional information about the processed packet (e.g. if it was a fragment or if it was the TCP segment which terminated the connection).

- The application protocol if it has been already identified, otherwise UNKNOWN protocol is returned.

- The *application flow data*, used by the callbacks to communicate to the rest of the application the result of the processing over the flow to which the packet belongs.

3.6 Demo application

A demo application has been implemented to show the potentialities of the framework and to test the developed code into a real environment.

The target of this application is to scan all the HTTP traffic and to search for well known threats signatures, similarly to what is done by Intrusion Detection and Prevention Systems (IDS/IPS) [41, 42]. As starting point, we used the open source application available in [84] and, with few changes, we modified it in order to read packets from a *pcap* file and to pass them to the framework. It's worth noting that, from a functional point of view, reading the packets from a file or from the network doesn't make any difference. Indeed, the framework has been developed in such a way that it can be independent from the specific technology used to read the packets and it only requires, for each packet, its length and a pointer to a memory area containing it.

Actually, to avoid that the packet reading turns out to be a bottleneck for the application, we first load all the packets in the main memory and then we pass to the framework, one after the other, the pointers to the packets. In this way, we can test the application under the highest load conditions possible on that target architecture.

After that the packets have been loaded, the application reads the set of virus signatures from a text file and insert them into a trie. The signatures check is done inside an HTTP callback similar to the one described in listing 3.1. This callback is executed on all the packets which contain a chunk of an HTTP *body* and, using a modified version of the Aho/Corasick pattern matching algorithm [85], searches if any of the viruses signatures loaded in the trie is present inside the packet. In order to allow a correct recognition of the signature, the framework must be run with TCP reordering enabled. Moreover, the algorithm is written in such a way that when the next *body* chunk is received, it starts the check from the point where it was left in the previous chunk.

It's important to notice that, from the programmer point of view, all the packet processing, TCP reordering, HTTP parsing and data extraction comes for free and it only needs few calls to the framework. In this way, the programmer can focus on what to do with the data instead of how to extract and manage the needed information from the traffic on the network.

As a further example, by changing a single line in the application code it could be modified in order to search patterns in other parts of the HTTP packet. For example, to implement an application which checks if an host is trying to connect to a blacklisted HTTP URL, the programmer needs to indicate that the same callback must be executed on the `Host` *field* instead

of the HTTP *body*.

This application has been validated by executing it over a *pcap* file containing the HTTP transfer of an infected file and by checking that it is successfully recognized by the application.

3.7 Parallel structure

We will now describe how we modified the framework in order to exploit the underlying multiprocessor architecture using the concepts introduced in the previous chapter.

3.7.1 Interface with the application

Differently from the sequential version, in this case the execution control is completely managed by the framework. Therefore, instead of calling the packet processing function directly, the application must specify a callback to let the framework read the packets from the network (*reading callback*) and another one to let it process the results of the identification process (*processing callback*). Moreover, it can specify a pointer to some *callback data* (e.g. a network socket) to be accessed from both the callbacks.

This model of interaction is depicted in figure 3.7, where the dashed boxes represent the callbacks specified by the application.

The *reading callback* has in input the *callback data* and returns to the framework a structure containing: a pointer to the packet, its length, the current time and an arbitrary pointer (*application pointer*). This can be used to associate some application information to the packet and therefore to the corresponding result.

For example, if the application reads the packets from multiple network interfaces, and the actions to take on the processing result changes depending on the interface from which the packet has been received, it may be used to associate the interface to the packet. In this way, when the result of the processing is obtained, the *processing callback* knows the interface from which the packet was received and therefore, the specific action to take on it.

It's important to point out that this pointer was not needed in the sequential framework because the model of interaction with the application was different.

If the returned packet pointer is equal to NULL, then it will be interpreted as an indication that the processing is finished and that the application wants to terminate the framework. This could be useful in case the application

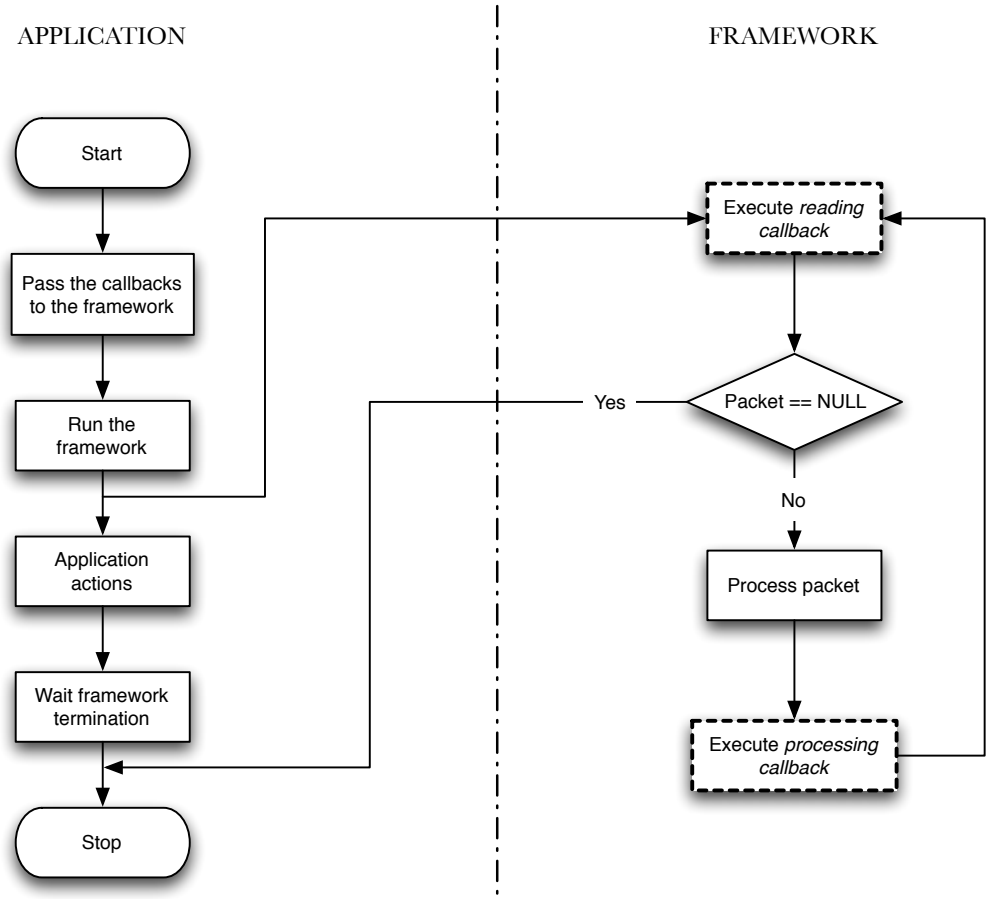

Figure 3.7: Interaction between the application and the parallel framework

received a termination signal from another process and want to provoke a "gentle termination".

The *processing callback* has in input: the *callback data*, the *application pointer* specified by the *reading callback* and the processing result, containing the same information's that were returned by the protocol identification function of the sequential framework.

After specifying the callbacks, the application can run the framework and, concurrently, keep doing some other work. For example, if the application behaves like a firewall, after starting the framework it could wait for changes in protocols filtering policies and, consequently, invoke over the framework the functions to enable or disable the corresponding inspectors.

Eventually, when the application has no more actions to do, it can synchronously wait for the framework termination.

3.7.2 Implementation details

From the point of view of the implementation, we used the `farm` and `pipeline` constructs provided by FastFlow to implement the concepts described in Chapter 2. FastFlow needs that the task that will travel over the computation graph are void pointers. However, to avoid to dynamically allocate and deallocate each of these task, we provided a feedback channel from the collector to the emitter of the farm in such a way that the tasks, when arrive to the end of the graph, can be sent back to the emitter to be reused again.

Regarding the parallelization of the emitter, one of the problems which may arise is that, in general, it could change the order of the received packets. Consequently, we could incur in the cost of TCP normalization also in the cases in which we originally received an ordered stream. To avoid this, we provided the possibility to define the L3 farm as an *order preserving* farm. This have been implemented by simply changing the scheduling strategy used by the emitter (using a variant of *round robin*) and the collection strategy used by the collector.

Therefore, with this small modification in emitter and collector strategies, the packets exit from L3 farm in the same order in which they entered. However, as we will see later, this will not lead in all the cases to an increase of the overall performances. Indeed, there are cases in which is preferable to incur in the cost of TCP reordering instead of blocking the L3 emitter or the L3 collector over the communication channels.

Moreover, we also provided the possibility to apply an *on demand* scheduling strategy for the L3 farm. We will analyze in section 4.3 the impact of

these different strategies.

Another important consideration to be done is about the management of IP defragmentation in the cases in which the processing of the network and transport headers is managed by the L3 farm. In this case indeed each worker may receive a fragment and therefore may need to access the table containing all the outstanding fragmented datagrams. Since it is shared among the different L3 workers, they need to access it in mutual exclusion. For this reason, we decided to protect the access to the datagram fragments table by using *spin locks*. It's important to notice that, since the amount of fragmented traffic is usually below the 1% of the total [86], this would not considerably harm the final performances of the framework. However, as a future work, this could be further optimized in order to reduce the critical section length.

Moreover, we should consider now the way in which the hash table has been split among the L7 workers. In principle, since each partition is independent, we could use n distinct hash tables of size $\frac{s}{n}$, where s is the size of the global table and n is the number of L7 workers. However, we decided to have only one table divided among the workers by means of two indexes for each worker. In this way, the worker i will only access the flows in the buckets of the table included in the bounds $[low_i, high_i[$.

The reason behind this decision can be found considering some of the possible future developments for this work. Indeed, having a global and partitioned structure it should be possible, in principle, to dynamically change the sizes of the partitions during the execution of the framework by simply modifying the bounds of each individual worker. This could be useful for two main purposes:

Dynamic workers reconfiguration This can be useful when we want to dynamically change the parallelism degree according to the current input bandwidth. Therefore, if the number of workers is increased or decreased, we need consequently to split again the table among the new number of workers.

Dynamic flow distribution This could be used in the cases in which the workers are unbalanced. Indeed, also if the hash functions we proposed present a good uniformity of the flow key, not all the flows require the same processing cost. This can depend from different factors like: protocol of the flow, its length or the average size of its packets. Accordingly, when the workers are unbalanced, the flows could be redistributed in a different way to try to mitigate this effect.

The last thing to point out is the way in which the callbacks that the

application defines over protocol data and metadata are invoked. Since they must be called after the extraction of the flow information from the hash table, they will be invoked by the L7 workers. Such that the packets belonging to the same flow will be processed sequentially, the data about the flow can be accessed by the callback without any need of synchronization. However, if the callback needs to access some data which is common to all the flows, then the application must consider that it could be accessed concurrently by multiple threads of the framework.

Chapter 4

Experimental results

In this chapter we assess the design with a complete set of experiments validating the different design choices as well as the overall performance of the framework. The performances of the framework have first been analyzed considering the case in which only the identification of the application protocol is required. After that, we studied the case in which, after that the application protocol has been identified, the extraction and processing of the data contained inside the application payload is required, by using the demo application described in section 3.6.

Unit tests have been developed together with the framework in order to check the correctness of some features like: parsing of network and transport headers, IP defragmentation, TCP stream reassembly and protocols inspectors. These tests work by reading network packets from some *pcap* [87] files provided with the framework and by comparing the obtained results with the expected ones.

All the performance related tests, in order to be reproducible, have been executed over traffic stored in *pcap* files. Moreover, in order to be able to test the framework under the maximum load conditions possible, we read the file containing the traffic directly from the local machine memory instead of reading them from the network. In this way we are able to avoid possible bottlenecks due to the specific packet reading technologies and we can isolate our results from external factors that are not dependent from our work. Furthermore, we would like to avoid to shift this problem to the one caused by the bottleneck due to the limited bandwidth of the I/O transfers from the disk. For this reason, when reading the file, we first load the entire *pcap* in main memory and then we start analyzing it.

It's important to point out that this doesn't affect the validity of our results and that, modelling the capture of the packets in this way, we are testing our framework in the worst conditions possible. In a real situation

indeed the latency to read a packet will be, in general, higher than the one that we have in our case where the packet is already present in memory and we simply need to dereference a pointer. Accordingly, the framework will have an input bandwidth lower than the one used to execute these tests that, consequently, can be considered as an analysis of the worst case.

For the different tests we used the following datasets:

CAIDA This dataset [88] contains traffic captured from a monitor located in a data center in Chicago, and connected to a 10GigE backbone link of a Tier1 ISP between Chicago and Seattle. The dataset has been kindly provided by CAIDA association [89] and contains about 24 millions of IPv4 packets distributed in 1428689 IPv4 flows[1]. The payload of the packets is not present for privacy reasons. However, this can be used to compare the proposed hash functions, as in this case we need to have data only up to the transport protocol header.

Sigcomm This dataset [90] contains a collection of the wireless IP traffic captured over the entire period of a three day conference. It contains about 16 millions of IPv4 packets distributed in 298449 IPv4 flows. Also in this case the payload has been cut off from the packets. Consequently, also this dataset will be used only to compare the different hash functions we proposed.

Synthetic This dataset contains 1428043 synthetic IPv4 packets. These packets are distributed over 13314 HTTP flows, each of which correspond to a transfer of a large file between two hosts.

Darpa This dataset [91] has been collected by the Cyber Systems and Technology Group of MIT Lincoln Laboratory and was commonly used to evaluate intrusion detection systems. It contains 1308081 IPv4 packets distributed in 38985 IPv4 flows.

Local This dataset contains traffic captured from a laptop connected to an home network. It contains 524761 IPv4 packets distributed in 17939 IPv4 flows and 34635 IPv6 packets distributed in 1670 IPv6 flows.

All the tests have been executed over a NUMA machine composed by two Intel Xeon E5-2650 @ 2.00GHz nodes with 8 hyperthreaded cores on each of them. Each NUMA node has 16GB of main memory, a shared 20MB L3 cache and private 256KB L2 and 32KB L1 caches for each core.

[1]The dataset contains a low number of IPv6 flows so the analysis is concentrated on IPv4 flows only.

In all the described experiments, when error bars are present, they represent the standard deviation from the mean. Error bars will not be shown for the results characterized by a negligible standard deviation.

4.1 Hash functions analysis

We now compare the hash functions we implemented to access the hash table containing the data about the flows. This comparison have been done in order to verify if more complicated hash functions, which in general are characterized by an higher latency, are also characterized by a better distribution uniformity with respect to simpler but faster hash functions.

First of all, we compare the distributions of the functions using the metric described in [92]. This metric evaluates the uniformity of an hash function using the following formula:

$$\frac{\sum_{j=0}^{m-1} \frac{b_j(b_j+1)}{2}}{\frac{n}{2m}(n+2m-1)}$$

where b_j is the number of items in $j-th$ slot, m is the number of slots, and n is the total number of items. The more this ratio is close to one, more the function is close to the uniform hash function.

In figure 4.1 we present the results obtained for the functions provided by the framework, varying the load factor of the hash table. For this comparison we used the **CAIDA** dataset. As we can see, all the four hash functions have a distribution which is very close to the uniform one.

Furthermore, in figure 4.2, we illustrate the comparison of the execution times of the parts of the framework directly influenced by the computation of the hash functions. As we can see, as a consequence of the good uniformity of all the proposed functions, the average time spent in accessing the collision lists is the almost the same in all the four cases.

On the other hand, the time spent in computing the function changes and, as we will see later, also if it may seem negligible with respect to the table access time, it has a non negligible effect on the bandwidth of the framework.

Similar results have been obtained also using the other datasets. However, due to space constraints, we will not show them here but we will directly show their effect on the total bandwidth of the framework (expressed in millions of packets per second) in figure 4.3.

As expected, the difference in the time spent to compute the hash function is reflected over the total bandwidth of the framework.

Moreover, from figure 4.3 we can analyze the behaviour of the framework under different kinds of traffic. As we can see, the dataset which exhibit

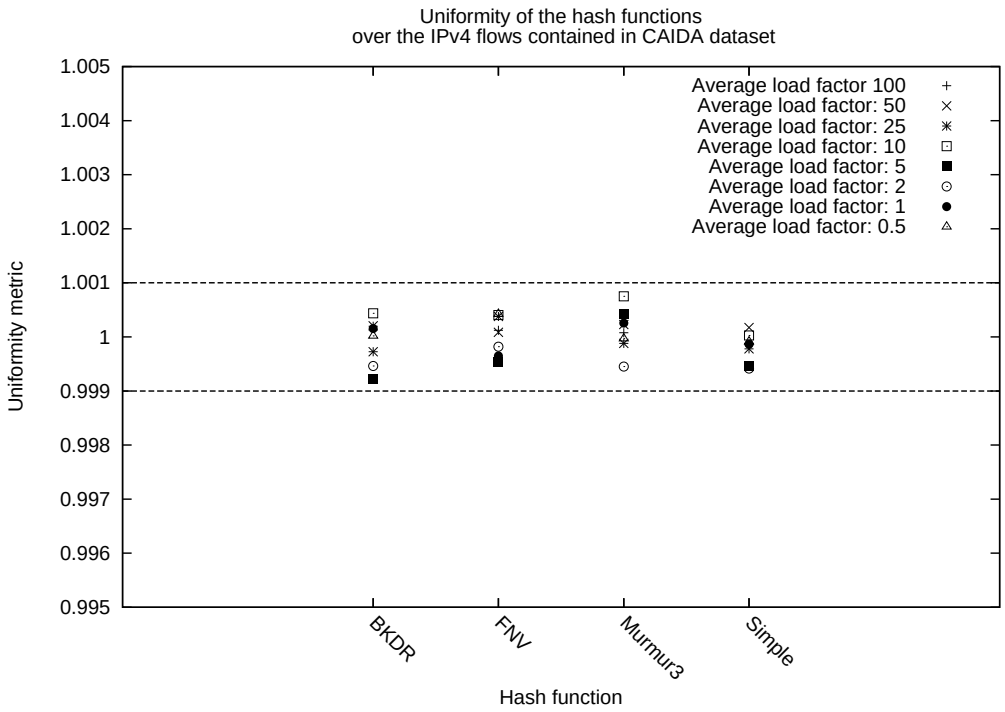

Figure 4.1: Hash functions uniformity over the flows contained in **CAIDA** dataset

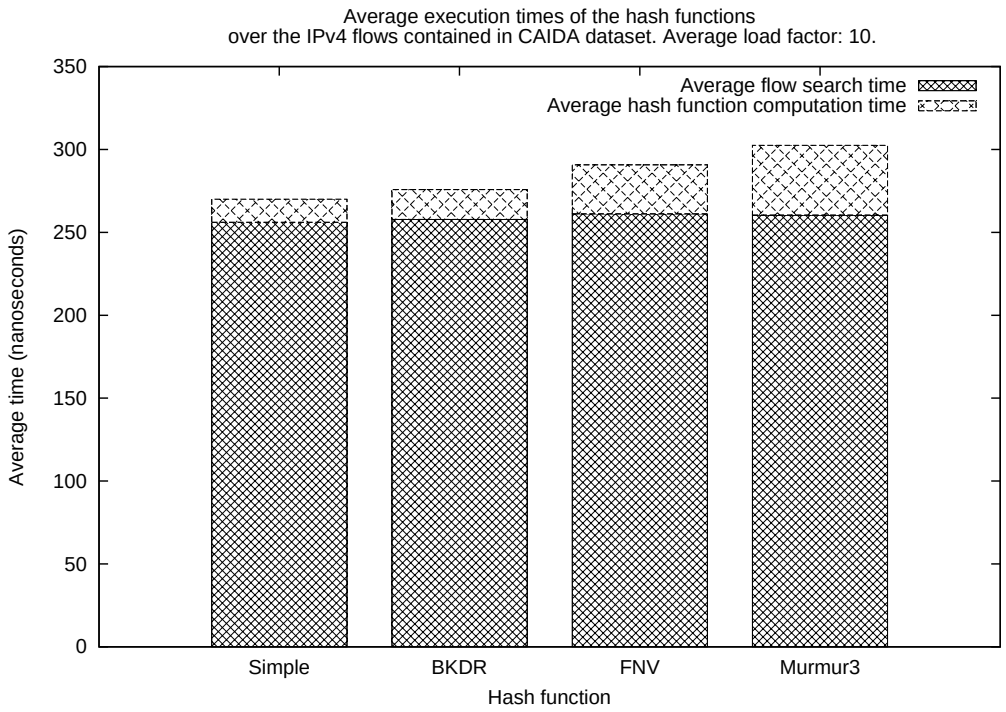

Figure 4.2: Hash functions execution times over the flows contained in **CAIDA** dataset

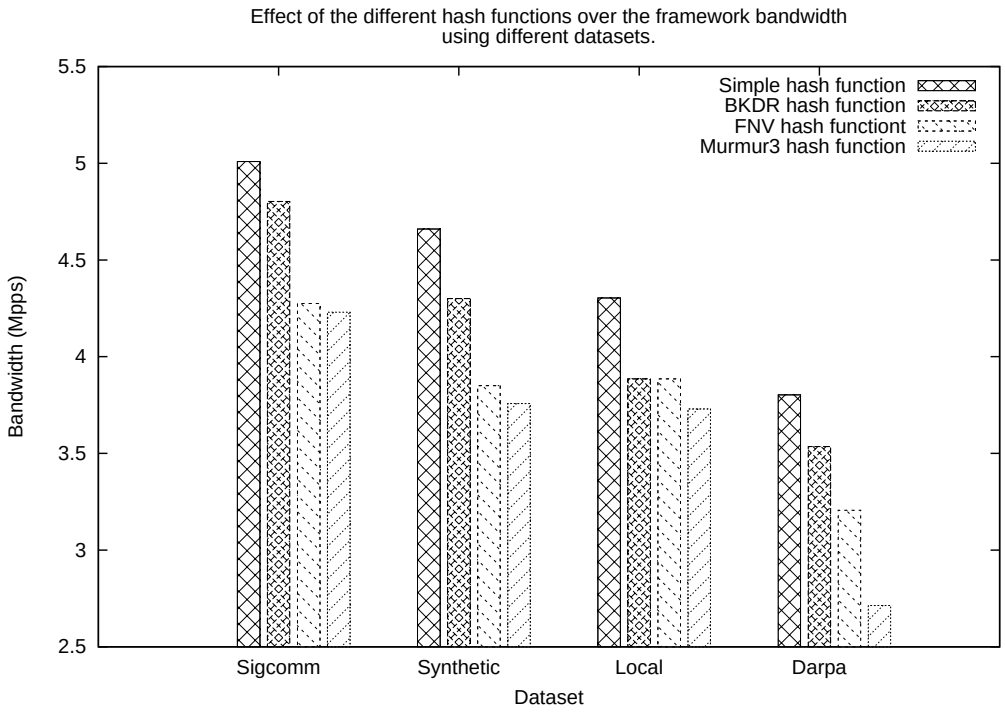

Figure 4.3: Comparison of the impact of the different hash functions over the framework bandwidth (in millions of packets per second)

the highest bandwidth is the **Sigcomm** dataset, due to the absence of the packet payload. Consequently, the protocol inspectors will never be called for this dataset, while we still execute the operations relative to network and transport header parsing, flow data retrieval and IP and TCP normalization.

Concerning the **Synthetic** dataset, it is characterized by flows with a long duration and thus, as we have described in Chapter 2, we will execute the inspectors only on the first packets of each flow. Then, once it has been identified, the inspectors will no more be called for the flow.

Eventually, **Local** and **Darpa** datasets require more computational effort to be processed, since they are characterized by shorter flows and thus the inspectors will be called more frequently.

In this section we have seen that the simplest and fastest hash function among the ones proposed present a distribution uniformity comparable to that of the other functions. Moreover, this has been verified both by analyzing in isolation the parts of the framework which are directly influenced by the hash function and by considering the impact on the overall bandwidth.

4.2 Analysis of Move To Front technique

As we described in section 3.3, the collision lists are kept sorted from the most recent to the least recent flow. This should have a twofold advantage:

- From one side it allows, in principle, to keep the most frequently updated flows in the first positions of the collision list, thus reducing the latency required to find a flow in the hash table.
- On the other hand, when scanning the table to find expired flows, we can usually avoid to scan each collision list entirely. Indeed, since each list is sorted, we can start scanning the list from the end up to the point in which we find a flow which is not expired.

Accordingly, we would like to evaluate if this technique actually increases the overall bandwidth of the framework with respect to the case in which it is not used. In order to assess the advantage of this technique, we made our test over the **Sigcomm** dataset varying the load factor of the hash table. Indeed, since the latency required to find a flow increases when the average length of the collision lists increase, the advantage should be more noticeable incrementing the load factor.

In figure 4.4 we can see that using MTF we achieve significantly better results. Moreover, as expected, this improvement increases while increasing the average load factor of the table.

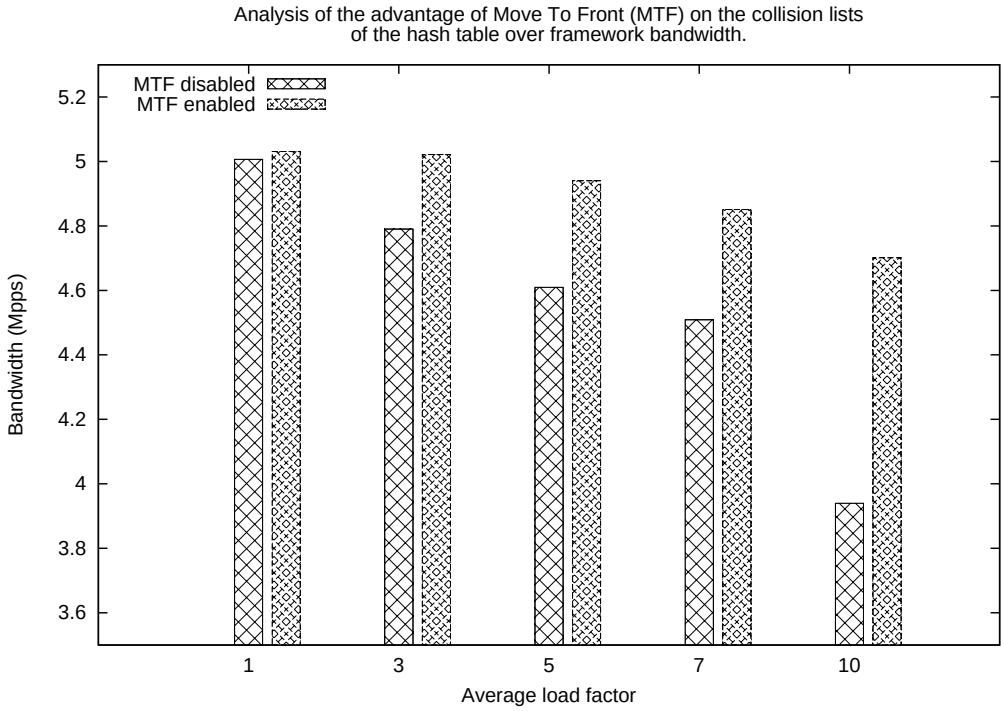

Figure 4.4: Analysis, using Sigcomm dataset, of the impact of MTF strategy over the global bandwidth of the framework varying the average load factor of the table

In this section we have thus validated the implementation choice we made in section 3.3 by analyzing its impact on the overall bandwidth of the framework.

4.3 Comparison of L3 farm scheduling strategies

As we described in section 2.2, when the emitter is parallelized and replaced by the L3 farm, we should consider the possibility that it could reorder the received packets. Therefore, also if they arrive in the correct order to the framework, they could be received out of order by the L7 worker and, consequently, we could incur in the latency caused by TCP normalization also when it is not really needed.

To avoid this problem, we proposed a scheduling strategy which preserves the order of the packets. For this reason in figure 4.5 we compare the two

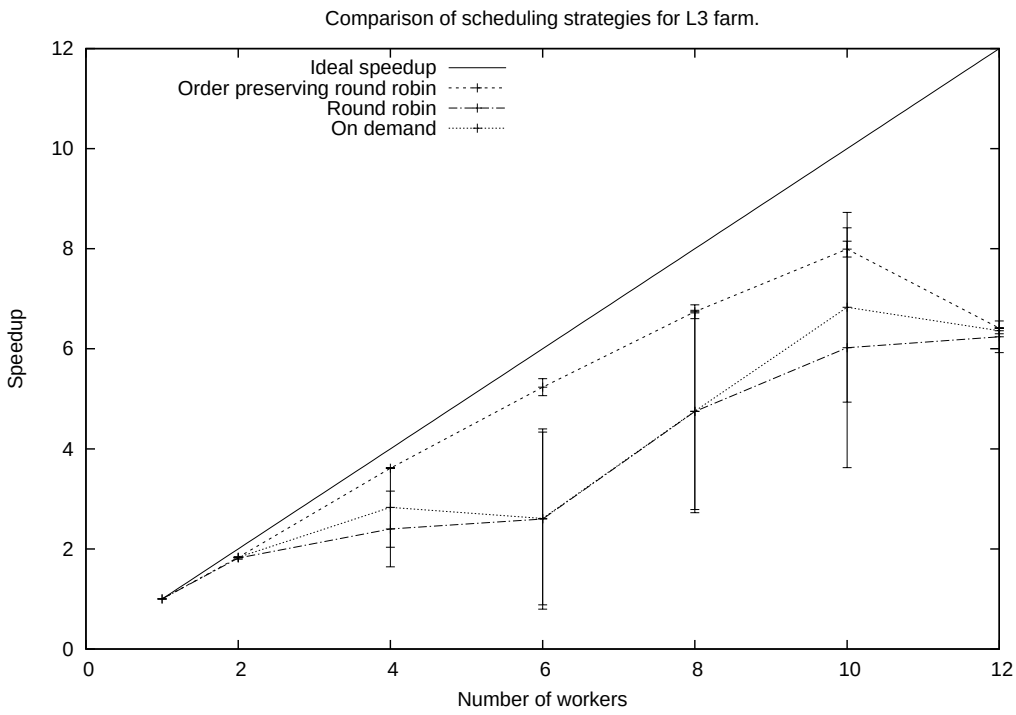

Figure 4.5: Comparison of the impact of the different scheduling strategies for the L3 farm

non ordering strategies (round robin and on demand), with respect to the ordering scheduling (ordered round robin).

As we can see, the ordering scheduling strategy provides better results with respect to the other two solutions. Moreover, when a strategy which doesn't preserve the order is used, the results are characterized by a much higher standard deviation. This happens because in the different runs the packets could be scheduled in different way and thus incurring in a different cost due to the TCP normalization. Furthermore, increasing the parallelism degree (and thus increasing also the number of workers activated for the L3 farm) we increase also the possible ways in which the packets can be scheduled and, as we can see from the figure, this is reflected on the standard deviation.

For these reasons, when TCP normalization is activated, an ordering strategy should be used to schedule the tasks in L3 farm. Indeed in this section we have shown that, when scheduling strategies which don't preserve the order of the packets are used, we could have a worse speedup due to an increase in the cost of TCP reordering. Consequently, the framework will

have a lower bandwidth with respect to the one we have when using an order preserving scheduling strategy.

4.4 Speedup

We will now analyze the speedup achieved thanks to the parallelization of the framework. In the analysis, we will separate the case in which we perform only the protocol identification, from the case in which Network Intelligence (NI) capabilities are required and thus all the packets will be processed also after that the protocol of the corresponding flow has been identified.

4.4.1 Protocol identification

First of all, we will analyze the speedup of the framework when only the protocol identification is performed.

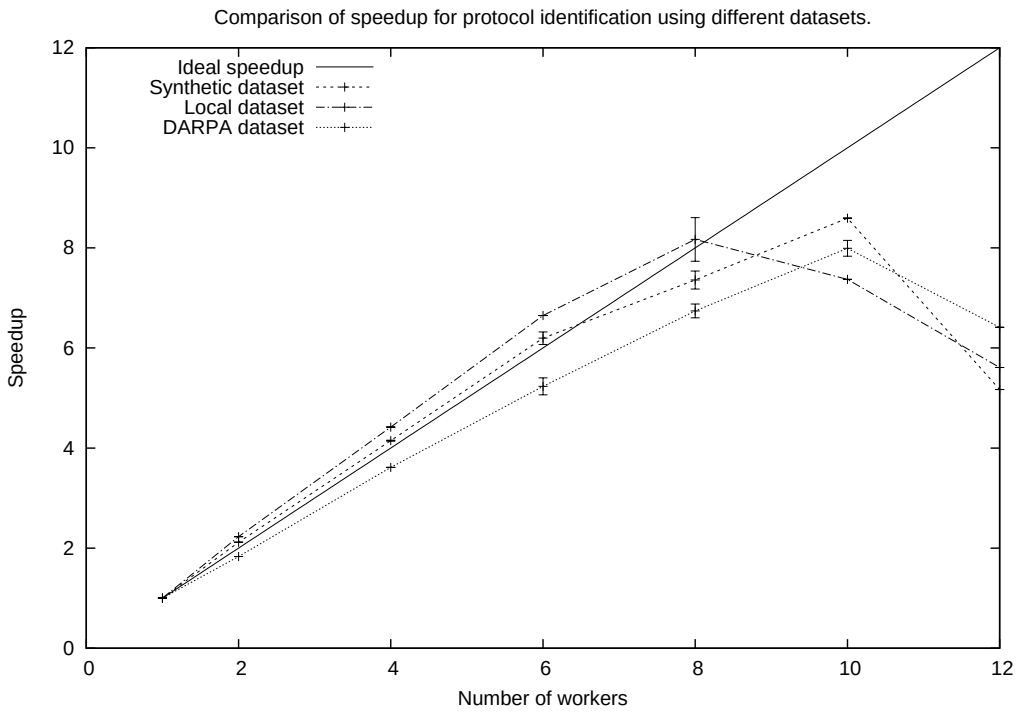

Figure 4.6: Comparison of the speedup for protocol identification using different datasets

Usually, the protocol of an application flow will be identified by inspecting

only the first packets carried by it. For example, considering an HTTP flow, the framework will usually identify the protocol by inspecting only the first packets of the flow (which carry the HTTP header). For this reason, for all the remaining packets of the flow, the cost of the inspection will not be payed and the framework will incur only in the latency due to the access to the hash table (which is always needed to check if the flow have been already identified). This is the reason why, in average, the processing of a packet is a very fine grained operation. For example, on the architecture used to run our tests, in many cases it has a latency of $O(100)$ ns. Consequently, as we can see from figure 4.6, there are cases in which we are not able to achieve the ideal speedup.

On the other side, there are cases in which the framework achieves a speedup greater than the ideal one. As we described in section 1.2, this is due to a better temporal locality exploitation. For example, let's consider the hash table that is partitioned among the workers of the L7 farm. In this case, when more than one node is used, each node will access only to a smaller part of the table which, hopefully, will fit in the lower levels of the memory hierarchy.

In this section we have shown that, despite the very low latency of the protocol identification process, the framework is still able to achieve a good speedup. However, due to the low latency of the protocol identification process, we reach a point where the emitters become bottlenecks and thus the framework reaches the saturation. Anyway, as we will see in section 4.4.2, when NI capabilities are required, the latency of the packet processing increases, allowing thus to achieve better results.

4.4.2 Processing of extracted data

We now analyze how the speedup is affected when packet processing capabilities are required, trying to find a lower bound to the processing grain which allows the framework to achieve a good speedup. In figure 4.7 we compare the speedup of the framework varying the latency of the processing function which will be executed for each identified packet.

As we can see, also for very fine grained processing functions, the framework is characterized by a good speedup. Moreover it's worth noting that, as we will see in section 4.4.3, "real" processing functions are characterized by latencies which may also be two order of magnitude greater than the ones we used to make this analysis. Consequently, as we will see, in real monitoring environments the framework is able to reach a speedup very close to the ideal one.

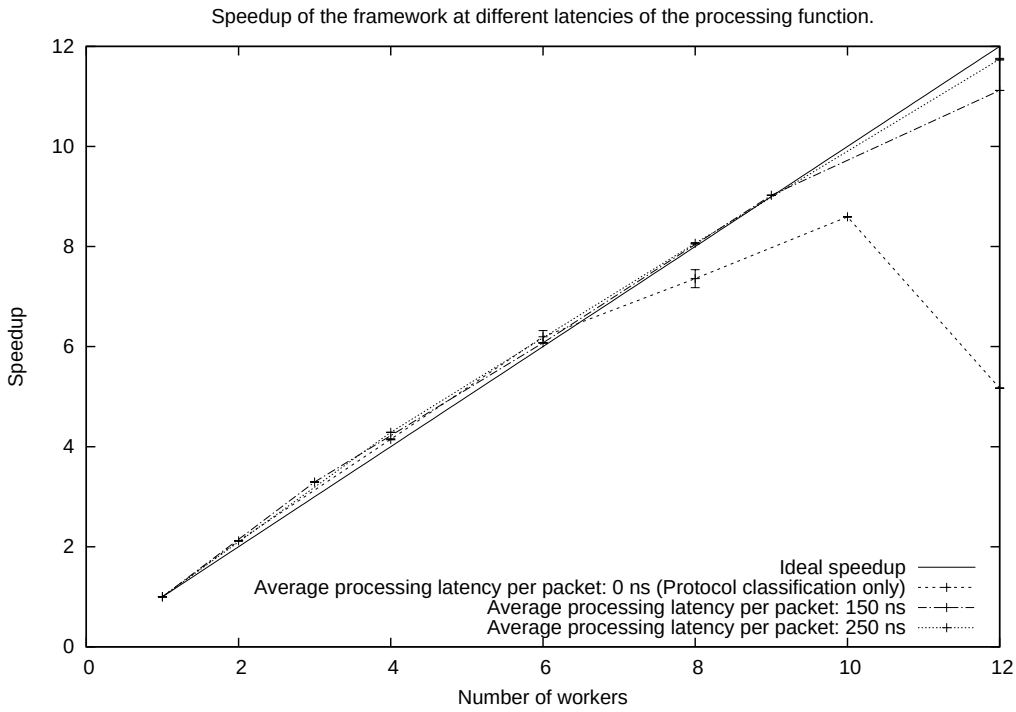

Figure 4.7: Comparison of the speedup when packet processing capabilities are required

4.4.3 Application speedup

Eventually, we analyze the capabilities of the framework under a real monitoring environment as the one described in section 3.6. We briefly recall that the application we built over the developed framework allows to search patterns inside the HTTP packets traveling over the network by using the Aho/Corasick algorithm. Our experiments shown that using a database of 1781 patterns with an average length of 274 characters each, in average, this operation has a latency of $O(10)$ microseconds per packet.

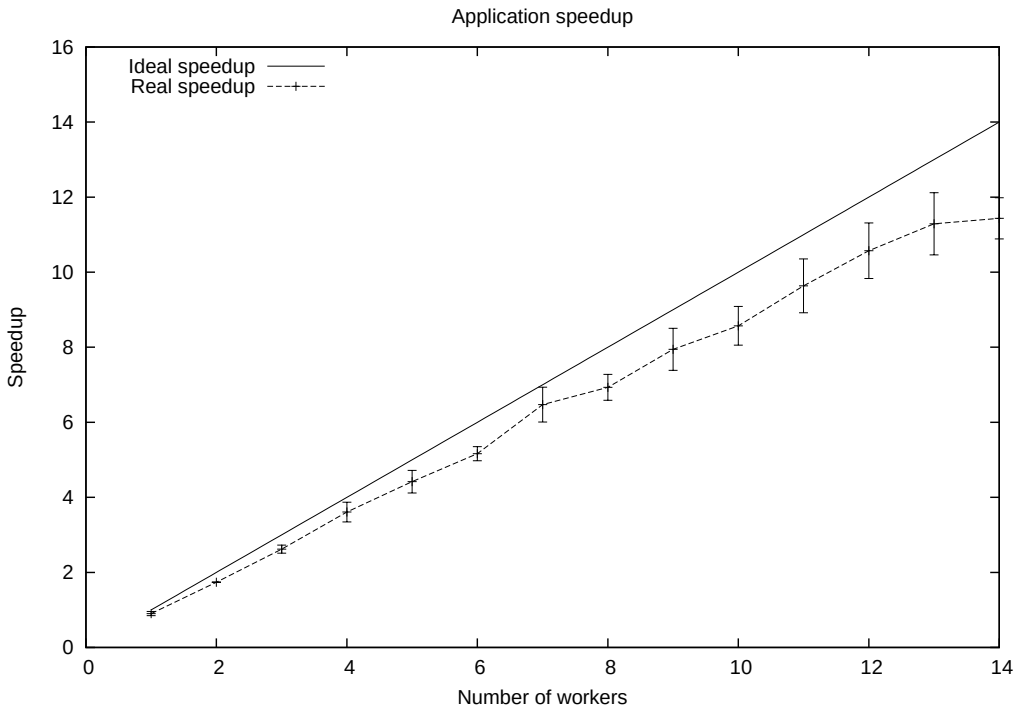

Figure 4.8: Comparison of the speedup of the HTTP payload pattern matching application

As we can see from figure 4.8, the application exhibits a good speedup up to 14 workers.

4.5 Assessment

In this section, we will evaluate the results we obtained using our framework, comparing them with the ones obtained with other existing software and hardware tools.

4.5.1 Comparison with other sequential software solutions

With respect to common and well known existing tools, as shown in Table 4.1, instead of focusing on the number of supported protocols we decided to characterize this work by providing a support for current multicore hardware and by providing to the application programmer the possibility to specify the callback to be used to process specific data and metadata carried by the protocol.

	Our framework	OpenDPI/ nDPI	libproto-ident	l7filter
IPv4 and IPv6 normalization (section 3.2.1)	Yes	No	Yes	Yes
"Stateful" mode (section 3.3)	Yes	No	Yes	Yes
TCP normalization (section 3.4.1)	Yes	No	Yes	Yes
Callbacks (section 3.5)	Yes	No	No	No
Multiprocessor support (Chapter 2)	Yes	No	No	No
Supported protocols	10	117/141	250	112

Table 4.1: Comparison of the features of our framework with respect to well known DPI libraries

Having illustrated the main contributions of our framework with respect to popular software tools, we now compare them from the point of view of the achieved performances.

Concerning nDPI, we made the comparison by using both nDPI and our framework over the same machine. Moreover, in order to have a fair comparison, we disabled from our framework the features that are not supported by nDPI and we disabled from nDPI the protocols inspectors not implemented in this work. Furthermore, since nDPI doesn't have its own way to store the flow data, we slightly modified the hash table used by our framework in order to store the data required by nDPI.

Under these conditions, we tested the two framework over different datasets, comparing them over both synthetic and real traffic. Moreover analyzing the code of nDPI we saw that, also if the protocol of the flow has been already identified, it still try to identify the other packets belonging to the same flow. Accordingly, we compared our results with two different versions of nDPI,

the native one and the one in which we call the library only if the flow has not been yet identified, obtaining the results shown in figure 4.9

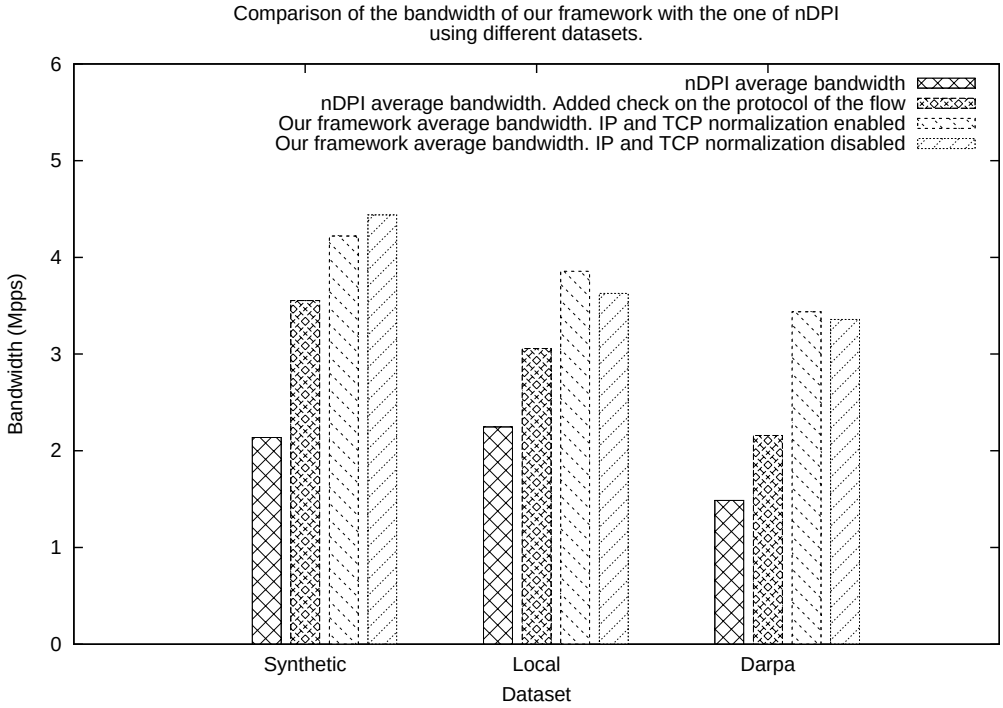

Figure 4.9: Comparison of the bandwidth (in millions of packets per second) of our framework with respect to nDPI over different datasets

As we can see, our framework exhibits always better results, also when IP and TCP normalization is enabled. Moreover it's worth noting that there are cases in which, enabling the TCP normalization, we get an higher bandwidth with respect to the cases in which it is disabled. This derives from the fact that in the former case, when out of order packets are received, we delay the analysis of the flow up to the point in which the stream is in the right order. Consequently, we execute the inspectors a lower number of times and over a longer segment and, since we usually doesn't need to scan the entire segment but only the first part, the cost due to the normalization is amortized and we globally have an higher bandwidth. However, this effect is not present for the **Synthetic** dataset since it contains already ordered data.

Furthermore in this comparison we should take into account that, although we disabled not required protocols, since nDPI has been designed to support more application protocols, its internal data structures (e.g. that used to store the flows information) are usually bigger than the ones we use in

our framework. Accordingly, we need to consider that this could be one of the reasons why nDPI exhibits lower performance, since it gains less advantages from spatial locality.

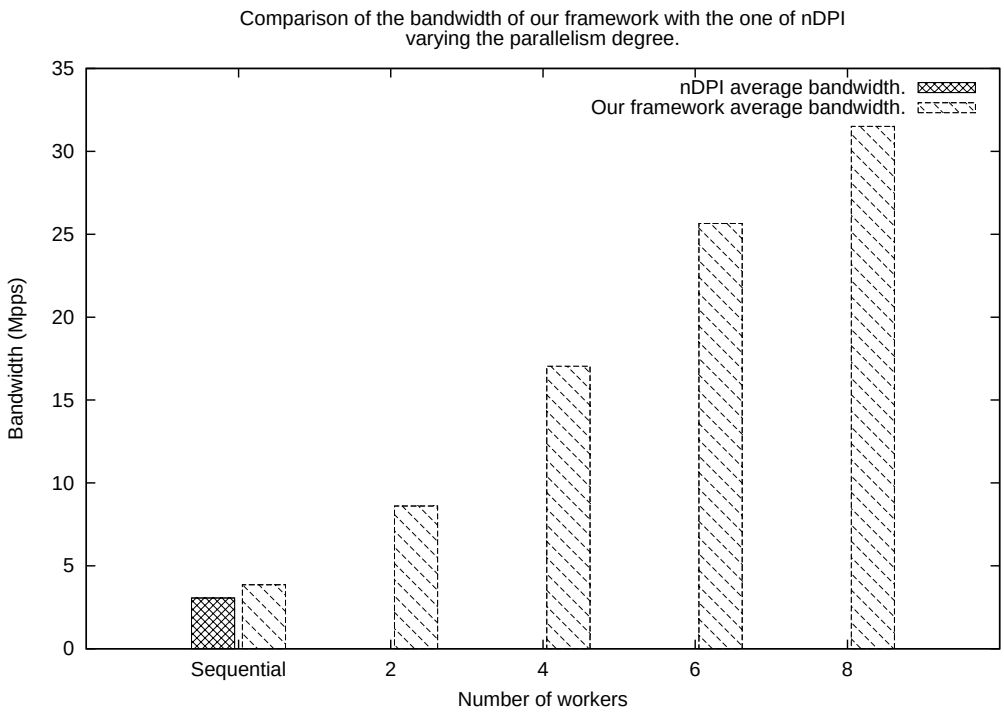

Figure 4.10: Comparison of the bandwidth (in millions of packets per second) of nDPI with respect to our framework varying the number of workers of the farm

Moreover nDPI provides only the possibility to process the packets sequentially and, if multicore support is required, it should be implemented from scratch. Accordingly, in figure 4.10 we compare the bandwidth manageable by nDPI with that of our framework, varying the number of workers of the farm.

Concerning the other tools, according to the authors of [22], l7filter and libprotoident achieve a bandwidth respectively of 2.421 Gbps and 7.752 Gbps. Under presumably similar hardware and using different kinds of real traffic dataset, as shown in Table 4.2 our framework has demonstrated to be able to reach from 5.89 Gbps up to 17.35 Gbps.

However, in this comparison we must consider that these tools support much more protocols with respect to the ones we implemented in our prototype.

Dataset	Bandwidth (Gbps)
Darpa	5.89
Local	17.35

Table 4.2: Bandwidth achieved by our framework using different real traffic datasets

Anyway, the qualitative results should still be valid and our approach, coherently with the considerations done in [22], from the point of view of the achieved bandwidth, should be located between libprotoident and l7filter. Indeed, since Libprotoident is a Lightweight Packet Inspection (LPI) approach and looks only at the first four bytes of the payload, it will probably have an higher bandwidth. However, from a functional point of view, analyzing only the first four bytes could be a limitation since they will not be able to sub classify the protocols or, in general, to extract the data carried by the application.

Concerning l7filter, it exhibits a significant lower bandwidth, mainly due to the inefficiency of regular expression matching with respect to fixed string matching as the one performed in the other cases.

Moreover the main advantage of our approach is that, as shown in section 4.4.1, differently from these tools we are able to exploit the underlying multicore architecture and thus to achieve an higher bandwidth.

Comparison of the speedup with existing software solutions To evaluate the validity of a *structured parallel programming* approach, we will compare the speedup achieved by our framework with that achieved by other DPI tools present in literature. However, since the source code of the other tools is not available, we make our consideration over the results found in the respective works. In Table 4.3 we show the best results obtained for each work in terms of speedup.

Tool	Number of threads	Speedup
[24]	16	10
[25]	14	6.25
[26]	5	1.75
[27]	3	1.98

Table 4.3: Speedup achieved by tools which provide multicore support

As we can see, the other existing tools suffer from limited speedup, also for relatively low parallelism degrees. On the other hand our framework,

as shown in section 4.4.1, is able to reach, in the best case a speedup of 8.17 using 8 farm workers when only the protocol identification is required. Moreover for more complex tasks, as the one described in section 3.6, we are able to reach a speedup of 12 with 14 farm workers.

Furthermore, as also stated from their developers, these problems are often related to design choices. Consequently, also if the results presented in the other works are collected using different machines from the one we used, we can still reasonably suppose that from a qualitative point of view, the results would not change significantly.

In this section, we have shown that we are able to often manage higher bandwidths with respect to the other existing DPI tools and, at the same time, to provide better results in term of speedup thanks to an accurate design which exploits the concepts of *structured parallel programming* theory. Although in some cases it is difficult to make a fair performance comparison that takes all the algorithmic, hardware and used datasets details into account, we clearly shown the efficiency of our approach with respect to those used in other works.

4.5.2 Comparison with hardware solutions

Many hardware solutions are proposed in literature. In most cases they perform pattern matching on the packets both by using Field Programmable Gate Arrays (FPGAs) [17, 37, 38] or by using Content Addressable Memories (CAMs) [39, 18, 19, 40].

Pattern matching can aim both to search for application signatures or to search generic patterns inside the payload, similarly to what the application we described in section 3.6 does.

Some of these solutions reach, in the best case, a bandwidth of 20Gbps [40]. Anyway, we need to consider that this is the rate in isolation of the pattern matching process and thus doesn't account the cost of all the processing related to the parsing of the network and transport headers, hash table lookup and TCP normalization. However, these are all tasks that should be considered and implemented in a real environment, since their absence could lead to the impossibility to find these patterns.

Considering only the protocol classification, we are able to reach the same rate of the CAM solution [40] (20Gbps) over the **Darpa** dataset by using only four workers. Similar results have also been obtained using the other datasets. However, we have to consider that the algorithms used in the two cases are different. Our solution indeed has a better knowledge of the protocols and of the structure of the packet and, consequently, each inspector usually needs only few byte before deciding if the protocol matches or not. Conversely, the

CAM solution searches for a signature inside all the packet payload and thus it usually scans it entirely before moving to the next signature.

For these reasons, a more significant comparison can be done by evaluate the two solutions over a pattern matching application as the one we described in section 3.6 and which have implemented using the framework developed in this thesis. In both cases, the results are taken by considering the matching over a database of 117 patterns with an average length of 67 characters each.

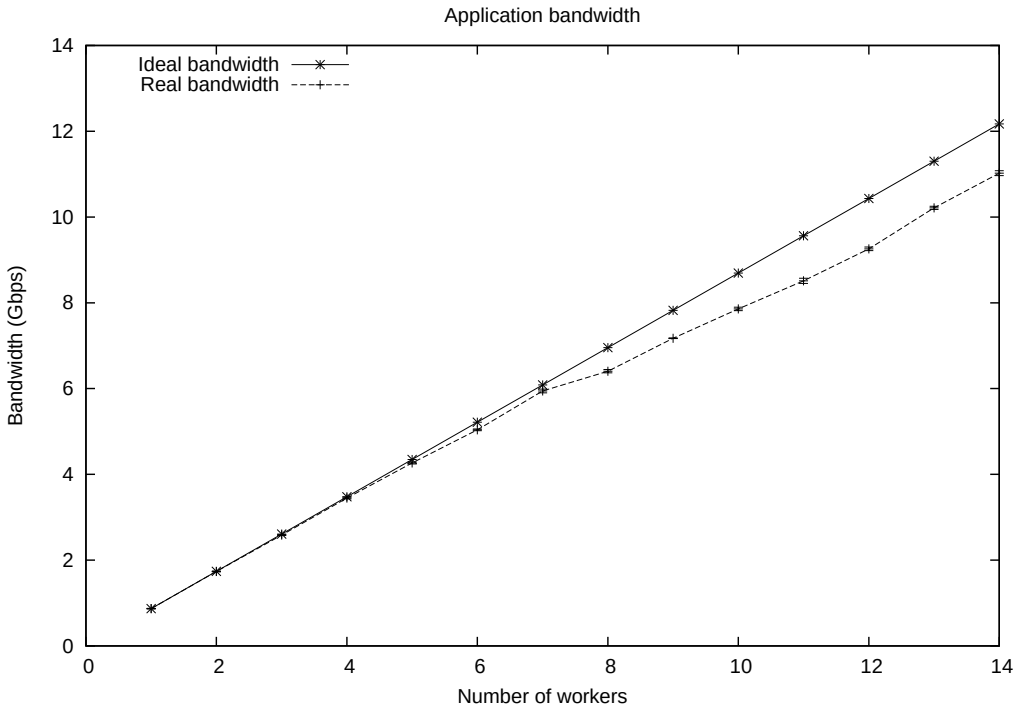

Figure 4.11: Bandwidth of the HTTP payload pattern matching application

Under these conditions, as we can see from figure 4.11, we are able to reach a rate of 11Gbps by using 14 workers. This is a positive result since we are able to achieve comparable rates to those achieved by dedicated hardware solutions and, at the same time, to provide the flexibility and ease of use which characterize software solutions with respect to the ones implemented through dedicated hardware. Moreover, as we said before, we should consider that our results also account the additional overhead due to all the other parts related to the packet processing and not only to pattern matching. From the point of view of the effectiveness, this allows our work to identify the patterns independently from the fact that the data are reordered or segmented while, using only a CAM or FPGA solution, this would not be possible since it

doesn't have any knowledge about the structure of the packets and simply search "blindly" in the traffic which travels over the network. Furthermore our solution allows not only to develop pattern matching applications but also any other type of traffic management applications which require to access and process specific parts of the packet payload.

Lastly it's important to notice that, in principle, nothing prevents our framework to take advantage of this kind of solutions for some specific parts of the protocol identification process by offloading them to FPGA or hardware coprocessors. As an example, considering the demo application described in section 3.6, the application programmer could decide, if needed, to offload the pattern search inside the HTTP payload to an hardware coprocessors and thus to take advantage from dedicated hardware performances.

Chapter 5

Conclusions

The original goal of this thesis was to explore the possibility to apply *structured parallel programming* theory to support the implementation of a DPI framework capable to manage current network rates by exploiting commodity multicore architectures. To achieve our target, we implemented these concepts using FastFlow, a parallel programming library which provides very low latency communication mechanisms to support high bandwidth streaming applications.

Our work allows to identify the application protocol carried by the packet by inspecting its contents up to the application layer and by checking the bytes situated in some specific locations or, in more complex cases, by implementing a full protocol parser. Since such kind of accurate identification requires to maintain some state about each application flow, we designed and implemented efficient data structures to store such data that could be easily partitioned among different threads when multicore support is required. Promising results have been obtained for both the sequential and the parallel version of the framework.

Concerning the sequential framework, we validated the approach by comparing the results we obtained with those achieved by well known DPI libraries: we are able to achieve comparable, and also better results in term of number of packets processed per second.

To support an high degree of flexibility so that the framework can be used for different monitoring applications, we allow the application programmer to specify which part of the protocol metadata or of the data carried by the protocol are actually needed and how to process them. To correctly perform this kind of tasks, it may often be required that the data are processed in the same way they are sent by the application. Consequently, we implemented IP and TCP normalization to rearrange and manage fragmented or out of order

data. Since this is a risky task which could be affected by security threats, this phase required careful design and implementation choices. We validated the flexibility and efficiency of our framework by using it to implement an application which searches some specified patterns inside all the HTTP traffic travelling over the network, performing thus a task similar to that performed by common Intrusion Detection/Prevention Systems (IDS/IPS).

In order to reach the target of this thesis and to efficiently exploit the underlying multicore architecture, the framework has been designed as a farm and, to avoid any kind of unnecessary synchronization mechanisms, the flow data have been partitioned among its workers. Since in some cases the emitter may be a bottleneck, we provided the possibility to parallelize it by means of another farm, obtaining thus a solution composed by two pipelined farms. To avoid the reordering of the packets, we proposed and validated an order preserving strategy for the L3 farm, allowing to prevent unnecessary costs due to TCP reordering and leading thus to better results.

We then validated our approach by showing that we are able to achieve a good speedup also when only protocol identification is required. However, since under certain traffic conditions it could be a very low latency task, we reach the saturation around 8-10 workers.

Moreover we studied the capabilities of our approach in the cases in which, after the protocol has been identified, further payload processing is required. Consequently, we searched the lower bound to the latency of the processing function which allows the framework to achieve a good speedup, founding that it can be achieved also by using functions characterized by a latency two or three order of magnitude lower than that required by real processing applications. Our results have been further validated by comparing the pattern matching application we developed using our framework, with a similar one implemented over dedicated hardware. The comparison showed that we are able to reach around 11 Gbps versus the 20 Gbps reached by the hardware solution. However, the dedicated hardware solution only offers pattern matching capabilities without any knowledge about the structure of the packets, thus providing a lower accuracy to the one we provided implementing also flow management and TCP normalization. Furthermore, our solution is characterized by the flexibility and ease of use which represent one of the biggest advantages of software solutions with respect to dedicated hardware ones. Anyway, nothing prevents our framework to use these hardware solutions to perform some specific tasks.

Our work could still be improved by addressing some problems such as:

Flow unbalancing Even if the hash functions we proposed present a good distribution of the flow key, the flows themselves could still be unbalanced. Accordingly, also if the number of flows managed by the different workers is equally distributed, we could still have an unbalance from the point of view of the managed packets. To solve this problem, the hash table we proposed in our implementation could be modified to dynamically resize is partitions accordingly to the current load of each worker of the farm.

Resizing of hash table Despite the hash functions we proposed are characterized by a good uniformity of their distributions, we could still have cases in which one collision list is particularly unbalanced with respect to the others. Accordingly, we could need some mechanism to dynamically resize the hash table and to redistribute the flows contained in it.

Adaptivity As we described in section 1.2.3 it is very difficult to foresee the average values of the interarrival time or the latencies of the stages of the processing, since they could be influenced by many different factors. Accordingly, when an high efficiency is required and under-provisioning and over-provisioning need to be avoided, our implementation could be modified in such a way that it can dynamically increase or decrease the number of activated threads and, possibly, also to change the parallelism form. This is one of the reasons why, instead of providing one distinct hash table for each worker of the farm, we decided to use a single hash table partitioned among them by means of bounds.

Support for more protocols The provided protocol inspectors may be improved and extended with data and metadata extraction capabilities. In this work, we provided the possibility to extract and process data and metadata only for HTTP protocol. However, the framework has been designed and implemented to support it possibly for all the implemented protocols. Furthermore, inspectors for other protocols could be added, either by using payload inspection techniques or by using statistical approaches.

These problems have not been addressed in this work due to time and size constraints of the thesis. The starting goal of the thesis has been fully achieved, proving the validity of a *structured parallel programming* approach to solve this kind of applications using commodity hardware.

Appendix A

Code of the sequential framework

A.1 API implementation

```
/*
 * api.h
 *
 *  Created on: 19/09/2012
 *  Author: Daniele De Sensi (d.desensi@gmail.com)
 *
 *
 *  Provided features:
 *      +Support for IPv6 and its optional headers
 *      +Tunneling: 6in6, 4in4, 6in4, 4in6
 *      +Protection against truncated or corrupted L3 and L4 packets.
 *       It can be enabled by DPI_ENABLE_L3_TRUNCATION_PROTECTION or
 *       DPI_ENABLE_L4_TRUNCATION_PROTECTION in the cases in which
 *       is not provided by the lower layers. This check is not done
 *       computing the checksum but only looking at the header/payload
 *       lengths.
 *      +Robust IPv4 and IPv6 defragmentation support tested against a
 *       large number of possible attacks.
 *      +TCP stream reassembly.
 *      +Possibility to activate and deactivate protocol inspector and
 *       callbacks at runtime.
 *      +Two possible ways to use the library: stateless and stateful.
 *          +Stateless is suited for applications which already have a
 *           concept of 'flow'. In this case the library stores almost
 *           no state about the flows. The user application should be
 *           modified in order to store with their own flow informations
 *           also the informations needed by the library to identify the
 *           protocols.
```

```
29  *          +Stateful is suited for applications which don't have a
30  *           concept of 'flow'. In this case the user simply pass to
31  *           the library a stream of packets without concerning about
32  *           how to store the flow. All the flow management and storing
33  *           will be done by the library.
34  *      +The protocol recognition can be done with
35  *       dpi_stateful_identify_application_protocol(...) giving simply a
36  *       pointer to the packet, or separating the two stages and using in
37  *       sequence dpi_parse_L3_L4_headers(...) and then
38  *       dpi_stateful_get_app_protocol_v4(...) (or _v6). The latter
39  *       solution gives to the user the possibility to use the L3 and L4
40  *       informations that maybe he already has (skipping the L3 and L4
41  *       info extraction) and to explicitly get a pointer to the beginning
42  *       of application flow in the case in which he wants to invoke its
43  *       own processing routines on the flow payload.
44  *      +Support for user defined callbacks on any HTTP header field and
45  *      HTTP body.
46  *
47  *
48  */
49
50 #ifndef DPI_API_H
51 #define DPI_API_H
52
53 #ifdef __cplusplus
54 extern "C" {
55 #endif
56
57 #include <sys/types.h>
58 #include "config.h"
59 #include "protocol_ids.h"
60 #include "utils.h"
61 #include "reassembly.h"
62 #include "inspectors/http_parser_joyent.h"
63
64 /** Errors **/
65 #define DPI_ERROR_WRONG_IPVERSION -1
66 #define DPI_ERROR_IPSEC_NOTSUPPORTED -2
67 #define DPI_ERROR_L3_TRUNCATED_PACKET -3
68 #define DPI_ERROR_L4_TRUNCATED_PACKET -4
69 #define DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED -5
70 #define DPI_ERROR_MAX_FLOWS -6
71
72 /** Statuses */
73 #define DPI_STATUS_OK 0
74 #define DPI_STATUS_IP_FRAGMENT 1
75 #define DPI_STATUS_IP_LAST_FRAGMENT 2
76 #define DPI_STATUS_TCP_OUT_OF_ORDER 3
77 #define DPI_STATUS_TCP_CONNECTION_TERMINATED 4
```

```
78
79  enum dpi_state_update_status{
80      DPI_STATE_UPDATE_SUCCESS
81     ,DPI_STATE_UPDATE_FAILURE
82  };
83
84
85  enum dpi_http_message_type{
86      DPI_HTTP_REQUEST=HTTP_REQUEST,
87      DPI_HTTP_RESPONSE=HTTP_RESPONSE
88  };
89
90  enum dpi_http_methods{
91  #define XX(num, name, string) DPI_HTTP_##name = num,
92    HTTP_METHOD_MAP(XX)
93  #undef XX
94   };
95
96
97  typedef struct dpi_identification_result{
98      int8_t status;
99      dpi_protocol_t protocol;
100     void* user_flow_data;
101 }dpi_identification_result_t;
102
103 typedef struct dpi_pkt_infos{
104     u_int16_t srcport; /** In network byte order. **/
105     u_int16_t dstport; /** In network byte order. **/
106     u_int8_t ip_version; /** 4 if IPv4, 6 in IPv6. **/
107     /**
108      * 0: From source to dest. 1: From dest to source
109      * (with respect to src and dst stored in the flow).
110      **/
111     u_int8_t direction;
112     /**
113      * Id corresponds to the id defined for IPv4 protocol
114      * field (IPv6 next header field).
115      **/
116     u_int8_t l4prot;
117
118     const unsigned char* pkt;
119     u_int16_t l4offset;
120     u_int16_t l7offset;
121     /**
122      * Length of the application data (from the end of L4 header to the
123      * end).
124      **/
125     u_int32_t data_length;
126     union src_addr{ /** Addresses mantained in network byte order. **/
```

```
127         struct in6_addr ipv6_srcaddr;
128         u_int32_t ipv4_srcaddr;
129     }src_addr_t;
130     union dst_addr{
131         struct in6_addr ipv6_dstaddr;
132         u_int32_t ipv4_dstaddr;
133     }dst_addr_t;
134     /** Time when the library started the processing (in seconds). **/
135     u_int32_t processing_time;
136 }dpi_pkt_infos_t;
137
138 typedef struct dpi_http_message_informations{
139     u_int8_t http_version_major;
140     u_int8_t http_version_minor;
141     /**
142      * HTTP method identifier if request_or_response==DPI_HTTP_REQUEST,
143      * otherwise is the HTTP status code. The method identifiers are
144      * named DPI_HTTP_METHOD_GET, DPI_HTTP_METHOD_POST, etc.
145      */
146     u_int16_t method_or_code;
147     /**
148      * DPI_HTTP_REQUEST if the header field belongs to an HTTP request,
149      * DPI_HTTP_RESPONSE otherwise.
150      **/
151     u_int8_t request_or_response;
152 }dpi_http_message_informations_t;
153
154 /**
155  * This callback is called when the flow is expired and deleted. It can be
156  * used by the user to clear flow_specific_user_data
157  * @param flow_specific_user_data A pointer to the user data specific ↙
        ↳ to this flow.
158  */
159 typedef void(dpi_flow_cleaner_callback)(void* flow_specific_user_data);
160
161 /**
162  * This callback is called when the corresponding header field is found.
163  * If the field is divided into more TCP segments it is reconstructed by
164  * the library and this callback is called only one time on the
165  * reconstructed field. In any case, the pointer to the value is valid
166  * only for the lifetime of the callback. It must be copied if the user
167  * needs it.
168  *
169  * @param url A pointer to the string representing the value of the url
170  *            It is valid only for the callback lifetime.
171  * @param url_length The length of the string representing the url.
172  * @param pkt_informations A pointer to the parsed packet.
173  * @param flow_specific_user_data A pointer to the user data specific
174  *                                to this flow.
```

```
175  * @param user_data A pointer to the global HTTP user data.
176  */
177 typedef void(dpi_http_header_url_callback)(
178                  const unsigned char* url,
179                  u_int32_t url_length,
180                  dpi_pkt_infos_t* pkt_informations,
181                  void** flow_specific_user_data,
182                  void* user_data);
183
184
185 /**
186  * This callback is called when the corresponding header field is found.
187  * If the field is divided into more TCP segments it is reconstructed by
188  * the library and this callback is called only one time on the
189  * reconstructed field. In any case, the pointer to the value is valid
190  * only for the lifetime of the callback. It must be copied if the user
191  * needs it.
192  *
193  * @param http_message_informations   A pointer to the struct containing
194  *                                    the message informations (method,
195  *                                    http version, etc..). The pointer
196  *                                    is valid only for the callback
197  *                                    lifetime.
198  * @param header_value                A pointer to the string representing
199  *                                    the value of the header field (NOTE:
200  *                                    Is not \0 terminated). It is valid
201  *                                    only for the callback lifetime.
202  * @param header_value_length         The length of the string ↙
       ↳ representing
203  *                                    the value of the header field.
204  * @param pkt_informations            A pointer to the parsed packet.
205  * @param flow_specific_user_data     A pointer to the user data specific
206  *                                    to this flow.
207  * @param user_data                   A pointer to the global HTTP user ↙
       ↳ data.
208  */
209 typedef void(dpi_http_header_field_callback)(
210                dpi_http_message_informations_t* http_message_informations,
211                const unsigned char* header_value,
212                u_int32_t header_value_length,
213                dpi_pkt_infos_t* pkt_informations,
214                void** flow_specific_user_data,
215                void* user_data);
216
217 /**
218  * This callback is called when the HTTP header processing is finished.
219  * This can be useful to distinguish the case in which a callback has
220  * not been invoked because the corresponding header field is not present
221  * or if it has not been invoked because the field is not yet been
```

```
222  * received.
223  *
224  * @param http_message_informations   A pointer to the struct containing
225  *                                    the message informations (method,
226  *                                    http version, etc..). The pointer is
227  *                                    valid only for the callback ↙
       ↳ lifetime.
228  * @param pkt_informations            A pointer to the parsed packet.
229  * @param flow_specific_user_data     A pointer to the user data ↙
       ↳ specific to
230  *                                    this flow.
231  * @param user_data                   A pointer to the global HTTP user ↙
       ↳ data.
232  */
233 typedef void(dpi_http_header_completion_callback)(
234                dpi_http_message_informations_t* http_message_informations,
235                dpi_pkt_infos_t* pkt_informations,
236                void** flow_specific_user_data,
237                void* user_data);
238
239 /**
240  * This callback is called when an HTTP body is found. If the body is
241  * divided in multiple TCP segments, then for each segment this callback
242  * is called with the content of that specific segment. When the last
243  * segment of the body arrives, the field last_chunk will be different
244  * from 0. The pointer to the body chunk is valid only for the lifetime
245  * of the callback.
246  *
247  * @param http_message_informations   A pointer to the struct containing
248  *                                    the message informations (method,
249  *                                    http version, etc..). The pointer
250  *                                    is valid only for the callback
251  *                                    lifetime.
252  * @param body_chunk A pointer to a chunk of the HTTP body present in the
253  *                   segment. It is valid only for the callback lifetime.
254  * @param body_chunk_length           The length of the chunk.
255  * @param pkt_informations            A pointer to the parsed packet.
256  * @param flow_specific_user_data     A pointer to the user data specific
257  *                                    to this flow.
258  * @param user_data A pointer to the global HTTP user data.
259  * @param last_chunk 0 if it is not the last chunk 1 otherwise.
260  */
261 typedef void(dpi_http_body_callback)(
262                dpi_http_message_informations_t* http_message_informations,
263                const unsigned char* body_chunk,
264                u_int32_t body_chunk_length,
265                dpi_pkt_infos_t* pkt_informations,
266                void** flow_specific_user_data,
267                void* user_data,
```

```
268                 u_int8_t last_chunk);
269
270 typedef struct dpi_http_callbacks{
271     /** Called on the HTTP request-URI. **/
272     dpi_http_header_url_callback* header_url_callback;
273
274     /** The names of the headers types that the user wants to inspect. **/
275     const char** header_names;
276
277     /** The number of headers types that the user wants to inspect. **/
278     u_int8_t num_header_types;
279
280     /**
281      * The callbacks that will be invoked for the specified header fields.
282      * header_types_callbacks[i] will be invoked when an header with name
283      * header_names[i] is found. The user cannot make any assumption on
284      * the order in which the callbacks will be invoked.
285      */
286     dpi_http_header_field_callback** header_types_callbacks;
287
288     /** Called when the header has been completely processed. **/
289     dpi_http_header_completion_callback* header_completion_callback;
290
291     /** Called on the entire HTTP body. **/
292     dpi_http_body_callback* http_body_callback;
293 }dpi_http_callbacks_t;
294
295 typedef struct dpi_http_internal_informations{
296     dpi_http_callbacks_t* callbacks;
297     dpi_pkt_infos_t* pkt_informations;
298     void** flow_specific_user_data;
299     void* user_data;
300     char* temp_buffer;
301     size_t temp_buffer_size;
302 }dpi_http_internal_informations_t;
303
304 /** This must be initialized to zero before use. **/
305 typedef struct dpi_tracking_informations{
306     /**
307      *  This data is passed to the user when a callback is invoked. It can
308      *  be used by the user to read/write flow specific informations or
309      *  informations which must be passed from one callback to another
310      *  (E.g. subprotocols informations). It is returned to the user when
311      *  dpi_state*_identify_application_protocol() is invoked.
312      **/
313     void* flow_specific_user_data;
314
315     /*********************************/
316     /** TCP Tracking informations.  **/
```

```
317     /**********************************/
318     /**
319      * The expected sequence numbers in the two directions.
320      * (Stored in host byte order).
321      **/
322     u_int32_t expected_seq_num[2];
323     /** A pointer to out of order segments. **/
324     dpi_reassembly_fragment_t* segments[2];
325
326     /** Three-way handshake tracking informations. **/
327     u_int8_t seen_syn:1;
328     u_int8_t seen_syn_ack:1;
329     u_int8_t seen_ack:1;
330
331     /** Connection termination tracking informations. **/
332     u_int8_t seen_fin:2;
333
334     u_int8_t first_packet_arrived:2;
335     u_int32_t highest_ack[2];
336
337     /**********************************/
338     /** HTTP Tracking informations. **/
339     /**********************************/
340     /** One HTTP parser per direction. **/
341     http_parser http[2];
342     dpi_http_internal_informations_t http_informations[2];
343
344     /**********************************/
345     /** SMTP Tracking informations. **/
346     /**********************************/
347     u_int8_t num_smtp_matched_messages:2;
348
349     /**********************************/
350     /** POP3 Tracking informations. **/
351     /**********************************/
352     u_int8_t num_pop3_matched_messages:2;
353 }dpi_tracking_informations_t;
354
355 typedef struct library_state dpi_library_state_t;
356
357 /**
358  * A generic protocol inspector.
359  * @param state          A pointer to the state of the library.
360  * @param pkt            A pointer to the parsed packet.
361  * @param app_data       A pointer to the application payload.
362  * @param data_length    The length of the application payload.
363  * @param tracking       A pointer to the protocols tracking informations.
364  * @return               DPI_PROTOCOL_MATCHES if the protocol matches.
365  *                       DPI_PROTOCOL_NO_MATCHES if the protocol doesn't
```

```
366  *                        matches.
367  *                        DPI_PROTOCOL_MORE_DATA_NEEDED if the inspector
368  *                        needs more data to decide.
369  *                        DPI_ERROR if an error occurred.
370  */
371 typedef u_int8_t(*dpi_inspector_callback)(
372                 dpi_library_state_t* state,
373                 dpi_pkt_infos_t* pkt,
374                 const unsigned char* app_data,
375                 u_int32_t data_length,
376                 dpi_tracking_informations_t* tracking);
377
378
379 struct library_state{
380     /*********************************************************************/
381     /** Created by dpi_init_state and never modified                    **/
382     /*********************************************************************/
383     void *db4;
384     void *db6;
385
386     /*********************************************************************/
387     /** Can be modified during the execution but only using the state  **/
388     /** update functions. They are never modified in other places      **/
389     /*********************************************************************/
390     char udp_protocols_to_inspect[BITNSLOTS(DPI_NUM_UDP_PROTOCOLS)];
391     char tcp_protocols_to_inspect[BITNSLOTS(DPI_NUM_TCP_PROTOCOLS)];
392
393     char udp_active_callbacks[BITNSLOTS(DPI_NUM_UDP_PROTOCOLS)];
394     char tcp_active_callbacks[BITNSLOTS(DPI_NUM_TCP_PROTOCOLS)];
395
396     dpi_l7_prot_id udp_active_protocols;
397     dpi_l7_prot_id tcp_active_protocols;
398
399     u_int16_t max_trials;
400
401     dpi_flow_cleaner_callback* flow_cleaner_callback;
402     /** HTTP callbacks. **/
403     void* http_callbacks;
404     void* http_callbacks_user_data;
405
406     u_int8_t tcp_reordering_enabled:1;
407
408     /*********************************************************************/
409     /** The content of these structures can be modified during the     **/
410     /** execution also in functions different from the state update    **/
411     /** functions. This is the reason why when multiprocessor support  **/
412     /** is used, we need to have one copy of these structures for each **/
413     /** worker or we need to protect the access with mutual exclusion  **/
414     /** mechanisms (e.g. locks).                                       **/
```

```
415     /*****************************************************************/
416     void* ipv4_frag_state;
417     void* ipv6_frag_state;
418 };
419
420 /**
421  * If stateless version is used, this structure the first time must be
422  * initialized with 'dpi_init_flow_infos'.
423  **/
424 typedef struct dpi_flow_infos{
425     /** The possible number of l7 protocols that match with this flow. **/
426     u_int8_t possible_protocols;
427
428     /**
429      * The protocol of this flow. It can be DPI_PROTOCOL_NOT_DETERMINED if
430      * it is not been yet determined; DPI_PROTOCOL_UNKNOWN if it is ↙
     ↳ unknown
431      * or the matching protocol identifier.
432      */
433     dpi_l7_prot_id l7prot;
434
435     /** Number of times that the library tried to guess the protocol. **/
436     u_int16_t trials;
437     /**
438      * Contains the possible matching protocols for the flow (At the first
439      * iteration the mask contains all the active protocols. During the
440      * successive iterations we remove from the mask the protocols which
441      * surely don't match).
442      **/
443     union possible_matching_protocols{
444         char udp[BITNSLOTS(DPI_NUM_UDP_PROTOCOLS)];
445         char tcp[BITNSLOTS(DPI_NUM_TCP_PROTOCOLS)];
446     }possible_matching_protocols_t;
447
448     /**
449      * In this way if a flow was created when TCP reordering was enabled,
450      * we will continue doing TCP reordering for this flow also if it is
451      * disabled. Basically the change in TCP reordering enabling/disabling
452      * will be applied only to new flows.
453      */
454     u_int8_t tcp_reordering_enabled:1;
455     dpi_tracking_informations_t tracking;
456 }dpi_flow_infos_t;
457
458
459 /**
460  * Initializes the state of the library. If not specified otherwise after
461  * the initialization, the library will consider all the protocols active.
462  * @param size_v4 Size of the array of pointers used to build the database
```

```
463  *                for v4 flows.
464  * @param size_v6 Size of the array of pointers used to build the database
465  *                for v6 flows.
466  * @param max_active_v4_flows The maximum number of IPv4 flows which can
467  *        be active at any time. After reaching this threshold, new flows
468  *        will not be created.
469  * @param max_active_v6_flows The maximum number of IPv6 flows which can
470  *        be active at any time. After reaching this threshold, new flows
471  *        will not be created.
472  * @return A pointer to the state of the library otherwise.
473  */
474 dpi_library_state_t* dpi_init_stateful(u_int32_t size_v4,
475                                        u_int32_t size_v6,
476                                        u_int32_t max_active_v4_flows,
477                                        u_int32_t max_active_v6_flows);
478
479 /**
480  * Initializes the state of the library. If not specified otherwise after
481  * the initialization, the library will consider all the protocols active.
482  * @return A pointer to the state of the library otherwise.
483  */
484 dpi_library_state_t* dpi_init_stateless(void);
485
486 /**
487  * Terminates the library.
488  * @param state A pointer to the state of the library.
489  */
490 void dpi_terminate(dpi_library_state_t *state);
491
492
493 /**
494  * Sets the maximum number of times that the library tries to guess the
495  * protocol. During the flow protocol identification, after this number
496  * of trials, in the case in which it cannot decide between two or more
497  * protocols, one of them will be chosen, otherwise DPI_PROTOCOL_UNKNOWN
498  * will be returned.
499  * @param state A pointer to the state of the library.
500  * @param max_trials Maximum number of trials. Zero will be consider as
501  *                   infinity.
502  *
503  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
504  *         otherwise.
505  */
506 u_int8_t dpi_set_max_trials(dpi_library_state_t *state,
507                             u_int16_t max_trials);
508
509
510 /**
511  * Enable IPv4 defragmentation.
```

```
512  * @param state        A pointer to the library state.
513  * @param table_size   The size of the table to be used to store IPv4
514  *                     fragments informations.
515  *
516  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
517  *         otherwise.
518  */
519 u_int8_t dpi_ipv4_fragmentation_enable(dpi_library_state_t *state,
520                                        u_int16_t table_size);
521
522 /**
523  * Enable IPv6 defragmentation.
524  * @param state        A pointer to the library state.
525  * @param table_size   The size of the table to be used to store IPv6
526  *                     fragments informations.
527  *
528  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
529  *         otherwise.
530  */
531 u_int8_t dpi_ipv6_fragmentation_enable(dpi_library_state_t *state,
532                                        u_int16_t table_size);
533
534 /**
535  * Sets the amount of memory that a single host can use for IPv4
536  * defragmentation.
537  * @param state                    A pointer to the library state.
538  * @param per_host_memory_limit    The maximum amount of memory that
539  *                                 any IPv4 host can use.
540  *
541  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
542  *         DPI_STATE_UPDATE_FAILURE otherwise.
543  */
544 u_int8_t dpi_ipv4_fragmentation_set_per_host_memory_limit(
545                dpi_library_state_t *state,
546                u_int32_t per_host_memory_limit);
547
548 /**
549  * Sets the amount of memory that a single host can use for IPv6
550  * defragmentation.
551  * @param state                    A pointer to the library state.
552  * @param per_host_memory_limit    The maximum amount of memory that
553  *                                  any IPv6 host can use.
554  *
555  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
556  *         DPI_STATE_UPDATE_FAILURE otherwise.
557  */
558 u_int8_t dpi_ipv6_fragmentation_set_per_host_memory_limit(
559                 dpi_library_state_t *state,
560                 u_int32_t per_host_memory_limit);
```

```
561
562 /**
563  * Sets the total amount of memory that can be used for IPv4
564  * defragmentation.
565  * If fragmentation is disabled and then enabled, this information must be
566  * passed again.
567  * Otherwise default value will be used.
568  * @param state               A pointer to the state of the library
569  * @param totel_memory_limit  The maximum amount of memory that can be ↙
       ↳ used
570  *                            for IPv4 defragmentation.
571  *
572  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
573  *         DPI_STATE_UPDATE_FAILURE otherwise.
574  */
575 u_int8_t dpi_ipv4_fragmentation_set_total_memory_limit(
576                 dpi_library_state_t *state,
577                 u_int32_t total_memory_limit);
578
579 /**
580  * Sets the total amount of memory that can be used for IPv6
581  * defragmentation. If fragmentation is disabled and then enabled, this
582  * information must be passed again. Otherwise default value will be used.
583  * @param state               A pointer to the state of the library
584  * @param total_memory_limit  The maximum amount of memory that can be
585  *                            used for IPv6 defragmentation.
586  *
587  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
588  *         DPI_STATE_UPDATE_FAILURE otherwise.
589  */
590 u_int8_t dpi_ipv6_fragmentation_set_total_memory_limit(
591                 dpi_library_state_t *state,
592                 u_int32_t total_memory_limit);
593
594 /**
595  * Sets the maximum time (in seconds) that can be spent to reassembly an
596  * IPv4 fragmented datagram. Is the maximum time gap between the first and
597  * last fragments of the datagram.
598  * @param state            A pointer to the state of the library.
599  * @param timeout_seconds  The reassembly timeout.
600  *
601  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
602  *         DPI_STATE_UPDATE_FAILURE otherwise.
603  */
604 u_int8_t dpi_ipv4_fragmentation_set_reassembly_timeout(
605                 dpi_library_state_t *state,
606                 u_int8_t timeout_seconds);
607
608 /**
```

```
609  * Sets the maximum time (in seconds) that can be spent to reassembly an
610  * IPv6 fragmented datagram. Is the maximum time gap between the first and
611  * last fragments of the datagram.
612  * @param state            A pointer to the state of the library.
613  * @param timeout_seconds  The reassembly timeout.
614  *
615  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
616  *         DPI_STATE_UPDATE_FAILURE otherwise.
617  */
618 u_int8_t dpi_ipv6_fragmentation_set_reassembly_timeout(
619                 dpi_library_state_t *state,
620                 u_int8_t timeout_seconds);
621
622 /**
623  * Disable IPv4 defragmentation.
624  * @param state A pointer to the state of the library.
625  *
626  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
627  *         DPI_STATE_UPDATE_FAILURE otherwise.
628  */
629 u_int8_t dpi_ipv4_fragmentation_disable(dpi_library_state_t *state);
630
631 /**
632  * Disable IPv6 defragmentation.
633  * @param state A pointer to the state of the library.
634  *
635  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
636  *         DPI_STATE_UPDATE_FAILURE otherwise.
637  */
638 u_int8_t dpi_ipv6_fragmentation_disable(dpi_library_state_t *state);
639
640 /**
641  * If enabled, the library will reorder out of order TCP packets
642  * (enabled by default).
643  * @param state  A pointer to the state of the library.
644  *
645  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
646  *         DPI_STATE_UPDATE_FAILURE otherwise.
647  */
648 u_int8_t dpi_tcp_reordering_enable(dpi_library_state_t* state);
649
650 /**
651  * If it is called, the library will not reorder out of order TCP packets.
652  * Out-of-order segments will be delivered to the inspector as they
653  * arrive. This means that the inspector may not be able to identify the
654  * application protocol. Moreover, if there are callbacks saved for TCP
655  * based protocols, if TCP reordering is disabled, the extracted
656  * informations could be erroneous or incomplete.
657  * @param state A pointer to the state of the library.
```

```
 *
 * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
 *         DPI_STATE_UPDATE_FAILURE otherwise.
 */
u_int8_t dpi_tcp_reordering_disable(dpi_library_state_t* state);

/**
 * Enable a protocol inspector.
 * @param state         A pointer to the state of the library.
 * @param protocol      The protocol to enable.
 *
 * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
 *         DPI_STATE_UPDATE_FAILURE otherwise.
 */
u_int8_t dpi_set_protocol(dpi_library_state_t *state,
                          dpi_protocol_t protocol);

/**
 * Disable a protocol inspector.
 * @param state       A pointer to the state of the library.
 * @param protocol    The protocol to disable.
 *
 * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
 *         DPI_STATE_UPDATE_FAILURE otherwise.
 */
u_int8_t dpi_delete_protocol(dpi_library_state_t *state,
                             dpi_protocol_t protocol);

/**
 * Enable all the protocol inspector.
 * @param state      A pointer to the state of the library.
 *
 * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
 *         DPI_STATE_UPDATE_FAILURE otherwise.
 */
u_int8_t dpi_inspect_all(dpi_library_state_t *state);

/**
 * Disable all the protocol inspector.
 * @param state      A pointer to the state of the library.
 *
 * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
 *         DPI_STATE_UPDATE_FAILURE otherwise.
 */
u_int8_t dpi_inspect_nothing(dpi_library_state_t *state);


/*
 * Try to detect the application protocol.
```

```
707  * @param   state The state of the library.
708  * @param   pkt The pointer to the beginning of IP header.
709  * @param   data_length Length of the packet (from the beginning of the IP
710  *          header, without L2 headers/trailers).
711  * @param   current_time The current time in seconds.
712  * @return  The status of the operation.  It gives additional informations
713  *          about the processing of the request. If lesser than 0, an ↙
       ↳ error
714  *          occurred. dpi_get_error_msg() can be used to get a textual
715  *          representation of the error. If greater or equal than 0 then
716  *          it should not be interpreted as an error but simply gives
717  *          additional informations (e.g. if the packet was IP fragmented,
718  *          if it was out of order in the TCP stream, if is a segment ↙
       ↳ of a
719  *          larger application request, etc..). dpi_get_status_msg() can
720  *          be used to get a textual representation of the status. Status
721  *          and error codes are defined above in this header file. If an
722  *          error occurred, the other returned fields are not meaningful.
723  *
724  *          The application protocol identifier plus the transport
725  *          protocol identifier. The application protocol identifier is
726  *          relative to the specific transport protocol.
727  *
728  *          The flow specific user data (possibly manipulated by the
729  *          user callbacks).
730  */
731 dpi_identification_result_t dpi_stateful_identify_application_protocol(
732                 dpi_library_state_t* state, const unsigned char* pkt,
733                 u_int32_t length, u_int32_t current_time);
734
735
736
737 /*
738  * Extract from the packet the informations about source and destination
739  * addresses, source and destination ports, L4 protocol and the offset
740  * where the application data starts.
741  * @param   state The state of the library.
742  * @param   pkt The pointer to the beginning of IP header.
743  * @param   data_length Length of the packet (from the beginning of the
744  *          IP header, without L2 headers/trailers).
745  * @param   pkt_infos The pointer to the packet infos. It will be filled
746  *          by the library.
747  * @param   current_time The current time in seconds. It must be
748  *          non-decreasing between two consecutive calls.
749  * @return  The status of the operation. It gives additional informations
750  *          about the processing of the request. If lesser than 0, an
751  *          error occurred. dpi_get_error_msg() can be used to get a
752  *          textual representation of the error. If greater or equal
753  *          than 0 then it should not be interpreted as an error but
```

```
754  *          simply gives additional informations (e.g. if the packet was
755  *          IP fragmented, if it was out of order in the TCP stream, if is
756  *          a segment of a larger application request, etc..).
757  *          dpi_get_status_msg() can be used to get a textual
758  *          representation of the status. Status and error codes are
759  *          defined above in this header file.
760  *
761  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
762  *          fragment. In this case, if IP fragmentation support is ↙
     ↳ enabled,
763  *          the library copied the content of the datagram, so if the user
764  *          wants, he can release the resources used to store the ↙
     ↳ datagram.
765  *
766  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
767  *          datagram allows the library to reconstruct a fragmented
768  *          datagram. In this case, pkt_infos->pkt will contain a pointer
769  *          to the recomposed datagram. This pointer will be different
770  *          from p_pkt. The user should free() this pointer when it is no
771  *          more needed (e.g. after calling
772  *          dpi_state*_get_app_protocol(..)).
773  */
774 int8_t dpi_parse_L3_L4_headers(dpi_library_state_t *state,
775                                const unsigned char* p_pkt,
776                                u_int32_t p_length,
777                                dpi_pkt_infos_t *pkt_infos,
778                                u_int32_t current_time);
779
780 /*
781  * Try to detect the application protocol. Before calling it, a check on
782  * L4 protocol should be done and the function should be called only if
783  * the packet is TCP or UDP.
784  * @param   state The pointer to the library state.
785  * @param   pkt_infos The pointer to the packet infos.
786  * @return  The status of the operation. It gives additional informations
787  *          about the processing of the request. If lesser than 0, an
788  *          error occurred. dpi_get_error_msg() can be used to get a
789  *          textual representation of the error. If greater or equal
790  *          than 0 then it should not be interpreted as an error but
791  *          simply gives additional informations (e.g. if the packet was
792  *          IP fragmented, if it was out of order in the TCP stream, if is
793  *          a segment of a larger application request, etc..).
794  *          dpi_get_status_msg() can be used to get a textual
795  *          representation of the status. Status and error codes are
796  *          defined above in this header file.
797  *
798  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
799  *          fragment. In this case, if IP fragmentation support is
800  *          enabled, the library copied the content of the datagram, so if
```

```
801  *          the user wants, he can release the resources used to store the
802  *          datagram.
803  *
804  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
805  *          datagram allows the library to reconstruct a fragmented
806  *          datagram. In this case, pkt_infos->pkt will contain a pointer
807  *          to the recomposed datagram. This pointer will be different
808  *          from p_pkt. The user should free() this pointer when it is no
809  *          more needed (e.g. after calling
810  *          dpi_state*_get_app_protocol(..)).
811  */
812 dpi_identification_result_t dpi_stateful_get_app_protocol(
813                 dpi_library_state_t *state, dpi_pkt_infos_t* pkt_infos);
814
815
816 /*
817  * Try to detect the application protocol. Before calling it, a check on
818  * L4 protocol should be done and the function should be called only if
819  * the packet is TCP or UDP. It should be used if the application already
820  * has the concept of 'flow'. In this case the first time that the flow is
821  * passed to the call, it must be initialized with
822  * dpi_init_flow_infos(...).
823  * @param   state The pointer to the library state.
824  * @param   flow The informations about the flow. They must be kept by the
825  *              user.
826  * @param   pkt_infos The pointer to the packet infos.
827  * @return  The status of the operation. It gives additional informations
828  *          about the processing of the request. If lesser than 0, an ↙
     ↳ error
829  *          occurred. dpi_get_error_msg() can be used to get a textual
830  *          representation of the error. If greater or equal than 0 then
831  *          it should not be interpreted as an error but simply gives
832  *          additional informations (e.g. if the packet was IP fragmented,
833  *          if it was out of order in the TCP stream, if is a segment of
834  *          a larger application request, etc..). dpi_get_status_msg()
835  *          can be used to get a textual representation of the status.
836  *          Status and error codes are defined above in this header file.
837  *
838  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
839  *          fragment. In this case, if IP fragmentation support is
840  *          enabled, the library copied the content of the datagram, so if
841  *          the user wants, he can release the resources used to store the
842  *          datagram.
843  *
844  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
845  *          datagram allows the library to reconstruct a fragmented
846  *          datagram. In this case, pkt_infos->pkt will contain a pointer
847  *          to the recomposed datagram. This pointer will be different
848  *          from p_pkt. The user should free() this pointer when it is no
```

```
849  *          more needed (e.g. after calling
850  *          dpi_state*_get_app_protocol(..)).
851  */
852 dpi_identification_result_t dpi_stateless_get_app_protocol(
853                dpi_library_state_t *state,
854                dpi_flow_infos_t *flow,
855                dpi_pkt_infos_t *pkt_infos);
856
857 /**
858  * Initialize the flow informations passed as argument.
859  * @param state       A pointer to the state of the library.
860  * @param flow_infos  The informations that will be initialized by the
861  *                    library.
862  * @param l4prot      The transport protocol identifier.
863  */
864 void dpi_init_flow_infos(
865                dpi_library_state_t* state,
866                dpi_flow_infos_t *flow_infos,
867                u_int8_t l4prot);
868
869 /**
870  * Try to guess the protocol looking only at source/destination ports.
871  * This could be erroneous because sometimes protocols run over ports
872  * which are not their well-known ports.
873  * @param    pkt_infos The pointer to the packet infos.
874  * @return   Returns the possible matching protocol.
875  */
876 dpi_protocol_t dpi_guess_protocol(dpi_pkt_infos_t* pkt_infos);
877
878 /**
879  * Get the string representing the error message associated to the
880  * specified error_code.
881  * @param   error_code The error code.
882  * @return  The error message.
883  */
884 const char* const dpi_get_error_msg(int8_t error_code);
885
886 /**
887  * Get the string representing the status message associated to the
888  * specified status_code.
889  * @param   status_code The status code.
890  * @return  The status message.
891  */
892 const char* const dpi_get_status_msg(int8_t status_code);
893
894
895 /**
896  * Returns a string corresponding to the given protocol.
897  * @param   protocol The protocol identifier.
```

```
898  * @return  A string representation of the given protocol.
899  */
900 const char* const dpi_get_protocol_name(dpi_protocol_t protocol);
901
902
903 /**
904  * Sets the callback that will be called when a flow expires.
905  * (Valid only if stateful API is used).
906  * @param state     A pointer to the state of the library.
907  * @param cleaner   The callback used to clear the user state.
908  *
909  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
910  *         DPI_STATE_UPDATE_FAILURE otherwise.
911  */
912 u_int8_t dpi_set_flow_cleaner_callback(
913                dpi_library_state_t* state,
914                dpi_flow_cleaner_callback* cleaner);
915
916 /**
917  * Activate HTTP callbacks. When a protocol is identified the default
918  * behavior is to not inspect the packets belonging to that flow anymore
919  * and keep simply returning the same protocol identifier.
920  *
921  * If a callback is enabled for a certain protocol, then we keep
922  * inspecting all the new flows with that protocol in order to invoke
923  * the callbacks specified by the user on the various parts of the
924  * message. Moreover, if the application protocol uses TCP, then we have
925  * the additional cost of TCP reordering for all the segments. Is highly
926  * recommended to enable TCP reordering if it is not already enabled
927  * (remember that is enabled by default). Otherwise the informations
928  * extracted could be erroneous/incomplete.
929  *
930  * The pointers to the data passed to the callbacks are valid only for the
931  * duration of the callback.
932  *
933  * @param state       A pointer to the state of the library.
934  * @param callbacks   A pointer to HTTP callbacks.
935  * @param user_data   A pointer to global user HTTP data. This pointer
936  *                    will be passed to any HTTP callback when it is
937  *                    invoked.
938  *
939  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
940  *         DPI_STATE_UPDATE_FAILURE otherwise.
941  *
942  **/
943 u_int8_t dpi_http_activate_callbacks(
944                dpi_library_state_t* state,
945                dpi_http_callbacks_t* callbacks,
946                void* user_data);
```

```
947
948 /**
949  * Disable the HTTP callbacks. user_data is not freed/modified.
950  * @param state       A pointer to the state of the library.
951  *
952  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
953  *         DPI_STATE_UPDATE_FAILURE otherwise.
954  */
955 u_int8_t dpi_http_disable_callbacks(dpi_library_state_t* state);
956
957
958
959 /****************************************/
960 /** Only to be used directly by mpdpi. **/
961 /****************************************/
962 dpi_library_state_t* dpi_init_stateful_num_partitions(
963                u_int32_t size_v4, u_int32_t size_v6,
964                u_int32_t max_active_v4_flows,
965                u_int32_t max_active_v6_flows,
966                u_int16_t num_table_partitions);
967 int8_t mp_dpi_extract_packet_infos(
968                dpi_library_state_t *state,
969                const unsigned char* p_pkt,
970                u_int32_t p_length, dpi_pkt_infos_t *pkt_infos,
971                u_int32_t current_time, int tid);
972
973 #ifdef __cplusplus
974 }
975 #endif
976
977 #endif
```

```
1  /*
2   * api.c
3   *
4   *  Created on: 19/09/2012
5   *  Author: Daniele De Sensi (d.desensi@gmail.com)
6   */
7
8  #include "api.h"
9  #include "flow_table.h"
10 #include "protocol_ids.h"
11 #include "hash_functions.h"
12 #include "utils.h"
13 #include "./inspectors/inspectors.h"
14 #include "tcp_stream_management.h"
15 #include "ipv4_reassembly.h"
16 #include "ipv6_reassembly.h"
17 #include <time.h>
```

```
18 #include <stdlib.h>
19 #include <strings.h>
20 #include <string.h>
21 #include <stdio.h>
22 #include <netinet/ip.h>
23 #include <netinet/ip6.h>
24 #include <netinet/udp.h>
25 #include <netinet/tcp.h>
26 #include <assert.h>
27
28
29 #include <arpa/inet.h>
30
31
32 #define debug_print(fmt, ...) \
33         do { if (DPI_DEBUG) fprintf(stderr, fmt, __VA_ARGS__); } while (0)
34
35 /**
36  * Designated initializers has been introduced with C99.
37  * gcc allows them also in c89.
38  **/
39 #if (defined(__STDC_VERSION__) && __STDC_VERSION__ >= 199901L || ↙
       ↳ defined(__GNUC__))
40 static const dpi_l7_prot_id const
41     dpi_well_known_ports_association_tcp[DPI_MAX_UINT_16+1]=
42         {[0 ... DPI_MAX_UINT_16]=DPI_PROTOCOL_UNKNOWN
43         ,[port_http]=DPI_PROTOCOL_TCP_HTTP
44         ,[port_bgp]=DPI_PROTOCOL_TCP_BGP
45         ,[port_smtp_1]=DPI_PROTOCOL_TCP_SMTP
46         ,[port_smtp_2]=DPI_PROTOCOL_TCP_SMTP
47         ,[port_pop3]=DPI_PROTOCOL_TCP_POP3};
48
49 static const dpi_l7_prot_id const
50     dpi_well_known_ports_association_udp[DPI_MAX_UINT_16+1]=
51         {[0 ... DPI_MAX_UINT_16]=DPI_PROTOCOL_UNKNOWN
52         ,[port_dns]=DPI_PROTOCOL_UDP_DNS
53         ,[port_mdns]=DPI_PROTOCOL_UDP_MDNS
54         ,[port_dhcp_1]=DPI_PROTOCOL_UDP_DHCP
55         ,[port_dhcp_2]=DPI_PROTOCOL_UDP_DHCP
56         ,[port_dhcpv6_1]=DPI_PROTOCOL_UDP_DHCPv6
57         ,[port_dhcpv6_2]=DPI_PROTOCOL_UDP_DHCPv6
58         ,[port_ntp]=DPI_PROTOCOL_UDP_NTP};
59
60
61 static const dpi_inspector_callback const
62     udp_inspectors[DPI_NUM_UDP_PROTOCOLS]=
63         {[DPI_PROTOCOL_UDP_DHCP]=check_dhcp
64         ,[DPI_PROTOCOL_UDP_DHCPv6]=check_dhcpv6
65         ,[DPI_PROTOCOL_UDP_DNS]=check_dns
```

```
66          ,[DPI_PROTOCOL_UDP_MDNS]=check_mdns
67          ,[DPI_PROTOCOL_UDP_NTP]=check_ntp};
68
69  static const dpi_inspector_callback const
70      tcp_inspectors[DPI_NUM_TCP_PROTOCOLS]=
71          {[DPI_PROTOCOL_TCP_BGP]=check_bgp
72          ,[DPI_PROTOCOL_TCP_HTTP]=check_http
73          ,[DPI_PROTOCOL_TCP_SMTP]=check_smtp
74          ,[DPI_PROTOCOL_TCP_POP3]=check_pop3};
75
76  static const dpi_inspector_callback const
77      udp_callbacks_manager[DPI_NUM_UDP_PROTOCOLS];
78
79  static const dpi_inspector_callback const
80      tcp_callbacks_manager[DPI_NUM_TCP_PROTOCOLS]=
81          {[DPI_PROTOCOL_TCP_HTTP]=invoke_callbacks_http
82          ,
83          };
84
85  #else
86  static dpi_l7_prot_id
87         dpi_well_known_ports_association_tcp[DPI_MAX_UINT_16+1];
88  static dpi_l7_prot_id
89         dpi_well_known_ports_association_udp[DPI_MAX_UINT_16+1];
90
91  static dpi_inspector_callback udp_inspectors[DPI_NUM_UDP_PROTOCOLS];
92  static dpi_inspector_callback tcp_inspectors[DPI_NUM_TCP_PROTOCOLS];
93
94  static dpi_inspector_callback
95         udp_callbacks_manager[DPI_NUM_UDP_PROTOCOLS];
96
97  static dpi_inspector_callback
98         tcp_callbacks_manager[DPI_NUM_TCP_PROTOCOLS];
99  #endif
100
101 /**
102  * Initializes the state of the library. If not specified otherwise after
103  * the initialization, the library will consider all the protocols active.
104  * Using this API, the hash table is divided in num_table_partitions
105  * partitions. These partitions can be accessed concurrently in a thread
106  * safe way from different threads if and only if each thread access only
107  * to its partition.
108  * @param size_v4 Size of the array of pointers used to build the database
109  *        for v4 flows.
110  * @param size_v6 Size of the array of pointers used to build the database
111  *        for v6 flows.
112  * @param max_active_v4_flows The maximum number of IPv4 flows which can
113  *        be active at any time. After reaching this threshold, new flows
114  *        will not be created.
```

```
115  * @param max_active_v6_flows The maximum number of IPv6 flows which can
116  *        be active at any time. After reaching this threshold, new flows
117  *        will not be created.
118  * @param num_table_partitions The number of partitions of the hash table.
119  * @return A pointer to the state of the library otherwise.
120  */
121 #if DPI_ENABLE_MP_API == 0
122 static
123 #endif
124 dpi_library_state_t* dpi_init_stateful_num_partitions(
125                u_int32_t size_v4, u_int32_t size_v6,
126                u_int32_t max_active_v4_flows,
127                u_int32_t max_active_v6_flows,
128                u_int16_t num_table_partitions){
129
130     dpi_library_state_t* state =
131              (dpi_library_state_t*) malloc(sizeof(dpi_library_state_t));
132
133     assert(state);
134
135     bzero(state, sizeof(dpi_library_state_t));
136
137 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
138     state->db4=dpi_flow_table_create_v4(size_v4, max_active_v4_flows,
139                           num_table_partitions,
140                           DPI_FLOW_TABLE_MEMORY_POOL_DEFAULT_SIZE_v4);
141     state->db6=dpi_flow_table_create_v6(size_v6, max_active_v6_flows,
142                           num_table_partitions,
143                           DPI_FLOW_TABLE_MEMORY_POOL_DEFAULT_SIZE_v6);
144 #else
145     state->db4=dpi_flow_table_create_v4(size_v4, max_active_v4_flows,
146                                         num_table_partitions);
147     state->db6=dpi_flow_table_create_v6(size_v6, max_active_v6_flows,
148                                         num_table_partitions);
149 #endif
150     dpi_set_max_trials(state, DPI_DEFAULT_MAX_TRIALS_PER_FLOW);
151     dpi_inspect_all(state);
152
153     dpi_ipv4_fragmentation_enable(state,
154                                 DPI_IPv4_FRAGMENTATION_DEFAULT_TABLE_SIZE);
155     dpi_ipv6_fragmentation_enable(state,
156                                 DPI_IPv6_FRAGMENTATION_DEFAULT_TABLE_SIZE);
157
158     dpi_tcp_reordering_enable(state);
159
160 #if !defined(__GNUC__) && (!defined(__STDC_VERSION__) || ↙
        ↳ __STDC_VERSION__ < 199901L)
161     memset(dpi_well_known_ports_association_tcp, DPI_PROTOCOL_UNKNOWN,
162            DPI_MAX_UINT_16+1);
```

```
163     memset(dpi_well_known_ports_association_udp, DPI_PROTOCOL_UNKNOWN,
164            DPI_MAX_UINT_16+1);
165     dpi_well_known_ports_association_tcp[port_http]=
166             DPI_PROTOCOL_TCP_HTTP;
167     dpi_well_known_ports_association_tcp[port_bgp]=
168             DPI_PROTOCOL_TCP_BGP;
169     dpi_well_known_ports_association_tcp[port_smtp_1]=
170             DPI_PROTOCOL_TCP_SMTP;
171     dpi_well_known_ports_association_tcp[port_smtp_2]=
172             DPI_PROTOCOL_TCP_SMTP;
173     dpi_well_known_ports_association_tcp[port_pop3]=
174             DPI_PROTOCOL_TCP_POP3;
175
176     dpi_well_known_ports_association_udp[port_dns]=
177             DPI_PROTOCOL_UDP_DNS;
178     dpi_well_known_ports_association_udp[port_mdns]=D
179             PI_PROTOCOL_UDP_MDNS;
180     dpi_well_known_ports_association_udp[port_dhcp_1]=
181             DPI_PROTOCOL_UDP_DHCP;
182     dpi_well_known_ports_association_udp[port_dhcp_2]=
183             DPI_PROTOCOL_UDP_DHCP;
184     dpi_well_known_ports_association_udp[port_dhcpv6_1]=
185             DPI_PROTOCOL_UDP_DHCPv6;
186     dpi_well_known_ports_association_udp[port_dhcpv6_2]=
187             DPI_PROTOCOL_UDP_DHCPv6;
188     dpi_well_known_ports_association_udp[port_ntp]=
189             DPI_PROTOCOL_UDP_NTP;
190
191
192     udp_inspectors[DPI_PROTOCOL_UDP_DHCP]=check_dhcp;
193     udp_inspectors[DPI_PROTOCOL_UDP_DHCPv6]=check_dhcpv;
194     udp_inspectors[DPI_PROTOCOL_UDP_DNS]=check_dns;
195     udp_inspectors[DPI_PROTOCOL_UDP_MDNS]=check_mdns;
196     udp_inspectors[DPI_PROTOCOL_UDP_NTP]=check_ntp;
197
198     tcp_inspectors[DPI_PROTOCOL_TCP_BGP]=check_bgp;
199     tcp_inspectors[DPI_PROTOCOL_TCP_HTTP]=check_http;
200     tcp_inspectors[DPI_PROTOCOL_TCP_SMTP]=check_smtp;
201     tcp_inspectors[DPI_PROTOCOL_TCP_POP3]=check_pop3;
202
203     tcp_callbacks_manager[DPI_PROTOCOL_TCP_HTTP]=invoke_callbacks_http;
204 #endif
205     return state;
206 }
207
208 dpi_library_state_t* dpi_init_stateful(
209                u_int32_t size_v4, u_int32_t size_v6,
210                u_int32_t max_active_v4_flows,
211                u_int32_t max_active_v6_flows){
```

```
212
213     return dpi_init_stateful_num_partitions(size_v4, size_v6,
214                                             max_active_v4_flows,
215                                             max_active_v6_flows, 1);
216 }
217
218
219
220 /**
221  * Initializes the state of the library. If not specified otherwise after
222  * the initialization, the library will consider all the protocols active.
223  * @return A pointer to the state of the library otherwise.
224  */
225 dpi_library_state_t* dpi_init_stateless(void){
226     return dpi_init_stateful(0,0,0,0);
227 }
228
229 /**
230  * Sets the maximum number of times that the library tries to guess the
231  * protocol. During the flow protocol identification, after this number
232  * of trials, in the case in which it cannot decide between two or more
233  * protocols, one of them will be chosen, otherwise DPI_PROTOCOL_UNKNOWN
234  * will be returned.
235  * @param state A pointer to the state of the library.
236  * @param max_trials Maximum number of trials. Zero will be consider as
237  *                   infinity.
238  *
239  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
240  *         otherwise.
241  */
242 u_int8_t dpi_set_max_trials(dpi_library_state_t *state, u_int16_t ↙
        ↳ max_trials){
243     state->max_trials=max_trials;
244     return DPI_STATE_UPDATE_SUCCESS;
245 }
246
247
248 /**
249  * Enable IPv4 defragmentation.
250  * @param state        A pointer to the library state.
251  * @param table_size   The size of the table to be used to store IPv4
252  *                     fragments informations.
253  *
254  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
255  *         otherwise.
256  */
257 u_int8_t dpi_ipv4_fragmentation_enable(dpi_library_state_t *state,
258                                        u_int16_t table_size){
259     if(likely(state)){
```

```
260         ↙
        ↳ state->ipv4_frag_state=dpi_reordering_enable_ipv4_fragmentation(table_size);
261         if(state->ipv4_frag_state)
262             return DPI_STATE_UPDATE_SUCCESS;
263         else
264             return DPI_STATE_UPDATE_FAILURE;
265     }else
266         return DPI_STATE_UPDATE_FAILURE;
267 }
268
269 /**
270  * Enable IPv6 defragmentation.
271  * @param state        A pointer to the library state.
272  * @param table_size   The size of the table to be used to store IPv6
273  *                     fragments informations.
274  *
275  * @return DPI_STATE_UPDATE_SUCCESS if succeeded, DPI_STATE_UPDATE_FAILURE
276  *         otherwise.
277  */
278 u_int8_t dpi_ipv6_fragmentation_enable(dpi_library_state_t *state,
279                                        u_int16_t table_size){
280     if(likely(state)){
281         state->ipv6_frag_state=dpi_reordering_enable_ipv6_fragmentation(
282                                 table_size);
283         if(state->ipv6_frag_state)
284             return DPI_STATE_UPDATE_SUCCESS;
285         else
286             return DPI_STATE_UPDATE_FAILURE;
287     }else{
288         return DPI_STATE_UPDATE_FAILURE;
289     }
290 }
291
292 /**
293  * Sets the amount of memory that a single host can use for IPv4
294  * defragmentation.
295  * @param state                   A pointer to the library state.
296  * @param per_host_memory_limit   The maximum amount of memory that
297  *                                any IPv4 host can use.
298  *
299  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
300  *         DPI_STATE_UPDATE_FAILURE otherwise.
301  */
302 u_int8_t dpi_ipv4_fragmentation_set_per_host_memory_limit(
303                 dpi_library_state_t *state,
304                 u_int32_t per_host_memory_limit){
305     if(likely(state && state->ipv4_frag_state)){
306         dpi_reordering_ipv4_fragmentation_set_per_host_memory_limit(
307                         state->ipv4_frag_state, per_host_memory_limit);
```

```
308         return DPI_STATE_UPDATE_SUCCESS;
309     }else{
310         return DPI_STATE_UPDATE_FAILURE;
311     }
312 }
313
314 /**
315  * Sets the amount of memory that a single host can use for IPv6
316  * defragmentation.
317  * @param state                   A pointer to the library state.
318  * @param per_host_memory_limit   The maximum amount of memory that
319  *                                 any IPv6 host can use.
320  *
321  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
322  *         DPI_STATE_UPDATE_FAILURE otherwise.
323  */
324 u_int8_t dpi_ipv6_fragmentation_set_per_host_memory_limit(
325                 dpi_library_state_t *state,
326                 u_int32_t per_host_memory_limit){
327     if(likely(state && state->ipv6_frag_state)){
328         dpi_reordering_ipv6_fragmentation_set_per_host_memory_limit(
329                       state->ipv6_frag_state, per_host_memory_limit);
330         return DPI_STATE_UPDATE_SUCCESS;
331     }else{
332         return DPI_STATE_UPDATE_FAILURE;
333     }
334 }
335
336 /**
337  * Sets the total amount of memory that can be used for IPv4
338  * defragmentation.
339  * If fragmentation is disabled and then enabled, this information must be
340  * passed again.
341  * Otherwise default value will be used.
342  * @param state               A pointer to the state of the library
343  * @param totel_memory_limit  The maximum amount of memory that can be ↙
     ↳ used
344  *                            for IPv4 defragmentation.
345  *
346  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
347  *         DPI_STATE_UPDATE_FAILURE otherwise.
348  */
349 u_int8_t dpi_ipv4_fragmentation_set_total_memory_limit(
350                 dpi_library_state_t *state,
351                 u_int32_t total_memory_limit){
352     if(likely(state && state->ipv4_frag_state)){
353         dpi_reordering_ipv4_fragmentation_set_total_memory_limit(
354                       state->ipv4_frag_state, total_memory_limit);
355         return DPI_STATE_UPDATE_SUCCESS;
```

```
356     }else{
357         return DPI_STATE_UPDATE_FAILURE;
358     }
359 }
360
361 /**
362 /**
363  * Sets the total amount of memory that can be used for IPv6
364  * defragmentation. If fragmentation is disabled and then enabled, this
365  * information must be passed again. Otherwise default value will be used.
366  * @param state               A pointer to the state of the library
367  * @param total_memory_limit  The maximum amount of memory that can be
368  *                            used for IPv6 defragmentation.
369  *
370  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
371  *         DPI_STATE_UPDATE_FAILURE otherwise.
372  */
373 u_int8_t dpi_ipv6_fragmentation_set_total_memory_limit(
374                 dpi_library_state_t *state,
375                 u_int32_t total_memory_limit){
376     if(likely(state && state->ipv6_frag_state)){
377         dpi_reordering_ipv6_fragmentation_set_total_memory_limit(
378                       state->ipv6_frag_state, total_memory_limit);
379         return DPI_STATE_UPDATE_SUCCESS;
380     }else{
381         return DPI_STATE_UPDATE_FAILURE;
382     }
383 }
384
385 /**
386  * Sets the maximum time (in seconds) that can be spent to reassembly an
387  * IPv4 fragmented datagram. Is the maximum time gap between the first and
388  * last fragments of the datagram.
389  * @param state            A pointer to the state of the library.
390  * @param timeout_seconds  The reassembly timeout.
391  *
392  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
393  *         DPI_STATE_UPDATE_FAILURE otherwise.
394  */
395 u_int8_t dpi_ipv4_fragmentation_set_reassembly_timeout(
396                 dpi_library_state_t *state,
397                 u_int8_t timeout_seconds){
398     if(likely(state && state->ipv4_frag_state)){
399         dpi_reordering_ipv4_fragmentation_set_reassembly_timeout(
400                       state->ipv4_frag_state, timeout_seconds);
401         return DPI_STATE_UPDATE_SUCCESS;
402     }else{
403         return DPI_STATE_UPDATE_FAILURE;
404     }
```

```
405 }
406
407 /**
408  * Sets the maximum time (in seconds) that can be spent to reassembly an
409  * IPv6 fragmented datagram. Is the maximum time gap between the first and
410  * last fragments of the datagram.
411  * @param state            A pointer to the state of the library.
412  * @param timeout_seconds  The reassembly timeout.
413  *
414  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
415  *         DPI_STATE_UPDATE_FAILURE otherwise.
416  */
417 u_int8_t dpi_ipv6_fragmentation_set_reassembly_timeout(
418                 dpi_library_state_t *state,
419                 u_int8_t timeout_seconds){
420     if(likely(state && state->ipv6_frag_state)){
421         dpi_reordering_ipv6_fragmentation_set_reassembly_timeout(
422                       state->ipv6_frag_state, timeout_seconds);
423         return DPI_STATE_UPDATE_SUCCESS;
424     }else{
425         return DPI_STATE_UPDATE_FAILURE;
426     }
427 }
428
429 /**
430  * Disable IPv4 defragmentation.
431  * @param state A pointer to the state of the library.
432  *
433  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
434  *         DPI_STATE_UPDATE_FAILURE otherwise.
435  */
436 u_int8_t dpi_ipv4_fragmentation_disable(dpi_library_state_t *state){
437     if(likely(state && state->ipv4_frag_state)){
438         dpi_reordering_disable_ipv4_fragmentation(state->ipv4_frag_state);
439         state->ipv4_frag_state=NULL;
440         return DPI_STATE_UPDATE_SUCCESS;
441     }else{
442         return DPI_STATE_UPDATE_FAILURE;
443     }
444 }
445
446 /**
447  * Disable IPv6 defragmentation.
448  * @param state A pointer to the state of the library.
449  *
450  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
451  *         DPI_STATE_UPDATE_FAILURE otherwise.
452  */
453 u_int8_t dpi_ipv6_fragmentation_disable(dpi_library_state_t *state){
```

```
454     if(likely(state && state->ipv6_frag_state)){
455         dpi_reordering_disable_ipv6_fragmentation(state->ipv6_frag_state);
456         state->ipv6_frag_state=NULL;
457         return DPI_STATE_UPDATE_SUCCESS;
458     }else{
459         return DPI_STATE_UPDATE_FAILURE;
460     }
461 }
462
463
464 /**
465  * If enabled, the library will reorder out of order TCP packets
466  * (enabled by default).
467  * @param state  A pointer to the state of the library.
468  *
469  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
470  *         DPI_STATE_UPDATE_FAILURE otherwise.
471  */
472 u_int8_t dpi_tcp_reordering_enable(dpi_library_state_t* state){
473     if(likely(state)){
474         state->tcp_reordering_enabled=1;
475         return DPI_STATE_UPDATE_SUCCESS;
476     }else{
477         return DPI_STATE_UPDATE_FAILURE;
478     }
479 }
480
481 /**
482  * If it is called, the library will not reorder out of order TCP packets.
483  * Out-of-order segments will be delivered to the inspector as they
484  * arrive. This means that the inspector may not be able to identify the
485  * application protocol. Moreover, if there are callbacks saved for TCP
486  * based protocols, if TCP reordering is disabled, the extracted
487  * informations could be erroneous or incomplete.
488  * @param state A pointer to the state of the library.
489  *
490  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
491  *         DPI_STATE_UPDATE_FAILURE otherwise.
492  */
493 u_int8_t dpi_tcp_reordering_disable(dpi_library_state_t* state){
494     if(likely(state)){
495         state->tcp_reordering_enabled=0;
496         return DPI_STATE_UPDATE_SUCCESS;
497     }else{
498         return DPI_STATE_UPDATE_FAILURE;
499     }
500 }
501
502 /**
```

```
503  * Enable a protocol inspector.
504  * @param state         A pointer to the state of the library.
505  * @param protocol      The protocol to enable.
506  *
507  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
508  *         DPI_STATE_UPDATE_FAILURE otherwise.
509  */
510 u_int8_t dpi_set_protocol(dpi_library_state_t *state,
511                           dpi_protocol_t protocol){
512     if(protocol.l4prot==IPPROTO_UDP){
513         BITSET(state->udp_protocols_to_inspect, protocol.l7prot);
514         ++state->udp_active_protocols;
515         return DPI_STATE_UPDATE_SUCCESS;
516     }else if(protocol.l4prot==IPPROTO_TCP){
517         BITSET(state->tcp_protocols_to_inspect, protocol.l7prot);
518         ++state->tcp_active_protocols;
519         return DPI_STATE_UPDATE_SUCCESS;
520     }else{
521         return DPI_STATE_UPDATE_FAILURE;
522     }
523 }
524
525 /**
526  * Disable a protocol inspector.
527  * @param state       A pointer to the state of the library.
528  * @param protocol    The protocol to disable.
529  *
530  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
531  *         DPI_STATE_UPDATE_FAILURE otherwise.
532  */
533 u_int8_t dpi_delete_protocol(dpi_library_state_t *state,
534                              dpi_protocol_t protocol){
535     char *protocol_mask;
536     char *inspector_mask;
537     if(protocol.l4prot==IPPROTO_UDP){
538         protocol_mask=state->udp_protocols_to_inspect;
539         inspector_mask=state->udp_active_callbacks;
540         --state->udp_active_protocols;
541     }else if(protocol.l4prot==IPPROTO_TCP){
542         protocol_mask=state->tcp_protocols_to_inspect;
543         inspector_mask=state->tcp_active_callbacks;
544         --state->tcp_active_protocols;
545     }else{
546         return DPI_STATE_UPDATE_FAILURE;
547     }
548
549     BITCLEAR(protocol_mask, protocol.l7prot);
550     BITCLEAR(inspector_mask, protocol.l7prot);
551     return DPI_STATE_UPDATE_SUCCESS;
```

```
552 }
553
554 /**
555  * Enable all the protocol inspector.
556  * @param state      A pointer to the state of the library.
557  *
558  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
559  *         DPI_STATE_UPDATE_FAILURE otherwise.
560  */
561 u_int8_t dpi_inspect_all(dpi_library_state_t *state){
562     unsigned char nonzero = ~0;
563     memset(state->udp_protocols_to_inspect, nonzero,
564            BITNSLOTS(DPI_NUM_UDP_PROTOCOLS));
565     memset(state->tcp_protocols_to_inspect, nonzero,
566            BITNSLOTS(DPI_NUM_TCP_PROTOCOLS));
567
568     state->udp_active_protocols=DPI_NUM_UDP_PROTOCOLS;
569     state->tcp_active_protocols=DPI_NUM_TCP_PROTOCOLS;
570     return DPI_STATE_UPDATE_SUCCESS;
571 }
572
573
574 /**
575  * Disable all the protocol inspector.
576  * @param state      A pointer to the state of the library.
577  *
578  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
579  *         DPI_STATE_UPDATE_FAILURE otherwise.
580  */
581 u_int8_t dpi_inspect_nothing(dpi_library_state_t *state){
582     bzero(state->udp_protocols_to_inspect, ↙
        ↳ BITNSLOTS(DPI_NUM_UDP_PROTOCOLS));
583     bzero(state->tcp_protocols_to_inspect, ↙
        ↳ BITNSLOTS(DPI_NUM_TCP_PROTOCOLS));
584
585     state->udp_active_protocols=0;
586     state->tcp_active_protocols=0;
587
588     bzero(state->tcp_active_callbacks, DPI_NUM_TCP_PROTOCOLS);
589     bzero(state->udp_active_callbacks, DPI_NUM_UDP_PROTOCOLS);
590     return DPI_STATE_UPDATE_SUCCESS;
591 }
592
593 /**
594  * Terminates the library.
595  * @param state A pointer to the state of the library.
596  */
597 void dpi_terminate(dpi_library_state_t *state){
598     if(likely(state)){
```

```
599         dpi_http_disable_callbacks(state);
600         dpi_ipv4_fragmentation_disable(state);
601         dpi_ipv6_fragmentation_disable(state);
602         dpi_tcp_reordering_disable(state);
603
604         dpi_flow_table_delete_v4(state->db4,
605                                  state->flow_cleaner_callback);
606         dpi_flow_table_delete_v6(state->db6,
607                                  state->flow_cleaner_callback);
608
609         free(state);
610     }
611 }
612
613
614 /*
615  * Try to detect the application protocol.
616  * @param   state The state of the library.
617  * @param   pkt The pointer to the beginning of IP header.
618  * @param   data_length Length of the packet (from the beginning of the IP
619  *          header, without L2 headers/trailers).
620  * @param   current_time The current time in seconds.
621  * @return  The status of the operation.  It gives additional informations
622  *          about the processing of the request. If lesser than 0, an ↙
     ↳ error
623  *          occurred. dpi_get_error_msg() can be used to get a textual
624  *          representation of the error. If greater or equal than 0 then
625  *          it should not be interpreted as an error but simply gives
626  *          additional informations (e.g. if the packet was IP fragmented,
627  *          if it was out of order in the TCP stream, if is a segment ↙
     ↳ of a
628  *          larger application request, etc..). dpi_get_status_msg() can
629  *          be used to get a textual representation of the status. Status
630  *          and error codes are defined above in this header file. If an
631  *          error occurred, the other returned fields are not meaningful.
632  *
633  *          The application protocol identifier plus the transport
634  *          protocol identifier. The application protocol identifier is
635  *          relative to the specific transport protocol.
636  *
637  *          The flow specific user data (possibly manipulated by the
638  *          user callbacks).
639  */
640 dpi_identification_result_t dpi_stateful_identify_application_protocol(
641                 dpi_library_state_t* state,
642                 const unsigned char* pkt,
643                 u_int32_t length,
644                 u_int32_t current_time){
645
```

```
646     dpi_identification_result_t r;
647     r.status=DPI_STATUS_OK;
648     dpi_pkt_infos_t infos;
649     u_int8_t l3_status;
650
651     r.status=dpi_parse_L3_L4_headers(state, pkt, length, &infos,
652                                      current_time);
653
654     if(unlikely(r.status==DPI_STATUS_IP_FRAGMENT || r.status<0)){
655         return r;
656     }
657
658     if(infos.l4prot!=IPPROTO_TCP && infos.l4prot!=IPPROTO_UDP){
659         r.status=DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
660         return r;
661     }
662
663     l3_status=r.status;
664     r.status=DPI_STATUS_OK;
665     /**
666      * We return the status of dpi_stateful_get_app_protocol call,
667      * without giving informations on status returned
668      * by dpi_parse_L3_L4_headers. Basically we return the status which
669      * provides more informations.
670      */
671     r=dpi_stateful_get_app_protocol(state, &infos);
672
673     if(l3_status==DPI_STATUS_IP_LAST_FRAGMENT){
674         free((unsigned char*) infos.pkt);
675     }
676
677     return r;
678 }
679
680
681 /*
682  * Extract from the packet the informations about source and destination
683  * addresses, source and destination ports, L4 protocol and the offset
684  * where the application data starts.
685  * @param   state The state of the library.
686  * @param   pkt The pointer to the beginning of IP header.
687  * @param   data_length Length of the packet (from the beginning of the
688  *          IP header, without L2 headers/trailers).
689  * @param   pkt_infos The pointer to the packet infos. It will be filled
690  *          by the library.
691  * @param   current_time The current time in seconds. It must be
692  *          non-decreasing between two consecutive calls.
693  * @param   tid The thread identifier.
694  * @return  The status of the operation. It gives additional informations
```

```
695  *          about the processing of the request. If lesser than 0, an
696  *          error occurred. dpi_get_error_msg() can be used to get a
697  *          textual representation of the error. If greater or equal than
698  *          0 then it should not be interpreted as an error but simply
699  *          gives additional informations (e.g. if the packet was IP
700  *          fragmented, if it was out of order in the TCP stream, if is a
701  *          segment of a larger application request, etc..).
702  *          dpi_get_status_msg() can be used to get a textual
703  *          representation of the status. Status and error codes are
704  *          defined above in this header file.
705  *
706  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
707  *          fragment. In this case, if IP fragmentation support is
708  *          enabled, the library copied the content of the datagram, so if
709  *          the user wants, he can release the resources used to store the
710  *          datagram.
711  *
712  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
713  *          datagram allows the library to reconstruct a fragmented
714  *          datagram. In this case, pkt_infos->pkt will contain a pointer
715  *          to the recomposed datagram. This pointer will be different
716  *          from p_pkt. The user should free() this pointer when it is no
717  *          more needed (e.g. after calling
718  *          dpi_state*_get_app_protocol(..)).
719  */
720 #if DPI_ENABLE_MP_API == 0
721 static
722 #if DPI_USE_INLINING == 1
723 inline
724 #endif
725 #endif
726 int8_t mp_dpi_extract_packet_infos(
727                dpi_library_state_t *state, const unsigned char* p_pkt,
728                u_int32_t p_length, dpi_pkt_infos_t *pkt_infos,
729                u_int32_t current_time, int tid){
730     if(unlikely(p_length==0)) return DPI_STATUS_OK;
731     u_int8_t version;
732 #if __BYTE_ORDER == __LITTLE_ENDIAN
733     version=(p_pkt[0]>>4)&0x0F;
734 #elif __BYTE_ORDER == __BIG_ENDIAN
735     version=(p_pkt[0]<<4)&0x0F;
736 #else
737 # error "Please fix <bits/endian.h>"
738 #endif
739
740     unsigned char* pkt=(unsigned char*) p_pkt;
741     u_int32_t length=p_length;
742     u_int16_t offset;
743     u_int8_t more_fragments;
```

```
744
745     pkt_infos->l4prot=0;
746     pkt_infos->srcport=0;
747     pkt_infos->dstport=0;
748
749     /** Offset starting from the beginning of p_pkt. **/
750     u_int32_t application_offset;
751     /**
752      * Offset starting from the last identified IPv4 or IPv6 header
753      * (used to support tunneling).
754      **/
755     u_int32_t relative_offset;
756     u_int32_t tmp;
757     u_int8_t next_header,stop=0;
758
759     int8_t to_return=DPI_STATUS_OK;
760
761     struct ip6_hdr* ip6=NULL;
762     struct iphdr* ip4=NULL;
763
764     if(version==DPI_IP_VERSION_4){  /** IPv4 **/
765         ip4=(struct iphdr*) (p_pkt);
766         u_int16_t tot_len=ntohs(ip4->tot_len);
767
768 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
769         if(unlikely(length<(sizeof(struct iphdr)) || tot_len>length ||
770                     tot_len<=((ip4->ihl)*4))){
771             return DPI_ERROR_L3_TRUNCATED_PACKET;
772         }
773 #endif
774         /**
775          * At this point we are sure that tot_len<=length, so we set
776          * length=tot_len. In some cases indeed there may be an L2 padding
777          * at the end of the packet, so capture length (length) may be
778          * greater than the effective datagram length.
779          */
780         length=tot_len;
781
782         offset=ntohs(ip4->frag_off);
783         if(unlikely((offset&DPI_IPv4_FRAGMENTATION_MF))){
784             more_fragments=1;
785         }else
786             more_fragments=0;
787
788         /*
789          * Offset is in 8-byte blocks. Multiplying by 8 correspond to a
790          * right shift by 3 position, but the offset was 13 bit, so it can
791          * still fit in a 16 bit integer.
792          */
```

```
793         offset=(offset & DPI_IPv4_FRAGMENTATION_OFFSET_MASK)*8;
794
795         if(likely((!more_fragments)&&(offset==0))){
796             pkt=(unsigned char*) p_pkt;
797         }else if(state->ipv4_frag_state!=NULL){
798             pkt=dpi_reordering_manage_ipv4_fragment(
799                     state->ipv4_frag_state, p_pkt, current_time,
800                     offset, more_fragments, tid);
801             if(pkt==NULL){
802                 return DPI_STATUS_IP_FRAGMENT;
803             }
804             to_return=DPI_STATUS_IP_LAST_FRAGMENT;
805             ip4=(struct iphdr*) (pkt);
806             length=ntohs(((struct iphdr*) (pkt))->tot_len);
807         }else{
808             return DPI_STATUS_IP_FRAGMENT;
809         }
810
811         pkt_infos->src_addr_t.ipv4_srcaddr=ip4->saddr;
812         pkt_infos->dst_addr_t.ipv4_dstaddr=ip4->daddr;
813
814         application_offset=(ip4->ihl)*4;
815         relative_offset=application_offset;
816
817         next_header=ip4->protocol;
818     }else if(version==DPI_IP_VERSION_6){ /** IPv6 **/
819         ip6=(struct ip6_hdr*) (pkt);
820         u_int16_t tot_len=ntohs(ip6->ip6_ctlun.ip6_un1.ip6_un1_plen)+
821                           sizeof(struct ip6_hdr);
822 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
823         if(unlikely(tot_len>length)){
824             return DPI_ERROR_L3_TRUNCATED_PACKET;
825         }
826 #endif
827
828         /**
829          * At this point we are sure that tot_len<=length, so we set
830          * length=tot_len. In some cases indeed there may be an L2 padding
831          * at the end of the packet, so capture length (length) may be
832          * greater than the effective datagram length.
833          */
834         length=tot_len;
835
836         pkt_infos->src_addr_t.ipv6_srcaddr=ip6->ip6_src;
837         pkt_infos->dst_addr_t.ipv6_dstaddr=ip6->ip6_dst;
838
839         application_offset=sizeof(struct ip6_hdr);
840         relative_offset=application_offset;
841         next_header=ip6->ip6_ctlun.ip6_un1.ip6_un1_nxt;
```

```
842     }else{
843         return DPI_ERROR_WRONG_IPVERSION;
844     }
845
846     while(!stop){
847         switch (next_header) {
848             case IPPROTO_TCP:{ /* TCP */
849                 struct tcphdr* tcp=(struct tcphdr*)
850                                     (pkt+application_offset);
851 #ifdef DPI_ENABLE_L4_TRUNCATION_PROTECTION
852                 if(unlikely(application_offset+
853                             sizeof(struct tcphdr)>length ||
854                             application_offset+tcp->doff*4>length)){
855                     if(unlikely(pkt!=p_pkt)) free(pkt);
856                     return DPI_ERROR_L4_TRUNCATED_PACKET;
857                 }
858 #endif
859                 pkt_infos->srcport=tcp->source;
860                 pkt_infos->dstport=tcp->dest;
861                 pkt_infos->l4offset=application_offset;
862                 application_offset+=(tcp->doff*4);
863                 stop=1;
864                 }
865                 break;
866             case IPPROTO_UDP:{ /* UDP */
867                 struct udphdr* udp=(struct udphdr*)
868                                     (pkt+application_offset);
869 #ifdef DPI_ENABLE_L4_TRUNCATION_PROTECTION
870                 if(unlikely(application_offset+
871                             sizeof(struct udphdr)>length ||
872                                    application_offset+
873                                    ntohs(udp->len)>length)){
874                     if(unlikely(pkt!=p_pkt)) free(pkt);
875                     return DPI_ERROR_L4_TRUNCATED_PACKET;
876                 }
877 #endif
878                 pkt_infos->srcport=udp->source;
879                 pkt_infos->dstport=udp->dest;
880                 pkt_infos->l4offset=application_offset;
881                 application_offset+=8;
882                 stop=1;
883                 }
884                 break;
885             case IPPROTO_HOPOPTS:{ /* Hop by hop options */
886 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
887                 if(unlikely(application_offset+
888                             sizeof(struct ip6_hbh)>length)){
889                     if(unlikely(pkt!=p_pkt)) free(pkt);
890                     return DPI_ERROR_L3_TRUNCATED_PACKET;
```

```
                }
#endif
                if(likely(version==6)){
                    struct ip6_hbh* hbh_hdr=(struct ip6_hbh*)
                                            (pkt+application_offset);
                    tmp=(8+hbh_hdr->ip6h_len*8);
                    application_offset+=tmp;
                    relative_offset+=tmp;
                    next_header=hbh_hdr->ip6h_nxt;
                }else{
                    if(unlikely(pkt!=p_pkt)) free(pkt);
                    return DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
                }
                }
                break;
            case IPPROTO_DSTOPTS:{ /* Destination options */
#ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
                if(unlikely(application_offset+
                            sizeof(struct ip6_dest)>length)){
                    if(unlikely(pkt!=p_pkt)) free(pkt);
                    return DPI_ERROR_L3_TRUNCATED_PACKET;
                }
#endif
                if(likely(version==6)){
                    struct ip6_dest* dst_hdr=(struct ip6_dest*)
                                             (pkt+application_offset);
                    tmp=(8+dst_hdr->ip6d_len*8);
                    application_offset+=tmp;
                    relative_offset+=tmp;
                    next_header=dst_hdr->ip6d_nxt;
                }else{
                    if(unlikely(pkt!=p_pkt)) free(pkt);
                    return DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
                }
                }
                break;
            case IPPROTO_ROUTING:{ /* Routing header */
#ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
                if(unlikely(application_offset+sizeof(struct ↙
    ↳ ip6_rthdr)>length)){
                    if(unlikely(pkt!=p_pkt)) free(pkt);
                    return DPI_ERROR_L3_TRUNCATED_PACKET;
                }
#endif
                if(likely(version==6)){
                    struct ip6_rthdr* rt_hdr=(struct ip6_rthdr*)
                                             (pkt+application_offset);
                    tmp=(8+rt_hdr->ip6r_len*8);
                    application_offset+=tmp;
```

```
939                     relative_offset+=tmp;
940                     next_header=rt_hdr->ip6r_nxt;
941                 }else{
942                     if(unlikely(pkt!=p_pkt)) free(pkt);
943                     return DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
944                 }
945                 }
946                 break;
947             case IPPROTO_FRAGMENT:{ /* Fragment header */
948 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
949                 if(unlikely(application_offset+
950                             sizeof(struct ip6_frag)>length)){
951                     if(unlikely(pkt!=p_pkt)) free(pkt);
952                     return DPI_ERROR_L3_TRUNCATED_PACKET;
953                 }
954 #endif
955                 if(likely(version==6)){
956                     if(state->ipv6_frag_state){
957                         struct ip6_frag* frg_hdr=(struct ip6_frag*)
958                                                 (pkt+application_offset);
959                         u_int16_t offset=((frg_hdr->ip6f_offlg &
960                                            IP6F_OFF_MASK)>>3)*8;
961                         u_int8_t more_fragments=((frg_hdr->ip6f_offlg &
962                                                   IP6F_MORE_FRAG))?1:0;
963                         offset=ntohs(offset);
964                         u_int32_t fragment_size=ntohs(
965                                 ip6->ip6_ctlun.ip6_un1.ip6_un1_plen)+
966                                 sizeof(struct ip6_hdr)-relative_offset-
967                                 sizeof(struct ip6_frag);
968
969                         /**
970                          * If this fragment has been obtained from a
971                          * defragmentation (e.g. tunneling), then delete
972                          * it after that the defragmentation support has
973                          * copied it.
974                          */
975                         unsigned char* to_delete=NULL;
976                         if(pkt!=p_pkt){
977                             to_delete=pkt;
978                         }
979
980                         /*
981                          * For our purposes, from the unfragmentable part
982                          * we need only the IPv6 header, any other
983                          * optional header can be discarded, for this
984                          * reason we copy only the IPv6 header bytes.
985                          */
986                         pkt=dpi_reordering_manage_ipv6_fragment(
987                                 state->ipv6_frag_state,
```

```
988                                 (unsigned char*) ip6,
989                                 sizeof(struct ip6_hdr),
990                                 ((unsigned char*) ip6)+
991                                     relative_offset+
992                                     sizeof(struct ip6_frag),
993                                 fragment_size, offset, more_fragments,
994                                 frg_hdr->ip6f_ident, frg_hdr->ip6f_nxt,
995                                 current_time, tid);
996
997                         if(to_delete) free(to_delete);
998
999                         if(pkt==NULL){
1000                            return DPI_STATUS_IP_FRAGMENT;
1001                        }
1002
1003                        to_return=DPI_STATUS_IP_LAST_FRAGMENT;
1004                        next_header=IPPROTO_IPV6;
1005                        length=((struct ip6_hdr*) (pkt))->
1006                                ip6_ctlun.ip6_un1.ip6_un1_plen+
1007                                sizeof(struct ip6_hdr);
1008                        /**
1009                         * Force the next iteration to analyze the
1010                         * reassembled IPv6 packet.
1011                         **/
1012                        application_offset=relative_offset=0;
1013                    }else{
1014                        return DPI_STATUS_IP_FRAGMENT;
1015                    }
1016                }else{
1017                    if(unlikely(pkt!=p_pkt)) free(pkt);
1018                    return DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
1019                }
1020                }
1021                break;
1022            case IPPROTO_IPV6: /** 6in4 and 6in6 tunneling **/
1023                /** The real packet is now ipv6. **/
1024                version=6;
1025                ip6=(struct ip6_hdr*) (pkt+application_offset);
1026 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
1027                if(unlikely(ntohs(ip6->ip6_ctlun.ip6_un1.ip6_un1_plen)+
1028                            sizeof(struct ip6_hdr)>length-
1029                            application_offset)){
1030                    if(unlikely(pkt!=p_pkt)) free(pkt);
1031                    return DPI_ERROR_L3_TRUNCATED_PACKET;
1032                }
1033 #endif
1034
1035                pkt_infos->src_addr_t.ipv6_srcaddr=ip6->ip6_src;
1036                pkt_infos->dst_addr_t.ipv6_dstaddr=ip6->ip6_dst;
```

```
1037
1038                 application_offset+=sizeof(struct ip6_hdr);
1039                 relative_offset=sizeof(struct ip6_hdr);
1040                 next_header=ip6->ip6_ctlun.ip6_un1.ip6_un1_nxt;
1041                 break;
1042             case 4: /* 4in4 and 4in6 tunneling */
1043                 /** The real packet is now ipv4. **/
1044                 version=4;
1045                 ip4=(struct iphdr*) (pkt+application_offset);
1046 #ifdef DPI_ENABLE_L3_TRUNCATION_PROTECTION
1047                 if(unlikely(application_offset+
1048                             sizeof(struct iphdr)>length ||
1049                             application_offset+((ip4->ihl)*4)>length ||
1050                             application_offset+
1051                             ntohs(ip4->tot_len)>length)){
1052                     if(unlikely(pkt!=p_pkt)) free(pkt);
1053                     return DPI_ERROR_L3_TRUNCATED_PACKET;
1054                 }
1055 #endif
1056                 pkt_infos->src_addr_t.ipv4_srcaddr=ip4->saddr;
1057                 pkt_infos->dst_addr_t.ipv4_dstaddr=ip4->daddr;
1058                 next_header=ip4->protocol;
1059                 tmp=(ip4->ihl)*4;
1060                 application_offset+=tmp;
1061                 relative_offset=tmp;
1062                 break;
1063             default:
1064                 stop=1;
1065                 pkt_infos->l4offset=application_offset;
1066                 break;
1067
1068             }
1069     }
1070
1071     pkt_infos->l4prot=next_header;
1072 #ifdef DPI_ENABLE_L4_TRUNCATION_PROTECTION
1073     if(unlikely(application_offset>length)){
1074         if(unlikely(pkt!=p_pkt)) free(pkt);
1075         return DPI_ERROR_L4_TRUNCATED_PACKET;
1076     }
1077 #endif
1078     pkt_infos->processing_time=current_time;
1079     pkt_infos->pkt=pkt;
1080     pkt_infos->l7offset=application_offset;
1081     pkt_infos->data_length= length-application_offset;
1082     pkt_infos->ip_version=version;
1083     return to_return;
1084 }
1085
```

```
1086 int8_t dpi_parse_L3_L4_headers(dpi_library_state_t *state,
1087                                const unsigned char* p_pkt,
1088                                u_int32_t p_length,
1089                                dpi_pkt_infos_t *pkt_infos,
1090                                u_int32_t current_time){
1091     /**
1092      * We can pass any thread id, indeed in this case we don't
1093      * need lock synchronization.
1094      **/
1095     return mp_dpi_extract_packet_infos(state, p_pkt, p_length, pkt_infos,
1096                                        current_time, 0);
1097 }
1098
1099
1100 /*
1101  * Try to detect the application protocol. Before calling it, a check on
1102  * L4 protocol should be done and the function should be called only if
1103  * the packet is TCP or UDP.
1104  * @param   state The pointer to the library state.
1105  * @param   pkt_infos The pointer to the packet infos.
1106  * @return  The status of the operation. It gives additional informations
1107  *          about the processing of the request. If lesser than 0, an
1108  *          error occurred. dpi_get_error_msg() can be used to get a
1109  *          textual representation of the error. If greater or equal
1110  *          than 0 then it should not be interpreted as an error but
1111  *          simply gives additional informations (e.g. if the packet was
1112  *          IP fragmented, if it was out of order in the TCP stream, if is
1113  *          a segment of a larger application request, etc..).
1114  *          dpi_get_status_msg() can be used to get a textual
1115  *          representation of the status. Status and error codes are
1116  *          defined above in this header file.
1117  *
1118  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
1119  *          fragment. In this case, if IP fragmentation support is
1120  *          enabled, the library copied the content of the datagram, so if
1121  *          the user wants, he can release the resources used to store the
1122  *          datagram.
1123  *
1124  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
1125  *          datagram allows the library to reconstruct a fragmented
1126  *          datagram. In this case, pkt_infos->pkt will contain a pointer
1127  *          to the recomposed datagram. This pointer will be different
1128  *          from p_pkt. The user should free() this pointer when it is no
1129  *          more needed (e.g. after calling
1130  *          dpi_state*_get_app_protocol(..)).
1131  */
1132 dpi_identification_result_t dpi_stateful_get_app_protocol(
1133                 dpi_library_state_t *state, dpi_pkt_infos_t* pkt_infos){
1134     dpi_identification_result_t r;
```

```
1135     r.status=DPI_STATUS_OK;
1136
1137     dpi_flow_infos_t* flow_infos=NULL;
1138     ipv4_flow_t* ipv4_flow=NULL;
1139     ipv6_flow_t* ipv6_flow=NULL;
1140
1141     if(pkt_infos->ip_version==DPI_IP_VERSION_4){
1142         ipv4_flow=dpi_flow_table_find_or_create_flow_v4(state, pkt_infos);
1143         flow_infos=&(ipv4_flow->infos);
1144     }else{
1145         ipv6_flow=dpi_flow_table_find_or_create_flow_v6(state, pkt_infos);
1146         flow_infos=&(ipv6_flow->infos);
1147     }
1148
1149     if(unlikely(flow_infos==NULL)){
1150         r.status=DPI_ERROR_MAX_FLOWS;
1151         return r;
1152     }
1153
1154     r=dpi_stateless_get_app_protocol(state, flow_infos, pkt_infos);
1155
1156     if(r.status==DPI_STATUS_TCP_CONNECTION_TERMINATED){
1157         if(ipv4_flow!=NULL){
1158             dpi_flow_table_delete_flow_v4(state->db4,
1159                                           state->flow_cleaner_callback,
1160                                           ipv4_flow);
1161         }else{
1162             dpi_flow_table_delete_flow_v6(state->db6,
1163                                           state->flow_cleaner_callback,
1164                                           ipv6_flow);
1165         }
1166     }
1167     return r;
1168 }
1169
1170
1171
1172 /**
1173  * Initialize the flow informations passed as argument.
1174  * @param state       A pointer to the state of the library.
1175  * @param flow_infos  The informations that will be initialized by
1176  *                    the library.
1177  * @param l4prot      The transport protocol identifier.
1178  */
1179 void dpi_init_flow_infos(
1180                dpi_library_state_t* state,
1181                dpi_flow_infos_t *flow_infos,
1182                u_int8_t l4prot){
1183     dpi_l7_prot_id i;
```

```
1184     if(l4prot==IPPROTO_TCP){
1185         for(i=0; i<BITNSLOTS(DPI_NUM_TCP_PROTOCOLS); i++){
1186             flow_infos->possible_matching_protocols_t.tcp[i]=
1187                           state->tcp_protocols_to_inspect[i];
1188         }
1189         flow_infos->possible_protocols=state->tcp_active_protocols;
1190     }else{
1191         for(i=0; i<BITNSLOTS(DPI_NUM_UDP_PROTOCOLS); i++){
1192             flow_infos->possible_matching_protocols_t.udp[i]=
1193                           state->udp_protocols_to_inspect[i];
1194         }
1195         flow_infos->possible_protocols=state->udp_active_protocols;
1196     }
1197     flow_infos->l7prot=DPI_PROTOCOL_NOT_DETERMINED;
1198     flow_infos->trials=0;
1199     flow_infos->tcp_reordering_enabled=state->tcp_reordering_enabled;
1200     bzero(&(flow_infos->tracking), sizeof(dpi_tracking_informations_t));
1201 }
1202
1203
1204 /*
1205  * Try to detect the application protocol. Before calling it, a check on
1206  * L4 protocol should be done and the function should be called only if
1207  * the packet is TCP or UDP. It should be used if the application already
1208  * has the concept of 'flow'. In this case the first time that the flow is
1209  * passed to the call, it must be initialized with
1210  * dpi_init_flow_infos(...).
1211  * @param   state The pointer to the library state.
1212  * @param   flow The informations about the flow. They must be kept by the
1213  *               user.
1214  * @param   pkt_infos The pointer to the packet infos.
1215  * @return  The status of the operation. It gives additional informations
1216  *          about the processing of the request. If lesser than 0, an ↙
      ↳ error
1217  *          occurred. dpi_get_error_msg() can be used to get a textual
1218  *          representation of the error. If greater or equal than 0 then
1219  *          it should not be interpreted as an error but simply gives
1220  *          additional informations (e.g. if the packet was IP fragmented,
1221  *          if it was out of order in the TCP stream, if is a segment of
1222  *          a larger application request, etc..). dpi_get_status_msg()
1223  *          can be used to get a textual representation of the status.
1224  *          Status and error codes are defined above in this header file.
1225  *
1226  *          The status is DPI_STATUS_IP_FRAGMENT if the datagram is a
1227  *          fragment. In this case, if IP fragmentation support is
1228  *          enabled, the library copied the content of the datagram, so if
1229  *          the user wants, he can release the resources used to store the
1230  *          datagram.
1231  *
```

```
1232  *          The status is DPI_STATUS_IP_LAST_FRAGMENT if the received
1233  *          datagram allows the library to reconstruct a fragmented
1234  *          datagram. In this case, pkt_infos->pkt will contain a pointer
1235  *          to the recomposed datagram. This pointer will be different
1236  *          from p_pkt. The user should free() this pointer when it is no
1237  *          more needed (e.g. after calling
1238  *          dpi_state*_get_app_protocol(..)).
1239  */
1240 dpi_identification_result_t dpi_stateless_get_app_protocol(
1241                dpi_library_state_t *state,
1242                dpi_flow_infos_t *flow,
1243                dpi_pkt_infos_t *pkt_infos){
1244     dpi_identification_result_t r;
1245     r.status=DPI_STATUS_OK;
1246     r.protocol.l4prot=pkt_infos->l4prot;
1247     r.user_flow_data=(flow->tracking.flow_specific_user_data);
1248     dpi_l7_prot_id i;
1249
1250     u_int8_t check_result=DPI_PROTOCOL_NO_MATCHES;
1251     dpi_l7_prot_id num_protocols;
1252     const dpi_l7_prot_id* well_known_ports;
1253     char *active_protocols_mask;
1254     const unsigned char* app_data=pkt_infos->pkt+pkt_infos->l7offset;
1255     u_int32_t data_length=pkt_infos->data_length;
1256     dpi_inspector_callback const *inspectors;
1257     dpi_tcp_reordering_reordered_segment_t seg;
1258     seg.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
1259     seg.data=NULL;
1260     seg.connection_terminated=0;
1261
1262     if(flow->l7prot<DPI_PROTOCOL_NOT_DETERMINED){
1263         r.protocol.l7prot=flow->l7prot;
1264         if(pkt_infos->l4prot==IPPROTO_TCP){
1265
1266             if(flow->tcp_reordering_enabled){
1267                 seg=dpi_reordering_tcp_track_connection(
1268                             pkt_infos, &(flow->tracking));
1269
1270                 if(seg.status==
1271                          DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER){
1272                     r.status=DPI_STATUS_TCP_OUT_OF_ORDER;
1273                     return r;
1274                 }else if(seg.status==
1275                          DPI_TCP_REORDERING_STATUS_REBUILT){
1276                     app_data=seg.data;
1277                     data_length=seg.data_length;
1278                 }
1279             }else{
1280                 seg.connection_terminated=
```

```
1281                        dpi_reordering_tcp_track_connection_light(
1282                                     pkt_infos, &(flow->tracking));
1283            }
1284
1285            if(BITTEST(state->tcp_active_callbacks, flow->l7prot) &&
1286                       data_length!=0){
1287                (*(tcp_callbacks_manager[flow->l7prot]))(
1288                         state, pkt_infos, app_data,
1289                         data_length, &(flow->tracking));
1290            }
1291
1292            if(seg.status==DPI_TCP_REORDERING_STATUS_REBUILT){
1293                free(seg.data);
1294            }
1295
1296        }else if(pkt_infos->l4prot==IPPROTO_UDP &&
1297                BITTEST(state->udp_active_callbacks, flow->l7prot)){
1298            (*(udp_callbacks_manager[flow->l7prot]))(
1299                         state, pkt_infos, app_data,
1300                         data_length, &(flow->tracking));
1301        }
1302
1303        if(seg.connection_terminated){
1304            r.status=DPI_STATUS_TCP_CONNECTION_TERMINATED;
1305        }
1306        return r;
1307    }else if(flow->l7prot==DPI_PROTOCOL_NOT_DETERMINED){
1308        if(pkt_infos->l4prot==IPPROTO_TCP &&
1309           state->tcp_active_protocols>0){
1310            active_protocols_mask=flow->possible_matching_protocols_t.tcp;
1311            inspectors=tcp_inspectors;
1312            num_protocols=DPI_NUM_TCP_PROTOCOLS;
1313            well_known_ports=dpi_well_known_ports_association_tcp;
1314            if(flow->tcp_reordering_enabled){
1315                seg=dpi_reordering_tcp_track_connection(
1316                          pkt_infos, &(flow->tracking));
1317
1318                if(seg.status==DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER){
1319                    r.status=DPI_STATUS_TCP_OUT_OF_ORDER;
1320                    r.protocol.l7prot=DPI_PROTOCOL_UNKNOWN;
1321                    return r;
1322                }else if(seg.status==DPI_TCP_REORDERING_STATUS_REBUILT){
1323                    app_data=seg.data;
1324                    data_length=seg.data_length;
1325                }
1326            }else{
1327                if(dpi_reordering_tcp_track_connection_light(
1328                          pkt_infos, &(flow->tracking)))
1329                    r.status=DPI_STATUS_TCP_CONNECTION_TERMINATED;;
```

```
1330            }
1331        }else if(pkt_infos->l4prot==IPPROTO_UDP &&
1332                 state->udp_active_protocols>0){
1333            active_protocols_mask=flow->possible_matching_protocols_t.udp;
1334            inspectors=udp_inspectors;
1335            num_protocols=DPI_NUM_UDP_PROTOCOLS;
1336            well_known_ports=dpi_well_known_ports_association_udp;
1337        }else{
1338            return r;
1339        }
1340
1341        /**
1342         * If we have no payload we don't do anything. We already
1343         * invoked the TCP reordering to update the connection state.
1344         */
1345        if(data_length==0){
1346            r.protocol.l7prot=flow->l7prot;
1347            return r;
1348        }
1349
1350        dpi_l7_prot_id first_protocol_to_check;
1351        dpi_l7_prot_id checked_protocols=0;
1352
1353        if((first_protocol_to_check=well_known_ports[pkt_infos->srcport])
1354            ==DPI_PROTOCOL_UNKNOWN &&
1355            (first_protocol_to_check=well_known_ports[pkt_infos->dstport])
1356            ==DPI_PROTOCOL_UNKNOWN){
1357            first_protocol_to_check=0;
1358        }
1359
1360        for(i=first_protocol_to_check; checked_protocols<num_protocols;
1361            i=(i+1)%num_protocols, ++checked_protocols){
1362            if(BITTEST(active_protocols_mask, i)){
1363                check_result=(*(inspectors[i]))(state, pkt_infos,
1364                                                app_data, data_length,
1365                                                &(flow->tracking));
1366                if(check_result==DPI_PROTOCOL_MATCHES){
1367                    flow->l7prot=i;
1368                    r.protocol.l7prot=flow->l7prot;
1369
1370                    if(seg.status==DPI_TCP_REORDERING_STATUS_REBUILT){
1371                        free(seg.data);
1372                    }
1373
1374                    if(seg.connection_terminated){
1375                        r.status=DPI_STATUS_TCP_CONNECTION_TERMINATED;
1376                    }
1377
1378                    return r;
```

```
1379                }else if(check_result==DPI_PROTOCOL_NO_MATCHES){
1380                    BITCLEAR(active_protocols_mask, i);
1381                    --(flow->possible_protocols);
1382                }
1383            }
1384        }
1385
1386        /**
1387         * If all the protocols don't match or if we still have
1388         * ambiguity after the maximum number of trials, then the
1389         * library was unable to identify the protocol.
1390         **/
1391        if(flow->possible_protocols==0 ||
1392           (state->max_trials!=0 &&
1393            unlikely(++flow->trials==state->max_trials))){
1394            flow->l7prot=DPI_PROTOCOL_UNKNOWN;
1395        }
1396
1397    }
1398
1399    r.protocol.l7prot=flow->l7prot;
1400
1401    if(seg.status==DPI_TCP_REORDERING_STATUS_REBUILT){
1402        free(seg.data);
1403    }
1404
1405    if(seg.connection_terminated){
1406        r.status=DPI_STATUS_TCP_CONNECTION_TERMINATED;
1407    }
1408    return r;
1409 }
1410
1411 /**
1412  * Try to guess the protocol looking only at source/destination ports.
1413  * This could be erroneous because sometimes protocols
1414  * run over ports which are not their well-known ports.
1415  * @param    pkt_infos The pointer to the packet infos.
1416  * @return   Returns the possible matching protocol.
1417  */
1418 dpi_protocol_t dpi_guess_protocol(dpi_pkt_infos_t* pkt_infos){
1419     dpi_protocol_t r;
1420     r.l4prot=pkt_infos->l4prot;
1421     if(pkt_infos->l4prot==IPPROTO_TCP){
1422         r.l7prot=dpi_well_known_ports_association_tcp[pkt_infos->srcport];
1423         if(r.l7prot==DPI_PROTOCOL_UNKNOWN)
1424             ↙
     ↳ r.l7prot=dpi_well_known_ports_association_tcp[pkt_infos->dstport];
1425     }else if(pkt_infos->l4prot==IPPROTO_UDP){
1426         r.l7prot=dpi_well_known_ports_association_udp[pkt_infos->srcport];
```

```
1427         if(r.l7prot==DPI_PROTOCOL_UNKNOWN)
1428             ↙
     ↳ r.l7prot=dpi_well_known_ports_association_udp[pkt_infos->dstport];
1429     }else{
1430         r.l7prot=DPI_PROTOCOL_UNKNOWN;
1431     }
1432     return r;
1433 }
1434
1435
1436 /**
1437  * Get the string representing the error message associated to the
1438  * specified error_code.
1439  * @param   error_code The error code.
1440  * @return  The error message.
1441  */
1442 const char* const dpi_get_error_msg(int8_t error_code){
1443     switch(error_code){
1444         case DPI_ERROR_WRONG_IPVERSION:
1445             return "ERROR: The packet is neither IPv4 nor IPv6.";
1446         case DPI_ERROR_IPSEC_NOTSUPPORTED:
1447             return "ERROR: The packet is encrypted using IPSEC. "
1448                    "IPSEC is not supported.";
1449         case DPI_ERROR_L3_TRUNCATED_PACKET:
1450             return "ERROR: The L3 packet is truncated or corrupted.";
1451         case DPI_ERROR_L4_TRUNCATED_PACKET:
1452             return "ERROR: The L4 packet is truncated or corrupted.";
1453         case DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED:
1454             return "ERROR: The transport protocol is not supported.";
1455         case DPI_ERROR_MAX_FLOWS:
1456             return "ERROR: The maximum number of active flows has been"
1457                    " reached.";
1458         default:
1459             return "ERROR: Not existing error code.";
1460     }
1461 }
1462
1463 /**
1464  * Get the string representing the status message associated to the
1465  * specified status_code.
1466  * @param   status_code The status code.
1467  * @return  The status message.
1468  */
1469 const char* const dpi_get_status_msg(int8_t status_code){
1470     switch(status_code){
1471         case DPI_STATUS_OK:
1472             return "STATUS: Everything is ok.";
1473         case DPI_STATUS_IP_FRAGMENT:
1474             return "STATUS: The received IP datagram is a fragment of a "
```

```
                    " bigger datagram.";
        case DPI_STATUS_IP_LAST_FRAGMENT:
            return "STATUS: The received IP datagram is the last fragment"
                   " of a bigger datagram. The original datagram has been"
                   " recomposed.";
        case DPI_STATUS_TCP_OUT_OF_ORDER:
            return "STATUS: The received TCP segment is out of order in "
                   " its stream. It will be buffered waiting for in order"
                   " segments.";
        case DPI_STATUS_TCP_CONNECTION_TERMINATED:
            return "STATUS: The TCP connection is terminated.";
        default:
            return "STATUS: Not existing status code.";
    }
}

/**
 * Returns a string corresponding to the given protocol.
 * @param   protocol The protocol identifier.
 * @return  A string representation of the given protocol.
 */
const char* const dpi_get_protocol_name(dpi_protocol_t protocol){
    if(protocol.l4prot==IPPROTO_TCP){
        switch(protocol.l7prot){
            case DPI_PROTOCOL_TCP_BGP:
                return "BGP";
            case DPI_PROTOCOL_TCP_HTTP:
                return "HTTP";
            case DPI_PROTOCOL_TCP_SMTP:
                return "SMTP";
            case DPI_PROTOCOL_TCP_POP3:
                return "POP3";
            default:
                return "Unknown";
        }
    }else if(protocol.l4prot==IPPROTO_UDP){
        switch(protocol.l7prot){
            case DPI_PROTOCOL_UDP_DHCP:
                return "DHCP";
            case DPI_PROTOCOL_UDP_DHCPv6:
                return "DHCPv6";
            case DPI_PROTOCOL_UDP_DNS:
                return "DNS";
            case DPI_PROTOCOL_UDP_MDNS:
                return "MDNS";
            case DPI_PROTOCOL_UDP_NTP:
                return "NTP";
            default:
                return "Unknown";
```

```
1524         }
1525     }else
1526         return "Unknown";
1527 }
1528
1529 /**
1530  * Sets the callback that will be called when a flow expires.
1531  * (Valid only if stateful API is used).
1532  * @param state     A pointer to the state of the library.
1533  * @param cleaner   The callback used to clear the user state.
1534  *
1535  * @return DPI_STATE_UPDATE_SUCCESS if succeeded,
1536  *         DPI_STATE_UPDATE_FAILURE otherwise.
1537  */
1538 u_int8_t dpi_set_flow_cleaner_callback(
1539             dpi_library_state_t* state,
1540             dpi_flow_cleaner_callback* cleaner){
1541     state->flow_cleaner_callback=cleaner;
1542     return DPI_STATE_UPDATE_SUCCESS;
1543 }
```

A.2 Flow table

```
1 /*
2  * flow_table.h
3  *
4  *  Created on: 22/10/2012
5  *  Author: Daniele De Sensi
6  */
7
8 #ifndef FLOW_TABLE_H_
9 #define FLOW_TABLE_H_
10
11 #include "api.h"
12 #include "config.h"
13
14 #ifdef __cplusplus
15 extern "C" {
16 #endif
17
18 typedef struct ipv4_flow ipv4_flow_t;
19 typedef struct ipv6_flow ipv6_flow_t;
20
21 struct ipv4_flow{
22     u_int16_t srcport;
23     u_int16_t dstport;
24     u_int32_t srcaddr;
25     u_int32_t dstaddr;
```

```
26      u_int8_t l4prot;
27
28      ipv4_flow_t* prev;
29      ipv4_flow_t* next;
30      dpi_flow_infos_t infos;
31      u_int32_t last_timestamp;
32  };
33
34  struct ipv6_flow{
35      u_int16_t srcport;
36      u_int16_t dstport;
37      struct in6_addr srcaddr;
38      struct in6_addr dstaddr;
39      u_int8_t l4prot;
40
41      ipv6_flow_t* prev;
42      ipv6_flow_t* next;
43      dpi_flow_infos_t infos;
44      u_int32_t last_timestamp;
45  };
46
47
48  typedef struct dpi_flow_DB_v4 dpi_flow_DB_v4_t;
49  typedef struct dpi_flow_DB_v6 dpi_flow_DB_v6_t;
50
51  #if DPI_FLOW_TABLE_USE_MEMORY_POOL
52  dpi_flow_DB_v4_t* dpi_flow_table_create_v4(u_int32_t size,
53                                             u_int32_t max_active_v4_flows,
54                                             u_int16_t num_partitions,
55                                             u_int32_t start_pool_size);
56  dpi_flow_DB_v6_t* dpi_flow_table_create_v6(u_int32_t size,
57                                             u_int32_t max_active_v6_flows,
58                                             u_int16_t num_partitions,
59                                             u_int32_t start_pool_size);
60
61  #else
62  dpi_flow_DB_v4_t* dpi_flow_table_create_v4(u_int32_t size,
63                                             u_int32_t max_active_v4_flows,
64                                             u_int16_t num_partitions);
65  dpi_flow_DB_v6_t* dpi_flow_table_create_v6(u_int32_t size,
66                                             u_int32_t max_active_v6_flows,
67                                             u_int16_t num_partitions);
68  #endif
69
70  void dpi_flow_table_delete_v4(
71          dpi_flow_DB_v4_t* db,
72          dpi_flow_cleaner_callback* flow_cleaner_callback);
73  void dpi_flow_table_delete_v6(
74          dpi_flow_DB_v6_t* db,
```

```
75          dpi_flow_cleaner_callback* flow_cleaner_callback);
76
77
78  /**
79   * Search for a flow in the table.
80   * @param state A pointer to the state of the library.
81   * @param index The hash index of the flow to search.
82   * @param pkt_infos The L3 and L4 packet's parsed informations.
83   * @return A pointer to the flow if it is present, NULL otherwise.
84   */
85  ipv4_flow_t* dpi_flow_table_find_flow_v4(dpi_library_state_t* state,
86                                           u_int32_t index,
87                                           dpi_pkt_infos_t* pkt_infos);
88
89  /**
90   * Search for a flow in the table.
91   * @param state A pointer to the state of the library.
92   * @param index The hash index of the flow to search.
93   * @param pkt_infos The L3 and L4 packet's parsed informations.
94   * @return A pointer to the flow if it is present, NULL otherwise.
95   */
96  ipv6_flow_t* dpi_flow_table_find_flow_v6(dpi_library_state_t* state,
97                                           u_int32_t index,
98                                           dpi_pkt_infos_t* pkt_infos);
99
100
101 /**
102  * Find the flow to which pkt_infos belongs or creates it if doesn't
103  * exists. Updates pkt_infos->direction field according to the direction
104  * of the stored flow.
105  * @return The informations about the flow.
106  */
107 ipv4_flow_t* dpi_flow_table_find_or_create_flow_v4(
108         dpi_library_state_t* state,
109         dpi_pkt_infos_t* pkt_infos);
110 ipv6_flow_t* dpi_flow_table_find_or_create_flow_v6(
111         dpi_library_state_t* state,
112         dpi_pkt_infos_t* pkt_infos);
113
114 void dpi_flow_table_delete_flow_v4(
115         dpi_flow_DB_v4_t* db,
116         dpi_flow_cleaner_callback* flow_cleaner_callback,
117         ipv4_flow_t* to_delete);
118
119 void dpi_flow_table_delete_flow_v6(
120         dpi_flow_DB_v6_t* db,
121         dpi_flow_cleaner_callback* flow_cleaner_callback,
122         ipv6_flow_t* to_delete);
123
```

```
124
125 /**
126  * They are used directly only in mpdpi. Should never be used directly
127  * by the user.
128  **/
129 u_int32_t dpi_compute_v4_hash_function(
130         dpi_flow_DB_v4_t *db,
131         const dpi_pkt_infos_t* const pkt_infos);
132
133 u_int32_t dpi_compute_v6_hash_function(
134         dpi_flow_DB_v6_t *db,
135         const dpi_pkt_infos_t* const pkt_infos);
136
137 ipv4_flow_t* mp_dpi_flow_table_find_or_create_flow_v4(
138         dpi_library_state_t* state, u_int16_t partition_id,
139         u_int32_t index, dpi_pkt_infos_t* pkt_infos);
140
141 ipv6_flow_t* mp_dpi_flow_table_find_or_create_flow_v6(
142         dpi_library_state_t* state, u_int16_t partition_id,
143         u_int32_t index, dpi_pkt_infos_t* pkt_infos);
144
145 void mp_dpi_flow_table_delete_flow_v4(
146         dpi_flow_DB_v4_t* db,
147         dpi_flow_cleaner_callback* flow_cleaner_callback,
148         u_int16_t partition_id, ipv4_flow_t* to_delete);
149
150 void mp_dpi_flow_table_delete_flow_v6(
151         dpi_flow_DB_v6_t* db,
152         dpi_flow_cleaner_callback* flow_cleaner_callback,
153         u_int16_t partition_id, ipv6_flow_t* to_delete);
154
155 #ifdef __cplusplus
156 }
157 #endif
158
159 #endif /* FLOW_TABLE_H_ */
```

```
1  /*
2   * flow_table.c
3   *
4   *  Created on: 22/10/2012
5   *  Author: Daniele De Sensi
6   */
7
8  #include "flow_table.h"
9  #include "hash_functions.h"
10 #include "tcp_stream_management.h"
11 #include "config.h"
12 #include "utils.h"
```

```
13 #include <stdlib.h>
14 #include <stdio.h>
15 #include <assert.h>
16 #include <strings.h>
17 #include <string.h>
18 #include <time.h>
19 #include <inttypes.h>
20 #include <math.h>
21
22 #if DPI_NUMA_AWARE
23 #include <numa.h>
24 #endif
25
26 #define DPI_CACHE_LINES_PADDING_REQUIRED(size)          \
27     (size%DPI_CACHE_LINE_SIZE==0?0:DPI_CACHE_LINE_SIZE- \
28     (size%DPI_CACHE_LINE_SIZE))
29
30 #define DPI_DEBUG_FLOW_TABLE 0
31 #define debug_print(fmt, ...)              \
32             do { if (DPI_DEBUG_FLOW_TABLE) \
33                 fprintf(stdout, fmt, __VA_ARGS__); } while (0)
34
35
36 #define DPI_FLOW_TABLE_MAX_IDLE_TIME 30 /** In seconds. **/
37
38 #define DPI_FLOW_TABLE_WALK_TIME 1 /** In seconds. **/
39
40 static inline ipv4_flow_t* v4_flow_alloc(){
41     void* r;
42 #if DPI_NUMA_AWARE
43     r=numa_alloc_onnode(sizeof(ipv4_flow_t),
44                         DPI_NUMA_AWARE_FLOW_TABLE_NODE);
45     assert(r);
46 #else
47     #if DPI_FLOW_TABLE_ALIGN_FLOWS
48         assert(posix_memalign((void**) &r, DPI_CACHE_LINE_SIZE,
49                sizeof(ipv4_flow_t))==0);
50     #else
51         r=malloc(sizeof(ipv4_flow_t));
52         assert(r);
53     #endif
54 #endif
55     return (ipv4_flow_t*) r;
56 }
57
58 static inline ipv6_flow_t* v6_flow_alloc(){
59     void* r;
60 #if DPI_NUMA_AWARE
61     r=numa_alloc_onnode(sizeof(ipv6_flow_t),
```

```
62                          DPI_NUMA_AWARE_FLOW_TABLE_NODE);
63     assert(r);
64 #else
65     #if DPI_FLOW_TABLE_ALIGN_FLOWS
66         assert(posix_memalign((void**) &r, DPI_CACHE_LINE_SIZE,
67                sizeof(ipv6_flow_t))==0);
68     #else
69         r=malloc(sizeof(ipv6_flow_t));
70         assert(r);
71     #endif
72 #endif
73     return (ipv6_flow_t*) r;
74 }
75
76 static inline void v4_flow_free(ipv4_flow_t* flow){
77 #if DPI_NUMA_AWARE
78     numa_free(flow, sizeof(ipv4_flow_t));
79 #else
80     free(flow);
81 #endif
82 }
83
84 static inline void v6_flow_free(ipv6_flow_t* flow){
85 #if DPI_NUMA_AWARE
86     numa_free(flow, sizeof(ipv6_flow_t));
87 #else
88     free(flow);
89 #endif
90 }
91
92
93 typedef u_int32_t(dpi_fnv_hash_function)(dpi_pkt_infos_t* in,
94                                          u_int8_t log);
95
96
97 typedef struct dpi_flow_DB_partition_specific_informations{
98     /** This table part is in the range [lowest_index, highest_index]. **/
99     u_int32_t lowest_index;
100    u_int32_t highest_index;
101    u_int32_t last_walk;
102    u_int32_t active_flows;
103    u_int32_t max_active_flows;
104 }dpi_flow_DB_partition_specific_informations_t;
105
106
107 typedef struct dpi_flow_DB_v4_partition{
108    struct dpi_flow_DB_v4_real_partition{
109        dpi_flow_DB_partition_specific_informations_t informations;
110    #if DPI_FLOW_TABLE_USE_MEMORY_POOL
```

```
111         /**
112          * If an integer x is contained in this array, then
113          * memory_chunk_lower_bound[i] can be used.
114          **/
115         u_int32_t* pool;
116         u_int32_t pool_size;
117         ipv4_flow_t* memory_chunk_lower_bound;
118         ipv4_flow_t* memory_chunk_upper_bound;
119     #endif
120     }partition;
121     /**
122      * Using padding each partition will go in a separate cache line
123      * avoiding false sharing between threads working on different
124      * partitions.
125      **/
126     char padding[DPI_CACHE_LINES_PADDING_REQUIRED
127                  (sizeof(struct dpi_flow_DB_v4_real_partition))];
128 }dpi_flow_DB_v4_partition_t;
129
130 typedef struct dpi_flow_DB_v6_partition{
131     struct dpi_flow_DB_v6_real_partition{
132         dpi_flow_DB_partition_specific_informations_t informations;
133     #if DPI_FLOW_TABLE_USE_MEMORY_POOL
134         /**
135          * If an integer x is contained in this array, then
136          * memory_chunk_lower_bound[i] can be used.
137          **/
138         u_int32_t* pool;
139         u_int32_t pool_size;
140         ipv6_flow_t* memory_chunk_lower_bound;
141         ipv6_flow_t* memory_chunk_upper_bound;
142     #endif
143     }partition;
144     /**
145      * Using padding each partition will go in a separate cache line
146      * avoiding false sharing between threads working on different
147      * partitions.
148      **/
149     char padding[DPI_CACHE_LINES_PADDING_REQUIRED
150                  (sizeof(struct dpi_flow_DB_v6_real_partition))];
151 }dpi_flow_DB_v6_partition_t;
152
153 struct dpi_flow_DB_v4{
154     /**
155      *  The flow table may be shared among multiple threads. In this
156      *  case each thread will access to a different part of 'table'.
157      *  We also have one dpi_flow_DB_v*_partition_t per thread containing
158      *  the thread's partition specific informations.
159      */
```

```
160     ipv4_flow_t* table;
161 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
162     u_int32_t seed;
163 #endif
164     u_int32_t total_size;
165     dpi_flow_DB_v4_partition_t* partitions;
166     u_int16_t num_partitions;
167 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
168     u_int32_t individual_pool_size;
169 #endif
170 };
171
172 struct dpi_flow_DB_v6{
173     /**
174      *  The flow table may be shared among multiple threads. In this
175      *  case each thread will access to a different part of 'table'.
176      *  We also have one dpi_flow_DB_v*_partition_t per thread containing
177      *  the thread's partition specific informations.
178      */
179     ipv6_flow_t* table;
180 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
181     u_int32_t seed;
182 #endif
183     u_int32_t total_size;
184     dpi_flow_DB_v6_partition_t* partitions;
185     u_int16_t num_partitions;
186 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
187     u_int32_t individual_pool_size;
188 #endif
189 };
190
191
192 #ifndef DPI_DEBUG
193 static
194 #endif
195 u_int8_t v4_equals(ipv4_flow_t* flow, dpi_pkt_infos_t* pkt_infos){
196     return ((flow->srcaddr==pkt_infos->src_addr_t.ipv4_srcaddr &&
197              flow->dstaddr==pkt_infos->dst_addr_t.ipv4_dstaddr &&
198              flow->srcport==pkt_infos->srcport &&
199              flow->dstport==pkt_infos->dstport) ||
200             (flow->srcaddr==pkt_infos->dst_addr_t.ipv4_dstaddr &&
201              flow->dstaddr==pkt_infos->src_addr_t.ipv4_srcaddr &&
202              flow->srcport==pkt_infos->dstport &&
203              flow->dstport==pkt_infos->srcport)) &&
204              flow->l4prot==pkt_infos->l4prot;
205
206 }
207
208 #ifndef DPI_DEBUG
```

```
209 static
210 #endif
211 u_int8_t v6_equals(ipv6_flow_t* flow, dpi_pkt_infos_t* pkt_infos){
212     u_int8_t i;
213
214     /*1: src=src and dst=dst. 2: src=dst and dst=src. */
215     u_int8_t direction=0;
216
217     for(i=0; i<16; i++){
218         if(direction!=2 &&
219            pkt_infos->src_addr_t.ipv6_srcaddr.s6_addr[i]==
220            flow->srcaddr.s6_addr[i] &&
221            pkt_infos->dst_addr_t.ipv6_dstaddr.s6_addr[i]==
222            flow->dstaddr.s6_addr[i]){
223             direction=1;
224         }else if(direction!=1 &&
225                  pkt_infos->src_addr_t.ipv6_srcaddr.s6_addr[i]==
226                  flow->dstaddr.s6_addr[i] &&
227                  pkt_infos->dst_addr_t.ipv6_dstaddr.s6_addr[i]==
228                  flow->srcaddr.s6_addr[i]){
229             direction=2;
230         }else
231             return 0;
232     }
233
234     if(direction==1)
235         return flow->srcport==pkt_infos->srcport &&
236                flow->dstport==pkt_infos->dstport &&
237                flow->l4prot==pkt_infos->l4prot;
238     else if(direction==2)
239         return flow->srcport==pkt_infos->dstport &&
240                flow->dstport==pkt_infos->srcport &&
241                flow->l4prot==pkt_infos->l4prot;
242     else
243         return 0;
244 }
245
246 #ifndef DPI_DEBUG
247 static
248 #endif
249 void dpi_flow_table_initialize_informations(
250         dpi_flow_DB_partition_specific_informations_t* table_informations,
251         u_int32_t lowest_index, u_int32_t highest_index,
252         u_int32_t max_active_flows){
253     table_informations->lowest_index=lowest_index;
254     table_informations->highest_index=highest_index;
255     table_informations->max_active_flows=max_active_flows;
256
257     table_informations->last_walk=0;
```

```
258     table_informations->active_flows=0;
259 }
260
261
262 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
263 dpi_flow_DB_v4_t* dpi_flow_table_create_v4(
264         u_int32_t size, u_int32_t max_active_v4_flows,
265         u_int16_t num_partitions, u_int32_t start_pool_size){
266 #else
267 dpi_flow_DB_v4_t* dpi_flow_table_create_v4(
268         u_int32_t size, u_int32_t max_active_v4_flows,
269         u_int16_t num_partitions){
270 #endif
271     u_int32_t i;
272     dpi_flow_DB_v4_t* table;
273
274     if(size!=0){
275         assert((table=(dpi_flow_DB_v4_t*)
276                   malloc(sizeof(dpi_flow_DB_v4_t)))!=NULL);
277         table->table=(ipv4_flow_t*)
278                   malloc(sizeof(ipv4_flow_t)*size);
279         assert(table->table);
280
281         for(i=0; i<size; i++){
282             /** Creation of sentinel node. **/
283             table->table[i].next=&(table->table[i]);
284             table->table[i].prev=&(table->table[i]);
285         }
286
287         u_int32_t partition_size=ceil((float)size/(float)num_partitions);
288         u_int32_t partition_max_active_v4_flows=
289                      max_active_v4_flows/num_partitions;
290 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
291         u_int32_t ↙
    ↳ partition_start_pool_size=start_pool_size/num_partitions;
292         table->individual_pool_size=
293                      partition_start_pool_size;
294 #endif
295
296 #if DPI_NUMA_AWARE
297         table->partitions=numa_alloc_onnode(
298                 sizeof(dpi_flow_DB_v4_partition_t)*num_partitions,
299                 DPI_NUMA_AWARE_FLOW_TABLE_NODE);
300         assert(table->partitions);
301 #else
302         assert(posix_memalign(
303                 (void**) &(table->partitions), DPI_CACHE_LINE_SIZE,
304                 sizeof(dpi_flow_DB_v4_partition_t)*num_partitions)==0);
305 #endif
```

```
306         table->num_partitions=num_partitions;
307         table->total_size=size;
308
309 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
310         srand((unsigned int) time(NULL));
311         table->seed=rand();
312 #endif
313
314         u_int16_t j;
315         u_int32_t lowest_index=0;
316         u_int32_t highest_index=lowest_index+partition_size-1;
317         for(j=0; j<num_partitions; ++j){
318             debug_print("[flow_table.c]: Created partition "
319                         "[%"PRIu32", %"PRIu32"]\n",
320                         lowest_index, highest_index);
321             dpi_flow_table_initialize_informations(
322                     &(table->partitions[j].partition.informations),
323                     lowest_index, highest_index,
324                     partition_max_active_v4_flows);
325             lowest_index=highest_index+1;
326             /**
327              * The last partition gets the entries up to the end of the
328              * table. Indeed, when the size is not a multiple of the
329              * number of partitions, the last partition may be smaller.
330              */
331             if(j==num_partitions-2)
332                 highest_index=size-1;
333             else
334                 highest_index+=partition_size;
335
336
337 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
338             ipv4_flow_t* flow_pool;
339 #if DPI_NUMA_AWARE
340             flow_pool=numa_alloc_onnode(
341                     sizeof(ipv4_flow_t)*partition_start_pool_size,
342                     DPI_NUMA_AWARE_FLOW_TABLE_NODE);
343             assert(flow_pool);
344             table->partitions[j].partition.pool=numa_alloc_onnode(
345                     sizeof(u_int32_t)*partition_start_pool_size,
346                     DPI_NUMA_AWARE_FLOW_TABLE_NODE);
347             assert(table->partitions[j].partition.pool);
348 #else
349             assert(posix_memalign(
350                    (void**) &flow_pool, DPI_CACHE_LINE_SIZE,
351                    (sizeof(ipv4_flow_t)*partition_start_pool_size)+
352                    DPI_CACHE_LINE_SIZE)==0);
353             assert(posix_memalign(
354                    (void**) &(table->partitions[j].partition.pool),
```

```
                    DPI_CACHE_LINE_SIZE,
                    (sizeof(u_int32_t)*partition_start_pool_size)+
                    DPI_CACHE_LINE_SIZE)==0);
#endif
            for(i=0; i<partition_start_pool_size; i++){
                table->partitions[j].partition.pool[i]=i;
            }
            table->partitions[j].partition.pool_size=
                        partition_start_pool_size;
            table->partitions[j].partition.memory_chunk_lower_bound=
                        flow_pool;
            table->partitions[j].partition.memory_chunk_upper_bound=
                        flow_pool+partition_start_pool_size;
#endif
        }
    }else
        table=NULL;
    return table;
}


#if DPI_FLOW_TABLE_USE_MEMORY_POOL
dpi_flow_DB_v6_t* dpi_flow_table_create_v6(u_int32_t size,
                                           u_int32_t max_active_v6_flows,
                                           u_int16_t num_partitions,
                                           u_int32_t start_pool_size){
#else
dpi_flow_DB_v6_t* dpi_flow_table_create_v6(u_int32_t size,
                                           u_int32_t max_active_v6_flows,
                                           u_int16_t num_partitions){
#endif
    u_int32_t i;
    dpi_flow_DB_v6_t* table;
    if(size!=0){
        assert((table=(dpi_flow_DB_v6_t*)
                       malloc(sizeof(dpi_flow_DB_v6_t)))!=NULL);
        table->table=(ipv6_flow_t*) malloc(sizeof(ipv6_flow_t)*size);
        assert(table->table);

        for(i=0; i<size; i++){
            /** Creation of sentinel node. **/
            table->table[i].next=&(table->table[i]);
            table->table[i].prev=&(table->table[i]);
        }

        u_int32_t partition_size=ceil((float)size/(float)num_partitions);
        u_int32_t partition_max_active_v6_flows=
                     max_active_v6_flows/num_partitions;
#if DPI_FLOW_TABLE_USE_MEMORY_POOL
```

```
404         u_int32_t partition_start_pool_size=
405                      start_pool_size/num_partitions;
406         table->individual_pool_size=partition_start_pool_size;
407 #endif
408
409 #if DPI_NUMA_AWARE
410         table->partitions=numa_alloc_onnode(
411                 sizeof(dpi_flow_DB_v6_partition_t)*num_partitions,
412                 DPI_NUMA_AWARE_FLOW_TABLE_NODE);
413         assert(table->partitions);
414 #else
415         assert(posix_memalign(
416                 (void**) &(table->partitions),
417                 DPI_CACHE_LINE_SIZE,
418                 sizeof(dpi_flow_DB_v6_partition_t)*num_partitions)==0);
419 #endif
420
421         table->num_partitions=num_partitions;
422         table->total_size=size;
423
424 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
425         srand((unsigned int) time(NULL));
426         table->seed=rand();
427 #endif
428
429         u_int16_t j;
430         u_int32_t lowest_index=0;
431         u_int32_t highest_index=lowest_index+partition_size-1;
432         for(j=0; j<num_partitions; ++j){
433             dpi_flow_table_initialize_informations(
434                     &(table->partitions[j].partition.informations),
435                     lowest_index, highest_index,
436                     partition_max_active_v6_flows);
437             lowest_index=highest_index+1;
438
439             /**
440              * The last partition gets the entries up to the end of the
441              * table. Indeed, when the size is not a multiple of the
442              * number of partitions, the last partition may be smaller.
443              */
444             if(j==num_partitions-2)
445                 highest_index=size-1;
446             else
447                 highest_index+=partition_size;
448
449 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
450             ipv6_flow_t* flow_pool;
451 #if DPI_NUMA_AWARE
452             flow_pool=numa_alloc_onnode(
```

```
453                         sizeof(ipv6_flow_t)*partition_start_pool_size,
454                         DPI_NUMA_AWARE_FLOW_TABLE_NODE);
455             assert(flow_pool);
456             table->partitions[j].partition.pool=numa_alloc_onnode(
457                         sizeof(u_int32_t)*partition_start_pool_size,
458                         DPI_NUMA_AWARE_FLOW_TABLE_NODE);
459             assert(table->partitions[j].partition.pool);
460 #else
461             assert(posix_memalign(
462                         (void**) &flow_pool,
463                         DPI_CACHE_LINE_SIZE,
464                         (sizeof(ipv6_flow_t)*partition_start_pool_size)+
465                         DPI_CACHE_LINE_SIZE)==0);
466             assert(posix_memalign(
467                         (void**) &(table->partitions[j].partition.pool),
468                         DPI_CACHE_LINE_SIZE,
469                         (sizeof(u_int32_t)*partition_start_pool_size)+
470                         DPI_CACHE_LINE_SIZE)==0);
471 #endif
472             for(i=0; i<partition_start_pool_size; i++){
473                 table->partitions[j].partition.pool[i]=i;
474             }
475             table->partitions[j].partition.pool_size=
476                     partition_start_pool_size;
477             table->partitions[j].partition.memory_chunk_lower_bound=
478                     flow_pool;
479             table->partitions[j].partition.memory_chunk_upper_bound=
480                     flow_pool+partition_start_pool_size;
481 #endif
482         }
483     }else
484         table=NULL;
485     return table;
486 }
487
488
489 void mp_dpi_flow_table_delete_flow_v4(
490         dpi_flow_DB_v4_t* db,
491         dpi_flow_cleaner_callback* flow_cleaner_callback,
492         u_int16_t partition_id, ipv4_flow_t* to_delete){
493     to_delete->prev->next=to_delete->next;
494     to_delete->next->prev=to_delete->prev;
495
496     if(flow_cleaner_callback)
497         (*(flow_cleaner_callback))(
498                 to_delete->infos.tracking.flow_specific_user_data);
499     --db->partitions[partition_id].partition.informations.active_flows;
500     free(to_delete->infos.tracking.http_informations[0].temp_buffer);
501     free(to_delete->infos.tracking.http_informations[1].temp_buffer);
```

```
502     dpi_reordering_tcp_delete_all_fragments(&(to_delete->infos.tracking));
503
504 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
505     if(likely(to_delete>=db->partitions[partition_id].partition.
506                  memory_chunk_lower_bound &&
507                 to_delete<db->partitions[partition_id].partition.
508                 memory_chunk_upper_bound)){
509         debug_print("%s\n", "[flow_table.c]: Reinserting the flow"
510                     " in the pool.");
511         db->partitions[partition_id].partition.pool
512             [db->partitions[partition_id].partition.pool_size]=
513             to_delete-db->partitions[partition_id].partition.
514             memory_chunk_lower_bound;
515         ++db->partitions[partition_id].partition.pool_size;
516     }else{
517         debug_print("%s\n", "[flow_table.c]: Poolsize exceeded,"
518                     " removing the flow.");
519         v4_flow_free(to_delete);
520     }
521 #else
522     v4_flow_free(to_delete);
523 #endif
524 }
525
526 void mp_dpi_flow_table_delete_flow_v6(
527         dpi_flow_DB_v6_t* db,
528         dpi_flow_cleaner_callback* flow_cleaner_callback,
529         u_int16_t partition_id,
530         ipv6_flow_t* to_delete){
531     to_delete->prev->next=to_delete->next;
532     to_delete->next->prev=to_delete->prev;
533
534     if(flow_cleaner_callback)
535         (*(flow_cleaner_callback))(to_delete->infos.
536                                    tracking.flow_specific_user_data);
537     --db->partitions[partition_id].partition.informations.active_flows;
538     free(to_delete->infos.tracking.http_informations[0].temp_buffer);
539     free(to_delete->infos.tracking.http_informations[1].temp_buffer);
540     dpi_reordering_tcp_delete_all_fragments(&(to_delete->infos.tracking));
541
542 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
543     if(likely(to_delete>=db->partitions[partition_id].
544                 partition.memory_chunk_lower_bound &&
545                 to_delete<db->partitions[partition_id].
546                 partition.memory_chunk_upper_bound)){
547         debug_print("%s\n", "[flow_table.c]: Reinserting the"
548                     " flow in the pool.");
549         db->partitions[partition_id].partition.pool
550                 [db->partitions[partition_id].partition.pool_size]=
```

```
551                 to_delete-db->partitions[partition_id].partition.
552                 memory_chunk_lower_bound;
553             ++db->partitions[partition_id].partition.pool_size;
554         }else{
555             debug_print("%s\n", "[flow_table.c]: Poolsize exceeded,"
556                         " removing the flow.");
557             v6_flow_free(to_delete);
558         }
559     #else
560         v6_flow_free(to_delete);
561     #endif
562     }
563
564     void dpi_flow_table_delete_flow_v4(
565             dpi_flow_DB_v4_t* db,
566             dpi_flow_cleaner_callback* flow_cleaner_callback,
567             ipv4_flow_t* to_delete){
568         mp_dpi_flow_table_delete_flow_v4(db, flow_cleaner_callback,
569                                          0, to_delete);
570     }
571
572     void dpi_flow_table_delete_flow_v6(
573             dpi_flow_DB_v6_t* db,
574             dpi_flow_cleaner_callback* flow_cleaner_callback,
575             ipv6_flow_t* to_delete){
576         mp_dpi_flow_table_delete_flow_v6(db, flow_cleaner_callback,
577                                          0, to_delete);
578     }
579
580
581
582     #ifndef DPI_DEBUG
583     static
584     #endif
585     void dpi_flow_table_check_expiration_v4(
586             dpi_flow_DB_v4_t* db,
587             dpi_flow_cleaner_callback* flow_cleaner_callback,
588             u_int16_t partition_id,
589             u_int32_t current_time){
590         u_int32_t i;
591         ipv4_flow_t* current;
592         for(i=db->partitions[partition_id].partition.
593                 informations.lowest_index;
594             i<=db->partitions[partition_id].partition.
595                 informations.highest_index; i++){
596             /**
597              * Set the last timestamp for the sentinel node in such
598              * a way that we simplify the loop over the list.
599              **/
```

```
600         db->table[i].last_timestamp=current_time;
601 #if DPI_USE_MTF
602         while(current_time-db->table[i].prev->last_timestamp>
603                 DPI_FLOW_TABLE_MAX_IDLE_TIME){
604             mp_dpi_flow_table_delete_flow_v4(db,
605                                              flow_cleaner_callback,
606                                              partition_id,
607                                              db->table[i].prev);
608         }
609 #else
610         current=db->table[i].prev;
611         while(current!=&(db->table[i])){
612             if(current_time-db->table[i].prev->last_timestamp>
613                  DPI_FLOW_TABLE_MAX_IDLE_TIME){
614                 mp_dpi_flow_table_delete_flow_v4(db,
615                                                  flow_cleaner_callback,
616                                                  partition_id,
617                                                  db->table[i].prev);
618             }
619             current=current->prev;
620         }
621 #endif
622     }
623 }
624
625 #ifndef DPI_DEBUG
626 static
627 #endif
628 void dpi_flow_table_check_expiration_v6(
629         dpi_flow_DB_v6_t* db,
630         dpi_flow_cleaner_callback* flow_cleaner_callback,
631         u_int16_t partition_id,
632         u_int32_t current_time){
633
634
635     u_int32_t i;
636     ipv6_flow_t* current;
637     for(i=db->partitions[partition_id].partition.
638           informations.lowest_index;
639         i<=db->partitions[partition_id].partition.
640           informations.highest_index; i++){
641
642         /**
643          *  Set the last timestamp for the sentinel node in such a way
644          *  that we simplify the loop over the list.
645          **/
646         db->table[i].last_timestamp=current_time;
647 #if DPI_USE_MTF
648         while(current_time-db->table[i].prev->last_timestamp>
```

```
649                     DPI_FLOW_TABLE_MAX_IDLE_TIME){
650             mp_dpi_flow_table_delete_flow_v6(db,
651                                              flow_cleaner_callback,
652                                              partition_id,
653                                              db->table[i].prev);
654         }
655 #else
656         current=db->table[i].prev;
657         while(current!=&(db->table[i])){
658             if(current_time-db->table[i].prev->last_timestamp>
659                     DPI_FLOW_TABLE_MAX_IDLE_TIME){
660                 mp_dpi_flow_table_delete_flow_v6(db,
661                                                  flow_cleaner_callback,
662                                                  partition_id,
663                                                  db->table[i].prev);
664             }
665             current=current->prev;
666         }
667 #endif
668     }
669 }
670
671 #if 0
672 #include <sys/socket.h>
673 #include <netinet/in.h>
674 #include <arpa/inet.h>
675
676 void print_flow(ipv4_flow_t* iterator) {
677     struct in_addr src, dst;
678     src.s_addr = iterator->srcaddr;
679     dst.s_addr = iterator->dstaddr;
680     printf("%s -> ", inet_ntoa(src));
681     printf("%s : ", inet_ntoa(dst));
682
683     printf("%u -> ", ntohs(iterator->srcport));
684     printf("%u : ", ntohs(iterator->dstport));
685     printf("%u\n", iterator->l4prot);
686 }
687 #endif
688
689
690 ipv4_flow_t* mp_dpi_flow_table_find_or_create_flow_v4(
691         dpi_library_state_t* state, u_int16_t partition_id,
692         u_int32_t index, dpi_pkt_infos_t *pkt_infos){
693
694
695     dpi_flow_DB_v4_t *db=(dpi_flow_DB_v4_t*) state->db4;
696
697     ipv4_flow_t* head=&(db->table[index]);
```

```
698     ipv4_flow_t* iterator=head->next;
699
700     debug_print("%s\n", "[flow_table.c]: "
701                 "dpi_flow_table_find_or_create_flow_v4 invoked.");
702
703     /** Flow searching. **/
704     while(iterator!=head && !v4_equals(iterator, pkt_infos)){
705         iterator=iterator->next;
706     }
707
708     /**Flow not found, add it after the head.**/
709     if(iterator==head){
710         if(unlikely(db->partitions[partition_id].partition.
711                     informations.active_flows==
712                     db->partitions[partition_id].partition.
713                     informations.max_active_flows))
714             return NULL;
715 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
716         if(likely(db->partitions[partition_id].partition.pool_size!=0)){
717             debug_print("%s\n", "[flow_table.c]: New flow created, "
718                         "extracting an empty flow from the pool.");
719             /** Remove the flow from the pool. **/
720             u_int32_t p=db->partitions[partition_id].
721                         partition.pool[--db->partitions[partition_id].
722                         partition.pool_size];
723             iterator=&(db->partitions[partition_id].partition.
724                        memory_chunk_lower_bound[p]);
725         }else{
726             debug_print("%s\n", "[flow_table.c]: New flow created, "
727                         " pool exhausted, allocating a new flow.");
728             iterator=v4_flow_alloc();
729         }
730 #else
731         iterator=v4_flow_alloc();
732 #endif
733         assert(iterator);
734
735         /**Creates new flow and inserts it in the list.**/
736         iterator->srcaddr=pkt_infos->src_addr_t.ipv4_srcaddr;
737         iterator->dstaddr=pkt_infos->dst_addr_t.ipv4_dstaddr;
738         iterator->srcport=pkt_infos->srcport;
739         iterator->dstport=pkt_infos->dstport;
740         iterator->l4prot=pkt_infos->l4prot;
741         iterator->prev=head;
742         iterator->next=head->next;
743         iterator->prev->next=iterator;
744         iterator->next->prev=iterator;
745         dpi_init_flow_infos(state, &(iterator->infos), pkt_infos->l4prot);
746
```

```
747         ++db->partitions[partition_id].partition.informations.
748               active_flows;
749     }
750 #if DPI_USE_MTF
751     else if(iterator->prev!=head){
752         /**
753          * Remove the flow from the current position. It will be inserted
754          * in the first position (Move to front). In this way collisions
755          * lists are sorted from the highest to the lowest 'last update'
756          * timestamp.
757          **/
758         iterator->prev->next=iterator->next;
759         iterator->next->prev=iterator->prev;
760         iterator->prev=head;
761         iterator->next=head->next;
762         iterator->prev->next=iterator;
763         iterator->next->prev=iterator;
764     }
765 #endif
766
767     iterator->last_timestamp=pkt_infos->processing_time;
768
769     if(unlikely(pkt_infos->processing_time-db->partitions[partition_id].
770                 partition.informations.last_walk>=
771                 DPI_FLOW_TABLE_WALK_TIME)){
772         dpi_flow_table_check_expiration_v4((dpi_flow_DB_v4_t*) state->db4,
773                                            state->flow_cleaner_callback,
774                                            partition_id,
775                                            pkt_infos->processing_time);
776         db->partitions[partition_id].partition.informations.last_walk=
777                                            pkt_infos->processing_time;
778     }
779
780     if(iterator->srcaddr==pkt_infos->src_addr_t.ipv4_srcaddr &&
781        iterator->srcport==pkt_infos->srcport){
782         pkt_infos->direction=0;
783     }else{
784         pkt_infos->direction=1;
785     }
786
787     return iterator;
788 }
789
790
791 u_int32_t dpi_compute_v4_hash_function(
792         dpi_flow_DB_v4_t *db,
793         const dpi_pkt_infos_t* const pkt_infos){
794 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_FNV_HASH
795     u_int32_t row=v4_fnv_hash_function(pkt_infos)%db->total_size;
```

```
796 #elif DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
797     u_int32_t row=v4_hash_murmur3(pkt_infos, db->seed)%db->total_size;
798 #elif DPI_FLOW_TABLE_HASH_VERSION == DPI_BKDR_HASH
799     u_int32_t row=v4_hash_function_bkdr(pkt_infos)%db->total_size;
800 #else /** Default hash function is the simplest one. **/
801     u_int32_t row=v4_hash_function_simple(pkt_infos)%db->total_size;
802 #endif
803     return row;
804 }
805
806 ipv4_flow_t* dpi_flow_table_find_or_create_flow_v4(
807         dpi_library_state_t* state,
808         dpi_pkt_infos_t* pkt_infos){
809     return mp_dpi_flow_table_find_or_create_flow_v4(
810             state, 0,
811             dpi_compute_v4_hash_function(
812                     (dpi_flow_DB_v4_t*) state->db4, pkt_infos),
813                     pkt_infos);
814 }
815
816 ipv6_flow_t* mp_dpi_flow_table_find_or_create_flow_v6(
817         dpi_library_state_t* state,
818         u_int16_t partition_id,
819         u_int32_t index,
820         dpi_pkt_infos_t* pkt_infos){
821     dpi_flow_DB_v6_t *db=(dpi_flow_DB_v6_t*) state->db6;
822
823     ipv6_flow_t* head=&(db->table[index]);
824     ipv6_flow_t* iterator=head->next;
825
826     debug_print("%s\n", "[flow_table.c]: "
827                 "dpi_flow_table_find_or_create_flow_v6 invoked.");
828
829     /** Flow searching. **/
830     while(iterator!=head && !v6_equals(iterator, pkt_infos)){
831         iterator=iterator->next;
832     }
833
834     /**Flow not found, add it after the head.**/
835     if(iterator==head){
836         if(unlikely(db->partitions[partition_id].partition.
837                     informations.active_flows==
838                     db->partitions[partition_id].partition.
839                     informations.max_active_flows))
840             return NULL;
841
842 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
843         if(likely(db->partitions[partition_id].partition.pool_size!=0)){
844             debug_print("%s\n", "[flow_table.c]: New flow created,"
```

```
845                         " extracting an empty flow from the pool.");
846             u_int32_t p=db->partitions[partition_id].partition.pool
847                         [--db->partitions[partition_id].
848                         partition.pool_size];
849             iterator=&(db->partitions[partition_id].partition.
850                        memory_chunk_lower_bound[p]);
851         }else{
852             debug_print("%s\n", "[flow_table.c]: New flow created, "
853                         "pool exhausted, allocating a new flow.");
854             iterator=v6_flow_alloc();
855         }
856 #else
857         iterator=v6_flow_alloc();
858 #endif
859         assert(iterator);
860
861         /**Creates new flow and inserts it in the list.**/
862         iterator->srcaddr=pkt_infos->src_addr_t.ipv6_srcaddr;
863         iterator->dstaddr=pkt_infos->dst_addr_t.ipv6_dstaddr;
864         iterator->srcport=pkt_infos->srcport;
865         iterator->dstport=pkt_infos->dstport;
866         iterator->l4prot=pkt_infos->l4prot;
867         iterator->prev=head;
868         iterator->next=head->next;
869         iterator->prev->next=iterator;
870         iterator->next->prev=iterator;
871         dpi_init_flow_infos(state, &(iterator->infos), pkt_infos->l4prot);
872
873         ++db->partitions[partition_id].partition.informations.
874                                                 active_flows;
875     }
876 #if DPI_USE_MTF
877     else if(iterator->prev!=head){
878         /**
879          * Remove the flow from the current position. It will be inserted
880          * in the first position (Move to front). In this way collisions
881          * lists are sorted from the highest to the lowest last update
882          * timestamp.
883          **/
884         debug_print("%s\n", "[flow_table.c]: Flow already exists, "
885                     "move to front.");
886         iterator->prev->next=iterator->next;
887         iterator->next->prev=iterator->prev;
888         iterator->prev=head;
889         iterator->next=head->next;
890         iterator->prev->next=iterator;
891         iterator->next->prev=iterator;
892     }
893 #endif
```

```
894
895     iterator->last_timestamp=pkt_infos->processing_time;
896
897     if(unlikely(pkt_infos->processing_time-
898                db->partitions[partition_id].partition.
899                informations.last_walk>=DPI_FLOW_TABLE_WALK_TIME)){
900         dpi_flow_table_check_expiration_v6(
901                 (dpi_flow_DB_v6_t*) state->db6,
902                 state->flow_cleaner_callback,
903                 partition_id,
904                 pkt_infos->processing_time);
905         db->partitions[partition_id].partition.informations.last_walk=
906                 pkt_infos->processing_time;
907     }
908
909     if(dpi_v6_addresses_equal(iterator->srcaddr,
910                               pkt_infos->src_addr_t.ipv6_srcaddr) &&
911                               iterator->srcport==pkt_infos->srcport){
912         pkt_infos->direction=0;
913     }else
914         pkt_infos->direction=1;
915
916     return iterator;
917 }
918
919
920 u_int32_t dpi_compute_v6_hash_function(
921         dpi_flow_DB_v6_t *db,
922         const dpi_pkt_infos_t* const pkt_infos){
923 #if DPI_FLOW_TABLE_HASH_VERSION == DPI_FNV_HASH
924     u_int32_t row=v6_fnv_hash_function(pkt_infos) % db->total_size;
925 #elif DPI_FLOW_TABLE_HASH_VERSION == DPI_MURMUR3_HASH
926     u_int32_t row=v6_hash_murmur3(pkt_infos, db->seed)%db->total_size;
927 #elif DPI_FLOW_TABLE_HASH_VERSION == DPI_BKDR_HASH
928     u_int32_t row=v6_hash_function_bkdr(pkt_infos)%db->total_size;
929 #else /** Default hash function is the simplest one. **/
930     u_int32_t row=v6_hash_function_simple(pkt_infos)%db->total_size;
931 #endif
932     return row;
933 }
934
935 ipv6_flow_t* dpi_flow_table_find_or_create_flow_v6(
936         dpi_library_state_t* state,
937         dpi_pkt_infos_t* pkt_infos){
938     return mp_dpi_flow_table_find_or_create_flow_v6(
939             state, 0,
940             dpi_compute_v6_hash_function(
941                     (dpi_flow_DB_v6_t*) state->db6, pkt_infos),
942                     pkt_infos);
```

```
943 }
944
945
946
947 /**
948  * Search for a flow in the table.
949  * @param state A pointer to the state of the library.
950  * @param index The hash index of the flow to search.
951  * @param pkt_infos The L3 and L4 packet's parsed informations.
952  * @return A pointer to the flow if it is present, NULL otherwise.
953  */
954 ipv4_flow_t* dpi_flow_table_find_flow_v4(
955         dpi_library_state_t* state,
956         u_int32_t index,
957         dpi_pkt_infos_t* pkt_infos){
958     dpi_flow_DB_v4_t *db=(dpi_flow_DB_v4_t*) state->db4;
959
960     ipv4_flow_t* head=&(db->table[index]);
961     ipv4_flow_t* iterator=head->next;
962
963     /** Flow searching. **/
964     while(iterator!=head && !v4_equals(iterator, pkt_infos)){
965         iterator=iterator->next;
966     }
967
968     if(iterator==head)
969         return NULL;
970     else
971         return iterator;
972 }
973
974 /**
975  * Search for a flow in the table.
976  * @param state A pointer to the state of the library.
977  * @param index The hash index of the flow to search.
978  * @param pkt_infos The L3 and L4 packet's parsed informations.
979  * @return A pointer to the flow if it is present, NULL otherwise.
980  */
981 ipv6_flow_t* dpi_flow_table_find_flow_v6(
982         dpi_library_state_t* state,
983         u_int32_t index,
984         dpi_pkt_infos_t* pkt_infos){
985     dpi_flow_DB_v6_t *db=(dpi_flow_DB_v6_t*) state->db6;
986
987     ipv6_flow_t* head=&(db->table[index]);
988     ipv6_flow_t* iterator=head->next;
989
990     /** Flow searching. **/
991     while(iterator!=head && !v6_equals(iterator, pkt_infos)){
```

```
992         iterator=iterator->next;
993     }
994
995     if(iterator==head)
996         return NULL;
997     else
998         return iterator;
999 }
1000
1001
1002
1003 void dpi_flow_table_delete_v4(
1004         dpi_flow_DB_v4_t* db,
1005         dpi_flow_cleaner_callback* flow_cleaner_callback){
1006
1007
1008     u_int32_t i;
1009     u_int16_t j;
1010     if(db!=NULL){
1011         if(db->table!=NULL){
1012             for(j=0; j<db->num_partitions; ++j){
1013                 for(i=db->partitions[j].partition.informations.
1014                           lowest_index;
1015                     i<=db->partitions[j].partition.informations.
1016                           highest_index; ++i){
1017                     while(db->table[i].next!=&(db->table[i])){
1018                         mp_dpi_flow_table_delete_flow_v4(
1019                                 db, flow_cleaner_callback,
1020                                 j, db->table[i].next);
1021                     }
1022                 }
1023 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
1024 #if DPI_NUMA_AWARE
1025                 numa_free(db->partitions[j].partition.
1026                                 memory_chunk_lower_bound,
1027                           sizeof(ipv6_flow_t)*db->individual_pool_size);
1028                 numa_free(db->partitions[j].partition.
1029                                  pool,
1030                           sizeof(u_int32_t)*db->individual_pool_size);
1031 #else
1032                 free(db->partitions[j].partition.
1033                          memory_chunk_lower_bound);
1034                 free(db->partitions[j].partition.pool);
1035 #endif
1036 #endif
1037             }
1038         }
1039
1040
```

```
1041
1042 #if DPI_NUMA_AWARE
1043         numa_free(db->partitions,
1044                   sizeof(dpi_flow_DB_v4_partition_t)*db->num_partitions);
1045         numa_free(db->table, sizeof(ipv4_flow_t)*db->total_size);
1046 #else
1047         free(db->partitions);
1048         free(db->table);
1049 #endif
1050         free(db);
1051     }
1052 }
1053
1054 void dpi_flow_table_delete_v6(
1055         dpi_flow_DB_v6_t* db,
1056         dpi_flow_cleaner_callback* flow_cleaner_callback){
1057     u_int32_t i;
1058     u_int16_t j;
1059     if(db!=NULL){
1060         if(db->table!=NULL){
1061             for(j=0; j<db->num_partitions; ++j){
1062                 for(i=db->partitions[j].partition.
1063                           informations.lowest_index;
1064                     i<=db->partitions[j].partition.
1065                            informations.highest_index; ++i){
1066                     while(db->table[i].next!=&(db->table[i])){
1067                         mp_dpi_flow_table_delete_flow_v6(
1068                                 db, flow_cleaner_callback,
1069                                 j, db->table[i].next);
1070                     }
1071                 }
1072 #if DPI_FLOW_TABLE_USE_MEMORY_POOL
1073 #if DPI_NUMA_AWARE
1074                 numa_free(db->partitions[j].partition.
1075                               memory_chunk_lower_bound,
1076                           sizeof(ipv4_flow_t)*db->individual_pool_size);
1077                 numa_free(db->partitions[j].partition.pool,
1078                           sizeof(u_int32_t)*db->individual_pool_size);
1079 #else
1080                 free(db->partitions[j].partition.
1081                          memory_chunk_lower_bound);
1082                 free(db->partitions[j].partition.pool);
1083 #endif
1084 #endif
1085             }
1086         }
1087
1088 #if DPI_NUMA_AWARE
1089         numa_free(db->table, sizeof(ipv6_flow_t)*db->total_size);
```

```
        numa_free(db->partitions,
                  sizeof(dpi_flow_DB_v6_partition_t)*db->num_partitions);
#else
        free(db->partitions);
        free(db->table);
#endif
        free(db);
    }
}
```

A.3 IPv4 defragmentation

```
/*
 * reassembly.h
 *
 *  Created on: 05/10/2012
 *  Author: Daniele De Sensi (d.desensi@gmail.com)
 *
 *  Contains data structures used for both IPv4 and IPv6
 *  fragmentation and for TCP stream reassembly. Here for
 *  'fragments' we mean both the IP fragments
 *  and the TCP segments.
 */

#ifndef REASSEMBLY_H_
#define REASSEMBLY_H_

#include <sys/types.h>
#include "utils.h"

#ifdef __cplusplus
extern "C" {
#endif

typedef struct dpi_reassembly_timer dpi_reassembly_timer_t;
typedef struct dpi_reassembly_fragment dpi_reassembly_fragment_t;


/**If tail insertion, then the head will be the first to expire. **/
struct dpi_reassembly_timer{
    dpi_reassembly_timer_t *prev;
    dpi_reassembly_timer_t *next;
    void* data;
    u_int32_t expiration_time;
};

/* Describe an IP fragment (or TCP segment). */
struct dpi_reassembly_fragment{
```

```
37      /* Offset of fragment. */
38      u_int32_t offset;
39      /* Last byte of data in fragment. */
40      u_int32_t end;
41      /**
42       * This is needed because when a segment contains a FIN,
43       * then the expected sequence number in the other direction
44       * will be incremented by one.
45       */
46      u_int8_t tcp_fin:1;
47      /* Pointer into real fragment data. */
48      unsigned char* ptr;
49      /* Linked list pointers to the other fragments. */
50      dpi_reassembly_fragment_t *next;
51      dpi_reassembly_fragment_t *prev;
52  };
53
54
55
56  /**
57   * Returns 1 if the sequence number x is before y, 0 otherwise.
58   * @param x First sequence number.
59   * @param y Second sequence number.
60   * @return 1 if x is before y, 0 otherwise.
61   */
62  u_int8_t dpi_reassembly_before(u_int32_t x, u_int32_t y);
63
64
65  /**
66   * Returns 1 if the sequence number x is before or equal y, 0 otherwise.
67   * @param x First sequence number.
68   * @param y Second sequence number.
69   * @return 1 if x is before or equal y, 0 otherwise.
70   */
71  u_int8_t dpi_reassembly_before_or_equal(u_int32_t x, u_int32_t y);
72
73  /**
74   * Returns 1 if the sequence number x is after y, 0 otherwise.
75   * @param x First sequence number.
76   * @param y Second sequence number.
77   * @return 1 if x is after y, 0 otherwise.
78   */
79  u_int8_t dpi_reassembly_after(u_int32_t x, u_int32_t y);
80
81  /**
82   * Returns 1 if the sequence number x is after or equal y, 0 otherwise.
83   * @param x First sequence number.
84   * @param y Second sequence number.
85   * @return 1 if x is after or equal y, 0 otherwise.
```

```
 */
u_int8_t dpi_reassembly_after_or_equal(u_int32_t x, u_int32_t y);

/**
 * Returns the length of a TCP segment (or IP fragment).
 * @param offset The offset where the segment starts.
 * @param end The last byte of the segment.
 * @return The length of the TCP segment (or IP fragment).
 */
u_int32_t dpi_reassembly_fragment_length(u_int32_t offset, u_int32_t end);

/**
 * Add a new timer to the list of IP reassembly timers.
 * @param head A pointer to the head of the timers list.
 * @param tail A pointer to the tail of the timers list.
 * @param timer The timer to insert.
 */
void dpi_reassembly_add_timer(dpi_reassembly_timer_t** head,
                              dpi_reassembly_timer_t** tail,
                              dpi_reassembly_timer_t* timer);

/**
 * Remove a timer to the list of IP reassembly timers.
 * @param head A pointer to the head of the timers list.
 * @param tail A pointer to the tail of the timers list.
 * @param timer The timer to remove.
 */
void dpi_reassembly_delete_timer(dpi_reassembly_timer_t** head,
                                 dpi_reassembly_timer_t** tail,
                                 dpi_reassembly_timer_t* timer);


/**
 * Insert a fragment in the correct position in the list of fragments,
 * considering overlaps, etc..
 * @param head The head of the list of fragments.
 * @param data The data contained in the fragment (without IP header).
 * @param offset The offset of this fragment.
 * @param end The end of this fragment.
 * @param bytes_removed The total number of bytes removed
 *                      (in the fragments) by this call.
 * @param bytes_inserted The total number of byte inserted
 *                       (in the fragments) by this call.
 *
 * @return The created fragment.
 */
dpi_reassembly_fragment_t* dpi_reassembly_insert_fragment(
        dpi_reassembly_fragment_t** head,
        const unsigned char* data,
```

```
135         u_int32_t offset, u_int32_t end, u_int32_t* bytes_removed,
136         u_int32_t* bytes_inserted);
137
138 /**
139  * See there is a train of contiguous fragments.
140  * @param head The pointer to the head of the list of fragments.
141  * @return 0 if there are missing fragments, 1 otherwise.
142  */
143 u_int8_t dpi_reassembly_ip_check_train_of_contiguous_fragments(
144         dpi_reassembly_fragment_t* head);
145
146 /**
147  * Compacts a train of contiguous fragments and returns it.
148  * @param head A pointer to the head of the train.
149  * @param where A pointer to a buffer where to put the data.
150  * @param len The data_length of the buffer where to put the data.
151  * @return The data_length of the recompacted data. If an error
152  * occurred (e.g. misbehaving packet), -1 is returned.
153  */
154 int32_t dpi_reassembly_ip_compact_fragments(
155         dpi_reassembly_fragment_t* head,
156         unsigned char** where,
157         u_int32_t len);
158
159 #ifdef __cplusplus
160 }
161 #endif
162
163 #endif /* REASSEMBLY_H_ */
```

```
1  /*
2   * reassembly.c
3   *
4   *  Created on: 06/10/2012
5   *  Author: Daniele De Sensi (d.desensi@gmail.com)
6   */
7
8  #include "reassembly.h"
9  #include "utils.h"
10 #include <stdlib.h>
11 #include <string.h>
12 #include <stdio.h>
13 #include <inttypes.h>
14
15 /**
16  * Add a new timer to the list of IP reassembly timers.
17  * @param head A pointer to the head of the timers list.
18  * @param tail A pointer to the tail of the timers list.
19  * @param timer The timer to insert.
```

```
20  */
21 void dpi_reassembly_add_timer(dpi_reassembly_timer_t** head,
22                               dpi_reassembly_timer_t** tail,
23                               dpi_reassembly_timer_t* timer){
24     if(*tail){
25         (*tail)->next=timer;
26         timer->prev=(*tail);
27         timer->next=NULL;
28         (*tail)=timer;
29     }else{
30         timer->prev=NULL;
31         timer->next=NULL;
32         (*tail)=(*head)=timer;
33     }
34 }
35
36 /**
37  * Remove a timer to the list of IP reassembly timers.
38  * @param head A pointer to the head of the timers list.
39  * @param tail A pointer to the tail of the timers list.
40  * @param timer The timer to remove.
41  */
42 void dpi_reassembly_delete_timer(dpi_reassembly_timer_t** head,
43                                  dpi_reassembly_timer_t** tail,
44                                  dpi_reassembly_timer_t* timer){
45     if(timer->prev)
46         timer->prev->next=timer->next;
47     else
48         (*head)=timer->next;
49
50     if(timer->next)
51         timer->next->prev=timer->prev;
52     else
53         (*tail)=timer->prev;
54 }
55
56
57 /**
58  * Returns 1 if the sequence number x is before y, 0 otherwise.
59  * @param x First sequence number.
60  * @param y Second sequence number.
61  * @return 1 if x is before y, 0 otherwise.
62  */
63 u_int8_t dpi_reassembly_before(u_int32_t x, u_int32_t y){
64     return x<y || (x-y)>DPI_TCP_MAX_IN_TRAVEL_DATA;
65 }
66
67
68 /**
```

```
69  * Returns 1 if the sequence number x is before or equal y, 0 otherwise.
70  * @param x First sequence number.
71  * @param y Second sequence number.
72  * @return 1 if x is before or equal y, 0 otherwise.
73  */
74 u_int8_t dpi_reassembly_before_or_equal(u_int32_t x, u_int32_t y){
75     return x<=y || (x-y)>=DPI_TCP_MAX_IN_TRAVEL_DATA;
76 }
77
78 /**
79  * Returns 1 if the sequence number x is after y, 0 otherwise.
80  * @param x First sequence number.
81  * @param y Second sequence number.
82  * @return 1 if x is after y, 0 otherwise.
83  */
84 u_int8_t dpi_reassembly_after(u_int32_t x, u_int32_t y){
85     return x>y || (y-x)>DPI_TCP_MAX_IN_TRAVEL_DATA;
86 }
87
88 /**
89  * Returns 1 if the sequence number x is after or equal y, 0 otherwise.
90  * @param x First sequence number.
91  * @param y Second sequence number.
92  * @return 1 if x is after or equal y, 0 otherwise.
93  */
94 u_int8_t dpi_reassembly_after_or_equal(u_int32_t x, u_int32_t y){
95     return x>=y || (y-x)>=DPI_TCP_MAX_IN_TRAVEL_DATA;
96 }
97
98 /**
99  * Returns the length of a TCP segment (or IP fragment).
100  * @param offset The offset where the segment starts.
101  * @param end The last byte of the segment.
102  * @return The length of the TCP segment (or IP fragment).
103  */
104 u_int32_t dpi_reassembly_fragment_length(u_int32_t offset, u_int32_t end){
105     if(end>=offset)
106         return end-offset;
107     else
108         return DPI_MAX_INT_32-offset+1+end;
109 }
110
111
112 #ifndef DPI_DEBUG
113 static
114 #endif
115 /**
116  * Builds a new fragment.
117  * @param offset The offset where the fragment start.
```

```
118  * @param end The last byte of the fragment.
119  * @param ptr The content of the fragment.
120  * @return The created fragment.
121  */
122 dpi_reassembly_fragment_t* dpi_reassembly_create_fragment(
123         u_int32_t offset, u_int32_t end, const unsigned char *ptr){
124     dpi_reassembly_fragment_t* fragment;
125     fragment=(dpi_reassembly_fragment_t*)
126                 calloc(1, sizeof(dpi_reassembly_fragment_t));
127     if(unlikely(fragment==NULL))
128         return NULL;
129
130     /* Fill in the structure. */
131     fragment->offset=offset;
132     fragment->end=end;
133     fragment->tcp_fin=0;
134
135     u_int32_t length=dpi_reassembly_fragment_length(offset, end);
136     fragment->ptr=(unsigned char*) malloc(sizeof(unsigned char)*(length));
137
138     if(unlikely(fragment->ptr==NULL)){
139         free(fragment);
140         return NULL;
141     }
142     memcpy(fragment->ptr, ptr, length);
143     return fragment;
144 }
145
146
147 /**
148  * Insert a fragment in the correct position in the list of fragments,
149  * considering overlaps, etc..
150  * @param head The head of the list of fragments.
151  * @param data The data contained in the fragment (without IP header).
152  * @param offset The offset of this fragment.
153  * @param end The end of this fragment.
154  * @param bytes_removed The total number of bytes removed (in the ↙
       ↳ fragments) by this call.
155  * @param bytes_inserted The total number of byte inserted (in the ↙
       ↳ fragments) by this call.
156  *
157  * @return The created fragment.
158  */
159 dpi_reassembly_fragment_t* dpi_reassembly_insert_fragment(
160         dpi_reassembly_fragment_t** head,
161         const unsigned char* data,
162         u_int32_t offset, u_int32_t end, u_int32_t* bytes_removed,
163         u_int32_t* bytes_inserted){
164     dpi_reassembly_fragment_t *prev, *next, *iterator, *tmp;
```

```
165     u_int32_t fragment_start=0;
166     *bytes_removed=0;
167     *bytes_inserted=0;
168
169     /*
170      * Find the position in which this fragment must be put.
171      */
172
173     prev=NULL;
174     for(next=(*head); next!=NULL; next=next->next){
175         if(dpi_reassembly_after_or_equal(next->offset, offset)){
176             break;
177         }
178         prev=next;
179     }
180
181     /**
182      *  If there are no overlaps, I should put the fragment after prev
183      *  and before next.
184      *
185      *  In the general case we will have (in sequence, some of this points
186      *  may be missing):
187      *   1. Partially or total overlap with the previous fragment.
188      *      (The overlapped part will be left in the old data).
189      *   2. A gap in which we have to insert a part of the new fragment.
190      *      (The new part will be inserted from the new fragment).
191      *   3. A train of completely overlapped fragments.
192      *      (Will be replaced by new fragment)
193      *   4. A partially overlapped fragment. (The overlapped part will
194      *      be left in the old data).
195      */
196
197     /**
198      *  Check for overlaps with the previous fragment. We will left the
199      *  previous fragment untouched and in case copy the new part.
200      **/
201     if(prev!=NULL && dpi_reassembly_before(offset, prev->end)){
202         /** If is contained in the previous fragment don't do anything.**/
203         if(dpi_reassembly_before_or_equal(end, prev->end)){
204             return NULL;
205         }
206         offset=prev->end;
207         fragment_start=dpi_reassembly_fragment_length(offset, prev->end);
208     }
209
210     /**
211      *  Check for overlaps with the subsequent segments (can be more
212      *  than one). If there are completely overlapped segments, they
213      *  will be removed and replaced by the new fragment. The
```

```
214      *  alternative was to add only the missing parts, but this could
215      *  lead to an high number of small fragments. Anyway, to avoid
216      *  this and to have a more compact list of fragments we incur in
217      *  the cost of reallocate and copy again already present data.
218      */
219     for(iterator=next; iterator!=NULL; ){
220         /**
221          * This data doesn't overlaps. The data are ordered so the
222          * successive will not overlap too.
223          **/
224         if(dpi_reassembly_after_or_equal(iterator->offset, end)){
225             break;
226         }
227
228         /**
229          * If the fragment 'iterator' is completely overlapped by the
230          * new fragment, then remove it.
231          **/
232         if(dpi_reassembly_after_or_equal(end, iterator->end)){
233             if (iterator->prev!=NULL){
234                 iterator->prev->next=iterator->next;
235             }else{
236                 (*head)=iterator->next;
237             }
238
239             if (iterator->next!=NULL)
240                 iterator->next->prev=iterator->prev;
241
242             tmp=iterator->next;
243
244             (*bytes_removed)+=dpi_reassembly_fragment_length(
245                     iterator->offset, iterator->end);
246
247             free(iterator->ptr);
248             free(iterator);
249
250             /**
251              * We deleted iterator, so we put the pointer back to the
252              * previous fragment. The for loop will then go to the
253              * successive data.
254              **/
255             iterator=tmp;
256             /** We have also to update next. **/
257             next=tmp;
258         }else{
259         /**
260          * If it is partially overlapped, then we left the overlapped
261          * part in the old fragment.
262          **/
```

```
263             end=iterator->offset;
264             iterator=iterator->next;
265         }
266
267     }
268
269     /** Insert the new fragment in the list. **/
270     tmp=dpi_reassembly_create_fragment(offset, end, data+fragment_start);
271
272     if(unlikely(tmp==NULL))
273         return NULL;
274
275     (*bytes_inserted)+=dpi_reassembly_fragment_length(offset, end);
276
277
278     tmp->prev=prev;
279     tmp->next=next;
280     if(prev)
281         prev->next=tmp;
282     else
283         (*head)=tmp;
284
285     if(next)
286         next->prev=tmp;
287     return tmp;
288 }
289
290
291 /**
292  * See there is a train of contiguous fragments.
293  * @param head The pointer to the head of the list of fragments.
294  * @return 0 if there are missing fragments, 1 otherwise.
295  */
296 u_int8_t dpi_reassembly_ip_check_train_of_contiguous_fragments(
297             dpi_reassembly_fragment_t* head){
298     if(!head) return 0;
299     u_int16_t offset=0;
300
301     /* Check all fragment offsets to see if they connect. */
302     while(head){
303         if(head->offset>offset)
304             return 0;
305         offset=head->end;
306         head=head->next;
307     }
308     return 1;
309 }
310
311
```

```
312 /**
313  * Compacts a train of contiguous fragments and returns it.
314  * @param head A pointer to the head of the train.
315  * @param where A pointer to a buffer where to put the data.
316  * @param len The data_length of the buffer where to put the data.
317  * @return The data_length of the recompacted data. If an error
318  *                          occurred (e.g. misbehaving packet),
319  *                          -1 is returned.
320  */
321 int32_t dpi_reassembly_ip_compact_fragments(
322         dpi_reassembly_fragment_t* head,
323         unsigned char** where,
324         u_int32_t len){
325     /* Copy the data portions of all fragments into the new buffer. */
326     u_int32_t fragment_length,count=0;
327     while(head!=NULL){
328         fragment_length=head->end-head->offset;
329         if(unlikely(head->offset+fragment_length>len)){
330             return -1;
331         }
332         memcpy(((*where)+head->offset), head->ptr, fragment_length);
333         count+=fragment_length;
334         head=head->next;
335     }
336     return count;
337 }
```

```
 1 /*
 2  * reordering.h
 3  *
 4  *  Created on: 03/10/2012
 5  *  Author: Daniele De Sensi (d.desensi@gmail.com)
 6  */
 7
 8 #ifndef REORDERING_H_
 9 #define REORDERING_H_
10
11 /* To get the 'fragment offset' part. **/
12 #define DPI_IPv4_FRAGMENTATION_OFFSET_MASK 0x1FFF
13 /* Flag: "More Fragments" */
14 #define DPI_IPv4_FRAGMENTATION_MF          0x2000
15
16 typedef struct dpi_ipv4_fragmentation_state
17                dpi_ipv4_fragmentation_state_t;
18
19 #ifdef __cplusplus
20 extern "C" {
21 #endif
22
```

```
23 /**
24  * Enables the IPv4 defragmentation.
25  * @param table_size  The size of the table used to
26  *                    store the fragments.
27  * @return            A pointer to the IPv4 defragmentation handle.
28  */
29 dpi_ipv4_fragmentation_state_t* dpi_reordering_enable_ipv4_fragmentation(
30                                 u_int16_t table_size);
31
32 /**
33  * Sets the maximum amount of memory that can be used to store
34  * fragments generated by the same source.
35  * @param frag_state A pointer to the IPv4 degragmentation handle.
36  * @param per_host_memory_limit  The maximum amount of memory that can
37  *                               be used to store fragments generated
38  *                               by the same source.
39  */
40 void dpi_reordering_ipv4_fragmentation_set_per_host_memory_limit(
41         dpi_ipv4_fragmentation_state_t *frag_state,
42         u_int32_t per_host_memory_limit);
43
44 /**
45  * Sets the maximum (global) amount of memory that can be used for
46  * defragmentation purposes.
47  * @param frag_state           A pointer to the IPv4 defragmentation
48  *                             handle.
49  * @param total_memory_limit   The global memory limit.
50  */
51 void dpi_reordering_ipv4_fragmentation_set_total_memory_limit(
52         dpi_ipv4_fragmentation_state_t *frag_state,
53         u_int32_t total_memory_limit);
54
55 /**
56  * Sets the maximum amount of time (seconds) which can elapse before
57  * the complete defragmentation of the datagram.
58  * @param frag_state        A pointer to the IPv4 defragmentation handle.
59  * @param timeout_seconds   The timeout (seconds).
60  */
61 void dpi_reordering_ipv4_fragmentation_set_reassembly_timeout(
62         dpi_ipv4_fragmentation_state_t *frag_state,
63         u_int8_t timeout_seconds);
64
65 /**
66  * Disables the IPv4 fragmentation and deallocates the handle.
67  * @param frag_state  A pointer to the IPv4 defragmentation handle.
68  */
69 void dpi_reordering_disable_ipv4_fragmentation(
70         dpi_ipv4_fragmentation_state_t* frag_state);
71
```

```
72  /**
73   * Reassemble the IP datagram if it is fragmented. It is thread safe
74   * if and only if DPI_THREAD_SAFETY_ENABLED == 1.
75   * @param state The state for fragmentation support.
76   * @param data A pointer to the beginning of IP header.
77   * @param current_time The current time, in seconds.
78   * @param offset The data offset specified in the ip header.
79   * @param more_fragments 1 if the MF flag is set, 0 otherwise.
80   * @param tid The thread id.
81   * @return Returns NULL if the datagram is a fragment but doesn't fill an
82   *         hole. In this case, the content of the datagram has been
83   *         copied, so if the user wants, he can release the resources
84   *         used to store the received packet.
85   *
86   *         Returns A pointer to the recomposed datagram if the datagram
87   *         is the last fragment of a bigger datagram. This pointer will be
88   *         different from data. The user should free() this pointer when
89   *         it is no more needed.
90   */
91  unsigned char* dpi_reordering_manage_ipv4_fragment(
92          dpi_ipv4_fragmentation_state_t *state,
93          const unsigned char* data,
94          u_int32_t current_time, u_int16_t offset,
95          u_int8_t more_fragments, int tid);
96
97  #ifdef __cplusplus
98  }
99  #endif
100 #endif /* REORDERING_H_ */
```

```
1   /*
2    * reordering.c
3    *
4    *  Support for IPv4 reassembly. Any modification done here should be ↙
       ↳ reflected on IPv6 reassembly.
5    *  Created on: 02/10/2012
6    *  Author: Daniele De Sensi (desensi@cli.di.unipi.it)
7    */
8
9   #include <sys/types.h>
10  #include <strings.h>
11  #include <string.h>
12  #include <stdlib.h>
13  #include <stdio.h>
14  #include <netinet/ip.h>
15  #include "ipv4_reassembly.h"
16  #include "reassembly.h"
17  #if DPI_THREAD_SAFETY_ENABLED == 1
18  #include <ff/spin-lock.hpp>
```

```
19 #endif
20
21
22 #define DPI_DEBUG_FRAGMENTATION_v4 0
23
24
25 #define debug_print(fmt, ...)                    \
26             do { if (DPI_DEBUG_FRAGMENTATION_v4) \
27             fprintf(stderr, fmt, __VA_ARGS__); } while (0)
28
29 #define DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE 65535
30 #define DPI_IPv4_FRAGMENTATION_MINIMUM_MTU 576
31
32
33 typedef struct dpi_ipv4_fragmentation_flow
34                dpi_ipv4_fragmentation_flow_t;
35 typedef struct dpi_ipv4_fragmentation_source
36                dpi_ipv4_fragmentation_source_t;
37
38
39 /* Is for a specific <(Source), Dest, Protocol, Identifier>. */
40 typedef struct dpi_ipv4_fragmentation_flow {
41     /* Pointer to IP header for the reassembled datagram. */
42     struct iphdr *iph;
43     /* Total data_length of the final datagram (without IP header). */
44     u_int16_t len;
45     u_int16_t id;
46     u_int32_t dest_ip;
47     u_int8_t protocol;
48     /* Linked list of received fragments. */
49     dpi_reassembly_fragment_t *fragments;
50     /*
51      * For a given source, pointer to the previous and next flows
52      * started from that source
53      */
54     dpi_ipv4_fragmentation_flow_t *next;
55     dpi_ipv4_fragmentation_flow_t *prev;
56     dpi_reassembly_timer_t timer;
57     dpi_ipv4_fragmentation_source_t *source;
58 }dpi_ipv4_fragmentation_flow_t;
59
60
61
62 /**
63  *  For each source IP which have fragments "in fly" stores the
64  *  fragments and the memory used by this flow.
65  **/
66 typedef struct dpi_ipv4_fragmentation_source{
67     dpi_ipv4_fragmentation_flow_t *flows;
```

```
68      u_int32_t source_used_mem;
69      u_int32_t src_ip;
70      u_int16_t row;
71      dpi_ipv4_fragmentation_source_t* prev;
72      dpi_ipv4_fragmentation_source_t* next;
73  }dpi_ipv4_fragmentation_source_t;
74
75
76  typedef struct dpi_ipv4_fragmentation_state{
77      /**
78       * Is an hash table containing associations between source IP
79       * address and fragments generated by that address.
80       **/
81      dpi_ipv4_fragmentation_source_t** table;
82      u_int32_t total_used_mem;
83      u_int16_t table_size;
84
85      /** List of timers. **/
86      dpi_reassembly_timer_t *timer_head, *timer_tail;
87
88      /** Memory limits. **/
89      u_int32_t per_source_memory_limit;
90      u_int32_t total_memory_limit;
91
92      /** Reassembly timeout. **/
93      u_int8_t timeout;
94  #if DPI_THREAD_SAFETY_ENABLED == 1
95      ff::CLHSpinLock lock;
96  #endif
97  }dpi_ipv4_fragmentation_state_t;
98
99  #ifndef DPI_DEBUG
100 static
101 #if DPI_USE_INLINING == 1
102 inline
103 #endif
104 #endif
105 void dpi_ipv4_fragmentation_delete_source(
106         dpi_ipv4_fragmentation_state_t* state,
107         dpi_ipv4_fragmentation_source_t* source);
108
109
110
111 /**
112  * Enables the IPv4 defragmentation.
113  * @param table_size  The size of the table used to store the fragments.
114  * @return            A pointer to the IPv4 defragmentation handle.
115  */
116 dpi_ipv4_fragmentation_state_t* dpi_reordering_enable_ipv4_fragmentation(
```

```
                                u_int16_t table_size){
    dpi_ipv4_fragmentation_state_t* r=(dpi_ipv4_fragmentation_state_t*)
                        calloc(1, sizeof(dpi_ipv4_fragmentation_state_t));
    if(unlikely(r==NULL))
        return NULL;
    r->table_size=table_size;
    r->table=(dpi_ipv4_fragmentation_source_t**)
            malloc(table_size * sizeof(dpi_ipv4_fragmentation_source_t*));
    if(unlikely(r->table==NULL)){
        free(r);
        return NULL;
    }
    u_int16_t i;
    for(i=0; i<table_size; i++){
        r->table[i]=NULL;
    }
    r->timer_head=NULL;
    r->timer_tail=NULL;
    r->per_source_memory_limit=
            DPI_IPv4_FRAGMENTATION_DEFAULT_PER_HOST_MEMORY_LIMIT;
    r->total_memory_limit=
            DPI_IPv4_FRAGMENTATION_DEFAULT_TOTAL_MEMORY_LIMIT;
    r->timeout=
            DPI_IPv4_FRAGMENTATION_DEFAULT_REASSEMBLY_TIMEOUT;
    r->total_used_mem=0;
#if DPI_THREAD_SAFETY_ENABLED == 1
    bzero(&(r->lock), sizeof(ff::CLHSpinLock));
    ff::init_unlocked(&(r->lock));
#endif
    return r;
}



/**
 * Sets the maximum amount of memory that can be used to store
 * fragments generated
 * by the same source.
 * @param frag_state  A pointer to the IPv4 degragmentation handle.
 * @param per_host_memory_limit  The maximum amount of memory that can be
 *                               used to store fragments generated by the
 *                               same source.
 */
void dpi_reordering_ipv4_fragmentation_set_per_host_memory_limit(
        dpi_ipv4_fragmentation_state_t *frag_state,
        u_int32_t per_host_memory_limit){
    frag_state->per_source_memory_limit=per_host_memory_limit;
}

```

```
166 /**
167  * Sets the maximum (global) amount of memory that can be used for
168  * defragmentation purposes.
169  * @param frag_state A pointer to the IPv4 defragmentation handle.
170  * @param total_memory_limit   The global memory limit.
171  */
172 void dpi_reordering_ipv4_fragmentation_set_total_memory_limit(
173         dpi_ipv4_fragmentation_state_t *frag_state,
174         u_int32_t total_memory_limit){
175     frag_state->total_memory_limit=total_memory_limit;
176 }
177
178 /**
179  * Sets the maximum amount of time (seconds) which can elapse before the
180  * complete defragmentation of the datagram.
181  * @param frag_state        A pointer to the IPv4 defragmentation handle.
182  * @param timeout_seconds   The timeout (seconds).
183  */
184 void dpi_reordering_ipv4_fragmentation_set_reassembly_timeout(
185         dpi_ipv4_fragmentation_state_t *frag_state,
186         u_int8_t timeout_seconds){
187     frag_state->timeout=timeout_seconds;
188 }
189
190
191 /**
192  * Disables the IPv4 fragmentation and deallocates the handle.
193  * @param frag_state  A pointer to the IPv4 defragmentation handle.
194  */
195 void dpi_reordering_disable_ipv4_fragmentation(
196         dpi_ipv4_fragmentation_state_t* frag_state) {
197     if(frag_state==NULL) return;
198     dpi_ipv4_fragmentation_source_t *source,*tmp_source;
199     if(frag_state->table){
200         u_int16_t i;
201         for(i=0; i<frag_state->table_size; i++){
202             if(frag_state->table[i]){
203                 source=frag_state->table[i];
204                 while(source){
205                     tmp_source=source->next;
206                     dpi_ipv4_fragmentation_delete_source(
207                             frag_state, source);
208                     source=tmp_source;
209                 }
210             }
211         }
212         free(frag_state->table);
213     }
214 #if DPI_THREAD_SAFETY_ENABLED == 1
```

```
215     frag_state->lock.~CLHSpinLock();
216 #endif
217     free(frag_state);
218 }
219
220 #ifndef DPI_DEBUG
221 static
222 #if DPI_USE_INLINING == 1
223 inline
224 #endif
225 #endif
226 /** Robert Jenkins' 32 bit integer hash function. **/
227 u_int16_t dpi_ipv4_fragmentation_hash_function(
228         dpi_ipv4_fragmentation_state_t* state,
229         u_int32_t src_ip){
230     src_ip=(src_ip+0x7ed55d16)+(src_ip<<12);
231     src_ip=(src_ip^0xc761c23c)^(src_ip>>19);
232     src_ip=(src_ip+0x165667b1)+(src_ip<<5);
233     src_ip=(src_ip+0xd3a2646c)^(src_ip<<9);
234     src_ip=(src_ip+0xfd7046c5)+(src_ip<<3);
235     src_ip=(src_ip^0xb55a4f09)^(src_ip>>16);
236     return src_ip%state->table_size;
237 }
238
239
240 #ifndef DPI_DEBUG
241 static
242 #endif
243 void dpi_ipv4_fragmentation_delete_flow(
244         dpi_ipv4_fragmentation_state_t* state,
245         dpi_ipv4_fragmentation_flow_t* flow){
246     dpi_reassembly_fragment_t *frag,*temp_frag;
247
248     dpi_ipv4_fragmentation_source_t* source=flow->source;
249
250     source->source_used_mem-=sizeof(dpi_ipv4_fragmentation_flow_t);
251     state->total_used_mem-=sizeof(dpi_ipv4_fragmentation_flow_t);
252
253     /* Stop the timer and delete it. */
254     dpi_reassembly_delete_timer(&(state->timer_head),
255                                 &(state->timer_tail), &(flow->timer));
256
257     /* Release all fragment data. */
258     frag=flow->fragments;
259     while(frag){
260         temp_frag=frag->next;
261         source->source_used_mem-=(frag->end-frag->offset);
262         state->total_used_mem-=(frag->end-frag->offset);
263
```

```
264         free(frag->ptr);
265         free(frag);
266         frag=temp_frag;
267     }
268
269     /** Delete the IP header. **/
270     if(flow->iph){
271         u_int8_t header_length=(flow->iph->ihl&0x0f)*4;
272         free(flow->iph);
273         source->source_used_mem-=header_length;
274         state->total_used_mem-=header_length;
275     }
276
277     /*
278      * Remove the flow from the list of the flows. If no more flows
279      * for this source, then delete the source.
280      */
281     if(flow->prev==NULL){
282         source->flows=flow->next;
283         if(source->flows!=NULL)
284             source->flows->prev=NULL;
285     }else{
286         flow->prev->next=flow->next;
287         if(flow->next)
288             flow->next->prev=flow->prev;
289     }
290     free(flow);
291 }
292
293 #ifndef DPI_DEBUG
294 static
295 #if DPI_USE_INLINING == 1
296 inline
297 #endif
298 #endif
299 void dpi_ipv4_fragmentation_delete_source(
300         dpi_ipv4_fragmentation_state_t* state,
301         dpi_ipv4_fragmentation_source_t* source) {
302     u_int16_t row=source->row;
303
304     /** Delete all the flows belonging to this source. **/
305     dpi_ipv4_fragmentation_flow_t* flow=source->flows,*temp_flow;
306     while(flow){
307         temp_flow=flow->next;
308         dpi_ipv4_fragmentation_delete_flow(state, flow);
309         flow=temp_flow;
310     }
311
312     /** Delete this source from the list. **/
```

```
313     if (source->prev)
314         source->prev->next=source->next;
315     else
316         state->table[row]=source->next;
317
318     if(source->next)
319         source->next->prev=source->prev;
320
321     free(source);
322     state->total_used_mem-=sizeof(dpi_ipv4_fragmentation_source_t);
323 }
324
325
326 #ifndef DPI_DEBUG
327 static
328 #endif
329 /**
330  * Try to find the specific source. If it is not find, then creates it.
331  * @param state The state of the defragmentation module.
332  * @param addr The source address.
333  * @return A pointer to the source.
334  */
335 dpi_ipv4_fragmentation_source_t*
336         dpi_ipv4_fragmentation_find_or_create_source(
337                 dpi_ipv4_fragmentation_state_t* state, u_int32_t addr){
338     u_int16_t hash_index=dpi_ipv4_fragmentation_hash_function(state,
339                                                               addr);
340     dpi_ipv4_fragmentation_source_t *source,*head;
341
342     head=state->table[hash_index];
343
344     for(source=head; source!=NULL; source=source->next){
345         if(source->src_ip==addr){
346             return source;
347         }
348     }
349
350     /** Not found, so create it. **/
351     source=(dpi_ipv4_fragmentation_source_t*)
352             malloc(sizeof(dpi_ipv4_fragmentation_source_t));
353     if(unlikely(source==NULL)){
354         return NULL;
355     }
356     source->row=hash_index;
357     source->flows=NULL;
358     source->src_ip=addr;
359     source->source_used_mem=sizeof(dpi_ipv4_fragmentation_source_t);
360     state->total_used_mem+=sizeof(dpi_ipv4_fragmentation_source_t);
361
```

```
362
363     /** Insertion at the beginning of the list. **/
364     source->prev=NULL;
365     source->next=head;
366     if(head)
367         head->prev=source;
368     state->table[hash_index]=source;
369
370     return source;
371 }
372
373
374 /*
375  * Find the flow. If it is not found creates it.
376  * @return A pointer to the flow.
377  */
378 #ifndef DPI_DEBUG
379 static
380 #endif
381 dpi_ipv4_fragmentation_flow_t* dpi_ipv4_fragmentation_find_or_create_flow(
382         dpi_ipv4_fragmentation_state_t* state,
383         dpi_ipv4_fragmentation_source_t* source,
384         const struct iphdr* iph, u_int32_t current_time){
385     dpi_ipv4_fragmentation_flow_t* flow;
386     for(flow = source->flows; flow != NULL; flow = flow->next) {
387         /**
388          * The source is matched for sure because all the
389          * flows will have the same source.
390          **/
391         if(iph->id==flow->id && iph->daddr==flow->dest_ip &&
392             iph->protocol==flow->protocol) {
393             return flow;
394         }
395     }
396
397     /** Not found, create a new flow. **/
398     flow=(dpi_ipv4_fragmentation_flow_t*)
399             malloc(sizeof(dpi_ipv4_fragmentation_flow_t));
400     if(unlikely(flow==NULL)){
401         return NULL;
402     }
403
404     source->source_used_mem+=sizeof(dpi_ipv4_fragmentation_flow_t);
405     state->total_used_mem+=sizeof(dpi_ipv4_fragmentation_flow_t);
406
407
408     flow->fragments=NULL;
409     flow->source=source;
410     flow->len=0;
```

```
411     /* Add this entry to the queue of flows. */
412     flow->prev=NULL;
413     flow->next=source->flows;
414     if(flow->next)
415         flow->next->prev=flow;
416     source->flows=flow;
417     /* Set the timer. */
418     flow->timer.expiration_time=current_time+state->timeout;
419     flow->timer.data = flow;
420     dpi_reassembly_add_timer(&(state->timer_head), &(state->timer_tail),
421                              &(flow->timer));
422     /* Fragments will be added later. */
423     flow->fragments=NULL;
424     flow->iph=NULL;
425     flow->id=iph->id;
426     flow->dest_ip=iph->daddr;
427     flow->protocol=iph->protocol;
428     return flow;
429 }
430
431 /**
432  * @return A pointer to the recompacted datagram or NULL if
433  *         an error occurred.
434  */
435 #ifndef DPI_DEBUG
436 static
437 #endif
438 unsigned char* dpi_ipv4_fragmentation_build_complete_datagram(
439         dpi_ipv4_fragmentation_state_t* state,
440         dpi_ipv4_fragmentation_flow_t* flow){
441     unsigned char *pkt_beginning, *pkt_data;
442     struct iphdr *iph;
443     u_int16_t len;
444     int32_t count;
445
446     /* Allocate a new buffer for the datagram. */
447     u_int8_t ihl=(flow->iph->ihl&0x0f)*4;
448     len=flow->len;
449
450     dpi_ipv4_fragmentation_source_t* source=flow->source;
451
452     u_int32_t tot_len=ihl+len;
453
454     if(tot_len>DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE){
455         dpi_ipv4_fragmentation_delete_flow(state, flow);
456         if(source->flows==NULL)
457             dpi_ipv4_fragmentation_delete_source(state,source);
458         return NULL;
459     }
```

```
460
461
462     if(unlikely((pkt_beginning=(unsigned char*) malloc(ihl+len))==NULL)){
463         dpi_ipv4_fragmentation_delete_flow(state, flow);
464         if(source->flows==NULL)
465             dpi_ipv4_fragmentation_delete_source(state,source);
466         return NULL;
467     }
468
469     memcpy(pkt_beginning, ((unsigned char*) flow->iph), ihl);
470     pkt_data=pkt_beginning+ihl;
471
472     count=dpi_reassembly_ip_compact_fragments(flow->fragments,
473                                               &pkt_data, len);
474
475     /**
476      * We recompacted the flow (datagram), so now
477      * we can delete it.
478      **/
479     dpi_ipv4_fragmentation_delete_flow(state, flow);
480     if(source->flows==NULL)
481         dpi_ipv4_fragmentation_delete_source(state,source);
482
483     /**
484      * Misbehaving packet, real size is different from that
485      * obtained from the last fragment.
486      **/
487     if(count==-1){
488         free(pkt_beginning);
489         return NULL;
490     }
491
492     /** Put the correct informations in the IP header. **/
493     iph=(struct iphdr *) pkt_beginning;
494     iph->frag_off=0;
495     iph->tot_len=htons(ihl+count);
496     return pkt_beginning;
497 }
498
499
500 /**
501  * Reassemble the IP datagram if it is fragmented. It is thread safe if
502  * and only if DPI_THREAD_SAFETY_ENABLED == 1.
503  * @param state The state for fragmentation support.
504  * @param data A pointer to the beginning of IP header.
505  * @param current_time The current time, in seconds.
506  * @param offset The data offset specified in the ip header.
507  * @param more_fragments 1 if the MF flag is set, 0 otherwise.
508  * @param tid The thread id.
```

```
509  * @return Returns NULL if the datagram is a fragment but doesn't fill
510  *         an hole. In this case, the content of the datagram has been
511  *         copied, so if the user wants, he can release the resources
512  *         used to store the received packet.
513  *
514  *         Returns A pointer to the recomposed datagram if the datagram
515  *         is the last fragment of a bigger datagram. This pointer will
516  *         be different from data. The user should free() this pointer
517  *         when it is no more needed.
518  */
519 unsigned char* dpi_reordering_manage_ipv4_fragment(
520         dpi_ipv4_fragmentation_state_t *state,
521         const unsigned char* data,
522         u_int32_t current_time, u_int16_t offset,
523         u_int8_t more_fragments, int tid){
524     struct iphdr *iph = (struct iphdr*) data;
525
526     dpi_ipv4_fragmentation_source_t* source,*tmpsource;
527     dpi_ipv4_fragmentation_flow_t* flow;
528
529     u_int16_t tot_len=ntohs(iph->tot_len);
530     /**
531      * Host are required to do not fragment datagrams with a total
532      * size up to 576 byte. If we received a fragment with a size <576
533      * it is maybe a forged fragment used to make an attack. We do this
534      * check only in non-debug situations because many of the test used
535      * to validate the IP reassembly contains small packets.
536      */
537 #ifndef DPI_DEBUG_FRAGMENTATION_v4
538     if(unlikely(tot_len<DPI_IPv4_FRAGMENTATION_MINIMUM_MTU)){
539         return NULL;
540     }
541 #endif
542
543     u_int8_t ihl=iph->ihl*4;
544     u_int16_t fragment_size=tot_len-ihl;
545     /** (end-1) is the last byte of the fragment. **/
546
547     u_int32_t end=offset+fragment_size;
548     /* Attempt to construct an oversize packet. */
549     if(unlikely(end>DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE)){
550         debug_print("%s\n","Attempt to build an oversized packet");
551         return NULL;
552     }
553
554 #if DPI_THREAD_SAFETY_ENABLED == 1
555     ff::spin_lock(&(state->lock), tid);
556 #endif
557     source=dpi_ipv4_fragmentation_find_or_create_source(state,
```

```
558                                                     iph->saddr);
559     if(unlikely(source==NULL)){
560         debug_print("%s\n","ERROR: Impossible to create the source.");
561 #if DPI_THREAD_SAFETY_ENABLED == 1
562         ff::spin_unlock(&(state->lock), tid);
563 #endif
564         return NULL;
565     }
566     debug_print("%s\n","Source found or created.");
567
568     debug_print("Total memory occupied: %d\n",state->total_used_mem);
569     debug_print("Source memory occupied: %d\n",state->total_used_mem);
570
571
572     /** If source limit exceeded, then delete flows from that source. **/
573     while(source->flows &&
574          (source->source_used_mem)>state->per_source_memory_limit){
575         debug_print("%s\n","Source limit exceeded, cleaning...");
576         dpi_ipv4_fragmentation_delete_flow(state, source->flows);
577         if(source->flows==NULL){
578             dpi_ipv4_fragmentation_delete_source(state, source);
579 #if DPI_THREAD_SAFETY_ENABLED == 1
580             ff::spin_unlock(&(state->lock), tid);
581 #endif
582             return NULL;
583         }
584     }
585
586     /**
587      * Control on global memory limit for ip fragmentation.
588      * The timer are sorted for the one which will expire sooner to the
589      * last that will expire. The loop stops when there are no more
590      * expired timers. dpi_ipv4_fragmentation_delete_flow(..) update
591      * the timer timer_head after deleting the timer_head if it is
592      * expired.
593      **/
594     while((state->timer_head) &&
595           ((state->timer_head->expiration_time<current_time) ||
596           (state->total_used_mem>=state->total_memory_limit))){
597         tmpsource=((dpi_ipv4_fragmentation_flow_t*)
598                     state->timer_head->data)->source;
599         dpi_ipv4_fragmentation_delete_flow(
600                 state,
601                 (dpi_ipv4_fragmentation_flow_t*) state->timer_head->data);
602         if(source->flows==NULL){
603             dpi_ipv4_fragmentation_delete_source(state,tmpsource);
604 #if DPI_THREAD_SAFETY_ENABLED == 1
605             ff::spin_unlock(&(state->lock), tid);
606 #endif
```

```
607             return NULL;
608         }
609     }
610
611
612     /* Find the flow. */
613     flow=dpi_ipv4_fragmentation_find_or_create_flow(state, source, iph,
614                                                     current_time);
615     debug_print("%s\n","Flow found or created.");
616
617     if(unlikely(flow==NULL)){
618         debug_print("%s\n","ERROR: Impossible to create the flow.");
619 #if DPI_THREAD_SAFETY_ENABLED == 1
620         ff::spin_unlock(&(state->lock), tid);
621 #endif
622         return NULL;
623     }
624
625     /**
626      * If is a malformed fragment which starts after the end
627      * of the entire datagram.
628      **/
629     if(unlikely(flow->len!=0 && offset>flow->len)){
630         debug_print("%s\n","Malformed fragment, starts after "
631                     "the end of the entire datagram.");
632 #if DPI_THREAD_SAFETY_ENABLED == 1
633         ff::spin_unlock(&(state->lock), tid);
634 #endif
635         return NULL;
636     }
637
638     /*
639      * If the first fragment is received for the first time,
640      * store the header
641      */
642     if(offset==0 && flow->iph==NULL){
643         debug_print("%s\n","Received fragment with offset zero");
644         flow->iph=(struct iphdr*) malloc(ihl*sizeof(unsigned char));
645         if(unlikely(flow->iph==NULL)){
646             dpi_ipv4_fragmentation_delete_flow(state, flow);
647 #if DPI_THREAD_SAFETY_ENABLED == 1
648             ff::spin_unlock(&(state->lock), tid);
649 #endif
650             return NULL;
651         }
652         state->total_used_mem+=ihl;
653         source->source_used_mem+=ihl;
654         memcpy(flow->iph, iph, ihl);
655     }
```

```
656
657     /**
658       * If is the final fragment, then we know the exact data_length
659       * of the original datagram.
660      **/
661     if(!more_fragments){
662         debug_print("%s\n","Last fragment received.");
663         /**
664          * If the data with MF flag=0 was already received then this
665          * fragment is useless.
666          **/
667         if(flow->len!=0){
668 #if DPI_THREAD_SAFETY_ENABLED == 1
669             ff::spin_unlock(&(state->lock), tid);
670 #endif
671             return NULL;
672         }
673         flow->len=end;
674     }
675
676     u_int32_t bytes_removed;
677     u_int32_t bytes_inserted;
678     dpi_reassembly_insert_fragment(&(flow->fragments), data+ihl,
679                                    offset, end, &(bytes_removed),
680                                    &(bytes_inserted));
681     state->total_used_mem+=bytes_inserted;
682     state->total_used_mem-=bytes_removed;
683
684     source->source_used_mem+=bytes_inserted;
685     source->source_used_mem-=bytes_removed;
686
687     debug_print("%s\n","Fragment inserted.");
688
689     /**
690      *  Check if with the new fragment that we inserted, the original
691      *  datagram is now complete.
692      *  (Only possible if we received the fragment with MF flag=0
693      *  (so the len is set) and if we have a train of contiguous
694      *   fragments).
695      **/
696     if(flow->len!=0 &&
697        dpi_reassembly_ip_check_train_of_contiguous_fragments(
698              flow->fragments)){
699         unsigned char* r;
700         debug_print("%s\n","Last fragment already received and train "
701                     "of contiguous fragments present, returing the "
702                     "recompacted datagram.");
703         r=dpi_ipv4_fragmentation_build_complete_datagram(state,flow);
704 #if DPI_THREAD_SAFETY_ENABLED == 1
```

```
705         ff::spin_unlock(&(state->lock), tid);
706 #endif
707         return r;
708     }
709 #if DPI_THREAD_SAFETY_ENABLED == 1
710     ff::spin_unlock(&(state->lock), tid);
711 #endif
712     return NULL;
713 }
```

```
 1 /*
 2  * ipv6_reassembly.h
 3  *
 4  *  Created on: 05/10/2012
 5  *  Author: Daniele De Sensi (d.desensi@gmail.com)
 6  */
 7
 8 #ifndef IPV6_REASSEMBLY_H_
 9 #define IPV6_REASSEMBLY_H_
10
11
12 #ifdef __cplusplus
13 extern "C" {
14 #endif
15
16 typedef struct dpi_ipv6_fragmentation_state
17                dpi_ipv6_fragmentation_state_t;
18
19 /**
20  * Enables the IPv6 defragmentation.
21  * @param table_size  The size of the table used to store the fragments.
22  * @return            A pointer to the IPv6 defragmentation handle.
23  */
24 dpi_ipv6_fragmentation_state_t*
25         dpi_reordering_enable_ipv6_fragmentation(u_int16_t table_size);
26
27 /**
28  * Sets the maximum amount of memory that can be used to store
29  * fragments generated by the same source.
30  * @param frag_state A pointer to the IPv6 degragmentation handle.
31  * @param per_host_memory_limit  The maximum amount of memory that can be
32  *                               used to store fragments generated by the
33  *                               same source.
34  */
35 void dpi_reordering_ipv6_fragmentation_set_per_host_memory_limit(
36         dpi_ipv6_fragmentation_state_t *frag_state,
37         u_int32_t per_host_memory_limit);
38
39 /**
```

```
40  * Sets the maximum (global) amount of memory that can be used for
41  * defragmentation purposes.
42  * @param frag_state A pointer to the IPv6 defragmentation handle.
43  * @param total_memory_limit   The global memory limit.
44  */
45 void dpi_reordering_ipv6_fragmentation_set_total_memory_limit(
46         dpi_ipv6_fragmentation_state_t *frag_state,
47         u_int32_t total_memory_limit);
48
49 /**
50  * Sets the maximum amount of time (seconds) which can elapse before the
51  * complete defragmentation of the datagram.
52  * @param frag_state        A pointer to the IPv6 defragmentation handle.
53  * @param timeout_seconds   The timeout (seconds).
54  */
55 void dpi_reordering_ipv6_fragmentation_set_reassembly_timeout(
56         dpi_ipv6_fragmentation_state_t *frag_state,
57         u_int8_t timeout_seconds);
58
59 /**
60  * Disables the IPv6 fragmentation and deallocates the handle.
61  * @param frag_state  A pointer to the IPv6 defragmentation handle.
62  */
63 void dpi_reordering_disable_ipv6_fragmentation(
64         dpi_ipv6_fragmentation_state_t* frag_state);
65
66 /**
67  * Reassemble the IP datagram if it is fragmented.
68  * @param state The state for fragmentation support.
69  * @param unfragmentable_start A pointer to the beginning of
70  * unfragmentable part. We suppose that unfragmentable and
71  * fragmentable data are in the same buffer.
72  * @param unfragmentable_size The size of the unfragmentable part.
73  * In principle this could obtained by difference (fragmentable_start-
74  * unfragmentable_start). By the way this additional level of flexibility
75  * allows to store only a part of the unfragmentable header (e.g. only the
76  * IPv6 header without additional headers).
77  * @param fragmentable_start A pointer to the beginning of the
78  * fragmentable part.
79  * @param fragmentable_size Size of the fragmentable part.
80  * @param offset The data offset specified in the IPv6 fragmentation
81  * header.
82  * @param more_fragments 1 if the MF flag is set, 0 otherwise.
83  * @param identification The identification of the datagram.
84  * @param next_header The header which follows the fragmentation
85  * header.
86  * @param current_time The current time, in seconds.
87  * @param tid The thread identifier.
88  * @return Returns NULL if the datagram is a fragment but doesn't fill
```

```
 * an hole. In this case, the content of the datagram has been copied,
 * so if the user wants, he can release the resources used to store the
 * received packet.
 * Returns a pointer to the recomposed datagram if the datagram is the
 * last fragment of a bigger datagram. This pointer will be different
 * from data. The user should free() this pointer when it is no more
 * needed.
 */
unsigned char* dpi_reordering_manage_ipv6_fragment(
        dpi_ipv6_fragmentation_state_t *state,
        const unsigned char* unfragmentable_start,
        u_int16_t unfragmentable_size,
        const unsigned char* fragmentable_start,
        u_int16_t fragmentable_size,
        u_int16_t offset, u_int8_t more_fragments,
        u_int32_t identification, u_int8_t next_header,
        u_int32_t current_time, int tid);

#ifdef __cplusplus
}
#endif
#endif /* IPV6_REASSEMBLY_H_ */
```

```
/*
 * ipv6_reassembly.c
 *
 *  Support for IPv6 reassembly. Any modification done here should
 *  be reflected on IPv4 reassembly.
 *
 *  Created on: 05/10/2012
 *  Author: Daniele De Sensi (d.desensi@gmail.com)
 */
#include <sys/types.h>
#include <strings.h>
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <netinet/ip6.h>
#include <inttypes.h>
#include "ipv6_reassembly.h"
#include "reassembly.h"
#include "utils.h"
#if DPI_THREAD_SAFETY_ENABLED == 1
#include <ff/spin-lock.hpp>
#endif

#define DPI_DEBUG_FRAGMENTATION_v6 0

#define debug_print(fmt, ...)                    \
```

```
27             do { if (DPI_DEBUG_FRAGMENTATION_v6) \
28             fprintf(stderr, fmt, __VA_ARGS__); } while (0)
29
30 #define DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE 65535
31 #define DPI_IPv6_FRAGMENTATION_MINIMUM_MTU 1280
32
33
34 typedef struct dpi_ipv6_fragmentation_flow
35                dpi_ipv6_fragmentation_flow_t;
36 typedef struct dpi_ipv6_fragmentation_source
37                dpi_ipv6_fragmentation_source_t;
38
39
40 typedef struct dpi_ipv6_fragmentation_flow {
41     /* Pointer to the unfragmentable part. */
42     unsigned char *unfragmentable;
43     u_int16_t unfragmentable_length;
44     /*Total length of the final datagram (without unfragmentable part).*/
45     u_int16_t len;
46     /*
47      * NOTE: Differently from ipv6, in IPv6 the key is <id, src, dest>
48      * (so no next header value).
49      */
50     u_int32_t id;
51     struct in6_addr dstaddr;
52     /* Linked list of received fragments. */
53     dpi_reassembly_fragment_t *fragments;
54     /*
55      * For a given source, pointer to the previous and next flows
56      * started from that source
57      */
58     dpi_ipv6_fragmentation_flow_t *next;
59     dpi_ipv6_fragmentation_flow_t *prev;
60     dpi_reassembly_timer_t timer;
61     dpi_ipv6_fragmentation_source_t *source;
62 }dpi_ipv6_fragmentation_flow_t;
63
64 typedef struct dpi_ipv6_fragmentation_source{
65     dpi_ipv6_fragmentation_flow_t *flows;
66     u_int32_t source_used_mem;
67     struct in6_addr ipv6_srcaddr;
68     u_int16_t row;
69     dpi_ipv6_fragmentation_source_t* prev;
70     dpi_ipv6_fragmentation_source_t* next;
71 }dpi_ipv6_fragmentation_source_t;
72
73 typedef struct dpi_ipv6_fragmentation_state{
74     /**
75      *  Is an hash table containing associations between source IP
```

```
76      *  address and fragments generated by that address.
77      **/
78     dpi_ipv6_fragmentation_source_t** table;
79     u_int32_t total_used_mem;
80     u_int16_t table_size;
81
82     /** List of timers. **/
83     dpi_reassembly_timer_t *timer_head, *timer_tail;
84
85     /** Memory limits. **/
86     u_int32_t per_source_memory_limit;
87     u_int32_t total_memory_limit;
88
89     /** Reassembly timeout. **/
90     u_int8_t timeout;
91 #if DPI_THREAD_SAFETY_ENABLED == 1
92     ff::CLHSpinLock lock;
93 #endif
94 }dpi_ipv6_fragmentation_state_t;
95
96
97 #ifndef DPI_DEBUG
98 static
99 #if DPI_USE_INLINING == 1
100 inline
101 #endif
102 #endif
103 void dpi_ipv6_fragmentation_delete_source(
104         dpi_ipv6_fragmentation_state_t* state,
105         dpi_ipv6_fragmentation_source_t* source);
106
107 /**
108  * Enables the IPv6 defragmentation.
109  * @param table_size  The size of the table used to store the fragments.
110  * @return            A pointer to the IPv6 defragmentation handle.
111  */
112 dpi_ipv6_fragmentation_state_t* dpi_reordering_enable_ipv6_fragmentation(
113         u_int16_t table_size){
114     dpi_ipv6_fragmentation_state_t* r=(dpi_ipv6_fragmentation_state_t*)
115             calloc(1, sizeof(dpi_ipv6_fragmentation_state_t));
116     if(r==NULL){
117         return NULL;
118     }
119     r->table_size=table_size;
120     r->table=(dpi_ipv6_fragmentation_source_t**)
121             malloc(table_size * sizeof(dpi_ipv6_fragmentation_source_t*));
122     if(r->table==NULL){
123         free(r);
124         return NULL;
```

```
125     }
126     u_int16_t i;
127     for(i=0; i<table_size; i++){
128         r->table[i]=NULL;
129     }
130     r->timer_head=NULL;
131     r->timer_tail=NULL;
132     r->per_source_memory_limit=
133             DPI_IPv6_FRAGMENTATION_DEFAULT_PER_HOST_MEMORY_LIMIT;
134     r->total_memory_limit=
135             DPI_IPv6_FRAGMENTATION_DEFAULT_TOTAL_MEMORY_LIMIT;
136     r->timeout=
137             DPI_IPv6_FRAGMENTATION_DEFAULT_REASSEMBLY_TIMEOUT;
138     r->total_used_mem=0;
139 #if DPI_THREAD_SAFETY_ENABLED == 1
140     bzero(&(r->lock), sizeof(ff::CLHSpinLock));
141     ff::init_unlocked(&(r->lock));
142 #endif
143     return r;
144 }
145
146
147 /**
148  * Sets the maximum amount of memory that can be used to store fragments
149  * generated
150  * by the same source.
151  * @param frag_state A pointer to the IPv6 degragmentation handle.
152  * @param per_host_memory_limit  The maximum amount of memory that can be
153  *                               used to store fragments generated by the
154  *                               same source.
155  */
156 void dpi_reordering_ipv6_fragmentation_set_per_host_memory_limit(
157         dpi_ipv6_fragmentation_state_t *frag_state,
158         u_int32_t per_host_memory_limit){
159     frag_state->per_source_memory_limit=per_host_memory_limit;
160 }
161
162 /**
163  * Sets the maximum (global) amount of memory that can be used for
164  * defragmentation purposes.
165  * @param frag_state A pointer to the IPv6 defragmentation handle.
166  * @param total_memory_limit   The global memory limit.
167  */
168 void dpi_reordering_ipv6_fragmentation_set_total_memory_limit(
169         dpi_ipv6_fragmentation_state_t *frag_state,
170         u_int32_t total_memory_limit){
171     frag_state->total_memory_limit=total_memory_limit;
172 }
173
```

```
174 /**
175  * Sets the maximum amount of time (seconds) which can elapse before the
176  * complete defragmentation of the datagram.
177  * @param frag_state        A pointer to the IPv6 defragmentation handle.
178  * @param timeout_seconds   The timeout (seconds).
179  */
180 void dpi_reordering_ipv6_fragmentation_set_reassembly_timeout(
181         dpi_ipv6_fragmentation_state_t *frag_state,
182         u_int8_t timeout_seconds){
183     frag_state->timeout=timeout_seconds;
184 }
185
186 /**
187  * Disables the IPv6 fragmentation and deallocates the handle.
188  * @param frag_state  A pointer to the IPv6 defragmentation handle.
189  */
190 void dpi_reordering_disable_ipv6_fragmentation(
191         dpi_ipv6_fragmentation_state_t* frag_state) {
192     if(frag_state==NULL) return;
193     dpi_ipv6_fragmentation_source_t *source,*tmp_source;
194     if(frag_state->table){
195         u_int16_t i;
196         for(i=0; i<frag_state->table_size; i++){
197             if(frag_state->table[i]){
198                 source=frag_state->table[i];
199                 while(source){
200                     tmp_source=source->next;
201                     dpi_ipv6_fragmentation_delete_source(frag_state,
202                                                          source);
203                     source=tmp_source;
204                 }
205             }
206         }
207         free(frag_state->table);
208     }
209 #if DPI_THREAD_SAFETY_ENABLED == 1
210     frag_state->lock.~CLHSpinLock();
211 #endif
212     free(frag_state);
213 }
214
215
216 #ifndef DPI_DEBUG
217 static
218 #if DPI_USE_INLINING == 1
219 inline
220 #endif
221 #endif
222 /**  Shift-Add-XOR hash. **/
```

```
223 u_int16_t dpi_ipv6_fragmentation_hash_function(
224         dpi_ipv6_fragmentation_state_t* state, struct in6_addr addr){
225     u_int16_t h=0;
226     u_int8_t i;
227
228     for(i=0; i<16; i++)
229         h^=(h<<5)+(h>>2)+addr.s6_addr[i];
230
231     return h%state->table_size;
232 }
233
234
235 #ifndef DPI_DEBUG
236 static
237 #endif
238 /**
239  * Try to find the specific source. If it is not find, then creates it.
240  * \param state The state of the defragmentation module.
241  * \param addr The source address.
242  * \returns A pointer to the source.
243  */
244 dpi_ipv6_fragmentation_source_t*
245        dpi_ipv6_fragmentation_find_or_create_source(
246                dpi_ipv6_fragmentation_state_t* state,
247                struct in6_addr addr){
248     u_int16_t hash_index=dpi_ipv6_fragmentation_hash_function(state,
249                                                               addr);
250     dpi_ipv6_fragmentation_source_t *source,*head;
251
252     head=state->table[hash_index];
253
254     for(source=head; source!=NULL; source=source->next){
255         if(dpi_v6_addresses_equal(source->ipv6_srcaddr, addr)){
256             return source;
257         }
258     }
259
260     /** Not found, so create it. **/
261     source=(dpi_ipv6_fragmentation_source_t*)
262             malloc(sizeof(dpi_ipv6_fragmentation_source_t));
263     if(unlikely(source==NULL)){
264         return NULL;
265     }
266     source->row=hash_index;
267     source->flows=NULL;
268
269     source->ipv6_srcaddr=addr;
270     source->source_used_mem=sizeof(dpi_ipv6_fragmentation_source_t);
271     state->total_used_mem+=sizeof(dpi_ipv6_fragmentation_source_t);
```

```
272
273
274     /** Insertion at the beginning of the list. **/
275     source->prev=NULL;
276     source->next=head;
277     if(head)
278         head->prev=source;
279     state->table[hash_index]=source;
280
281     return source;
282 }
283
284 #ifndef DPI_DEBUG
285 static
286 #endif
287 void dpi_ipv6_fragmentation_delete_flow(
288         dpi_ipv6_fragmentation_state_t* state,
289         dpi_ipv6_fragmentation_flow_t* flow){
290     dpi_reassembly_fragment_t *frag,*temp_frag;
291
292     dpi_ipv6_fragmentation_source_t* source=flow->source;
293
294     source->source_used_mem-=sizeof(dpi_ipv6_fragmentation_flow_t);
295     state->total_used_mem-=sizeof(dpi_ipv6_fragmentation_flow_t);
296
297     /* Stop the timer and delete it. */
298     dpi_reassembly_delete_timer(&(state->timer_head),
299                                 &(state->timer_tail), &(flow->timer));
300
301     /* Release all fragment data. */
302     frag = flow->fragments;
303     while(frag){
304         temp_frag=frag->next;
305         source->source_used_mem-=(frag->end-frag->offset);
306         state->total_used_mem-=(frag->end-frag->offset);
307
308         free(frag->ptr);
309         free(frag);
310         frag=temp_frag;
311     }
312
313     /** Delete the IP header. **/
314     if(flow->unfragmentable){
315         source->source_used_mem-=flow->unfragmentable_length;
316         state->total_used_mem-=flow->unfragmentable_length;
317         free(flow->unfragmentable);
318     }
319
320     /*
```

```
321      * Remove the flow from the list of the flows. If no more flows for
322      * this source, then delete the source.
323      */
324     if(flow->prev==NULL){
325         source->flows=flow->next;
326         if(source->flows!=NULL)
327             source->flows->prev=NULL;
328     }else{
329         flow->prev->next=flow->next;
330         if(flow->next)
331             flow->next->prev=flow->prev;
332     }
333     free(flow);
334 }
335
336
337 /*
338  * Find the flow. If it is not found creates it.
339  * \returns A pointer to the flow.
340  */
341 #ifndef DPI_DEBUG
342 static
343 #endif
344 dpi_ipv6_fragmentation_flow_t* dpi_ipv6_fragmentation_find_or_create_flow(
345         dpi_ipv6_fragmentation_state_t* state,
346         dpi_ipv6_fragmentation_source_t* source,
347         u_int32_t id, struct in6_addr dstaddr,
348         u_int32_t current_time){
349     dpi_ipv6_fragmentation_flow_t* flow;
350     for(flow=source->flows; flow!=NULL; flow=flow->next){
351         /**
352          * The source is matched for sure because all the flows
353          * will have the same source.
354          **/
355         if(id==flow->id && dpi_v6_addresses_equal(dstaddr,
356                                                   flow->dstaddr)){
357             return flow;
358         }
359     }
360
361     /** Not found, create a new flow. **/
362     flow=(dpi_ipv6_fragmentation_flow_t*)
363             malloc(sizeof(dpi_ipv6_fragmentation_flow_t));
364     if(unlikely(flow==NULL)){
365         return NULL;
366     }
367
368     source->source_used_mem+=sizeof(dpi_ipv6_fragmentation_flow_t);
369     state->total_used_mem+=sizeof(dpi_ipv6_fragmentation_flow_t);
```

```
370
371
372     flow->fragments=NULL;
373     flow->source=source;
374     flow->len=0;
375     /* Add this entry to the queue of flows. */
376     flow->prev=NULL;
377     flow->next=source->flows;
378     if(flow->next)
379         flow->next->prev=flow;
380     source->flows=flow;
381     /* Set the timer. */
382     flow->timer.expiration_time=current_time+state->timeout;
383     flow->timer.data = flow;
384     dpi_reassembly_add_timer(&(state->timer_head), &(state->timer_tail),
385                              &(flow->timer));
386     /* Fragments will be added later. */
387     flow->fragments=NULL;
388     flow->unfragmentable=NULL;
389     flow->id=id;
390     flow->dstaddr=dstaddr;
391     return flow;
392 }
393
394
395 /**
396  * @return A pointer to the recompacted datagram or NULL if an error
397  * occurred.
398  */
399 #ifndef DPI_DEBUG
400 static
401 #endif
402 unsigned char* dpi_ipv6_fragmentation_build_complete_datagram(
403         dpi_ipv6_fragmentation_state_t* state,
404         dpi_ipv6_fragmentation_flow_t* flow){
405     unsigned char *pkt_beginning, *pkt_data;
406     struct ip6_hdr *iph;
407     u_int16_t len;
408     int32_t count;
409
410     /* Allocate a new buffer for the datagram. */
411     len=flow->len;
412
413     dpi_ipv6_fragmentation_source_t* source=flow->source;
414
415     u_int32_t tot_len=flow->unfragmentable_length+len;
416
417     if(unlikely(tot_len>DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE)){
418         dpi_ipv6_fragmentation_delete_flow(state, flow);
```

```
419         if(source->flows==NULL)
420             dpi_ipv6_fragmentation_delete_source(state,source);
421         return NULL;
422     }
423
424
425     if(unlikely((pkt_beginning=(unsigned char*)
426             malloc(flow->unfragmentable_length+len))==NULL)){
427         dpi_ipv6_fragmentation_delete_flow(state, flow);
428         if(source->flows==NULL)
429             dpi_ipv6_fragmentation_delete_source(state,source);
430         return NULL;
431     }
432
433     memcpy(pkt_beginning, (flow->unfragmentable),
434            flow->unfragmentable_length);
435     pkt_data=pkt_beginning+flow->unfragmentable_length;
436
437     count=dpi_reassembly_ip_compact_fragments(flow->fragments,
438                                               &pkt_data, len);
439
440     /**
441      * Misbehaving packet, real size is different from that obtained
442      * from the last fragment.
443      **/
444     if(count==-1){
445         free(pkt_beginning);
446         return NULL;
447     }
448
449     /** Put the correct informations in the IP header. **/
450     iph=(struct ip6_hdr *) pkt_beginning;
451     iph->ip6_ctlun.ip6_un1.ip6_un1_plen=
452             htons(count+flow->unfragmentable_length-
453                   sizeof(struct ip6_hdr));
454
455     /** We recompacted the flow (datagram), so now we can delete it. **/
456     dpi_ipv6_fragmentation_delete_flow(state, flow);
457     if(source->flows==NULL)
458         dpi_ipv6_fragmentation_delete_source(state,source);
459
460     return pkt_beginning;
461 }
462
463
464 #ifndef DPI_DEBUG
465 static
466 #if DPI_USE_INLINING == 1
467 inline
```

```
468 #endif
469 #endif
470 void dpi_ipv6_fragmentation_delete_source(
471         dpi_ipv6_fragmentation_state_t* state,
472         dpi_ipv6_fragmentation_source_t* source) {
473     u_int16_t row=source->row;
474 
475     /** Delete all the flows belonging to this source. **/
476     dpi_ipv6_fragmentation_flow_t* flow=source->flows,*temp_flow;
477     while(flow){
478         temp_flow=flow->next;
479         dpi_ipv6_fragmentation_delete_flow(state, flow);
480         flow=temp_flow;
481     }
482 
483     /** Delete this source from the list. **/
484     if (source->prev)
485         source->prev->next=source->next;
486     else
487         state->table[row]=source->next;
488 
489     if(source->next)
490         source->next->prev=source->prev;
491 
492     free(source);
493     state->total_used_mem-=sizeof(dpi_ipv6_fragmentation_source_t);
494 }
495 
496 
497 
498 /**
499  * Reassemble the IP datagram if it is fragmented.
500  * @param state The state for fragmentation support.
501  * @param unfragmentable_start A pointer to the beginning of
502  * unfragmentable part. We suppose that unfragmentable and
503  * fragmentable data are in the same buffer.
504  * @param unfragmentable_size The size of the unfragmentable part.
505  * In principle this could obtained by difference (fragmentable_start-
506  * unfragmentable_start). By the way this additional level of flexibility
507  * allows to store only a part of the unfragmentable header (e.g. only the
508  * IPv6 header without additional headers).
509  * @param fragmentable_start A pointer to the beginning of the
510  * fragmentable part.
511  * @param fragmentable_size Size of the fragmentable part.
512  * @param offset The data offset specified in the IPv6 fragmentation
513  * header.
514  * @param more_fragments 1 if the MF flag is set, 0 otherwise.
515  * @param identification The identification of the datagram.
516  * @param next_header The header which follows the fragmentation
```

```
517  * header.
518  * @param current_time The current time, in seconds.
519  * @param tid The thread identifier.
520  * @return Returns NULL if the datagram is a fragment but doesn't fill
521  * an hole. In this case, the content of the datagram has been copied,
522  * so if the user wants, he can release the resources used to store the
523  * received packet.
524  * Returns a pointer to the recomposed datagram if the datagram is the
525  * last fragment of a bigger datagram. This pointer will be different
526  * from data. The user should free() this pointer when it is no more
527  * needed.
528  */
529 unsigned char* dpi_reordering_manage_ipv6_fragment(
530         dpi_ipv6_fragmentation_state_t *state,
531         const unsigned char* unfragmentable_start,
532         u_int16_t unfragmentable_size,
533         const unsigned char* fragmentable_start,
534         u_int16_t fragmentable_size,
535         u_int16_t offset, u_int8_t more_fragments,
536         u_int32_t identification, u_int8_t next_header,
537         u_int32_t current_time, int tid){
538     dpi_ipv6_fragmentation_source_t* source,*tmpsource;
539     dpi_ipv6_fragmentation_flow_t* flow;
540
541     struct ip6_hdr* ip6=(struct ip6_hdr*) unfragmentable_start;
542     /**
543      * Host are required to do not fragment datagrams with a total size
544      * up to 576 byte. If we received a fragment with a size <576 it is
545      * maybe a forged fragment used to make an attack. We do this check
546      * only in non-debug situations because many of the test used to
547      * validate the ip reassembly contains small packets.
548      */
549 #ifndef DPI_DEBUG_FRAGMENTATION_v6
550     if(unlikely(fragmentable_start+fragmentable_size-unfragmentable_start<
551                 DPI_IPv6_FRAGMENTATION_MINIMUM_MTU)){
552         return NULL;
553     }
554 #endif
555
556     /** (end-1) is the last byte of the fragment. **/
557     u_int32_t end=offset+fragmentable_size;
558
559     /* Attempt to construct an oversize packet. */
560     if(unlikely(end>DPI_IP_FRAGMENTATION_MAX_DATAGRAM_SIZE)){
561         debug_print("%s\n","Attempt to build an oversized packet");
562         return NULL;
563     }
564
565 #if DPI_THREAD_SAFETY_ENABLED == 1
```

```
566     ff::spin_lock(&(state->lock), tid);
567 #endif
568     source=dpi_ipv6_fragmentation_find_or_create_source(
569              state, ip6->ip6_src);
570
571     if(unlikely(source==NULL)){
572         debug_print("%s\n","ERROR: Impossible to create the source. "
573                     "Memory exhausted.");
574 #if DPI_THREAD_SAFETY_ENABLED == 1
575         ff::spin_unlock(&(state->lock), tid);
576 #endif
577         return NULL;
578     }
579     debug_print("%s\n","Source found or created.");
580
581     debug_print("Total memory occupied: %d\n", state->total_used_mem);
582     debug_print("Source memory occupied: %d\n", state->total_used_mem);
583
584
585     /**
586      * If I exceeded the source limit, then delete flows from that
587      * source.
588      **/
589     while(source->flows && (source->source_used_mem)>
590           state->per_source_memory_limit){
591         debug_print("%s\n","Source limit exceeded, cleaning...");
592         dpi_ipv6_fragmentation_delete_flow(state, source->flows);
593         if(source->flows==NULL){
594             dpi_ipv6_fragmentation_delete_source(state, source);
595 #if DPI_THREAD_SAFETY_ENABLED == 1
596             ff::spin_unlock(&(state->lock), tid);
597 #endif
598             return NULL;
599         }
600     }
601
602     /**
603      * Control on global memory limit for ip fragmentation.
604      * The timer are sorted for the one which will expire sooner to the
605      * last that will expire. The loop stops when there are no more
606      * expired timers. dpi_ipv4_fragmentation_delete_flow(..) update
607      * the timer timer_head after deleting the timer_head if it is
608      * expired.
609      **/
610     while((state->timer_head) &&
611           ((state->timer_head->expiration_time<current_time) ||
612           (state->total_used_mem>=state->total_memory_limit))){
613         tmpsource=((dpi_ipv6_fragmentation_flow_t*)
614                      state->timer_head->data)->source;
```

```
615         dpi_ipv6_fragmentation_delete_flow(state,
616                                            (dpi_ipv6_fragmentation_flow_t*)
617                                            state->timer_head->data);
618         if(source->flows==NULL){
619             dpi_ipv6_fragmentation_delete_source(state,tmpsource);
620 #if DPI_THREAD_SAFETY_ENABLED == 1
621             ff::spin_unlock(&(state->lock), tid);
622 #endif
623             return NULL;
624         }
625     }
626
627
628     /* Find the flow. */
629     flow=dpi_ipv6_fragmentation_find_or_create_flow(state, source,
630                                                     identification,
631                                                     ip6->ip6_dst,
632                                                     current_time);
633
634     if(unlikely(flow==NULL)){
635         debug_print("%s\n","ERROR: Impossible to create the flow.");
636 #if DPI_THREAD_SAFETY_ENABLED == 1
637         ff::spin_unlock(&(state->lock), tid);
638 #endif
639         return NULL;
640     }
641
642     debug_print("%s\n","Flow found or created.");
643
644     /**
645      * If is a malformed fragment which starts after the end of
646      * the entire datagram.
647      **/
648     if(unlikely(flow->len!=0 && offset>flow->len)){
649 #if DPI_THREAD_SAFETY_ENABLED == 1
650         ff::spin_unlock(&(state->lock), tid);
651 #endif
652         return NULL;
653     }
654
655     /*
656      * The unfragmentable part is the same for all the fragments. So,
657      * differently from IPv4, we don't have to check that the offset is
658      * zero to store it but only that it is not already present.
659      */
660     if(flow->unfragmentable==NULL){
661         flow->unfragmentable=(unsigned char*)
662                 malloc(unfragmentable_size*sizeof(unsigned char));
663         if(unlikely(flow->unfragmentable==NULL)){
```

```
664             dpi_ipv6_fragmentation_delete_flow(state, flow);
665 #if DPI_THREAD_SAFETY_ENABLED == 1
666             ff::spin_unlock(&(state->lock), tid);
667 #endif
668             return NULL;
669         }
670         flow->unfragmentable_length=unfragmentable_size;
671         state->total_used_mem+=unfragmentable_size;
672         source->source_used_mem+=unfragmentable_size;
673         memcpy(flow->unfragmentable, unfragmentable_start,
674                unfragmentable_size);
675         ((struct ip6_hdr*) flow->unfragmentable)->ip6_ctlun.
676                             ip6_un1.ip6_un1_nxt=next_header;
677     }
678
679     debug_print("More fragments: %d\n", more_fragments);
680     /**
681      * If is the final fragment, then we know the exact data_length
682      * of the original datagram.
683      **/
684     if(!more_fragments){
685         debug_print("%s\n","Last fragment received.");
686         /**
687          * If the data with MF flag=0 was already received then this
688          * fragment is useless.
689          **/
690         if(flow->len!=0){
691 #if DPI_THREAD_SAFETY_ENABLED == 1
692             ff::spin_unlock(&(state->lock), tid);
693 #endif
694             return NULL;
695         }
696         flow->len=end;
697     }
698
699     u_int32_t bytes_removed;
700     u_int32_t bytes_inserted;
701     dpi_reassembly_insert_fragment(&(flow->fragments), fragmentable_start,
702                                    offset, end, &bytes_removed,
703                                    &bytes_inserted);
704
705     state->total_used_mem+=bytes_inserted;
706     state->total_used_mem-=bytes_removed;
707
708     source->source_used_mem+=bytes_inserted;
709     source->source_used_mem-=bytes_removed;
710
711     debug_print("%s\n","Fragment inserted.");
712
```

```
713     /**
714      *  Check if with the new fragment that we inserted, the original
715      *  datagram is now complete. (Only possible if we received the
716      *  fragment with MF flag=0 (so the len is set) and if we have a
717      *  train of contiguous fragments).
718      **/
719     if(flow->len!=0 &&
720        dpi_reassembly_ip_check_train_of_contiguous_fragments(
721                flow->fragments)){
722         unsigned char* r;
723         debug_print("%s\n","Last fragment already received and train of "
724                     "contiguous fragments present, returing the ↙
    ↳ recompacted "
725                     "datagram.");
726         r=dpi_ipv6_fragmentation_build_complete_datagram(state,flow);
727 #if DPI_THREAD_SAFETY_ENABLED == 1
728         ff::spin_unlock(&(state->lock), tid);
729 #endif
730         return r;
731     }
732 #if DPI_THREAD_SAFETY_ENABLED == 1
733     ff::spin_unlock(&(state->lock), tid);
734 #endif
735     return NULL;
736
737 }
```

A.4 TCP stream management

```
1  /*
2   * tcp_stream_management.h
3   *
4   *  Created on: 06/10/2012
5   *  Author: Daniele De Sensi
6   */
7
8  #ifndef TCP_STREAM_MANAGEMENT_H_
9  #define TCP_STREAM_MANAGEMENT_H_
10
11 #include <sys/types.h>
12 #include "api.h"
13
14 #ifdef __cplusplus
15 extern "C" {
16 #endif
17
18 /**
19  * If the sequence number of the received segment is 'x',
```

```
20  * and the expected sequence number is 'y',
21  * if x-y > DPI_TCP_MAX_OUT_OF_ORDER_BYTES, the segment will
22  * not be buffered.
23  */
24 #define DPI_TCP_MAX_OUT_OF_ORDER_BYTES 32768
25
26
27 enum dpi_tcp_reordering_statuses{
28     DPI_TCP_REORDERING_STATUS_IN_ORDER=0
29    ,DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER
30    ,DPI_TCP_REORDERING_STATUS_REBUILT
31 };
32
33
34
35 typedef struct dpi_tcp_reordering_reordered_segment{
36     unsigned char* data;
37     u_int32_t data_length;
38     u_int8_t status:2;
39     u_int8_t connection_terminated:1;
40 }dpi_tcp_reordering_reordered_segment_t;
41
42
43 /**
44  * Deletes all the pendent fragments belonging to a certain flow.
45  * @param victim
46  */
47 void dpi_reordering_tcp_delete_all_fragments(
48         dpi_tracking_informations_t *victim);
49
50
51 /**
52  * Tracks the TCP connection.
53  * @param pkt The informations about the packet.
54  * @param tracking A pointer to the structure containing the information
55  *                 about the TCP connection.
56  * @return If the packet is out of order, if it has already been received
57  *         or if an error occurred, the returned structure contains
58  *         DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER in the 'status' field.
59  *         The 'data' and 'data_length' fields in this case have no
60  *         meaning.
61  *
62  *         If the data is in order but doesn't fill an 'hole' in the
63  *         segment space, the returned structure contains
64  *         DPI_TCP_REORDERING_STATUS_IN_ORDER in the 'status' field.
65  *         The 'data' and 'data_length' fields in this case have no
66  *         meaning.
67  *
68  *         If the received data is in order and fills an 'hole' in
```

```
69  *         the segment space, the returned structure contains
70  *         DPI_TCP_REORDERING_STATUS_REBUILT in the 'status' field.
71  *         The 'data' field will contain a pointer to the new (longer)
72  *         segment. This pointer must be freed after the use. The
73  *         'data_length' field contains the length of the new (longer)
74  *         segment.
75  */
76 dpi_tcp_reordering_reordered_segment_t dpi_reordering_tcp_track_connection
77     (dpi_pkt_infos_t* pkt, dpi_tracking_informations_t* tracking);
78
79 /**
80  * Only checks if the connection terminates.
81  * @param pkt The informations about the packet.
82  * @param tracking A pointer to the structure containing the informations
83  *                 about the TCP connection.
84  * @return 1 if the connection is terminated, 0 otherwise.
85  */
86 u_int8_t dpi_reordering_tcp_track_connection_light
87     (dpi_pkt_infos_t* pkt, dpi_tracking_informations_t* tracking);
88
89 #ifdef __cplusplus
90 }
91 #endif
92
93 #endif /* TCP_STREAM_MANAGEMENT_H_ */
```

```
1  /*
2   * tcp_stream_management.c
3   *
4   *  Created on: 06/10/2012
5   *  Author: Daniele De Sensi (d.desensi@gmail.com)
6   */
7
8
9  #include "tcp_stream_management.h"
10 #include "reassembly.h"
11 #include "utils.h"
12 #include <stdlib.h>
13 #include <strings.h>
14 #include <string.h>
15 #include <stdio.h>
16 #include <netinet/tcp.h>
17 #include <arpa/inet.h>
18 #include <inttypes.h>
19 #include <assert.h>
20
21
22 #define DPI_DEBUG_TCP_REORDERING 0
23 #define debug_print(fmt, ...)                    \
```

```
24             do { if (DPI_DEBUG_TCP_REORDERING) \
25             fprintf(stdout, fmt, __VA_ARGS__); } while (0)
26
27
28 /**
29  * Deletes all the pendent fragments belonging to a certain flow.
30  * @param victim
31  */
32 void dpi_reordering_tcp_delete_all_fragments(
33         dpi_tracking_informations_t *victim){
34     if(victim){
35         dpi_reassembly_fragment_t *frag=victim->segments[0],
36                                   *temp_frag;
37
38         while(frag){
39             temp_frag=frag->next;
40             free(frag->ptr);
41             free(frag);
42             frag=temp_frag;
43         }
44
45         frag=victim->segments[1];
46         while(frag){
47             temp_frag=frag->next;
48             free(frag->ptr);
49             free(frag);
50             frag=temp_frag;
51         }
52     }
53 }
54
55 #ifndef DPI_DEBUG
56 static
57 #endif
58 u_int32_t dpi_reordering_tcp_length_contiguous_segments(
59             dpi_reassembly_fragment_t* head){
60     u_int32_t r=0;
61     u_int32_t last_end=head->offset;
62     while(head){
63         if(head->offset!=last_end){
64             return r;
65         }
66         r+=dpi_reassembly_fragment_length(head->offset, head->end);
67         if(head->tcp_fin){
68             ++r;
69         }
70         last_end=head->end;
71         head=head->next;
72     }
```

```
73      return r;
74  }
75
76
77  /**
78   * Group a certain number of contiguous segments copying them in
79   * where and freeing the old structures used to store them.
80   */
81  #ifndef DPI_DEBUG
82  static
83  #endif
84  void dpi_reordering_tcp_group_contiguous_segments(
85              dpi_reassembly_fragment_t** head, unsigned char** where){
86      /* Copy the data portions of all segments into the new buffer. */
87      u_int32_t fragment_length;
88      u_int32_t offset=0, last_end=(*head)->offset;
89      dpi_reassembly_fragment_t *fragment=*head;
90      dpi_reassembly_fragment_t *tmp;
91      while(fragment!=NULL && fragment->offset==last_end){
92          fragment_length=dpi_reassembly_fragment_length(fragment->offset,
93                                                         fragment->end);
94          if(fragment_length!=0)
95              memcpy(((*where)+offset), fragment->ptr, fragment_length);
96          offset+=fragment_length;
97          tmp=fragment->next;
98          last_end=fragment->end;
99          free(fragment->ptr);
100         free(fragment);
101         fragment=tmp;
102     }
103     *head=fragment;
104     if(fragment) fragment->prev=NULL;
105 }
106
107 /**
108  * According to RFC 1072 and RFC 793:
109  * "TCP determines if a data segment is "old" or "new" by testing if
110  * its sequence number is within 2**31 bytes of the left edge of the
111  * window.  If not, the data is "old" and discarded."
112  *
113  * The left edge of the receiver correspond to its highest ack sent.
114  * For this reason to ensure that the segment is new, we should check
115  * that its sequence number is between highest_ack[1-pkt->direction] and
116  * highest_ack[1-pkt->direction]+DPI_TCP_MAX_IN_TRAVEL_DATA.
117  *
118  * Anyway, expected_seq_num[direction] is in general >=
119  * highest_ack_num[1-direction], so we consider
120  * as left margin of the window expected_seq_num[direction]
121  *
```

```
122  **/
123 #ifndef DPI_DEBUG
124 static
125 #endif
126 u_int8_t dpi_reordering_tcp_is_new_segment(
127         u_int32_t seqnum, dpi_tracking_informations_t* tracking,
128         u_int8_t direction){
129     u_int32_t lowest=tracking->expected_seq_num[direction];
130     u_int32_t highest=lowest+DPI_TCP_MAX_IN_TRAVEL_DATA;
131
132     debug_print("Window: [%"PRIu32", %"PRIu32"]\n", lowest, highest);
133     if(lowest<=highest){
134         return seqnum>=lowest && seqnum<=highest;
135     }else{
136         return !(seqnum<lowest && seqnum>highest);
137     }
138 }
139
140 #ifndef DPI_DEBUG
141 static
142 #endif
143 void dpi_reordering_tcp_analyze_out_of_order(
144         dpi_pkt_infos_t* pkt, dpi_tracking_informations_t* tracking,
145         u_int32_t received_seq_num){
146     u_int32_t end=received_seq_num+pkt->data_length;
147     struct tcphdr* tcph=(struct tcphdr*) ((pkt->pkt)+(pkt->l4offset));
148     /**
149      * We pass a dummy variable because the memory bound on TCP
150      * reordering is not implemented at the moment.
151      **/
152     u_int32_t dummy;
153     dpi_reassembly_fragment_t* frag;
154
155     if(pkt->data_length==0){
156         debug_print("%s\n", "The segment has no payload");
157         if(tcph->fin==1 &&
158            !BIT_IS_SET(tracking->seen_fin, pkt->direction) &&
159            (frag=dpi_reassembly_insert_fragment(
160                    &(tracking->segments[pkt->direction]),
161             pkt->pkt+pkt->l7offset, received_seq_num, end,
162             &dummy, &dummy))){
163             frag->tcp_fin=1;
164             SET_BIT(tracking->seen_fin, pkt->direction);
165         }
166         return;
167     }
168
169     frag=dpi_reassembly_insert_fragment(
170             &(tracking->segments[pkt->direction]),
```

```
171             pkt->pkt+pkt->l7offset,
172             received_seq_num, end, &dummy, &dummy);
173     if(frag && tcph->fin==1){
174         frag->tcp_fin=1;
175         SET_BIT(tracking->seen_fin, pkt->direction);
176     }
177 }
178
179
180
181 /**
182  * Analyze the sequence numbers and puts the data in order.
183  * @param state The state of TCP reordering module.
184  * @param pkt The informations about the packet.
185  * @param tracking A pointer to the structure containing the
186  *                 informations about the TCP connection.
187  * @return Returns NULL if the packet is out of order, if it
188  *         has already been received or if an error occurred.
189  *         Otherwise it returns the reordered data. If the
190  *         data is in order but doesn't fill an 'hole' in the
191  *         segment space, the field 'status' is zero, the field
192  *         'data' points to the application data and 'data_length'
193  *         is the data_length of the application data. If the data
194  *         fills an hole in the sequence numbers space, the field
195  *         'status' is one, 'data' will contain a pointer to the
196  *         status segment (it must be freed after using it) and
197  *         'data_length' will contain the data_length of the new
198  *         reordered segment (data part only).
199  */
200 #ifndef DPI_DEBUG
201 static
202 #endif
203 dpi_tcp_reordering_reordered_segment_t
204         dpi_reordering_tcp_analyze_sequence_numbers(
205                 dpi_pkt_infos_t* pkt,
206                 dpi_tracking_informations_t* tracking){
207
208     dpi_tcp_reordering_reordered_segment_t to_return;
209     to_return.data=NULL;
210     to_return.data_length=0;
211     to_return.connection_terminated=0;
212     to_return.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
213
214     struct tcphdr* tcph=(struct tcphdr*) ((pkt->pkt)+(pkt->l4offset));
215     u_int32_t received_seq_num=ntohl(tcph->seq);
216     u_int32_t expected_seq_num=tracking->expected_seq_num[pkt->direction];
217     /** Automatically wrapped when exceed the 32bit limit. **/
218     u_int32_t end=received_seq_num+pkt->data_length;
219
```

```
220     debug_print("Direction: %d\n",pkt->direction);
221     debug_print("Received Seq Num: %"PRIu32" Expected: %"PRIu32"\n",
222                 received_seq_num,
223                 tracking->expected_seq_num[pkt->direction]);
224
225     u_int32_t ack=ntohl(tcph->ack_seq);
226
227     if(received_seq_num==expected_seq_num){
228         debug_print("%s\n", "Received in order segment");
229         to_return.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
230         tracking->expected_seq_num[pkt->direction]=end;
231         if(tcph->fin==1){
232             ++tracking->expected_seq_num[pkt->direction];
233             SET_BIT(tracking->seen_fin, pkt->direction);
234         }
235
236         /**
237          * If both FIN segments arrived and there are no more out of
238          * order segments, then the TCP connection is terminated and
239          * we can delete the flow informations.
240          **/
241         if(BIT_IS_SET(tracking->seen_fin, 0) &&
242            BIT_IS_SET(tracking->seen_fin, 1) &&
243            tracking->segments[0]==NULL &&
244            tracking->segments[1]==NULL){
245             to_return.connection_terminated=1;
246         }
247
248         if(pkt->data_length==0){
249             debug_print("%s\n", "The segment has no payload");
250             return to_return;
251         }
252
253         /**
254          * If there was out of order segments and this segment fills an
255          * hole, then group the segment together to make a bigger ordered
256          * segment. We check offset<=end because the received fragment
257          * could overlap with the pool_head of the segments.
258          **/
259         if(tracking->segments[pkt->direction] &&
260            tracking->segments[pkt->direction]->offset<=end){
261             u_int32_t overlap=
262                     end-tracking->segments[pkt->direction]->offset;
263             u_int32_t pkt_length=pkt->data_length-overlap;
264
265             debug_print("%s\n", "The segment fills an 'hole'");
266
267             u_int32_t new_length=
268                     dpi_reordering_tcp_length_contiguous_segments(
```

```
269                             tracking->segments[pkt->direction])+
270                             pkt_length;
271             unsigned char* buffer=(unsigned char*)
272                     malloc(sizeof(char)*(new_length));
273             assert(buffer);
274
275             memcpy(buffer, pkt->pkt+pkt->l7offset, pkt_length);
276             unsigned char* where=buffer+pkt_length;
277             dpi_reordering_tcp_group_contiguous_segments(
278                     &(tracking->segments[pkt->direction]), &where);
279
280             to_return.data=buffer;
281             to_return.data_length=new_length;
282             to_return.status=DPI_TCP_REORDERING_STATUS_REBUILT;
283
284             /**Update expected sequence number. **/
285             tracking->expected_seq_num[pkt->direction]=
286                     received_seq_num+new_length;
287         }else{
288             debug_print("%s\n", "The segment doesn't fill an 'hole'");
289         }
290         return to_return;
291     }else if(dpi_reordering_tcp_is_new_segment(received_seq_num,
292                                                tracking, pkt->direction)){
293         /** Out of order segment. **/
294         debug_print("Received out of order segment. Expected: %"PRIu32""
295                      " Received: %"PRIu32"\n", expected_seq_num,
296                      received_seq_num);
297         to_return.status=DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER;
298
299         if(dpi_reassembly_fragment_length(
300                 expected_seq_num, received_seq_num)>
301                     DPI_TCP_MAX_OUT_OF_ORDER_BYTES){
302             return to_return;
303         }else{
304             dpi_reordering_tcp_analyze_out_of_order(pkt, tracking,
305                                                     received_seq_num);
306         }
307         return to_return;
308     }else{
309         debug_print("Received old segment. SeqNum: %"PRIu32"\n",
310                     received_seq_num);
311         to_return.status=DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER;
312         return to_return;
313     }
314 }
315
316 /**
317  * Only checks if the connection terminates.
```

```
318  * @param pkt The informations about the packet.
319  * @param tracking A pointer to the structure containing the
320  *                 informations about the TCP connection.
321  * @return 1 if the connection is terminated, 0 otherwise.
322  */
323 u_int8_t dpi_reordering_tcp_track_connection_light(
324             dpi_pkt_infos_t* pkt, dpi_tracking_informations_t* tracking){
325     struct tcphdr* tcph=(struct tcphdr*) ((pkt->pkt)+(pkt->l4offset));
326     if(tcph->fin==1){
327         SET_BIT(tracking->seen_fin, pkt->direction);
328     }
329     /**
330      * If both FIN segments arrived, then the TCP connection is
331      * terminated and we can delete the flow informations.
332      **/
333     if(BIT_IS_SET(tracking->seen_fin, 0) &&
334        BIT_IS_SET(tracking->seen_fin, 1)){
335         return 1;
336     }
337     return 0;
338 }
339
340 /**
341  * Tracks the TCP connection.
342  * @param pkt The informations about the packet.
343  * @param tracking A pointer to the structure containing the information
344  *                 about the TCP connection.
345  * @return If the packet is out of order, if it has already been received
346  *         or if an error occurred, the returned structure contains
347  *         DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER in the 'status' field.
348  *         The 'data' and 'data_length' fields in this case have no
349  *         meaning.
350  *
351  *         If the data is in order but doesn't fill an 'hole' in the
352  *         segment space, the returned structure contains
353  *         DPI_TCP_REORDERING_STATUS_IN_ORDER in the 'status' field.
354  *         The 'data' and 'data_length' fields in this case have no
355  *         meaning.
356  *
357  *         If the received data is in order and fills an 'hole' in
358  *         the segment space, the returned structure contains
359  *         DPI_TCP_REORDERING_STATUS_REBUILT in the 'status' field.
360  *         The 'data' field will contain a pointer to the new (longer)
361  *         segment. This pointer must be freed after the use. The
362  *         'data_length' field contains the length of the new (longer)
363  *         segment.
364  */
365 dpi_tcp_reordering_reordered_segment_t ↙
        ↳ dpi_reordering_tcp_track_connection(
```

```
366         dpi_pkt_infos_t* pkt, dpi_tracking_informations_t* tracking){
367     dpi_tcp_reordering_reordered_segment_t to_return;
368     to_return.data=NULL;
369     to_return.data_length=0;
370     to_return.connection_terminated=0;
371     to_return.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
372
373     struct tcphdr* tcph=(struct tcphdr*) ((pkt->pkt)+(pkt->l4offset));
374
375     if(tracking->seen_ack){
376         debug_print("%s\n", "Connection already established, check "
377                     "sequence numbers");
378
379         return dpi_reordering_tcp_analyze_sequence_numbers(pkt, tracking);
380     }else if(tcph->syn!=0 && tcph->ack==0 && tracking->seen_syn==0 &&
381              tracking->seen_syn_ack==0 && tracking->seen_ack==0){
382         tracking->seen_syn=1;
383         tracking->expected_seq_num[pkt->direction]=ntohl(tcph->seq)+1;
384
385         debug_print("%s\n", "Syn received.");
386         debug_print("Chosen sequence number: %"PRIu32" for direction: "
387                     "%d\n", tracking->expected_seq_num[pkt->direction],
388                     pkt->direction);
389
390         to_return.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
391         return to_return;
392     }else if(tcph->syn!=0 && tcph->ack!=0 && tracking->seen_syn==1 &&
393              tracking->seen_syn_ack==0  && tracking->seen_ack==0){
394         tracking->seen_syn_ack=1;
395         tracking->expected_seq_num[pkt->direction]=ntohl(tcph->seq)+1;
396
397         debug_print("%s\n", "SynAck received.");
398         debug_print("Chosen sequence number: %"PRIu32" for direction: "
399                     "%d\n", tracking->expected_seq_num[pkt->direction],
400                     pkt->direction);
401
402         to_return.status=DPI_TCP_REORDERING_STATUS_IN_ORDER;
403         return to_return;
404     }else if(tcph->syn==0 && tcph->ack==1 && tracking->seen_syn==1 &&
405              tracking->seen_syn_ack==1 && tracking->seen_ack==0){
406         debug_print("%s\n", "Ack received, handshake concluded.");
407         tracking->seen_ack=1;
408         /**
409          * Some TCP implementations carry data with the 3-rd handshake
410          * segment. Moreover if the 3-rd handshake segment is lost,
411          * this branch of the 'if' will be triggered by the successive
412          * segments, which can carry data. For this reason we don't
413          * exit from the function and we analyze the sequence numebers.
414          */
```

```
415         return dpi_reordering_tcp_analyze_sequence_numbers(pkt, tracking);
416     }else{
417         /**
418          *  Received segments from connections from which we didn't see
419          *  the syn. We observe the sequence numbers and acknowledgment
420          *  numbers in the two direction up to the point in which we
421          *  are able to understand which is the real status of the
422          *  connection (when seq and ack numbers in the two directions
423          *  coincide).
424          *  Notice that doing it in this way we will lose the data of
425          *  the first packets.
426          */
427         to_return.status=DPI_TCP_REORDERING_STATUS_OUT_OF_ORDER;
428
429         u_int32_t seq=ntohl(tcph->seq);
430         u_int32_t ack=ntohl(tcph->ack_seq);
431         debug_print("NOSYN branch. Direction: %d\n", pkt->direction);
432
433         if(!BIT_IS_SET(tracking->first_packet_arrived, pkt->direction)){
434             if(!tcph->syn)
435                 tracking->expected_seq_num[pkt->direction]=
436                         seq+pkt->data_length;
437             else
438                 tracking->expected_seq_num[pkt->direction]=
439                         seq+1;
440
441             tracking->highest_ack[pkt->direction]=ack;
442
443             SET_BIT(tracking->first_packet_arrived, pkt->direction);
444         }else{
445             if(dpi_reassembly_after(
446                     seq, tracking->expected_seq_num[pkt->direction])){
447                 tracking->expected_seq_num[pkt->direction]=
448                         seq+pkt->data_length;
449             }
450
451             if(dpi_reassembly_after(
452                     ack, tracking->highest_ack[pkt->direction])){
453                 tracking->highest_ack[pkt->direction]=ack;
454             }
455         }
456
457         debug_print("expected[0]: %"PRIu32" highestack[1]: %"PRIu32"\n",
458                 tracking->expected_seq_num[0], tracking->highest_ack[1]);
459         debug_print("expected[1]: %"PRIu32" highestack[0]: %"PRIu32"\n",
460                 tracking->expected_seq_num[1], tracking->highest_ack[0]);
461
462         /** Hooked! **/
463         if(BIT_IS_SET(tracking->first_packet_arrived, 0) &&
```

```
464             BIT_IS_SET(tracking->first_packet_arrived, 1) &&
465             tracking->expected_seq_num[0]==tracking->highest_ack[1] &&
466             tracking->expected_seq_num[1]==tracking->highest_ack[0]){
467              debug_print("%s\n", "Hooked!!!");
468              /**
469               * We left seen_syn=0 because in this way we can signal that
470               * this is a connection that we didn't start looking
471               * from the beginning.
472               */
473              tracking->seen_ack=1;
474              return dpi_reordering_tcp_analyze_sequence_numbers(
475                      pkt, tracking);
476         }else
477              return to_return;
478     }
479 }
```

Appendix B

Code of the parallel implementation

B.1 Nodes definition

```
/*
 * worker.hpp
 *
 *  Created on: 12/11/2012
 *  Author: Daniele De Sensi (d.desensi@gmail.com)
 */

#ifndef WORKER_HPP_
#define WORKER_HPP_

#include <ff/farm.hpp>
#include <ff/svector.hpp>

#include "../DPI/api.h"
#include "mp_dpi_api.h"
#include "configure.hpp"


#define DPI_DEBUG_MP_WORKER 0
#define worker_debug_print(fmt, ...)      \
            do { if (DPI_DEBUG_MP_WORKER) \
            fprintf(stdout, fmt, __VA_ARGS__); } while (0)

namespace dpi{

typedef struct L3_L4_input_task{
    const unsigned char* pkt;
    u_int32_t length;
```

```
29      u_int32_t current_time;
30      void* user_pointer;
31  }L3_L4_input_task_struct;
32
33  typedef struct L3_L4_output_task{
34      u_int32_t hash_result;
35      u_int16_t destination_worker;
36      int8_t status;
37      dpi_pkt_infos_t pkt_infos;
38      void* user_pointer;
39  }L3_L4_output_task_struct;
40
41
42  typedef struct L7_output_task{
43      dpi_identification_result_t result;
44      void* user_pointer;
45  }L7_output_task_struct;
46
47  #define DPI_CACHE_LINES_PADDING_REQUIRED(size)             \
48          (size%DPI_CACHE_LINE_SIZE==0?0:DPI_CACHE_LINE_SIZE \
49           -(size%DPI_CACHE_LINE_SIZE))
50
51
52  typedef struct mp_dpi_task{
53      union input_output_task{
54          L3_L4_input_task_struct
55              L3_L4_input_task_t[DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE];
56          L3_L4_output_task_struct
57              L3_L4_output_task_t[DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE];
58          L7_output_task_struct
59              L7_output_task_t[DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE];
60      }input_output_task_t;
61      char padding[DPI_CACHE_LINES_PADDING_REQUIRED(
62              sizeof(input_output_task_t))];
63  }mp_dpi_task_t;
64
65
66  /*****************************************************/
67  /*                      L3_L4 nodes.                 */
68  /*****************************************************/
69
70  class dpi_L3_L4_emitter: public ff::ff_node{
71  private:
72      char padding1[DPI_CACHE_LINE_SIZE];
73      dpi_mp_packet_reading_callback** const cb;
74      void** user_data;
75      u_int8_t* freeze_flag;
76      u_int8_t* terminating;
77      const u_int16_t proc_id;
```

```
78      ff::SWSR_Ptr_Buffer* tasks_pool;
79      u_int8_t initialized;
80      char padding2[DPI_CACHE_LINE_SIZE];
81  public:
82      dpi_L3_L4_emitter(dpi_mp_packet_reading_callback** cb,
83                        void** user_data,
84                        u_int8_t* freeze_flag,
85                        u_int8_t* terminating,
86                        u_int16_t proc_id,
87                        ff::SWSR_Ptr_Buffer* tasks_pool);
88      ~dpi_L3_L4_emitter();
89
90      int svc_init();
91      void* svc(void*);
92  };
93
94
95
96  class dpi_L3_L4_worker: public ff::ff_node{
97  private:
98      char padding1[DPI_CACHE_LINE_SIZE];
99      dpi_library_state_t* const state;
100     L3_L4_input_task_struct* in;
101     const u_int32_t v4_worker_table_size;
102     const u_int32_t v6_worker_table_size;
103     const uint worker_id;
104     const u_int16_t proc_id;
105     char padding2[DPI_CACHE_LINE_SIZE];
106 public:
107     dpi_L3_L4_worker(dpi_library_state_t* state,
108                      uint worker_id,
109                      u_int16_t proc_id,
110                      u_int32_t v4_worker_table_size,
111                      u_int32_t v6_worker_table_size);
112     ~dpi_L3_L4_worker();
113
114     int svc_init();
115     void* svc(void*);
116 };
117
118 class dpi_L3_L4_collector: public ff::ff_node{
119 private:
120     char padding1[DPI_CACHE_LINE_SIZE];
121     const u_int16_t proc_id;
122     char padding2[DPI_CACHE_LINE_SIZE];
123 public:
124     dpi_L3_L4_collector(u_int16_t proc_id);
125
126     int svc_init();
```

```
127     void* svc(void*);
128 };
129
130
131 /*****************************************************/
132 /*                        L7 nodes.                  */
133 /*****************************************************/
134
135 class dpi_L7_scheduler: public ff::ff_loadbalancer{
136 private:
137     char padding1[DPI_CACHE_LINE_SIZE];
138     int victim;
139     char padding2[DPI_CACHE_LINE_SIZE];
140 protected:
141     inline int selectworker(){
142         worker_debug_print("[worker.hpp]: select_worker: %u\n",
143                            victim);
144         return victim;
145     }
146 public:
147     dpi_L7_scheduler(int max_num_workers):
148             ff_loadbalancer(max_num_workers), victim(0){}
149     void set_victim(int v){
150         victim=v;
151         worker_debug_print("[worker.hpp]: set_victim: %u\n",
152                            victim);
153     }
154 };
155
156 class dpi_L7_emitter: public ff::ff_node{
157 private:
158     char padding1[DPI_CACHE_LINE_SIZE];
159     dpi_L7_scheduler* const lb;
160     mp_dpi_task_t* partially_filled;
161     uint* partially_filled_sizes;
162     mp_dpi_task_t** waiting_tasks;
163     u_int16_t waiting_tasks_size;
164     const u_int16_t proc_id;
165     char padding2[DPI_CACHE_LINE_SIZE];
166 public:
167     dpi_L7_emitter(dpi_L7_scheduler* lb,
168                    u_int16_t num_L7_workers,
169                    u_int16_t proc_id);
170     ~dpi_L7_emitter();
171     int svc_init();
172     void* svc(void* task);
173 };
174
175 class dpi_L7_worker: public ff::ff_node{
```

```
176 private:
177     char padding1[DPI_CACHE_LINE_SIZE];
178     dpi_library_state_t* const state;
179     L3_L4_output_task_struct* temp;
180     const u_int16_t worker_id;
181     const u_int16_t proc_id;
182     char padding2[DPI_CACHE_LINE_SIZE];
183 public:
184     dpi_L7_worker(dpi_library_state_t* state,
185                   u_int16_t worker_id,
186                   u_int16_t proc_id);
187     ~dpi_L7_worker();
188     int svc_init();
189     void* svc(void*);
190 };
191
192
193 class dpi_L7_collector: public ff::ff_node{
194 private:
195     char padding1[DPI_CACHE_LINE_SIZE];
196     dpi_mp_processing_result_callback** const cb;
197     void** user_data;
198     const u_int16_t proc_id;
199     ff::SWSR_Ptr_Buffer* tasks_pool;
200     u_int8_t initialized;
201     char padding2[DPI_CACHE_LINE_SIZE];
202 public:
203     dpi_L7_collector(dpi_mp_processing_result_callback** cb,
204                      void** user_data, u_int16_t proc_id,
205                      ff::SWSR_Ptr_Buffer* tasks_pool);
206     int svc_init();
207     void* svc(void*);
208     void svc_end();
209 };
210
211
212 class dpi_collapsed_emitter: public dpi::dpi_L7_emitter{
213 private:
214     char padding1[DPI_CACHE_LINE_SIZE];
215     dpi_L3_L4_emitter* L3_L4_emitter;
216     dpi_L3_L4_worker* L3_L4_worker;
217     u_int16_t proc_id;
218     char padding2[DPI_CACHE_LINE_SIZE];
219 public:
220     dpi_collapsed_emitter(dpi_mp_packet_reading_callback** cb,
221                           void** user_data, u_int8_t* freeze_flag,
222                           u_int8_t* terminating,
223                           ff::SWSR_Ptr_Buffer* tasks_pool,
224                           dpi_library_state_t* state,
```

```
225                             u_int32_t v4_worker_table_size,
226                             u_int32_t v6_worker_table_size,
227                             dpi_L7_scheduler* lb,
228                             u_int16_t num_workers,
229                             u_int16_t proc_id);
230     ~dpi_collapsed_emitter();
231     int svc_init();
232     void* svc(void*);
233     void svc_end();
234 };
235
236 }
237
238
239 #endif /* WORKER_HPP_ */
```

```
1  /*
2   * worker.cpp
3   *
4   *  Created on: 12/11/2012
5   *  Author: Daniele De Sensi (d.desensi@gmail.com)
6   */
7
8  #include "worker.hpp"
9  #include "mp_dpi_api.h"
10 #include "../DPI/flow_table.h"
11 #include <pthread.h>
12 #include <stdlib.h>
13 #include <ff/mapping_utils.hpp>
14
15 #if DPI_NUMA_AWARE
16 #include <numa.h>
17 #endif
18
19 namespace dpi{
20
21 #ifndef DPI_DEBUG
22 static inline
23 #endif
24 mp_dpi_task_t* dpi_allocate_task(){
25     mp_dpi_task_t* r;
26 #if DPI_NUMA_AWARE
27     r=(mp_dpi_task_t*) numa_alloc_onnode(sizeof(mp_dpi_task_t),
28                        DPI_NUMA_AWARE_TASKS_NODE);
29 #else
30     #if DPI_MULTIPROCESSOR_ALIGN_TASKS
31         assert(posix_memalign((void**) &r, DPI_CACHE_LINE_SIZE,
32                sizeof(mp_dpi_task_t))==0);
33     #else
```

```
34          r=new mp_dpi_task_t;
35      #endif
36  #endif
37      assert(r);
38      return r;
39  }
40
41  #ifndef DPI_DEBUG
42  static inline
43  #endif
44  void dpi_free_task(mp_dpi_task_t* task){
45  #if DPI_NUMA_AWARE
46      numa_free(task, sizeof(mp_dpi_task_t));
47  #else
48      #if DPI_MULTIPROCESSOR_ALIGN_TASKS
49          free(task);
50      #else
51          delete task;
52      #endif
53  #endif
54  }
55
56
57
58
59  /*****************************************************/
60  /*                      L3_L4 nodes.                 */
61  /*****************************************************/
62  dpi_L3_L4_emitter::dpi_L3_L4_emitter(dpi_mp_packet_reading_callback** cb,
63                                       void** user_data,
64                                       u_int8_t* freeze_flag,
65                                       u_int8_t* terminating,
66                                       u_int16_t proc_id,
67                                       ff::SWSR_Ptr_Buffer* tasks_pool)
68                                       :cb(cb), user_data(user_data),
69                                        freeze_flag(freeze_flag),
70                                        terminating(terminating),
71                                        proc_id(proc_id),
72                                        tasks_pool(tasks_pool),
73                                        initialized(0){
74      ;
75  }
76
77  int dpi_L3_L4_emitter::svc_init(){
78      worker_debug_print("[worker.cpp]: L3_L4 emitter mapped on "
79                         "processor: %d\n", proc_id);
80      ff_mapThreadToCpu(proc_id,-20);
81
82      if(!initialized){
```

```
83          /** Fill the task pool. **/
84  #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
85          for(uint i=0; i<DPI_MULTIPROCESSOR_TASKS_POOL_SIZE; i++){
86              tasks_pool->push(dpi_allocate_task());
87          }
88  #endif
89          initialized=1;
90      }
91      return 0;
92  }
93
94  void* dpi_L3_L4_emitter::svc(void* task){
95      dpi_mp_packet_reading_result_t packet;
96      mp_dpi_task_t* r=NULL;
97
98      if(unlikely(*freeze_flag)){
99          worker_debug_print("%s\n", "[worker.cpp]: Freeze message ↙
        ↳ received,"
100                                 " terminating.");
101         return (void*) ff::FF_EOS;
102     }
103
104 #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
105     if(!tasks_pool->empty()){
106         tasks_pool->pop((void**) &r);
107     }else{
108         r=dpi_allocate_task();
109     }
110 #else
111     r=dpi_allocate_task();
112 #endif
113
114     for(uint i=0; i<DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE; i++){
115         packet=(*(*cb))(*user_data);
116         if(unlikely(packet.pkt==NULL)){
117             worker_debug_print("%s\n", "[worker.cpp]: No more task to "
118                                 "process, terminating.");
119             *terminating=1;
120             return (void*) ff::FF_EOS;
121         }
122
123         r->input_output_task_t.L3_L4_input_task_t[i].user_pointer=
124                 packet.user_pointer;
125         r->input_output_task_t.L3_L4_input_task_t[i].current_time=
126                 packet.current_time;
127         r->input_output_task_t.L3_L4_input_task_t[i].length=
128                 packet.length;
129         r->input_output_task_t.L3_L4_input_task_t[i].pkt=
130                 packet.pkt;
```

```
131         __builtin_prefetch(
132                 &(r->input_output_task_t.L3_L4_input_task_t[i+5]), 1, 0);
133     }
134     return (void*) r;
135 }
136
137 dpi_L3_L4_emitter::~dpi_L3_L4_emitter(){
138     ;
139 }
140
141
142 dpi_L3_L4_worker::dpi_L3_L4_worker(dpi_library_state_t* state,
143                                    uint worker_id,
144                                    u_int16_t proc_id,
145                                    u_int32_t v4_worker_table_size,
146                                    u_int32_t v6_worker_table_size)
147                                   :state(state), worker_id(worker_id),
148                                    proc_id(proc_id),
149                                    ↙
    ↳ v4_worker_table_size(v4_worker_table_size),
150                                    ↙
    ↳ v6_worker_table_size(v6_worker_table_size){
151     assert(posix_memalign((void**) &in, DPI_CACHE_LINE_SIZE,
152            sizeof(L3_L4_input_task_struct)*
153            DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE)==0);
154 }
155
156 dpi_L3_L4_worker::~dpi_L3_L4_worker(){
157     free(in);
158 }
159
160 int dpi_L3_L4_worker::svc_init(){
161     worker_debug_print("[worker.cpp]: L3_L4 worker %d mapped "
162                        "on processor: %d\n", worker_id, proc_id);
163     ff_mapThreadToCpu(proc_id,-20);
164     return 0;
165 }
166
167
168 void* dpi_L3_L4_worker::svc(void* task){
169     mp_dpi_task_t* real_task=(mp_dpi_task_t*) task;
170
171     /**
172      * Here we need a copy. Indeed, the task is a union and, if
173      * we do not make the copy, we overwrite the input tasks with
174      * the generated output tasks.
175      **/
176     memcpy(in, real_task->input_output_task_t.L3_L4_input_task_t,
177             DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE*
```

```
178             sizeof(L3_L4_input_task_struct));
179
180     dpi_pkt_infos_t pkt_infos;
181     for(uint i=0; i<DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE; i++){
182         __builtin_prefetch(&(in[i+2]), 0, 0);
183         __builtin_prefetch((in[i+2]).pkt, 0, 0);
184
185         real_task->input_output_task_t.L3_L4_output_task_t[i].status=
186                 mp_dpi_extract_packet_infos(this->state, in[i].pkt,
187                                             in[i].length, &pkt_infos,
188                                             in[i].current_time,
189                                             worker_id);
190
191         /* To have always a consistent value temp L7 worker selection. */
192         real_task->input_output_task_t.
193             L3_L4_output_task_t[i].hash_result=0;
194         real_task->input_output_task_t.
195             L3_L4_output_task_t[i].destination_worker=0;
196         real_task->input_output_task_t.
197             L3_L4_output_task_t[i].user_pointer=in[i].user_pointer;
198
199         if(likely(real_task->input_output_task_t.
200                     L3_L4_output_task_t[i].status>=0)){
201
202             if(pkt_infos.l4prot!=IPPROTO_TCP &&
203                pkt_infos.l4prot!=IPPROTO_UDP){
204                 real_task->input_output_task_t.
205                     L3_L4_output_task_t[i].status=
206                             DPI_ERROR_TRANSPORT_PROTOCOL_NOTSUPPORTED;
207                 continue;
208             }
209
210             if(likely(real_task->input_output_task_t
211                         .L3_L4_output_task_t[i].status!=
212                                 DPI_STATUS_IP_FRAGMENT)){
213                 real_task->input_output_task_t.
214                         L3_L4_output_task_t[i].pkt_infos=pkt_infos;
215                 if(pkt_infos.ip_version==DPI_IP_VERSION_4){
216                     real_task->input_output_task_t.L3_L4_output_task_t[i].
217                         hash_result=dpi_compute_v4_hash_function(
218                                 (dpi_flow_DB_v4*) this->state->db4,
219                                 &pkt_infos);
220                     real_task->input_output_task_t.L3_L4_output_task_t[i].
221                         destination_worker=real_task->input_output_task_t.
222                         L3_L4_output_task_t[i].
223                         hash_result/v4_worker_table_size;
224                 }else{
225                     real_task->input_output_task_t.L3_L4_output_task_t[i].
226                     hash_result=dpi_compute_v6_hash_function(
```

```
227                         (dpi_flow_DB_v6*) this->state->db6, &pkt_infos);
228                     real_task->input_output_task_t.L3_L4_output_task_t[i].
229                     destination_worker=real_task->input_output_task_t.
230                     L3_L4_output_task_t[i].
231                     hash_result/v6_worker_table_size;
232                 }
233             }
234         }
235     }
236     return real_task;
237 }
238
239
240 dpi_L3_L4_collector::dpi_L3_L4_collector(u_int16_t proc_id):
241         proc_id(proc_id){
242     ;
243 }
244
245 int dpi_L3_L4_collector::svc_init(){
246     worker_debug_print("[worker.cpp]: L3_L4 collector mapped "
247                        "on processor: %d\n", proc_id);
248     ff_mapThreadToCpu(proc_id,-20);
249     return 0;
250 }
251
252 void* dpi_L3_L4_collector::svc(void* task){
253     return task;
254 }
255
256
257 /*****************************************************************/
258 /*                          L7 nodes.                            */
259 /*****************************************************************/
260
261 dpi_L7_emitter::dpi_L7_emitter(dpi_L7_scheduler* lb,
262                                u_int16_t num_L7_workers,
263                                u_int16_t proc_id)
264                               :lb(lb), proc_id(proc_id){
265     assert(posix_memalign((void**) &partially_filled_sizes,
266           DPI_CACHE_LINE_SIZE, (sizeof(uint)*num_L7_workers)+
267           DPI_CACHE_LINE_SIZE)==0);
268     bzero(partially_filled_sizes, sizeof(uint)*num_L7_workers);
269
270     assert(posix_memalign((void**) &partially_filled,
271           DPI_CACHE_LINE_SIZE,
272           (sizeof(mp_dpi_task_t)*num_L7_workers)+
273           DPI_CACHE_LINE_SIZE)==0);
274     bzero(partially_filled, sizeof(mp_dpi_task_t)*num_L7_workers);
275
```

```
276     assert(posix_memalign((void**) &waiting_tasks,
277            DPI_CACHE_LINE_SIZE,
278            (sizeof(mp_dpi_task_t*)*num_L7_workers*2)+
279            DPI_CACHE_LINE_SIZE)==0);
280
281     waiting_tasks_size=0;
282     for(uint i=0; i<num_L7_workers; i++){
283         waiting_tasks[i]=dpi_allocate_task();
284         ++waiting_tasks_size;
285     }
286 }
287
288 dpi_L7_emitter::~dpi_L7_emitter(){
289     free(partially_filled_sizes);
290     free(partially_filled);
291 }
292
293 int dpi_L7_emitter::svc_init(){
294     worker_debug_print("[worker.cpp]: L7 emitter mapped "
295                        "on processor: %d\n", proc_id);
296     ff_mapThreadToCpu(proc_id,-20);
297     return 0;
298 }
299
300 void* dpi_L7_emitter::svc(void* task){
301     mp_dpi_task_t* real_task=(mp_dpi_task_t*) task;
302     u_int16_t destination_worker;
303     mp_dpi_task_t* out;
304     uint pfs;
305
306     for(uint i=0; i<DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE; i++){
307         __builtin_prefetch(&(real_task->input_output_task_t.
308                 L3_L4_output_task_t[i+4]), 0, 0);
309         destination_worker=real_task->input_output_task_t.
310                 L3_L4_output_task_t[i].destination_worker;
311         worker_debug_print("[worker.cpp]: L7 emitter: Inserted"
312                            " a task into the queue of worker: "
313                            "%d\n", destination_worker);
314         pfs=partially_filled_sizes[destination_worker];
315         __builtin_prefetch(&(partially_filled[destination_worker].
316                 input_output_task_t.L3_L4_output_task_t[pfs]), 1, 0);
317
318         if(pfs+1==DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE){
319             assert(waiting_tasks_size!=0);
320             out=waiting_tasks[--waiting_tasks_size];
321             memcpy(out->input_output_task_t.L3_L4_output_task_t,
322                    partially_filled[destination_worker].
323                        input_output_task_t.L3_L4_output_task_t,
324                    sizeof(L3_L4_output_task_struct)*
```

```
325                         (DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE-1));
326             out->input_output_task_t.L3_L4_output_task_t
327                 [DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE-1]=
328                         real_task->input_output_task_t.
329                             L3_L4_output_task_t[i];
330             lb->set_victim(destination_worker);
331             while(ff_send_out((void*) out)==false);
332             partially_filled_sizes[destination_worker]=0;
333         }else{
334             partially_filled[destination_worker].input_output_task_t.
335                 L3_L4_output_task_t[pfs]=real_task->input_output_task_t.
336                     L3_L4_output_task_t[i];
337             ++partially_filled_sizes[destination_worker];
338         }
339     }
340     waiting_tasks[waiting_tasks_size]=real_task;
341     ++waiting_tasks_size;
342     return (void*) ff::FF_GO_ON;
343 }
344
345
346
347 dpi_L7_worker::dpi_L7_worker(dpi_library_state_t* state,
348                              u_int16_t worker_id,
349                              u_int16_t proc_id):
350                              state(state),
351                              worker_id(worker_id),
352                              proc_id(proc_id){
353     assert(posix_memalign((void**) &this->temp, DPI_CACHE_LINE_SIZE,
354                           (sizeof(L3_L4_output_task_struct)*
355                                   DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE)+
356                           DPI_CACHE_LINE_SIZE)==0);
357 }
358
359 dpi_L7_worker::~dpi_L7_worker(){
360     free(temp);
361 }
362
363 int dpi_L7_worker::svc_init(){
364     worker_debug_print("[worker.cpp]: L7 worker %d mapped on"
365                        " processor: %d\n", worker_id, proc_id);
366     ff_mapThreadToCpu(proc_id,-20);
367     return 0;
368 }
369
370
371 void* dpi_L7_worker::svc(void* task){
372     mp_dpi_task_t* real_task=(mp_dpi_task_t*) task;
373     int8_t l3_status;
```

```
374     dpi_pkt_infos_t infos;
375     dpi_flow_infos_t* flow_infos;
376     ipv4_flow_t* ipv4_flow;
377     ipv6_flow_t* ipv6_flow;
378
379     memcpy(temp, real_task->input_output_task_t.L3_L4_output_task_t,
380            DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE*
381                sizeof(L3_L4_output_task_struct));
382
383     for(uint i=0; i<DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE; i++){
384         real_task->input_output_task_t.L7_output_task_t[i].user_pointer=
385                 temp[i].user_pointer;
386         ipv4_flow=NULL;
387         ipv6_flow=NULL;
388
389         l3_status=temp[i].status;
390         if(unlikely(l3_status<0 || l3_status==DPI_STATUS_IP_FRAGMENT)){
391             real_task->input_output_task_t.L7_output_task_t[i].
392                 result.status=l3_status;
393             continue;
394         }
395         infos=temp[i].pkt_infos;
396         __builtin_prefetch(temp[i+1].pkt_infos.pkt+temp[i+1].
397                 pkt_infos.l7offset, 0, 0);
398
399         if(infos.ip_version==DPI_IP_VERSION_4){
400             ipv4_flow=mp_dpi_flow_table_find_or_create_flow_v4(
401                     state, this->worker_id, temp[i].hash_result,
402                     &(infos));
403             flow_infos=&(ipv4_flow->infos);
404         }else{
405             ipv6_flow=mp_dpi_flow_table_find_or_create_flow_v6(
406                     state, this->worker_id, temp[i].hash_result,
407                     &(infos));
408             flow_infos=&(ipv6_flow->infos);
409         }
410
411         real_task->input_output_task_t.L7_output_task_t[i].result.
412             status=DPI_STATUS_OK;
413         if(unlikely(flow_infos==NULL)){
414             real_task->input_output_task_t.L7_output_task_t[i].result.
415                 status=DPI_ERROR_MAX_FLOWS;
416             if(unlikely(l3_status==DPI_STATUS_IP_LAST_FRAGMENT)){
417                 free((unsigned char*) infos.pkt);
418             }
419             break;
420         }
421
422         real_task->input_output_task_t.L7_output_task_t[i].result=
```

```
423                 dpi_stateless_get_app_protocol(state, flow_infos,
424                                                &(infos));
425         if(real_task->input_output_task_t.L7_output_task_t[i].result.
426                 status==DPI_STATUS_TCP_CONNECTION_TERMINATED){
427             if(ipv4_flow!=NULL){
428                 mp_dpi_flow_table_delete_flow_v4(
429                         (dpi_flow_DB_v4_t*) state->db4,
430                         state->flow_cleaner_callback,
431                         this->worker_id,
432                         ipv4_flow);
433             }else{
434                 mp_dpi_flow_table_delete_flow_v6(
435                         (dpi_flow_DB_v6_t*) state->db6,
436                         state->flow_cleaner_callback,
437                         this->worker_id,
438                         ipv6_flow);
439             }
440         }
441
442         if(unlikely(l3_status==DPI_STATUS_IP_LAST_FRAGMENT)){
443             free((unsigned char*) infos.pkt);
444         }
445     }
446     return real_task;
447 }
448
449
450 dpi_L7_collector::dpi_L7_collector(dpi_mp_processing_result_callback** cb,
451                                    void** user_data,
452                                    u_int16_t proc_id,
453                                    ff::SWSR_Ptr_Buffer* tasks_pool)
454                                    :cb(cb), user_data(user_data),
455                                     proc_id(proc_id),
456                                     tasks_pool(tasks_pool),
457                                     initialized(0){
458     ;
459 }
460
461 int dpi_L7_collector::svc_init(){
462     worker_debug_print("[worker.cpp]: L7 collector"
463                        " mapped on processor: %d\n", proc_id);
464     ff_mapThreadToCpu(proc_id,-20);
465     if(!initialized){
466         initialized=1;
467     }
468     return 0;
469 }
470
471 void* dpi_L7_collector::svc(void* task){
```

```
472     mp_dpi_processing_result_t r;
473     mp_dpi_task_t* real_task=(mp_dpi_task_t*) task;
474     for(uint i=0; i<DPI_MULTIPROCESSOR_DEFAULT_GRAIN_SIZE; i++){
475         r.result=real_task->input_output_task_t.L7_output_task_t[i].
476                 result;
477         r.user_pointer=real_task->input_output_task_t.
478                 L7_output_task_t[i].user_pointer;
479         (*(*cb))(&r, *user_data);
480     }
481 #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
482     if(tasks_pool->available()){
483         tasks_pool->push(task);
484     }else{
485         dpi_free_task(real_task);
486     }
487 #else
488     dpi_free_task(real_task, allocator);
489 #endif
490     return (void*) ff::FF_GO_ON;
491 }
492
493 void dpi_L7_collector::svc_end(){
494 #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
495     mp_dpi_task_t* task;
496     while(!tasks_pool->empty()){
497         tasks_pool->pop((void**) &task);
498         dpi_free_task(task);
499     }
500 #endif
501 }
502
503
504 dpi_collapsed_emitter::dpi_collapsed_emitter(
505         dpi_mp_packet_reading_callback** cb,
506         void** user_data, u_int8_t* freeze_flag,
507         u_int8_t* terminating,
508         ff::SWSR_Ptr_Buffer* tasks_pool,
509         dpi_library_state_t* state,
510         u_int32_t v4_worker_table_size,
511         u_int32_t v6_worker_table_size,
512         dpi_L7_scheduler* lb,
513         u_int16_t num_workers, u_int16_t proc_id):
514                 dpi_L7_emitter(lb, num_workers, proc_id),
515                 proc_id(proc_id){
516     L3_L4_emitter=new dpi::dpi_L3_L4_emitter(cb, user_data,
517                                              freeze_flag,
518                                              terminating,
519                                              proc_id,
520                                              tasks_pool);
```

```
521     L3_L4_worker=new dpi::dpi_L3_L4_worker(state, 0, proc_id,
522                                            v4_worker_table_size,
523                                            v6_worker_table_size);
524 }
525
526 dpi_collapsed_emitter::~dpi_collapsed_emitter(){
527     delete L3_L4_emitter;
528     delete L3_L4_worker;
529 }
530
531 int dpi_collapsed_emitter::svc_init(){
532     L3_L4_emitter->svc_init();
533     L3_L4_worker->svc_init();
534     dpi_L7_emitter::svc_init();
535     worker_debug_print("[worker.cpp]: collapsed emitter mapped "
536                        "on processor: %d\n", proc_id);
537     return 0;
538 }
539 void* dpi_collapsed_emitter::svc(void* task){
540     void* r=L3_L4_emitter->svc(task);
541     if(unlikely(r==(void*) ff::FF_EOS)){
542         return r;
543     }else{
544         return dpi_L7_emitter::svc(L3_L4_worker->svc(r));
545     }
546 }
547
548 void dpi_collapsed_emitter::svc_end(){
549     L3_L4_emitter->svc_end();
550     L3_L4_worker->svc_end();
551     dpi_L7_emitter::svc_end();
552 }
553
554
555 }
```

B.2 API implementation

```
1  /*
2   * mp_dpi_api.h
3   *
4   * This file contains the API of the multiprocessor version of DPI
5   * library. The state of the library can be used by multiple threads
6   * if and only if it is accessed in mutual exclusion. The reason of
7   * this is that the library has been designed to be used mainly by
8   * single threaded applications because the multithreaded part is
9   * already embedded in the library. In some cases the library could
10  * be used by multiple threads without harming the performances,
```

```
11  * e.g.:
12  *
13  * +In passive mode we could have one thread to offload/load and another
14  * thread which monitors the performances and enable/disable library
15  * features/inspectors according to them.
16  *
17  * +In the same way, in active mode, the thread which runs the library
18  * could enable/disable library features while the library is running.
19  * Notice that in this case is not needed any lock because only one
20  * thread is modifying the state (the thread of the library which
21  * executes the callback doesn't matter).
22  *
23  *  Created on: 12/11/2012
24  *  Author: Daniele De Sensi (d.desensi@gmail.com)
25  */
26
27 #ifndef MP_DPI_API_H_
28 #define MP_DPI_API_H_
29
30 #include "configure.hpp"
31 #include "../DPI/api.h"
32 #include "benchmark.hpp"
33
34 typedef struct dpi_mp_library_state dpi_mp_library_state_t;
35
36 typedef struct mp_dpi_processing_result{
37     void* user_pointer;
38     dpi_identification_result_t result;
39 }mp_dpi_processing_result_t;
40
41 typedef struct dpi_mp_packet_reading_result{
42     const unsigned char* pkt;
43     u_int32_t length;
44     u_int32_t current_time;
45     void* user_pointer;
46 }dpi_mp_packet_reading_result_t;
47
48 /**
49  * @struct dpi_mp_parallelism_details_t
50  * @brief Represents some details that can be specified by the user
51  * during initializations.
52  *
53  * @var available processors The maximum number of processors available
54  *                           for the library. If 0, all the processors
55  *                           will be considered available.
56  * @var mapping An array of processors identifiers on which the library
57  *              can be mapped. If NULL, a linear mapping will be
58  *              applied.
59  *
```

```
60   * @var automatic_derivation If 1, the best parallelism form will be
61   *                           automatically computed. Otherwise, the one
62   *                           defined by the programmer will be used.
63   *
64   * @var parallelism_form DPI_MP_PARELLELISM_FORM_DOUBLE_FARM or
65   *                       DPI_MP_PARALLELISM_FORM_ONE_FARM. By default it
66   *                       is equal to DPI_MP_PARALLELISM_FORM_ONE_FARM.
67   * @var double_farm_num_L3_workers   If parallelism_form==
68   *                                   DPI_MP_PARELLELISM_FORM_DOUBLE_FARM,
69   *                                   it represents the number of workers
70   *                                   to activate for the first farm. It
71   *                                   must be different from 0.
72   * @var double_farm_num_L7_workers   If parallelism_form==
73   *                                   DPI_MP_PARELLELISM_FORM_DOUBLE_FARM,
74   *                                   it represents the number of workers
75   *                                   to activate for the second farm. It
76   *                                   must be different from 0.
77   * @var single_farm_num_workers      If parallelism_form==
78   *                                   DPI_MP_PARALLELISM_FORM_ONE_FARM, it
79   *                                   represents the number of workers to
80   *                                   activate. It must be different
81   *                                   from 0.
82   */
83  typedef struct dpi_mp_parallelism_details{
84      /** Mapping informations. **/
85      u_int16_t available_processors;
86      u_int16_t* mapping;
87      /** If 1 the best parallelism form will be automatically computed. **/
88      u_int8_t automatic_derivation;
89      /** User manual specification of parallelism form. **/
90      u_int8_t parallelism_form;
91      u_int16_t double_farm_num_L3_workers;
92      u_int16_t double_farm_num_L7_workers;
93      u_int16_t single_farm_num_workers;
94  }dpi_mp_parallelism_details_t;
95
96
97  /**
98   * This function will be called by the library (active mode only) to read
99   * a packet from the network.
100  * @param callback_data   A pointer to user specified data (e.g.
101  *                        network socket).
102  * @return                The packet read. If the pkt field is NULL, then
103  *                        there are no more data to read and the library
104  *                        will terminate. The user must never try to
105  *                        modify the state after that he returned
106  *                        pkt=NULL, otherwise the behaviour is not
107  *                        defined.
108  */
```

```
109 typedef dpi_mp_packet_reading_result_t(dpi_mp_packet_reading_callback)
110             (void* callback_data);
111
112 /**
113  * This function will be called by the library (active mode only) to
114  * process the result of the protocol identification.
115  * @param processing_result   A pointer to the result of the library
116  *                            processing.
117  * @param callback_data       A pointer to user specified data (e.g.
118  *                            network socket).
119  */
120 typedef void(dpi_mp_processing_result_callback)
121             (mp_dpi_processing_result_t* processing_result,
122              void* callback_data);
123
124 /**
125  * Initializes the library and sets the parallelism degree according to
126  * the cost model obtained from the parameters that the user specifies.
127  * If not specified otherwise after the initialization, the library will
128  * consider all the protocols active.
129  *
130  * @param size_v4 Size of the array of pointers used to build the database
131  *                for v4 flows.
132  * @param size_v6 Size of the array of pointers used to build the database
133  *                for v6 flows.
134  * @param max_active_v4_flows The maximum number of IPv4 flows which can
135  *                            be active at any time. After reaching this
136  *                            threshold, new flows will not be created.
137  * @param max_active_v6_flows The maximum number of IPv6 flows which can
138  *                            be active at any time. After reaching this
139  *                            threshold, new flows will not be created.
140  * @param parallelism_details Details about the parallelism form. Must be
141  *                            zeroed and then filled by the user.
142  * @return A pointer to the state of the library.
143  */
144 dpi_mp_library_state_t* mp_dpi_init_stateful(
145         u_int32_t size_v4, u_int32_t size_v6,
146         u_int32_t max_active_v4_flows,
147         u_int32_t max_active_v6_flows,
148         dpi_mp_parallelism_details_t parallelism_details);
149
150
151
152 /**
153  * Sets the reading and processing callbacks. It can be done only after
154  * that the state has been initialized and before calling run().
155  *
156  * @param state                 A pointer to the state of the library.
157  * @param reading_callback      A pointer to the reading callback. It must
```

```
158  *                                be different from NULL.
159  * @param processing_callback   A pointer to the processing callback. It
160  *                                must be different from NULL.
161  * @param user_data             A pointer to the user data to be passed to
162  *                                the callbacks.
163  */
164 void mp_dpi_set_read_and_process_callbacks(
165         dpi_mp_library_state_t* state,
166         dpi_mp_packet_reading_callback* reading_callback,
167         dpi_mp_processing_result_callback* processing_callback,
168         void* user_data);
169
170
171 /**
172  * Starts the library.
173  * @param state A pointer to the state of the library.
174  */
175 void mp_dpi_run(dpi_mp_library_state_t* state);
176
177 /**
178  * Wait the end of the data processing.
179  * @param state A pointer to the state of the library.
180  */
181 void mp_dpi_wait_end(dpi_mp_library_state_t* state);
182
183 /**
184  * Prints execution's statistics.
185  * @param state A pointer to the state of the library.
186  */
187 void mp_dpi_print_stats(dpi_mp_library_state_t* state);
188
189 /**
190  * Terminates the library.
191  * @param state A pointer to the state of the library.
192  */
193 void mp_dpi_terminate(dpi_mp_library_state_t *state);
194
195
196 /*************************************************/
197 /*          Status change API calls              */
198 /*************************************************/
199
200 /**
201  * Sets the maximum number of times that the library tries to guess the
202  * protocol. During the flow protocol identification, after this number
203  * of trials, in the case in which it cannot decide between two or more
204  * protocols, one of them will be chosen, otherwise DPI_PROTOCOL_UNKNOWN
205  * will be returned.
206  * @param state       A pointer to the state of the library.
```

```
207  * @param max_trials  The maximum number of trials.
208  *
209  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
210  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
211  *         been changed because a problem happened.
212  */
213 u_int8_t mp_dpi_set_max_trials(dpi_mp_library_state_t *state,
214                                u_int16_t max_trials);
215
216
217 /**
218  * Enable IPv4 defragmentation.
219  * @param state        A pointer to the library state.
220  * @param table_size   The size of the table to be used to store IPv4
221  *                     fragments informations.
222  *
223  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
224  *          updated. DPI_STATE_UPDATE_FAILURE if the state has not
225  *         been changed because a problem happened.
226  */
227 u_int8_t mp_dpi_ipv4_fragmentation_enable(dpi_mp_library_state_t *state,
228                                           u_int16_t table_size);
229
230 /**
231  * Enable IPv6 defragmentation.
232  * @param state        A pointer to the library state.
233  * @param table_size   The size of the table to be used to store IPv6
234  *                     fragments informations.
235  *
236  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
237  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
238  *         been changed because a problem happened.
239  */
240 u_int8_t mp_dpi_ipv6_fragmentation_enable(dpi_mp_library_state_t *state,
241                                           u_int16_t table_size);
242
243 /**
244  * Sets the amount of memory that a single host can use for IPv4
245  * defragmentation.
246  * @param state                   A pointer to the library state.
247  * @param per_host_memory_limit   The maximum amount of memory that
248  *                                 any IPv4 host can use.
249  *
250  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
251  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
252  *         been changed because a problem happened.
253  */
254 u_int8_t mp_dpi_ipv4_fragmentation_set_per_host_memory_limit(
255         dpi_mp_library_state_t *state,
```

```
256         u_int32_t per_host_memory_limit);
257
258 /**
259  * Sets the amount of memory that a single host can use for IPv6
260  * defragmentation.
261  * @param state                   A pointer to the library state.
262  * @param per_host_memory_limit   The maximum amount of memory that
263  *                                any IPv6 host can use.
264  *
265  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
266  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
267  *         been changed because a problem happened.
268  */
269 u_int8_t mp_dpi_ipv6_fragmentation_set_per_host_memory_limit(
270         dpi_mp_library_state_t *state,
271         u_int32_t per_host_memory_limit);
272
273 /**
274  * Sets the total amount of memory that can be used for IPv4
275  * defragmentation.
276  * If fragmentation is disabled and then enabled, this information
277  * must be passed again.
278  * Otherwise default value will be used.
279  * @param state               A pointer to the state of the library
280  * @param totel_memory_limit  The maximum amount of memory that can
281  *                             be used for IPv4 defragmentation.
282  *
283  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
284  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
285  *         been changed because a problem happened.
286  */
287 u_int8_t mp_dpi_ipv4_fragmentation_set_total_memory_limit(
288         dpi_mp_library_state_t *state,
289         u_int32_t total_memory_limit);
290
291 /**
292  * Sets the total amount of memory that can be used for
293  * IPv6 defragmentation.
294  * If fragmentation is disabled and then enabled, this information
295  * must be passed again.
296  * Otherwise default value will be used.
297  * @param state               A pointer to the state of the library
298  * @param totel_memory_limit  The maximum amount of memory that can
299  *                            be used for IPv6 defragmentation.
300  *
301  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
302  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
303  *         been changed because a problem happened.
304  */
```

```
305 u_int8_t mp_dpi_ipv6_fragmentation_set_total_memory_limit(
306         dpi_mp_library_state_t *state,
307         u_int32_t total_memory_limit);
308
309 /**
310  * Sets the maximum time (in seconds) that can be spent to
311  * reassembly an IPv4 fragmented datagram.
312  * Is the maximum time gap between the first and last fragments
313  * of the datagram.
314  * @param state            A pointer to the state of the library.
315  * @param timeout_seconds  The reassembly timeout.
316  *
317  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
318  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the
319  *         state has not been changed because a problem happened.
320  */
321 u_int8_t mp_dpi_ipv4_fragmentation_set_reassembly_timeout(
322         dpi_mp_library_state_t *state,
323         u_int8_t timeout_seconds);
324
325 /**
326  * Sets the maximum time (in seconds) that can be spent to reassembly
327  * an IPv6 fragmented datagram.
328  * Is the maximum time gap between the first and last fragments of
329  * the datagram.
330  * @param state            A pointer to the state of the library.
331  * @param timeout_seconds  The reassembly timeout.
332  *
333  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
334  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
335  *         been changed because a problem happened.
336  */
337 u_int8_t mp_dpi_ipv6_fragmentation_set_reassembly_timeout(
338         dpi_mp_library_state_t *state,
339         u_int8_t timeout_seconds);
340
341 /**
342  * Disable IPv4 defragmentation.
343  * @param state A pointer to the state of the library.
344  *
345  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
346  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the
347  *         state has not been changed because a problem happened.
348  */
349 u_int8_t mp_dpi_ipv4_fragmentation_disable(dpi_mp_library_state_t *state);
350
351 /**
352  * Disable IPv6 defragmentation.
353  * @param state A pointer to the state of the library.
```

```
354  *
355  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
356  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
357  *         been changed because a problem happened.
358  */
359 u_int8_t mp_dpi_ipv6_fragmentation_disable(dpi_mp_library_state_t *state);
360
361
362
363 /**
364  * If enabled, the library will reorder out of order TCP packets
365  * (enabled by default).
366  * @param state  A pointer to the state of the library.
367  *
368  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
369  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the state
370  *         has not been changed because a problem happened.
371  */
372 u_int8_t mp_dpi_tcp_reordering_enable(dpi_mp_library_state_t* state);
373
374 /**
375  * If it is called, the library will not reorder out of order TCP packets.
376  * Out-of-order segments will be delivered to the inspector as they
377  * arrive. This means that the inspector may not be able to identify
378  * the application protocol. Moreover, if there are callbacks saved
379  * for TCP based protocols, if TCP reordering is disabled, the
380  * extracted informations could be erroneous or incomplete.
381  * @param state A pointer to the state of the library.
382  *
383  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
384  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
385  *         been changed because a problem happened.
386  */
387 u_int8_t mp_dpi_tcp_reordering_disable(dpi_mp_library_state_t* state);
388
389
390 /**
391  * Enable a protocol inspector.
392  * @param state         A pointer to the state of the library.
393  * @param protocol      The protocol to enable.
394  *
395  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
396  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
397  *         been changed because a problem happened.
398  */
399 u_int8_t mp_dpi_set_protocol(dpi_mp_library_state_t *state,
400                              dpi_protocol_t protocol);
401
402 /**
```

```
403  * Disable a protocol inspector.
404  * @param state       A pointer to the state of the library.
405  * @param protocol    The protocol to disable.
406  *
407  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
408  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
409  *         been changed because a problem happened.
410  */
411 u_int8_t mp_dpi_delete_protocol(dpi_mp_library_state_t *state,
412                                 dpi_protocol_t protocol);
413
414 /**
415  * Enable all the protocol inspector.
416  * @param state      A pointer to the state of the library.
417  *
418  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
419  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
420  *         been changed because a problem happened.
421  */
422 u_int8_t mp_dpi_inspect_all(dpi_mp_library_state_t *state);
423
424 /**
425  * Disable all the protocol inspector.
426  * @param state      A pointer to the state of the library.
427  *
428  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
429  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
430  *         been changed because a problem happened.
431  */
432 u_int8_t mp_dpi_inspect_nothing(dpi_mp_library_state_t *state);
433
434
435 /**
436  * Sets the callback that will be called when a flow expires.
437  * (Valid only if stateful API is used).
438  * @param state     A pointer to the state of the library.
439  * @param cleaner   The callback used to clear the user state.
440  *
441  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
442  *         successfully updated. DPI_STATE_UPDATE_FAILURE if
443  *         the state has not been changed because a problem
444  *         happened.
445  */
446 u_int8_t mp_dpi_set_flow_cleaner_callback(
447         dpi_mp_library_state_t* state,
448         dpi_flow_cleaner_callback* cleaner);
449
450 /**
451  * Sets callbacks informations. When a protocol is identified the
```

```
452  * default behavior is to not inspect the packets belonging to that
453  * flow anymore and keep simply returning the same protocol identifier.
454  *
455  * If a callback is enabled for a certain protocol, then we keep
456  * inspecting all the new flows with that protocol in order to
457  * invoke the callbacks specified by the user on the various parts
458  * of the message. Moreover, if the application protocol uses TCP,
459  * then we have the additional cost of TCP reordering for all the
460  * segments. Is highly recommended to enable TCP reordering if it is
461  * not already enabled (remember that is enabled by default).
462  * Otherwise the informations extracted could be erroneous/incomplete.
463  *
464  * The pointers to the data passed to the callbacks are valid only for
465  * the duration of the callback.
466  *
467  * @param state       A pointer to the state of the library.
468  * @param callbacks   A pointer to HTTP callbacks.
469  * @param user_data   A pointer to global user HTTP data. This pointer
470  *                    will be passed to any HTTP callback when it is
471  *                    invoked.
472  *
473  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
474  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
475  *         been changed because a problem happened.
476  *
477  **/
478 u_int8_t mp_dpi_http_activate_callbacks(
479         dpi_mp_library_state_t* state,
480         dpi_http_callbacks_t* callbacks,
481         void* user_data);
482
483 /**
484  * Remove the internal structure used to store callbacks informations.
485  * user_data is not freed/modified.
486  * @param state       A pointer to the state of the library.
487  *
488  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
489  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
490  *         been changed because a problem happened.
491  */
492 u_int8_t mp_dpi_http_disable_callbacks(dpi_mp_library_state_t* state);
493
494
495 #endif /* MP_DPI_API_H_ */
```

```
1  /*
2   * mp_dpi_api.cpp
3   *
4   *  Created on: 12/11/2012
```

```
5   *  Author: Daniele De Sensi (d.desensi@gmail.com)
6   */
7
8
9  #include "configure.hpp"
10 #include "mp_dpi_api.h"
11 #include "utils.hpp"
12 #include "worker.hpp"
13 #include <stddef.h>
14 #include <vector>
15 #include <math.h>
16
17 #include <ff/farm.hpp>
18 #include <ff/pipeline.hpp>
19 #include <ff/buffer.hpp>
20
21 #define DPI_DEBUG_MP_API 0
22 #define debug_print(fmt, ...)          \
23             do { if (DPI_DEBUG_MP_API) \
24             fprintf(stdout, fmt, __VA_ARGS__); } while (0)
25
26
27 enum tids{
28     DPI_MULTIPROCESSOR_STATUS_UPDATER_TID=1
29    ,DPI_MULTIPROCESSOR_WORKER_RECONFIGURATOR_TID
30 };
31
32
33
34 typedef struct dpi_mp_library_state{
35     dpi_library_state_t* sequential_state;
36     ff::SWSR_Ptr_Buffer* tasks_pool;
37
38     u_int8_t parallel_module_type;
39     /*****************************************************/
40     /*                     Callbacks                     */
41     /*****************************************************/
42     dpi_mp_packet_reading_callback* reading_callback;
43     dpi_mp_processing_result_callback* processing_callback;
44     void* read_process_callbacks_user_data;
45
46     u_int8_t is_frozen:1;
47     u_int8_t freeze_flag;
48     u_int8_t terminating;
49
50     /**
51      * This lock keeps the state locked between a freeze and
52      * the successive run. **/
53     ff::CLHSpinLock state_update_lock;
```

```
54      /*****************************************************/
55      /*                  Nodes for single farm.             */
56      /*****************************************************/
57      ff::ff_farm<dpi::dpi_L7_scheduler>* single_farm;
58      std::vector<ff::ff_node*>* single_farm_workers;
59      dpi::dpi_collapsed_emitter* single_farm_emitter;
60      dpi::dpi_L7_collector* single_farm_collector;
61
62      u_int16_t single_farm_active_workers;
63
64      /*****************************************************/
65      /*                  Nodes for double farm.             */
66      /*****************************************************/
67      dpi::dpi_L3_L4_emitter* L3_L4_emitter;
68  #if DPI_MULTIPROCESSOR_L3_L4_FARM_TYPE == \
69      DPI_MULTIPROCESSOR_L3_L4_ORDERED_FARM
70      ff::ff_ofarm* L3_L4_farm;
71  #else
72      ff::ff_farm<>* L3_L4_farm;
73  #endif
74      std::vector<ff::ff_node*>* L3_L4_workers;
75      dpi::dpi_L3_L4_collector* L3_L4_collector;
76
77      dpi::dpi_L7_emitter* L7_emitter;
78      ff::ff_farm<dpi::dpi_L7_scheduler>* L7_farm;
79      std::vector<ff::ff_node*>* L7_workers;
80      dpi::dpi_L7_collector* L7_collector;
81      ff::ff_pipeline* pipeline;
82      u_int16_t double_farm_L3_L4_active_workers;
83      u_int16_t double_farm_L7_active_workers;
84  }dpi_mp_library_state_t;
85
86
87
88
89  #define DPI_MP_NUM_SUPPORT_THREADS 2 /** emitter + collector. **/
90
91
92  #ifndef DPI_DEBUG
93  static inline
94  #endif
95  void mp_dpi_create_double_farm(dpi_mp_library_state_t* state,
96                                 u_int16_t available_procs,
97                                 u_int16_t* mapping,
98                                 u_int32_t size_v4,
99                                 u_int32_t size_v6){
100     u_int16_t last_mapped=0;
101     /****************************************/
102     /*         Create the first farm.         */
```

```
103     /****************************************/
104     void* tmp;
105 #if DPI_MULTIPROCESSOR_L3_L4_FARM_TYPE == \
106     DPI_MULTIPROCESSOR_L3_L4_ORDERED_FARM
107     tmp=malloc(sizeof(ff::ff_ofarm));
108     assert(tmp);
109     state->L3_L4_farm=new (tmp) ff::ff_ofarm(
110             false,
111             DPI_MULTIPROCESSOR_L3_L4_FARM_INPUT_BUFFER_SIZE,
112             DPI_MULTIPROCESSOR_L3_L4_FARM_OUTPUT_BUFFER_SIZE,
113             false, available_procs, true);
114     tmp=malloc(sizeof(dpi::dpi_L3_L4_emitter));
115     assert(tmp);
116     state->L3_L4_emitter=new (tmp) dpi::dpi_L3_L4_emitter(
117             &(state->reading_callback),
118             &(state->read_process_callbacks_user_data),
119             &(state->freeze_flag), &(state->terminating),
120             mapping[last_mapped], state->tasks_pool);
121     last_mapped=(last_mapped+1)%available_procs;
122     state->L3_L4_farm->setEmitterF(state->L3_L4_emitter);
123 #else
124     tmp=malloc(sizeof(ff::ff_farm<>));
125     assert(tmp);
126     state->L3_L4_farm=new (tmp) ff::ff_farm<>(
127             false,
128             DPI_MULTIPROCESSOR_L3_L4_FARM_INPUT_BUFFER_SIZE,
129             DPI_MULTIPROCESSOR_L3_L4_FARM_OUTPUT_BUFFER_SIZE,
130             false, available_procs, true);
131     tmp=malloc(sizeof(dpi::dpi_L3_L4_emitter));
132     assert(tmp);
133     state->L3_L4_emitter=new (tmp) dpi::dpi_L3_L4_emitter(
134             &(state->reading_callback),
135             &(state->read_process_callbacks_user_data),
136             &(state->freeze_flag), &(state->terminating),
137             mapping[last_mapped], state->tasks_pool);
138     last_mapped=(last_mapped+1)%available_procs;
139     state->L3_L4_farm->add_emitter(state->L3_L4_emitter);
140 #if DPI_MULTIPROCESSOR_L3_L4_FARM_TYPE == \
141     DPI_MULTIPROCESSOR_L3_L4_ON_DEMAND
142     state->L3_L4_farm->set_scheduling_ondemand(1024);
143 #endif
144 #endif
145
146     state->L3_L4_workers=new std::vector<ff::ff_node*>;
147     dpi::dpi_L3_L4_worker* w1;
148     for(uint i=0; i<state->double_farm_L3_L4_active_workers; i++){
149         tmp=malloc(sizeof(dpi::dpi_L3_L4_worker));
150         assert(tmp);
151         w1=new (tmp) dpi::dpi_L3_L4_worker(state->sequential_state, i,
```

```
152             mapping[last_mapped],
153             ceil((float)size_v4/
154                    (float)state->double_farm_L7_active_workers),
155             ceil((float)size_v6/
156                    (float)state->double_farm_L7_active_workers));
157         state->L3_L4_workers->push_back(w1);
158         last_mapped=(last_mapped+1)%available_procs;
159     }
160     assert(state->L3_L4_farm->add_workers(*(state->L3_L4_workers))==0);
161 
162     tmp=malloc(sizeof(dpi::dpi_L3_L4_collector));
163     assert(tmp);
164     state->L3_L4_collector=new (tmp)
165                            dpi::dpi_L3_L4_collector(mapping[last_mapped]);
166     assert(state->L3_L4_collector);
167     last_mapped=(last_mapped+1)%available_procs;
168 #if DPI_MULTIPROCESSOR_L3_L4_FARM_TYPE == \
169     DPI_MULTIPROCESSOR_L3_L4_ORDERED_FARM
170     state->L3_L4_farm->setCollectorF(state->L3_L4_collector);
171 #else
172     assert(state->L3_L4_farm->add_collector(state->L3_L4_collector)>=0);
173 #endif
174 
175     /*************************************/
176     /*      Create the second farm.       */
177     /*************************************/
178     tmp=malloc(sizeof(ff::ff_farm<dpi::dpi_L7_scheduler>));
179     assert(tmp);
180     state->L7_farm=new (tmp) ff::ff_farm<dpi::dpi_L7_scheduler>(
181             false, DPI_MULTIPROCESSOR_L7_FARM_INPUT_BUFFER_SIZE,
182             DPI_MULTIPROCESSOR_L7_FARM_OUTPUT_BUFFER_SIZE, false,
183             available_procs, true);
184 
185     tmp=malloc(sizeof(dpi::dpi_L7_emitter));
186     assert(tmp);
187     state->L7_emitter=new (tmp) dpi::dpi_L7_emitter(
188             state->L7_farm->getlb(),
189             state->double_farm_L7_active_workers,
190             mapping[last_mapped]);
191     last_mapped=(last_mapped+1)%available_procs;
192     state->L7_farm->add_emitter(state->L7_emitter);
193 
194     state->L7_workers=new std::vector<ff::ff_node*>;
195     dpi::dpi_L7_worker* w2;
196     for(uint i=0; i<state->double_farm_L7_active_workers; i++){
197         tmp=malloc(sizeof(dpi::dpi_L7_worker));
198         assert(tmp);
199         w2=new (tmp) dpi::dpi_L7_worker(state->sequential_state, i,
200                                         mapping[last_mapped]);
```

```
201         state->L7_workers->push_back(w2);
202         last_mapped=(last_mapped+1)%available_procs;
203     }
204     assert(state->L7_farm->add_workers(*(state->L7_workers))==0);
205 
206     tmp=malloc(sizeof(dpi::dpi_L7_collector));
207     assert(tmp);
208     state->L7_collector=new (tmp) dpi::dpi_L7_collector(
209             &(state->processing_callback),
210             &(state->read_process_callbacks_user_data),
211             mapping[last_mapped], state->tasks_pool);
212     last_mapped=(last_mapped+1)%available_procs;
213     assert(state->L7_farm->add_collector(state->L7_collector)>=0);
214 
215     /*********************************/
216     /*     Create the pipeline.     */
217     /*********************************/
218     tmp=malloc(sizeof(ff::ff_pipeline));
219     assert(tmp);
220     state->pipeline=new (tmp) ff::ff_pipeline(
221             false,
222             DPI_MULTIPROCESSOR_PIPELINE_INPUT_BUFFER_SIZE,
223             DPI_MULTIPROCESSOR_PIPELINE_OUTPUT_BUFFER_SIZE,
224             true);
225 
226     state->pipeline->add_stage(state->L3_L4_farm);
227     state->pipeline->add_stage(state->L7_farm);
228     state->parallel_module_type=DPI_MP_PARELLELISM_FORM_DOUBLE_FARM;
229 }
230 
231 #ifndef DPI_DEBUG
232 static inline
233 #endif
234 void mp_dpi_create_single_farm(dpi_mp_library_state_t* state,
235                                u_int16_t available_procs,
236                                u_int16_t* mapping,
237                                u_int32_t size_v4, u_int32_t size_v6){
238     u_int16_t last_mapped=0;
239     state->single_farm=new ff::ff_farm<dpi::dpi_L7_scheduler>(
240             false,
241             DPI_MULTIPROCESSOR_L7_FARM_INPUT_BUFFER_SIZE,
242             DPI_MULTIPROCESSOR_L7_FARM_OUTPUT_BUFFER_SIZE,
243             false, available_procs, true);
244     assert(state->single_farm);
245 
246     state->single_farm_emitter=new dpi::dpi_collapsed_emitter(
247             &(state->reading_callback),
248             &(state->read_process_callbacks_user_data),
249             &(state->freeze_flag), &(state->terminating),
```

```
250             state->tasks_pool, state->sequential_state,
251             ceil((float)size_v4/(float)state->single_farm_active_workers),
252             ceil((float)size_v6/(float)state->single_farm_active_workers),
253             state->single_farm->getlb(),
254             state->single_farm_active_workers, mapping[last_mapped]);
255     assert(state->single_farm_emitter);
256     last_mapped=(last_mapped+1)%available_procs;
257     state->single_farm->add_emitter(state->single_farm_emitter);
258
259     state->single_farm_workers=new std::vector<ff::ff_node*>;
260     dpi::dpi_L7_worker* w;
261     for(u_int16_t i=0; i<state->single_farm_active_workers; i++){
262         w=new dpi::dpi_L7_worker(state->sequential_state, i,
263                                  mapping[last_mapped]);
264         assert(w);
265         state->single_farm_workers->push_back(w);
266         last_mapped=(last_mapped+1)%available_procs;
267     }
268
269     assert(state->single_farm->add_workers(
270                 *(state->single_farm_workers))==0);
271     state->single_farm_collector=new dpi::dpi_L7_collector(
272             &(state->processing_callback),
273             &(state->read_process_callbacks_user_data),
274             mapping[last_mapped], state->tasks_pool);
275     assert(state->single_farm_collector);
276     last_mapped=(last_mapped+1)%available_procs;
277     assert(state->single_farm->add_collector(
278             state->single_farm_collector)>=0);
279     state->parallel_module_type=DPI_MP_PARALLELISM_FORM_ONE_FARM;
280 }
281
282
283
284 /**
285  * Initializes the library and sets the parallelism degree according to
286  * the cost model obtained from the parameters that the user specifies.
287  * If not specified otherwise after the initialization, the library will
288  * consider all the protocols active.
289  *
290  * @param size_v4 Size of the array of pointers used to build the database
291  *                for v4 flows.
292  * @param size_v6 Size of the array of pointers used to build the database
293  *                for v6 flows.
294  * @param max_active_v4_flows The maximum number of IPv4 flows which can
295  *                            be active at any time. After reaching this
296  *                            threshold, new flows will not be created.
297  * @param max_active_v6_flows The maximum number of IPv6 flows which can
298  *                            be active at any time. After reaching this
```

```
299  *                            threshold, new flows will not be created.
300  * @param parallelism_details Details about the parallelism form. Must be
301  *                            zeroed and then filled by the user.
302  * @return A pointer to the state of the library.
303  */
304 dpi_mp_library_state_t* mp_dpi_init_stateful(
305         u_int32_t size_v4, u_int32_t size_v6,
306         u_int32_t max_active_v4_flows,
307         u_int32_t max_active_v6_flows,
308         dpi_mp_parallelism_details_t parallelism_details){
309     dpi_mp_library_state_t* state;
310     assert(posix_memalign((void**) &state, DPI_CACHE_LINE_SIZE,
311            sizeof(dpi_mp_library_state_t)+DPI_CACHE_LINE_SIZE)==0);
312     bzero(state, sizeof(dpi_mp_library_state_t));
313
314     if(parallelism_details.available_processors==0)
315         parallelism_details.available_processors=
316                 dpi_get_num_of_processors();
317
318     assert(parallelism_details.available_processors>0);
319     assert(parallelism_details.available_processors>=
320             DPI_MP_NUM_SUPPORT_THREADS+1);
321
322     u_int8_t delete_mapping=0;
323     if(parallelism_details.mapping==NULL){
324         parallelism_details.mapping=
325                 new u_int16_t[parallelism_details.available_processors];
326         dpi_generate_mapping(parallelism_details.mapping,
327                              parallelism_details.available_processors);
328         delete_mapping=1;
329     }
330
331     state->freeze_flag=0;
332     state->is_frozen=1;
333     state->terminating=0;
334     ff::init_unlocked(&(state->state_update_lock));
335     ff::spin_lock(&(state->state_update_lock),
336                   DPI_MULTIPROCESSOR_STATUS_UPDATER_TID);
337
338     u_int8_t parallelism_form;
339
340     if(parallelism_details.automatic_derivation==1){
341         dpi::Benchmark benchmark;
342         parallelism_form=benchmark.analyze_cost_model();
343         state->double_farm_L3_L4_active_workers=
344                 benchmark.double_farm_get_num_L3_workers();
345         state->double_farm_L7_active_workers=
346                 benchmark.double_farm_get_num_L7_workers();
347         state->single_farm_active_workers=
```

```
348                 benchmark.single_farm_get_num_workers();
349     }else{
350         parallelism_form=parallelism_details.parallelism_form;
351         state->double_farm_L3_L4_active_workers=
352                 parallelism_details.double_farm_num_L3_workers;
353         state->double_farm_L7_active_workers=
354                 parallelism_details.double_farm_num_L7_workers;
355         state->single_farm_active_workers=
356                 parallelism_details.single_farm_num_workers;
357         if(parallelism_form==DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
358             assert(state->double_farm_L3_L4_active_workers!=0 &&
359                    state->double_farm_L7_active_workers!=0);
360         }else{
361             assert(state->single_farm_active_workers!=0);
362         }
363     }
364
365     u_int16_t hash_table_partitions;
366     if(parallelism_form==DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
367         debug_print("%s\n","[mp_dpi_api.cpp]: A pipeline of two "
368                     "farms will be activated.");
369         hash_table_partitions=state->double_farm_L7_active_workers;
370     }else{
371         debug_print("%s\n","[mp_dpi_api.cpp]: Only one farm will "
372                     "be activated.");
373         hash_table_partitions=state->single_farm_active_workers;
374     }
375
376
377     state->sequential_state=dpi_init_stateful_num_partitions(
378             size_v4,
379             size_v6,
380             max_active_v4_flows,
381             max_active_v6_flows,
382             hash_table_partitions);
383
384
385     /******************************/
386     /*   Create the tasks pool.   */
387     /******************************/
388 #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
389     void* tmp;
390     assert(posix_memalign((void**) &tmp, DPI_CACHE_LINE_SIZE,
391            sizeof(ff::SWSR_Ptr_Buffer)+DPI_CACHE_LINE_SIZE)==0);
392     state->tasks_pool=new (tmp) ff::SWSR_Ptr_Buffer(
393             DPI_MULTIPROCESSOR_TASKS_POOL_SIZE);
394     state->tasks_pool->init();
395 #endif
396
```

```
397     if(parallelism_form==DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
398         mp_dpi_create_double_farm(
399                 state,
400                 parallelism_details.available_processors,
401                 parallelism_details.mapping,
402                 size_v4, size_v6);
403     }else{
404         mp_dpi_create_single_farm(
405                 state,
406                 parallelism_details.available_processors,
407                 parallelism_details.mapping,
408                 size_v4, size_v6);
409     }
410
411     if(delete_mapping)
412         delete[] parallelism_details.mapping;
413
414     return state;
415 }
416
417 /**
418  * Prints execution's statistics.
419  * @param state A pointer to the state of the library.
420  */
421 void mp_dpi_print_stats(dpi_mp_library_state_t* state){
422     if(state){
423         if(state->parallel_module_type==
424                 DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
425             state->pipeline->ffStats(std::cout);
426             std::cout << "Completion time: " <<
427                     std::max(ff::diffmsec(
428                             state->L3_L4_farm->getwstoptime(),
429                             state->L3_L4_farm->getwstartime()),
430                             ff::diffmsec(
431                                     state->L7_farm->getwstoptime(),
432                                     state->L7_farm->getwstartime()))
433             << std::endl;
434         }else{
435             state->single_farm->ffStats(std::cout);
436             std::cout << "Completion time: " <<
437                     ff::diffmsec(state->single_farm->getwstoptime(),
438                             state->single_farm->getwstartime())
439             << std::endl;
440         }
441     }
442 }
443
444
445 /**
```

```
446  * Terminates the library.
447  * @param state A pointer to the state of the library.
448  */
449 void mp_dpi_terminate(dpi_mp_library_state_t *state){
450     if(likely(state)){
451         if(state->parallel_module_type==
452                 DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
453             state->L3_L4_emitter->~dpi_L3_L4_emitter();
454             free(state->L3_L4_emitter);
455 #if DPI_MULTIPROCESSOR_L3_L4_FARM_TYPE ==\
456     DPI_MULTIPROCESSOR_L3_L4_ORDERED_FARM
457             state->L3_L4_farm->~ff_ofarm();
458 #else
459             state->L3_L4_farm->~ff_farm();
460 #endif
461             free(state->L3_L4_farm);
462             free(state->L3_L4_collector);
463
464             while(!state->L3_L4_workers->empty()){
465                 ((dpi::dpi_L3_L4_worker*) state->
466                         L3_L4_workers->back())->~dpi_L3_L4_worker();
467                 free((dpi::dpi_L3_L4_worker*) state->
468                         L3_L4_workers->back());
469                 state->L3_L4_workers->pop_back();
470             }
471             delete state->L3_L4_workers;
472
473             state->L7_emitter->~dpi_L7_emitter();
474             free(state->L7_emitter);
475             state->L7_farm->~ff_farm();
476             free(state->L7_farm);
477             free(state->L7_collector);
478
479             while(!state->L7_workers->empty()){
480                 ((dpi::dpi_L7_worker*) state->
481                         L7_workers->back())->~dpi_L7_worker();
482                 free((dpi::dpi_L7_worker*) state->L7_workers->back());
483                 state->L7_workers->pop_back();
484             }
485             delete state->L7_workers;
486
487             state->pipeline->~ff_pipeline();
488             free(state->pipeline);
489         }else{
490             delete state->single_farm_emitter;
491             delete state->single_farm;
492             delete state->single_farm_collector;
493             while(!state->single_farm_workers->empty()){
494                 delete (dpi::dpi_L7_worker*) state->
```

```
495                         single_farm_workers->back();
496                 state->single_farm_workers->pop_back();
497             }
498             delete state->single_farm_workers;
499         }
500         dpi_terminate(state->sequential_state);
501
502 #if DPI_MULTIPROCESSOR_USE_TASKS_POOL
503         state->tasks_pool->~SWSR_Ptr_Buffer();
504         free(state->tasks_pool);
505 #endif
506
507         free(state);
508     }
509
510 }
511
512 /**
513  * Sets the reading and processing callbacks. It can be done only after
514  * that the state has been initialized and before calling run().
515  *
516  * @param state                 A pointer to the state of the library.
517  * @param reading_callback      A pointer to the reading callback. It must
518  *                              be different from NULL.
519  * @param processing_callback   A pointer to the processing callback. It
520  *                              must be different from NULL.
521  * @param user_data             A pointer to the user data to be passed to
522  *                              the callbacks.
523  */
524 void mp_dpi_set_read_and_process_callbacks(
525         dpi_mp_library_state_t* state,
526         dpi_mp_packet_reading_callback* reading_callback,
527         dpi_mp_processing_result_callback* processing_callback,
528         void* user_data){
529     state->reading_callback=reading_callback;
530     state->processing_callback=processing_callback;
531     state->read_process_callbacks_user_data=user_data;
532 }
533
534
535 /****************************************/
536 /*          Other API calls             */
537 /****************************************/
538
539
540 /**
541  * This call freezes the library. Basically all the threads are
542  * suspended until another mp_dpi_run(..) is called.
543  * When this call returns all the threads are freezed.
```

```
544  * @param state A pointer to the state of the library.
545  */
546 #ifndef DPI_DEBUG
547 static inline
548 #endif
549 void mp_dpi_freeze(dpi_mp_library_state_t* state){
550 #ifdef DPI_DEBUG
551     assert(!state->is_frozen);
552 #endif
553     if(likely(!state->is_frozen)){
554         debug_print("%s\n","[mp_dpi_api.cpp]: Acquiring freeze lock.");
555         ff::spin_lock(&(state->state_update_lock),
556                 DPI_MULTIPROCESSOR_STATUS_UPDATER_TID);
557         debug_print("%s\n","[mp_dpi_api.cpp]: Freeze lock acquired, "
558                     "sending freeze message.");
559         if(state->parallel_module_type==
560                 DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
561             state->pipeline->freeze();
562         }else{
563             state->single_farm->freeze();
564         }
565
566         state->freeze_flag=1;
567
568         debug_print("%s\n","[mp_dpi_api.cpp]: Freeze message sent, "
569                     "wait for freezing.");
570         if(state->parallel_module_type==
571                 DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
572             state->pipeline->wait_freezing();
573             assert(state->pipeline->isfrozen());
574         }else{
575             state->single_farm->wait_freezing();
576             assert(state->single_farm->isfrozen());
577         }
578         debug_print("%s\n","[mp_dpi_api.cpp]: Skeleton freezed.");
579         state->is_frozen=1;
580     }
581 }
582
583 /**
584  * Starts the library.
585  * @param state A pointer to the state of the library.
586  */
587 void mp_dpi_run(dpi_mp_library_state_t* state){
588 #ifdef DPI_DEBUG
589     assert(state->is_frozen);
590 #endif
591     if(state->is_frozen){
592         /**
```

```
593          * If is frozen because it has been terminated, then we can't
594          * run it, so return.
595          **/
596         if(unlikely(state->terminating)){
597             debug_print("%s\n","[mp_dpi_api.cpp]: ATTENTION! Attempt "
598                         "to run the library after that it terminated.");
599             return;
600         }
601         state->freeze_flag=0;
602
603         debug_print("%s\n","[mp_dpi_api.cpp]: Starting...");
604         if(state->parallel_module_type==
605                 DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
606             assert(state->pipeline->run_then_freeze()>=0);
607         }else{
608             assert(state->single_farm->run_then_freeze()>=0);
609         }
610         debug_print("%s\n","[mp_dpi_api.cpp]: Skeleton started, "
611                     "unlocking state update lock.");
612         ff::spin_unlock(&(state->state_update_lock),
613                 DPI_MULTIPROCESSOR_STATUS_UPDATER_TID);
614         debug_print("%s\n\n","[mp_dpi_api.cpp]: State update "
615                     "lock unlocked.");
616         state->is_frozen=0;
617     }
618 }
619
620 /**
621  * Wait the end of the data processing.
622  * @param state A pointer to the state of the library.
623  */
624 void mp_dpi_wait_end(dpi_mp_library_state_t* state){
625     if(state->parallel_module_type==DPI_MP_PARELLELISM_FORM_DOUBLE_FARM){
626         assert(state->pipeline->wait()>=0);
627     }else{
628         assert(state->single_farm->wait()>=0);
629     }
630     state->is_frozen=1;
631 }
632
633
634 /****************************************/
635 /*        Status change API calls       */
636 /****************************************/
637
638
639 /**
640  * Sets the maximum number of times that the library tries to guess the
641  * protocol. During the flow protocol identification, after this number
```

```
642  * of trials, in the case in which it cannot decide between two or more
643  * protocols, one of them will be chosen, otherwise DPI_PROTOCOL_UNKNOWN
644  * will be returned.
645  * @param state       A pointer to the state of the library.
646  * @param max_trials  The maximum number of trials.
647  *
648  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
649  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
650  *         been changed because a problem happened.
651  */
652 u_int8_t mp_dpi_set_max_trials(dpi_mp_library_state_t *state,
653                                u_int16_t max_trials){
654     if(state->is_frozen){
655         return dpi_set_max_trials(state->sequential_state, max_trials);
656     }else{
657         u_int8_t r;
658         mp_dpi_freeze(state);
659         r=dpi_set_max_trials(state->sequential_state, max_trials);
660         mp_dpi_run(state);
661         return r;
662     }
663 }
664
665
666
667 /**
668  * Enable IPv4 defragmentation.
669  * @param state        A pointer to the library state.
670  * @param table_size   The size of the table to be used to store IPv4
671  *                     fragments informations.
672  *
673  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
674  *          updated. DPI_STATE_UPDATE_FAILURE if the state has not
675  *         been changed because a problem happened.
676  */
677 u_int8_t mp_dpi_ipv4_fragmentation_enable(dpi_mp_library_state_t *state,
678                                           u_int16_t table_size){
679     if(state->is_frozen){
680         return dpi_ipv4_fragmentation_enable(state->sequential_state,
681                                              table_size);
682     }else{
683         u_int8_t r;
684         mp_dpi_freeze(state);
685         r=dpi_ipv4_fragmentation_enable(state->sequential_state,
686                                         table_size);
687         mp_dpi_run(state);
688         return r;
689     }
690 }
```

```
691
692 /**
693  * Enable IPv6 defragmentation.
694  * @param state        A pointer to the library state.
695  * @param table_size   The size of the table to be used to store IPv6
696  *                     fragments informations.
697  *
698  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
699  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
700  *         been changed because a problem happened.
701  */
702 u_int8_t mp_dpi_ipv6_fragmentation_enable(dpi_mp_library_state_t *state,
703                                           u_int16_t table_size){
704     if(state->is_frozen){
705         return dpi_ipv6_fragmentation_enable(state->sequential_state,
706                                              table_size);
707     }else{
708         u_int8_t r;
709         mp_dpi_freeze(state);
710         r=dpi_ipv6_fragmentation_enable(state->sequential_state,
711                                         table_size);
712         mp_dpi_run(state);
713         return r;
714     }
715 }
716
717 /**
718  * Sets the amount of memory that a single host can use for IPv4
719  * defragmentation.
720  * @param state                   A pointer to the library state.
721  * @param per_host_memory_limit   The maximum amount of memory that
722  *                                 any IPv4 host can use.
723  *
724  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
725  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
726  *         been changed because a problem happened.
727  */
728 u_int8_t mp_dpi_ipv4_fragmentation_set_per_host_memory_limit(
729         dpi_mp_library_state_t *state,
730         u_int32_t per_host_memory_limit){
731     if(state->is_frozen){
732         return dpi_ipv4_fragmentation_set_per_host_memory_limit(
733                 state->sequential_state,
734                 per_host_memory_limit);
735     }else{
736         u_int8_t r;
737         mp_dpi_freeze(state);
738         r=dpi_ipv4_fragmentation_set_per_host_memory_limit(
739                 state->sequential_state,
```

```
740                 per_host_memory_limit);
741         mp_dpi_run(state);
742         return r;
743     }
744 }
745
746 /**
747  * Sets the amount of memory that a single host can use for IPv6
748  * defragmentation.
749  * @param state                    A pointer to the library state.
750  * @param per_host_memory_limit    The maximum amount of memory that
751  *                                 any IPv6 host can use.
752  *
753  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
754  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
755  *         been changed because a problem happened.
756  */
757 u_int8_t mp_dpi_ipv6_fragmentation_set_per_host_memory_limit(
758         dpi_mp_library_state_t *state,
759         u_int32_t per_host_memory_limit){
760     if(state->is_frozen){
761         return dpi_ipv6_fragmentation_set_per_host_memory_limit(
762                 state->sequential_state,
763                 per_host_memory_limit);
764     }else{
765         u_int8_t r;
766         mp_dpi_freeze(state);
767         r=dpi_ipv6_fragmentation_set_per_host_memory_limit(
768                 state->sequential_state,
769                 per_host_memory_limit);
770         mp_dpi_run(state);
771         return r;
772     }
773 }
774
775 /**
776  * Sets the total amount of memory that can be used for IPv4
777  * defragmentation.
778  * If fragmentation is disabled and then enabled, this information
779  * must be passed again.
780  * Otherwise default value will be used.
781  * @param state               A pointer to the state of the library
782  * @param totel_memory_limit  The maximum amount of memory that can
783  *                             be used for IPv4 defragmentation.
784  *
785  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
786  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
787  *         been changed because a problem happened.
788  */
```

```
789 u_int8_t mp_dpi_ipv4_fragmentation_set_total_memory_limit(
790         dpi_mp_library_state_t *state,
791         u_int32_t total_memory_limit){
792     if(state->is_frozen){
793         return dpi_ipv4_fragmentation_set_total_memory_limit(
794                 state->sequential_state, total_memory_limit);
795     }else{
796         u_int8_t r;
797         mp_dpi_freeze(state);
798         r=dpi_ipv4_fragmentation_set_total_memory_limit(
799                 state->sequential_state, total_memory_limit);
800         mp_dpi_run(state);
801         return r;
802     }
803 }
804
805 /**
806  * Sets the total amount of memory that can be used for
807  * IPv6 defragmentation.
808  * If fragmentation is disabled and then enabled, this information
809  * must be passed again.
810  * Otherwise default value will be used.
811  * @param state               A pointer to the state of the library
812  * @param totel_memory_limit  The maximum amount of memory that can
813  *                            be used for IPv6 defragmentation.
814  *
815  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
816  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
817  *         been changed because a problem happened.
818  */
819 u_int8_t mp_dpi_ipv6_fragmentation_set_total_memory_limit(
820         dpi_mp_library_state_t *state,
821         u_int32_t total_memory_limit){
822     if(state->is_frozen){
823         return dpi_ipv6_fragmentation_set_total_memory_limit(
824                 state->sequential_state, total_memory_limit);
825     }else{
826         u_int8_t r;
827         mp_dpi_freeze(state);
828         r=dpi_ipv6_fragmentation_set_total_memory_limit(
829                 state->sequential_state, total_memory_limit);
830         mp_dpi_run(state);
831         return r;
832     }
833 }
834
835 /**
836  * Sets the maximum time (in seconds) that can be spent to
837  * reassembly an IPv4 fragmented datagram.
```

```
838  * Is the maximum time gap between the first and last fragments
839  * of the datagram.
840  * @param state            A pointer to the state of the library.
841  * @param timeout_seconds  The reassembly timeout.
842  *
843  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
844  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the
845  *         state has not been changed because a problem happened.
846  */
847 u_int8_t mp_dpi_ipv4_fragmentation_set_reassembly_timeout(
848         dpi_mp_library_state_t *state,
849         u_int8_t timeout_seconds){
850     if(state->is_frozen){
851         return dpi_ipv4_fragmentation_set_reassembly_timeout(
852                 state->sequential_state, timeout_seconds);
853     }else{
854         u_int8_t r;
855         mp_dpi_freeze(state);
856         r=dpi_ipv4_fragmentation_set_reassembly_timeout(
857                 state->sequential_state, timeout_seconds);
858         mp_dpi_run(state);
859         return r;
860     }
861 }
862
863 /**
864  * Sets the maximum time (in seconds) that can be spent to reassembly
865  * an IPv6 fragmented datagram.
866  * Is the maximum time gap between the first and last fragments of
867  * the datagram.
868  * @param state            A pointer to the state of the library.
869  * @param timeout_seconds  The reassembly timeout.
870  *
871  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
872  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
873  *         been changed because a problem happened.
874  */
875 u_int8_t mp_dpi_ipv6_fragmentation_set_reassembly_timeout(
876         dpi_mp_library_state_t *state,
877         u_int8_t timeout_seconds){
878     if(state->is_frozen){
879         return dpi_ipv6_fragmentation_set_reassembly_timeout(
880                 state->sequential_state, timeout_seconds);
881     }else{
882         u_int8_t r;
883         mp_dpi_freeze(state);
884         r=dpi_ipv6_fragmentation_set_reassembly_timeout(
885                 state->sequential_state, timeout_seconds);
886         mp_dpi_run(state);
```

```
887         return r;
888     }
889 }
890
891 /**
892  * Disable IPv4 defragmentation.
893  * @param state A pointer to the state of the library.
894  *
895  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
896  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the
897  *         state has not been changed because a problem happened.
898  */
899 u_int8_t mp_dpi_ipv4_fragmentation_disable(dpi_mp_library_state_t *state){
900     if(state->is_frozen){
901         return dpi_ipv4_fragmentation_disable(state->sequential_state);
902     }else{
903         u_int8_t r;
904         mp_dpi_freeze(state);
905         r=dpi_ipv4_fragmentation_disable(state->sequential_state);
906         mp_dpi_run(state);
907         return r;
908     }
909 }
910
911 /**
912  * Disable IPv6 defragmentation.
913  * @param state A pointer to the state of the library.
914  *
915  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
916  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
917  *         been changed because a problem happened.
918  */
919 u_int8_t mp_dpi_ipv6_fragmentation_disable(dpi_mp_library_state_t *state){
920     if(state->is_frozen){
921         return dpi_ipv6_fragmentation_disable(state->sequential_state);
922     }else{
923         u_int8_t r;
924         mp_dpi_freeze(state);
925         r=dpi_ipv6_fragmentation_disable(state->sequential_state);
926         mp_dpi_run(state);
927         return r;
928     }
929 }
930
931
932
933 /**
934  * If enabled, the library will reorder out of order TCP packets
935  * (enabled by default).
```

```
936  * @param state  A pointer to the state of the library.
937  *
938  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
939  *         successfully updated. DPI_STATE_UPDATE_FAILURE if the state
940  *         has not been changed because a problem happened.
941  */
942 u_int8_t mp_dpi_tcp_reordering_enable(dpi_mp_library_state_t* state){
943     if(state->is_frozen){
944         return dpi_tcp_reordering_enable(state->sequential_state);
945     }else{
946         u_int8_t r;
947         mp_dpi_freeze(state);
948         r=dpi_tcp_reordering_enable(state->sequential_state);
949         mp_dpi_run(state);
950         return r;
951     }
952 }
953
954 /**
955  * If it is called, the library will not reorder out of order TCP packets.
956  * Out-of-order segments will be delivered to the inspector as they
957  * arrive. This means that the inspector may not be able to identify
958  * the application protocol. Moreover, if there are callbacks saved
959  * for TCP based protocols, if TCP reordering is disabled, the
960  * extracted informations could be erroneous or incomplete.
961  * @param state A pointer to the state of the library.
962  *
963  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
964  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
965  *         been changed because a problem happened.
966  */
967 u_int8_t mp_dpi_tcp_reordering_disable(dpi_mp_library_state_t* state){
968     if(state->is_frozen){
969         return dpi_tcp_reordering_disable(state->sequential_state);
970     }else{
971         u_int8_t r;
972         mp_dpi_freeze(state);
973         r=dpi_tcp_reordering_disable(state->sequential_state);
974         mp_dpi_run(state);
975         return r;
976     }
977 }
978
979 /**
980  * Enable a protocol inspector.
981  * @param state         A pointer to the state of the library.
982  * @param protocol      The protocol to enable.
983  *
984  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
```

```
985  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
986  *         been changed because a problem happened.
987  */
988 u_int8_t mp_dpi_set_protocol(dpi_mp_library_state_t *state,
989                              dpi_protocol_t protocol){
990     if(state->is_frozen){
991         return dpi_set_protocol(state->sequential_state, protocol);
992     }else{
993         u_int8_t r;
994         mp_dpi_freeze(state);
995         r=dpi_set_protocol(state->sequential_state, protocol);
996         mp_dpi_run(state);
997         return r;
998     }
999 }
1000
1001 /**
1002  * Disable a protocol inspector.
1003  * @param state       A pointer to the state of the library.
1004  * @param protocol    The protocol to disable.
1005  *
1006  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
1007  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
1008  *         been changed because a problem happened.
1009  */
1010 u_int8_t mp_dpi_delete_protocol(dpi_mp_library_state_t *state,
1011                                 dpi_protocol_t protocol){
1012     if(state->is_frozen){
1013         return dpi_delete_protocol(state->sequential_state, protocol);
1014     }else{
1015         u_int8_t r;
1016         mp_dpi_freeze(state);
1017         r=dpi_delete_protocol(state->sequential_state, protocol);
1018         mp_dpi_run(state);
1019         return r;
1020     }
1021 }
1022
1023 /**
1024  * Enable all the protocol inspector.
1025  * @param state      A pointer to the state of the library.
1026  *
1027  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
1028  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
1029  *         been changed because a problem happened.
1030  */
1031 u_int8_t mp_dpi_inspect_all(dpi_mp_library_state_t *state){
1032     if(state->is_frozen){
1033         return dpi_inspect_all(state->sequential_state);
```

```
1034     }else{
1035         u_int8_t r;
1036         mp_dpi_freeze(state);
1037         r=dpi_inspect_all(state->sequential_state);
1038         mp_dpi_run(state);
1039         return r;
1040     }
1041 }
1042
1043 /**
1044  * Disable all the protocol inspector.
1045  * @param state      A pointer to the state of the library.
1046  *
1047  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
1048  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
1049  *         been changed because a problem happened.
1050  */
1051 u_int8_t mp_dpi_inspect_nothing(dpi_mp_library_state_t *state){
1052     if(state->is_frozen){
1053         return dpi_inspect_nothing(state->sequential_state);
1054     }else{
1055         u_int8_t r;
1056         mp_dpi_freeze(state);
1057         r=dpi_inspect_nothing(state->sequential_state);
1058         mp_dpi_run(state);
1059         return r;
1060     }
1061 }
1062
1063
1064 /**
1065  * Sets the callback that will be called when a flow expires.
1066  * (Valid only if stateful API is used).
1067  * @param state     A pointer to the state of the library.
1068  * @param cleaner   The callback used to clear the user state.
1069  *
1070  * @return DPI_STATE_UPDATE_SUCCESS If the state has been
1071  *         successfully updated. DPI_STATE_UPDATE_FAILURE if
1072  *         the state has not been changed because a problem
1073  *         happened.
1074  */
1075 u_int8_t mp_dpi_set_flow_cleaner_callback(
1076         dpi_mp_library_state_t* state,
1077         dpi_flow_cleaner_callback* cleaner){
1078     if(state->is_frozen){
1079         return dpi_set_flow_cleaner_callback(
1080                 state->sequential_state, cleaner);
1081     }else{
1082         u_int8_t r;
```

```
1083         mp_dpi_freeze(state);
1084         r=dpi_set_flow_cleaner_callback(
1085                 state->sequential_state, cleaner);
1086         mp_dpi_run(state);
1087         return r;
1088     }
1089 }
1090
1091 /**
1092  * Sets callbacks informations. When a protocol is identified the
1093  * default behavior is to not inspect the packets belonging to that
1094  * flow anymore and keep simply returning the same protocol identifier.
1095  *
1096  * If a callback is enabled for a certain protocol, then we keep
1097  * inspecting all the new flows with that protocol in order to
1098  * invoke the callbacks specified by the user on the various parts
1099  * of the message. Moreover, if the application protocol uses TCP,
1100  * then we have the additional cost of TCP reordering for all the
1101  * segments. Is highly recommended to enable TCP reordering if it is
1102  * not already enabled (remember that is enabled by default).
1103  * Otherwise the informations extracted could be erroneous/incomplete.
1104  *
1105  * The pointers to the data passed to the callbacks are valid only for
1106  * the duration of the callback.
1107  *
1108  * @param state       A pointer to the state of the library.
1109  * @param callbacks   A pointer to HTTP callbacks.
1110  * @param user_data   A pointer to global user HTTP data. This pointer
1111  *                    will be passed to any HTTP callback when it is
1112  *                    invoked.
1113  *
1114  * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
1115  *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
1116  *         been changed because a problem happened.
1117  *
1118  **/
1119 u_int8_t mp_dpi_http_activate_callbacks(
1120         dpi_mp_library_state_t* state,
1121         dpi_http_callbacks_t* callbacks,
1122         void* user_data){
1123     if(state->is_frozen){
1124         return dpi_http_activate_callbacks(state->sequential_state,
1125                                            callbacks,
1126                                            user_data);
1127     }else{
1128         u_int8_t r;
1129         mp_dpi_freeze(state);
1130         r=dpi_http_activate_callbacks(state->sequential_state,
1131                                       callbacks,
```

```
                                        user_data);
        mp_dpi_run(state);
        return r;
    }
}

/**
 * Remove the internal structure used to store callbacks informations.
 * user_data is not freed/modified.
 * @param state       A pointer to the state of the library.
 *
 * @return DPI_STATE_UPDATE_SUCCESS If the state has been successfully
 *         updated. DPI_STATE_UPDATE_FAILURE if the state has not
 *         been changed because a problem happened.
 */
u_int8_t mp_dpi_http_disable_callbacks(dpi_mp_library_state_t* state){
    if(state->is_frozen){
        return dpi_http_disable_callbacks(state->sequential_state);
    }else{
        u_int8_t r;
        mp_dpi_freeze(state);
        r=dpi_http_disable_callbacks(state->sequential_state);
        mp_dpi_run(state);
        return r;
    }
}
```

B.3 Test

B.3.1 Performance test

```
/*
 * test.c
 *
 *  Created on: 12/nov/2012
 *      Author: daniele
 */


#define NUMBER_L3_WORKERS 1
#define NUMBER_L7_WORKERS 2

#include <time.h>



#include "../mp_dpi_api.h"
#include <pcap.h>
```

```
18 #include <net/ethernet.h>
19 #include <time.h>
20 #include <sys/socket.h>
21 #include <netinet/in.h>
22 #include <arpa/inet.h>
23 #include <unistd.h>
24 #include <stdlib.h>
25 #include <string.h>
26 #include <stdio.h>
27 #include <inttypes.h>
28 #include <assert.h>
29 #include <ff/mapping_utils.hpp>
30 #include <ff/utils.hpp>
31
32 #if DPI_NUMA_AWARE
33 #include <numa.h>
34 #endif
35
36 #define CLOCK_FREQ 2000000000L
37
38 typedef struct pcap_packets_memory{
39     unsigned char* const * packets;
40     const u_int32_t* sizes;
41     u_int32_t num_packets;
42     u_int32_t next_packet_to_sent;
43     u_int32_t performed_iterations;
44     u_int32_t iterations_to_perform;
45 }pcap_packets_memory_t;
46
47 void dummy_callback(
48         dpi_http_message_informations_t* http_message_informations,
49         const unsigned char* body_chunk, u_int32_t body_chunk_length,
50         dpi_pkt_infos_t* pkt_informations,
51         void** flow_specific_user_data, void* user_data,
52         u_int8_t last_chunk){
53     return;
54 }
55
56 dpi_mp_packet_reading_result_t reading_cb(void* user_data){
57   static unsigned long last_ts=getticks();
58   static u_int32_t last_sec=0;
59   pcap_packets_memory_t* packets=(pcap_packets_memory_t*) user_data;
60
61   dpi_mp_packet_reading_result_t r;
62
63   if(packets->next_packet_to_sent==packets->num_packets){
64     ++packets->performed_iterations;
65     packets->next_packet_to_sent=0;
66   }
```

```
67
68    if(packets->performed_iterations<packets->iterations_to_perform){
69      r.pkt=packets->packets[packets->next_packet_to_sent];
70    }else{
71      r.pkt=NULL;
72      printf("Sending EOS!\n");
73      fflush(stdout);
74    }
75    if(getticks()-last_ts>CLOCK_FREQ){
76      last_ts=getticks();
77      ++last_sec;
78    }
79    r.current_time=last_sec;
80    r.length=packets->sizes[packets->next_packet_to_sent];
81    r.user_pointer=NULL;
82    ++packets->next_packet_to_sent;
83    return r;
84  }
85
86
87  void processing_cb(mp_dpi_processing_result_t* processing_result,
88                     void* user_data){
89      ;
90  }
91
92  #define CAPACITY_CHUNK 1000
93  #define AVAILABLE_PROCESSORS 16
94  static u_int16_t mapping[AVAILABLE_PROCESSORS]=
95      {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
96
97
98  int main(int argc, char** argv){
99      if(argc!=3){
100         fprintf(stderr, "Usage: %s pcap_file num_iterations\n", argv[0]);
101         return -1;
102     }
103     ff_mapThreadToCpu(mapping[0], -20);
104     char* pcap_filename=argv[1];
105     u_int32_t iterations_to_perform=atoi(argv[2]);
106
107     pcap_t *handle;
108     char errbuf[PCAP_ERRBUF_SIZE];
109
110     printf("Open offline.\n");
111     handle=pcap_open_offline(pcap_filename, errbuf);
112
113     if(handle==NULL){
114         fprintf(stderr, "Couldn't open device %s: %s\n",
115                 pcap_filename, errbuf);
```

```
116         return (2);
117     }
118
119
120     int datalink_type=pcap_datalink(handle);
121
122     if(datalink_type!=DLT_EN10MB){
123         fprintf(stderr, "Datalink type not supported\n");
124         exit(-1);
125     }
126
127
128     const u_char* packet;
129     struct pcap_pkthdr header;
130     unsigned char** packets;
131     u_int32_t* sizes;
132     u_int32_t num_packets=0;
133     u_int32_t current_capacity=0;
134     assert(posix_memalign((void**) &packets, DPI_CACHE_LINE_SIZE,
135             (sizeof(unsigned char*)*CAPACITY_CHUNK)+
136                 DPI_CACHE_LINE_SIZE)==0);
137     assert(posix_memalign((void**) &sizes, DPI_CACHE_LINE_SIZE,
138             (sizeof(u_int32_t)*CAPACITY_CHUNK)+
139                 DPI_CACHE_LINE_SIZE)==0);
140
141     current_capacity+=CAPACITY_CHUNK;
142     while((packet=pcap_next(handle, &header))!=NULL){
143         if((((struct ether_header*) packet)->ether_type)!=
144                 htons(ETHERTYPE_IP) &&
145             (((struct ether_header*) packet)->ether_type!=
146                     htons(ETHERTYPE_IPV6))){
147             continue;
148         }
149
150         if(num_packets==current_capacity){
151           current_capacity+=CAPACITY_CHUNK;
152           void* tmp;
153
154           tmp=realloc(packets, sizeof(unsigned char*)*current_capacity);
155           assert(tmp);
156           packets=(unsigned char**) tmp;
157
158           tmp=realloc(sizes, sizeof(u_int32_t)*current_capacity);
159           assert(tmp);
160           sizes=(u_int32_t*) tmp;
161
162         }
163
164         assert(header.len>sizeof(struct ether_header));
```

```
165         sizes[num_packets]=(header.len-sizeof(struct ether_header));
166 #if DPI_NUMA_AWARE
167         packets[num_packets]=(unsigned char*) numa_alloc_onnode(
168                 sizeof(unsigned char)*sizes[num_packets],
169                        DPI_NUMA_AWARE_PACKETS_NODE);
170         assert(packets[num_packets]);
171 #else
172         assert(posix_memalign((void**) &(packets[num_packets]),
173                 DPI_CACHE_LINE_SIZE,
174                 sizeof(unsigned char)*sizes[num_packets])==0);
175 #endif
176         assert(packets[num_packets]);
177         memcpy(packets[num_packets],
178                packet+sizeof(struct ether_header),
179                sizes[num_packets]);
180         ++num_packets;
181     }
182     printf("Read %"PRIu32" packets.\n", num_packets);
183     pcap_close(handle);
184     pcap_packets_memory_t *x;
185
186     posix_memalign((void**) &x, DPI_CACHE_LINE_SIZE,
187                    sizeof(pcap_packets_memory_t)+DPI_CACHE_LINE_SIZE);
188
189     x->packets=packets;
190     x->sizes=sizes;
191     x->performed_iterations=0;
192     x->next_packet_to_sent=0;
193     x->iterations_to_perform=iterations_to_perform;
194     x->num_packets=num_packets;
195
196
197     dpi_mp_parallelism_details_t details;
198     bzero(&details, sizeof(dpi_mp_parallelism_details_t));
199     details.available_processors=AVAILABLE_PROCESSORS;
200     details.mapping=mapping;
201
202     details.parallelism_form=DPI_MP_PARELLELISM_FORM_DOUBLE_FARM;
203     details.double_farm_num_L3_workers=NUMBER_L3_WORKERS;
204     details.double_farm_num_L7_workers=NUMBER_L7_WORKERS;
205
206     printf("Initializing the library...\n");
207     dpi_mp_library_state_t* state=mp_dpi_init_stateful(
208             32771, 32771, 1000000, 1000000, details);
209     printf("Library initialized...\n");
210
211     /**
212      * The callbacks actually does nothing. Anyway this is needed for
213      * the following reason: once a flow protocol is identified, if a
```

```
214      * callback is not present, the packets belonging to the identified
215      * flow are no more parsed. Using a callback we 'force' the library to
216      * always parse the packet to extract informations needed for the
217      * callback.
218      */
219     dpi_http_callbacks_t cback={0, 0, 0, 0, 0, &dummy_callback};
220     //mp_dpi_http_activate_callbacks(state, &cback, NULL);
221
222
223     //mp_dpi_tcp_reordering_disable(state);
224     //mp_dpi_ipv4_fragmentation_disable(state);
225     //mp_dpi_ipv6_fragmentation_disable(state);
226
227     mp_dpi_set_read_and_process_callbacks(
228             state, &reading_cb, &processing_cb, (void*) x);
229     printf("Running the library...\n");
230     mp_dpi_run(state);
231
232     mp_dpi_wait_end(state);
233     mp_dpi_print_stats(state);
234     mp_dpi_terminate(state);
235
236     u_int32_t i;
237     for(i=0; i<num_packets; i++){
238 #if DPI_NUMA_AWARE
239         numa_free(packets[i], sizes[i]);
240 #else
241         free(packets[i]);
242 #endif
243     }
244     free(packets);
245     free(sizes);
246
247     return 0;
248 }
```

B.3.2 Demo application

```
1  #define _POSIX_C_SOURCE 1
2  #include <cstdlib>
3  #include <cstring>
4  #include <fstream>
5  #include <iomanip>
6  #include <iostream>
7  #include <stdexcept>
8  #include <utility>
9  #include <typeinfo>
10 #include "timer.h"
11 #include "trie.h"
```

```
12 #include "signatures.h"
13 /** Starting with demo-only includes. **/
14 #include "../mp_dpi_api.h"
15 #include <pcap.h>
16 #include <netinet/in.h>
17 #include <net/ethernet.h>
18 #include <stdint.h>
19 #include <ff/mapping_utils.hpp>
20 #include <ff/utils.hpp>
21 #include <ff/ubuffer.hpp>
22
23 using namespace antivirus;
24 #define CAPACITY_CHUNK 1000
25 #define SCANNER_POOL_SIZE 4096
26 #define CLOCK_FREQ 2000000000L
27
28
29 #define AVAILABLE_CORES 16
30 static u_int16_t mapping[AVAILABLE_CORES]=
31     {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
32
33 static ff::uSWSR_Ptr_Buffer* scanner_pool;
34
35 typedef struct pcap_packets_memory{
36     unsigned char** packets;
37     u_int32_t* sizes;
38     u_int32_t num_packets;
39     u_int32_t next_packet_to_sent;
40     u_int32_t performed_iterations;
41     u_int32_t iterations_to_perform;
42 }pcap_packets_memory_t;
43
44 dpi_mp_packet_reading_result_t reading_cb(void* user_data){
45     static unsigned long last_ts=getticks();
46     static u_int32_t last_sec=0;
47
48     dpi_mp_packet_reading_result_t r;
49
50     pcap_packets_memory_t* packets=(pcap_packets_memory_t*) user_data;
51
52     if(packets->next_packet_to_sent==packets->num_packets){
53         ++packets->performed_iterations;
54         packets->next_packet_to_sent=0;
55     }
56
57     if(packets->performed_iterations<packets->iterations_to_perform){
58         r.pkt=packets->packets[packets->next_packet_to_sent];
59     }else{
60         r.pkt=NULL;
```

```
61          printf("Sending EOS!\n");
62          fflush(stdout);
63      }
64      if(getticks()-last_ts>CLOCK_FREQ){
65          last_ts=getticks();
66          ++last_sec;
67      }
68
69      r.current_time=last_sec;
70      r.length=packets->sizes[packets->next_packet_to_sent];
71      ++packets->next_packet_to_sent;
72      return r;
73  }
74
75  void processing_cb(mp_dpi_processing_result_t* processing_result,
76                     void* user_data){
77      ;
78  }
79
80  static void match_found(string::size_type position,
81                          trie::value_type const &match){
82      using namespace std;
83      cout << "Matched '" << match.second << "' at " << position << endl;
84  }
85
86  void body_cb(dpi_http_message_informations_t* http_informations,
87               const u_char* app_data, u_int32_t data_length,
88               dpi_pkt_infos_t* pkt,
89               void** flow_specific_user_data, void* user_data,
90               u_int8_t last){
91      if(*flow_specific_user_data==NULL){
92          if(scanner_pool->mc_pop(flow_specific_user_data)==false){
93              *flow_specific_user_data=
94                      new byte_scanner(*((trie*) user_data), match_found);
95          }
96      }
97      byte_scanner* scanner=(byte_scanner*) (*flow_specific_user_data);
98      for(u_int32_t i=0; i<data_length; i++){
99          scanner->match(app_data[i]);
100     }
101 }
102
103 void flow_cleaner(void* flow_specific_user_data){
104     if(scanner_pool->mp_push(flow_specific_user_data)==false){
105         delete (byte_scanner*) flow_specific_user_data;
106     }
107 }
108
109
```

```
110 int main(int argc, char **argv){
111     using namespace std;
112     ff_mapThreadToCpu(mapping[0], -20);
113
114     try {
115         if (argc<5){
116             cerr << "Usage: " << argv[0] <<
117                     " virus-signatures-file input-file "
118                     "num_iterations par_degree\n";
119             exit(EXIT_FAILURE);
120         }
121
122         string::size_type trie_depth=DEFAULT_TRIE_MAXIMUM_DEPTH;
123
124         char const *virus_signatures_file_name=argv[1];
125         char const *input_file_name=argv[2];
126         u_int16_t num_iterations=atoi(argv[3]);
127         u_int16_t num_workers=atoi(argv[4]);
128
129
130         ifstream signatures;
131         signatures.open(virus_signatures_file_name);
132         if(!signatures){
133             cerr << argv[0] << ": failed to open '" <<
134                     virus_signatures_file_name << "'\n";
135             exit(EXIT_FAILURE);
136         }
137
138         signature_reader reader(signatures);
139
140         cout << "reading '" << virus_signatures_file_name << "'... ";
141
142         timer read_signatures_timer;
143         read_signatures_timer.start();
144         trie t(trie_depth);
145         while (reader.next()) {
146             t.insert(make_pair(reader.current_pattern(),
147                                reader.current_name()));
148         }
149         read_signatures_timer.stop();
150         cout << setiosflags(ios_base::fixed) << setprecision(3) <<
151                 read_signatures_timer.real_time() << " seconds.\n";
152         cout << "preparing '" << virus_signatures_file_name << "'... ";
153         timer prepare_signatures_timer;
154         prepare_signatures_timer.start();
155         t.prepare();
156         prepare_signatures_timer.stop();
157         cout << setiosflags(ios_base::fixed)
158              << setprecision(3)
```

```
159             << prepare_signatures_timer.real_time() << " seconds.\n";
160
161         cout << "# of allocated trie nodes: " << t.node_count() << endl;
162
163         timer full_timer;
164         full_timer.start();
165         /*****************************************************/
166         /*             Start scanning the files.             */
167         /*****************************************************/
168         pcap_t *handle;
169         char errbuf[PCAP_ERRBUF_SIZE];
170
171         dpi_mp_parallelism_details_t details;
172         bzero(&details, sizeof(dpi_mp_parallelism_details_t));
173         details.available_processors=AVAILABLE_CORES;
174         details.mapping=mapping;
175
176         details.single_farm_num_workers=num_workers;
177         dpi_mp_library_state_t* state=mp_dpi_init_stateful(
178                 32767, 32767, 1000000, 1000000, details);
179
180         printf("Open offline.\n");
181         handle=pcap_open_offline(input_file_name, errbuf);
182
183         if(handle==NULL){
184             fprintf(stderr, "Couldn't open device %s: %s\n",
185                     input_file_name, errbuf);
186             exit(EXIT_FAILURE);
187         }
188         const u_char* packet;
189         struct pcap_pkthdr header;
190         unsigned char** packets;
191         u_int32_t* sizes;
192         u_int32_t num_packets=0;
193         u_int32_t current_capacity=0;
194         packets=(unsigned char**)
195                 malloc(sizeof(unsigned char*)*CAPACITY_CHUNK);
196         sizes=(u_int32_t*)
197                 malloc((sizeof(u_int32_t))*CAPACITY_CHUNK);
198         assert(packets);
199         assert(sizes);
200         current_capacity+=CAPACITY_CHUNK;
201         while((packet=pcap_next(handle, &header))!=NULL){
202             if((((struct ether_header*) packet)->ether_type)!=
203                     htons(ETHERTYPE_IP) &&
204                 (((struct ether_header*) packet)->ether_type!=
205                         htons(ETHERTYPE_IPV6))){
206                 continue;
207             }
```

```
208
209             if(num_packets==current_capacity){
210                 packets=(unsigned char**)
211                         realloc(packets, sizeof(unsigned char*)*
212                                 (current_capacity+CAPACITY_CHUNK));
213                 sizes=(u_int32_t*)
214                         realloc(sizes, sizeof(u_int32_t)*
215                                 (current_capacity+CAPACITY_CHUNK));
216                 current_capacity+=CAPACITY_CHUNK;
217                 assert(packets);
218                 assert(sizes);
219             }
220
221             assert(header.len>sizeof(struct ether_header));
222
223             posix_memalign((void**) &(packets[num_packets]),
224                            DPI_CACHE_LINE_SIZE,
225                            sizeof(unsigned char)*
226                                (header.len-sizeof(struct ether_header)));
227             assert(packets[num_packets]);
228             memcpy(packets[num_packets],
229                    packet+sizeof(struct ether_header),
230                    (header.len-sizeof(struct ether_header)));
231             sizes[num_packets]=
232                     (header.len-sizeof(struct ether_header));
233             ++num_packets;
234         }
235         std::cout << "Read " << num_packets << " packets." <<
236                 std::endl;
237         pcap_close(handle);
238         pcap_packets_memory_t x;
239         x.packets=packets;
240         x.sizes=sizes;
241         x.performed_iterations=0;
242         x.next_packet_to_sent=0;
243         x.iterations_to_perform=num_iterations;
244         x.num_packets=num_packets;
245
246
247         scanner_pool=new ff::uSWSR_Ptr_Buffer(SCANNER_POOL_SIZE);
248         scanner_pool->init();
249         for(uint i=0; i<SCANNER_POOL_SIZE; i++){
250             scanner_pool->push(new byte_scanner(t, match_found));
251         }
252         mp_dpi_set_read_and_process_callbacks(
253                 state, &reading_cb, &processing_cb, (void*) &x);
254         mp_dpi_set_flow_cleaner_callback(state, &flow_cleaner);
255         dpi_http_callbacks_t callback={0, 0, 0, 0, 0, &body_cb};
256         mp_dpi_http_activate_callbacks(state, &callback, (void*)(&t));
```

```
257         timer scan_timer;
258         scan_timer.start();
259         mp_dpi_run(state);
260
261
262         mp_dpi_wait_end(state);
263         mp_dpi_print_stats(state);
264         scan_timer.stop();
265
266         byte_scanner* bs;
267         while(!scanner_pool->empty()){
268             scanner_pool->pop((void**) &bs);
269             delete bs;
270         }
271         /* And close the session */
272         mp_dpi_terminate(state);
273         delete scanner_pool;
274
275         full_timer.stop();
276
277         cout << "Completion time: "
278              << setiosflags(ios_base::fixed) << setprecision(3)
279              << full_timer.real_time() << " seconds.\n";
280
281         u_int32_t i;
282         for(i=0; i<num_packets; i++){
283             free(packets[i]);
284         }
285         free(packets);
286         free(sizes);
287         exit(EXIT_SUCCESS);
288     }catch(exception const &ex){
289         cout.flush();
290         cerr << typeid(ex).name() << " exception: " << ex.what() << "\n";
291         throw;
292     }
293 }
```

Bibliography

[1] FastFlow - Website. http://calvados.di.unipi.it/dokuwiki/doku.php?id=ffnamespace:about, 2012. [Online; accessed 23-December-2012].

[2] Inc. eSoft. White Paper - Modern Network Security: The Migration to Deep Packet Inspection. www.esoft.com/content/pdf/dpi-migration-whitepaper.pdf, 2012. [Online; accessed 23-December-2012].

[3] Elfiq App Optimizer. http://www.elfiq.com/appoptimizer. [Online; accessed 14-January-2013].

[4] Meraki - Application QoS. http://www.meraki.com/technologies/application-qos. [Online; accessed 14-January-2013].

[5] Clifton Phua. Protecting organisations from personal data breaches. *Computer Fraud and Security*, 2009(1):13 – 18, 2009.

[6] Martin H. Bosworth. ChoicePoint Settles Data Breach Lawsuit. http://www.consumeraffairs.com/choicepoint. [Online; accessed 14-January-2013].

[7] US Federal Communications Commission. Children's Internet Protection Act. http://www.fcc.gov/guides/childrens-internet-protection-act, 2012. [Online; accessed 23-December-2012].

[8] Research note - Facebook: Measuring the cost to business of social notworking. http://nucleusresearch.com/research/notes-and-reports/facebook-measuring-the-cost-to-business-of-social-notworking/, 2009. [Online; accessed 23-December-2012].

[9] Cisco IOS Netflow. http://www.cisco.com/en/US/products/ps6601/products_ios_protocol_group_home.html, 2012. [Online; accessed 23-December-2012].

[10] Qixiang Sun, Daniel R. Simon, Yi-Min Wang, Wilf Russell, Venkata N. Padmanabhan, and Lili Qiu. Statistical identification of encrypted web browsing traffic. In *IEEE Symposium on Security and Privacy*. Society Press, 2002.

[11] Sebastian Zander, Thuy Nguyen, and Grenville Armitage. Automated traffic classification and application identification using machine learning. In *Proceedings of the The IEEE Conference on Local Computer Networks 30th Anniversary*. IEEE Computer Society, 2005.

[12] Manuel Crotti, Maurizio Dusi, Francesco Gringoli, and Luca Salgarelli. Traffic classification through simple statistical fingerprinting. *SIGCOMM Computer Communication Review*, 37(1):5–16, January 2007.

[13] Craig Labovitz, Scott Iekel-Johnson, Danny McPherson, Jon Oberheide, and Farnam Jahanian. Internet inter-domain traffic. *SIGCOMM Computer Communication Review*, 41(4), August 2010.

[14] Qosmos DeepFlow Probes - Ready-to-Use Network Intelligence. `http://www.qosmos.com/new-qosmos-deepflow-probes-deliver-the-power-of-network-intelligence-in-ready-to-use-formats/`. [Online; accessed 14-January-2013].

[15] G. Ziemba, D. Reed, and P. Traina. RFC 1858 - Security Considerations for IP Fragment Filtering. `http://tools.ietf.org/rfc/rfc1858.txt`, 2012. [Online; accessed 23-December-2012].

[16] Sarang Dharmapurikar and Vern Paxson. Robust TCP stream reassembly in the presence of adversaries. In *Proceedings of the 14th conference on USENIX Security Symposium - Volume 14*. USENIX Association, 2005.

[17] Monther Aldwairi, Thomas Conte, and Paul Franzon. Configurable string matching hardware for speeding up intrusion detection. *SIGARCH Computer Architecture News*, 33(1):99–107, March 2005.

[18] Yaron Weinsberg, Shimrit Tzur-david, and Danny Dolev. High performance string matching algorithm for a network intrusion prevention system (nips). In *High Performance Switching and Routing*, 2006.

[19] Sherif Yusuf and Wayne Luk. Bitwise optimised CAM for Network Intrusion Detection Systems. In *Proceedings of the 2005 International Conference on Field Programmable Logic and Applications*. IEEE, 2005.

[20] OpenDPI - Website. `http://www.opendpi.org/`, 2012. [Online; accessed 23-December-2012].

[21] l7filter - Website. `http://l7-filter.clearfoundation.com/`, 2012. [Online; accessed 23-December-2012].

[22] Shane Alcock and Richard Nelson. Libprotoident: traffic classification using Lightweight Packet Inspection. `www.wand.net.nz/~salcock/lpi/lpi.pdf`. [Online; accessed 12-February-2013].

[23] nDPI - Website. `http://www.ntop.org/products/ndpi/`, 2012. [Online; accessed 23-December-2012].

[24] Yunchun Li and Xinxin Qiao. A parallel packet processing method on multi-core systems. In *Proceedings of the 2011 10th International Symposium on Distributed Computing and Applications to Business, Engineering and Science*. IEEE Computer Society, 2011.

[25] Terry Nelms and Mustaque Ahamad. Packet scheduling for deep packet inspection on multi-core architectures. In *Proceedings of the 6th ACM/IEEE Symposium on Architectures for Networking and Communications Systems*. ACM, 2010.

[26] Tan Li, Fan Yang, Jie Yang, Yinan Dou, and Huanhao Zou. Research of dpi optimization on multi-core platform. In *Broadband Network and Multimedia Technology (IC-BNMT), 2010 3rd IEEE International Conference on*, 2010.

[27] Junchang Wang, Haipeng Cheng, Bei Hua, and Xinan Tang. Practice of parallelizing network applications on multi-core architectures. In *Proceedings of the 23rd international conference on Supercomputing*. ACM, 2009.

[28] Marco Aldinucci, Marco Danelutto, Peter Kilpatrick, and Massimo Torquati. FastFlow: high-level and efficient streaming on multi-core. In Sabri Pllana and Fatos Xhafa, editors, *Programming Multi-core and Many-core Computing Systems*, Parallel and Distributed Computing, chapter 13. Wiley, January 2013.

[29] Maurizio Dusi, Francesco Gringoli, and Luca Salgarelli. Quantifying the accuracy of the ground truth associated with Internet traffic traces. *Computer Networks*, 55(5):1158–1167, April 2011.

[30] Andrew W. Moore and Konstantina Papagiannaki. Toward the accurate identification of network applications. In *PAM*, 2005.

[31] Patrick Haffner, Subhabrata Sen, Oliver Spatscheck, and Dongmei Wang. ACAS: automated construction of application signatures. In *Proceedings of the 2005 ACM SIGCOMM workshop on Mining network data*. ACM, 2005.

[32] James F. Kurose and Keith W. Ross. *Computer Networking: A Top-Down Approach*. Addison-Wesley Publishing Company, USA, 5th edition, 2009.

[33] Mark Handley, Vern Paxson, and Christian Kreibich. Network intrusion detection: evasion, traffic normalization, and end-to-end protocol semantics. In *Proceedings of the 10th conference on USENIX Security Symposium - Volume 10*. USENIX Association, 2001.

[34] Thomas H. Ptacek and Timothy N. Newsham. Insertion, evasion, and denial of service: Eluding network intrusion detection. Technical report, 1998. [Online; accessed 14-January-2013].

[35] Gianni Antichi, Domenico Ficara, Stefano Giordano, Gregorio Procissi, and Fabio Vitucci. Counting bloom filters for pattern matching and anti-evasion at the wire speed. *IEEE Network*, 23(1):30–35, January 2009.

[36] George Varghese, J. Andrew Fingerhut, and Flavio Bonomi. Detecting evasion attacks at high speeds without reassembly. *SIGCOMM Computer Communication Review*, 36(4):327–338, August 2006.

[37] Gerald Tripp. A finite-state-machine based string matching system for intrusion detection on high-speed networks. In *EICAR 2005 Conference Proceedings*, 2005.

[38] Tran Ngoc Thinh, Tran Trung Hieu, Van Quoc Dung, and S. Kittitornkun. A FPGA-based deep packet inspection engine for Network Intrusion Detection System. In *9th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, 2012.

[39] Fang Yu, Randy H. Katz, and T. V. Lakshman. Gigabit rate packet pattern-matching using TCAM. In *Proceedings of the 12th IEEE International Conference on Network Protocols*. IEEE Computer Society, 2004.

[40] Mansoor Alicherry, M. Muthuprasanna, and Vijay Kumar. High speed pattern matching for network IDS/IPS. In *Proceedings of the 2006 IEEE International Conference on Network Protocols*. IEEE Computer Society, 2006.

[41] Snort - Intrusion Detection and Prevention system. `http://www.snort.org/`, 2012. [Online; accessed 23-December-2012].

[42] Martin Roesch. Snort - lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX conference on System administration*. USENIX Association, 1999.

[43] Chaofan Shen and Leijun Huang. On detection accuracy of L7-filter and OpenDPI. *Third International Conference on Networking and Distributed Computing*, pages 119–123, 2012.

[44] WAND - Network Research Group. The case against L7 Filter. `http://www.wand.net.nz/content/case-against-l7-filter`, December 2012. [Online; accessed 14-January-2013].

[45] Murray Cole. *Algorithmic skeletons: structured management of parallel computation*. MIT Press, Cambridge, MA, USA, 1991.

[46] Marco Vanneschi. The programming model of ASSIST, an environment for parallel and distributed portable applications. *Parallel Computing*, 28(12):1709–1732, December 2002.

[47] Bruno Bacci, Marco Danelutto, Salvatore Orlando, Susanna Pelagatti, and Marco Vanneschi. P3L: A structured high level programming language and its structured support. *Concurrency Practice and Experience*, 7(3):225–255, May 1995.

[48] Marco Aldinucci and Marco Danelutto. Skeleton-based parallel programming: Functional and parallel semantics in a single shot. *Computer Languages, Systems and Structures*, 33(3-4):179–192, October 2007.

[49] M. Vanneschi. *Course Notes of High Performance Systems and Enabling Platforms - Master Program in Computer Science and Networking*. 2011.

[50] The Muenster Skeleton Library Muesli - A Comprehensive Overview. `http://www.ercis.org/de/node/230`, 2009. [Online; accessed 13-February-2013].

[51] Philipp Ciechanowicz and Herbert Kuchen. Enhancing muesli's data parallel skeletons for multi-core computer architectures. In *Proceedings of the 2010 IEEE 12th International Conference on High Performance Computing and Communications*. IEEE Computer Society, 2010.

[52] OpenMP - Website. `http://openmp.org/wp/`. [Online; accessed 14-January-2013].

[53] Message Passing Interface (MPI) - Website. `http://www.mcs.anl.gov/research/projects/mpi/`. [Online; accessed 14-January-2013].

[54] S. Ernsting and H. Kuchen. Data parallel skeletons for GPU clusters and multi-GPU systems. In *Proceedings of ParCo 2011*. IOS Press, 2012.

[55] Marco Aldinucci, Massimo Torquati, and Massimiliano Meneghin. Fastflow: Efficient parallel streaming applications on multi-core. *Computing Research Repository*, abs/0909.1187, 2009.

[56] Marco Aldinucci, Massimiliano Meneghin, and Massimo Torquati. Efficient smith-waterman on multi-core with fastflow. In Marco Danelutto, Tom Gross, and Julien Bourgeois, editors, *Proceedings of International Euromicro PDP 2010: Parallel Distributed and network-based Processing*, Pisa, Italy, February 2010. IEEE.

[57] Kiminori Matsuzaki, Hideya Iwasaki, Kento Emoto, and Zhenjiang Hu. A library of constructive skeletons for sequential style of parallel programming. In *Proceedings of the 1st international conference on Scalable information systems*. ACM, 2006.

[58] Kento Emoto, Kiminori Matsuzaki, Zhenjiang Hu, and Masato Takeichi. Domain-specific optimization strategy for skeleton programs. In *Proceedings of Euro-Par 2007*. Springer, 2007.

[59] Akimasa Morihata, Kiminori Matsuzaki, Zhenjiang Hu, and Masato Takeichi. The third homomorphism theorem on trees: downward & upward lead to divide-and-conquer. *SIGPLAN Notices*, 44(1):177–185, January 2009.

[60] Johan Enmyren and Christoph W. Kessler. SkePU: a multi-backend skeleton programming library for multi-GPU systems. In *Proceedings of the fourth international workshop on High-level parallel programming and applications*. ACM, 2010.

[61] CUDA - Website. http://www.nvidia.com/object/cuda_home_new.html. [Online; accessed 14-January-2013].

[62] OpenCL - Website. http://www.khronos.org/opencl/. [Online; accessed 14-January-2013].

[63] M. Danelutto, L. Deri, and D. De Sensi. Network monitoring on multicores with algorithmic skeletons. In *Proceedings of ParCo 2011*. IOS Press, 2012.

[64] Massimo Torquati. Single-Producer/Single-Consumer queues on shared cache multi-core systems. *Computing Research Repository*, abs/1012.1824, 2010.

[65] Marco Aldinucci, Marco Danelutto, Peter Kilpatrick, Massimiliano Meneghin, and Massimo Torquati. An efficient unbounded lock-free queue for multi-core systems. In *Proceedings of 18th International Euro-Par 2012 Parallel Processing*. Springer, 2012.

[66] Leslie Lamport. Specifying concurrent program modules. *ACM Transactions on Programming Languages and Systems*, 5(2):190–222, April 1983.

[67] John Giacomoni. Fastforward for efficient pipeline parallelism: A cache-optimized concurrent lock-free queue. In *Proceedings of the The 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM Press, 2008.

[68] HPC Advisory Council. Interconnect Analysis: 10GigE and InfiniBand in High Performance Computing. http://www.mellanox.com/blog/2009/10/interconnect-analysis-infiniband-and-10gige-in-high-performance/, 2009. [Online; accessed 12-February-2013].

[69] Mellanox Technologies. InfiniBand clustering. Delivering better price/performance than Ethernet. http://www.mellanox.com/pdf/whitepapers/IB_vs_Ethernet_Clustering_WP_100.pdf. [Online; accessed 12-February-2013].

[70] Infiniband trade association Website. http://www.infinibandta.org/. [Online; accessed 2-January-2013].

[71] Myricom Website. http://www.myricom.com/. [Online; accessed 2-January-2013].

[72] Libzero. http://www.ntop.org/products/pf_ring/libzero-for-dna/. [Online; accessed 2-January-2013].

[73] PFRing with Direct NIC Access. http://www.ntop.org/products/pf_ring/dna/. [Online; accessed 2-January-2013].

[74] Augonnet Cédric. Interval-based registration cache for zero-copy protocols. http://cedric-augonnet.com/wp-content/uploads/2012/02/Rapport-StageM1.pdf, 2007. [Online; accessed 12-February-2013].

[75] Hiroshi Tezuka, Francis O'Carroll, Atsushi Hori, and Yutaka Ishikawa. Pin-down cache: A virtual memory management technique for zero-copy communication. In *Proceedings of the 12th. International Parallel Processing Symposium on International Parallel Processing Symposium*. IEEE Computer Society, 1998.

[76] Myrinet. Myri10GE performances. http://www.myricom.com/scs/performance/Myri10GE/. [Online; accessed 2-January-2013].

[77] Performances of PFRing with Direct NIC Access. http://www.ntop.org/pf_ring/pf_ring-dna-rfc-2544-benchmark/. [Online; accessed 2-January-2013].

[78] W. Simpson. RFC 1853 - IP in IP Tunneling. http://tools.ietf.org/html/rfc1853, 2012. [Online; accessed 23-December-2012].

[79] Information Sciences Institute University of Southern California. RFC 791 - Internet Protocol Specification. http://tools.ietf.org/rfc/rfc791.txt, 2012. [Online; accessed 23-December-2012].

[80] Fowler/Noll/Vo hash function. http://www.isthe.com/chongo/tech/comp/fnv/. [Online; accessed 2-January-2013].

[81] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall Professional Technical Reference, 2nd edition, 1988.

[82] Murmur3 hash function. http://code.google.com/p/smhasher/. [Online; accessed 2-January-2013].

[83] Joyent Inc. HTTP Parser. https://github.com/joyent/http-parser, 2012. [Online; accessed 11-January-2013].

[84] NLnet Labs - Signature based antivirus. http://www.nlnetlabs.nl/downloads/antivirus/, 2012. [Online; accessed 14-January-2013].

[85] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: an aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, June 1975.

[86] D. Moore, C. Shannon, and k. claffy. Characteristics of fragmented traffic on Internet links. In *Internet Measurement Workshop (IMW)*, 2001.

[87] Libpcap - Website. `http://www.tcpdump.org/pcap.htm`, 2012. [Online; accessed 14-January-2013].

[88] CAIDA Association. The CAIDA UCSD Anonymized Internet Traces 2011 - 19/05/2011. `http://www.caida.org/data/passive/passive_2011_dataset.xml`. [Online; accessed 23-December-2012].

[89] CAIDA - Website. `http://www.caida.org/home/`, 2012. [Online; accessed 23-December-2012].

[90] Anand Balachandran, Geoffrey M. Voelker, Paramvir Bahl, and P. Venkat Rangan. CRAWDAD trace ucsd/sigcomm2001/tcpdump/08292005 (v. 2002-04-23). `http://crawdad.cs.dartmouth.edu/ucsd/sigcomm2001/tcpdump/08292005`, April 2002. [Online; accessed 14-January-2013].

[91] Massachusetts Institute of Technology: Lincoln Laboratory. DARPA intrusion detection evaluation data set. `www.ll.mit.edu/mission/communications/cyber/CSTcorpora/ideval/data/1999/training/week1/index.html`. [Online; accessed 14-January-2013].

[92] Alfred Aho. *Compilers, principles, techniques, and tools.* Addison-Wesley Pub. Co, Reading, Mass, 1986.